LLM-SERVER-LEETCODE-150 · NANO-SERVE

写一个推理服务器，顺手做完 117 道 LeetCode

系统设计 · 逐题算法讲解 · 对比讲解（contrastive explanation）

117
个练习，全部在引擎里被真实调用

63
道来自 Top Interview 150 ★

54
道其它 LeetCode

858
个测试（672 单元 + 186 集成）

M1 单请求生成 | M2 服务器与调度 | M3 分页 KV Cache | M4 前缀缓存与截止时间索引 | M5 模型 IR 与配置 | M6 解码与评测 | M7 可观测性与扩缩容

github.com/yiyuanli939/LLM-Server-Leetcode-150 · 2026 年 10 月

目录

如何使用这份讲义 2

第一部分　系统设计 **3**

第 0 章　总览：一个由 117 道题拼成的推理服务器 4

第 1 章　单请求生成：分词、采样、流式输出、stop 检测与扣留 10

第 2 章　服务器：连续 batching、准入控制、集群与离线规划 18

第 3 章　分页 KV Cache：块表、共享、驱逐与自检 26

第 4 章　前缀缓存 + EDF 截止时间队列 + checkpoint：让服务器记住前缀、按截止时间排队、能停… 34

第 5 章　模型 IR 编译 + 配置系统：把模型变成可分析的算子图，把部署变成可校验的配置 42

第 6 章　解码与评测：推测解码、离线评测、基准网格与穷举 fuzz 50

第 7 章　可观测性与自动扩缩容：文本面板、健康感知路由与闭环容量规划 57

第二部分　117 道题逐题讲解 **65**

M1　单请求生成（12 题） 66

M2　服务器与调度（25 题） 86

M3　分页 KV Cache（30 题） 129

M4　前缀缓存与截止时间索引（16 题） 181

M5　模型 IR 与配置（17 题） 208

M6　解码与评测（8 题） 238

M7　可观测性与扩缩容（9 题） 252

第三部分　按模式索引：训练「认模式」 **269**

附录 A　117 题完整对应表 282

附录 B　被删掉的 87 道 Top Interview 150 题 286

如何使用这份讲义

这份讲义配套仓库 `github.com/yiyuanli939/LLM-Server-Leetcode-150`（nano-serve）。它包含参考实现，**是剧透**：建议做题时只看仓库里 stub 的 docstring，做完或卡住再翻到对应的讲解页。

第一部分

系统设计

总览 + 七个里程碑各一章：问题、架构、关键取舍（每个都与替代方案对比）、文件导读、实测数字。

第二部分

117 道题逐题讲解

63 道 ★ Top Interview 150 + 54 道其它 LeetCode。每题：引擎角色、题目、洞见、算法、复杂度、参考实现、对比讲解、易错点、模式信号。

第三部分

按模式索引

同一模式的题放在一起，配识别信号与相邻模式的分界，用来复盘和训练「认模式」。

每道题的版面

引擎里的角色：调用点（文件::函数）、输入从哪来、输出给谁、为什么需要这个复杂度

核心洞见：把问题化归成什么

对比讲解：为什么用这个而不是那个；什么条件下另一种更好；与兄弟练习的信号差别

易错点：边界条件与常见错误

颜色

M1 单请求生成　M2 服务器与调度　M3 分页 KV Cache　M4 前缀缓存与截止时间索引

M5 模型 IR 与配置　M6 解码与评测　M7 可观测性与扩缩容

「★」标记来自 LeetCode Top Interview 150 的题。LC 编号与英文题名便于你去 LeetCode 上冷启动重写。参考实现由脚本从仓库的 `reference/` 自动抽取，与 `solutions` 标签中的参考解一致。

第一部分

系统设计

总览，以及七个里程碑各自的问题、架构与设计取舍。

第 0 章　总览：一个由 117 道题拼成的推理服务器

> nano-serve 是一个能端到端跑通的迷你 LLM 推理服务器；它的 117 个关键函数/类各等价于一道 LeetCode，而其余的框架代码负责把它们接成一个真实系统。

这个项目想解决什么问题

刷题和做系统是两种割裂的训练：刷题时你知道这是一道「堆」题，但不知道真实系统里哪里会用到堆；做系统时你会用到堆，但往往直接 `import heapq` 或者抄一个库，算法本身没有被训练到。

nano-serve 把两者缝在一起：它是一个结构上对标 vLLM / SGLang 的推理服务器（连续 batching、分页 KV cache、前缀缓存、投机解码、可观测性、自动扩缩容），但其中每一个「算法性」的零件都被抠出来做成练习。练习的题面只写领域需求（例如「运行中 batch 的槽位分配器：总是分配编号最小的空闲槽」），不写题号。你得先从需求里认出模式（这里是「最小堆 + 计数器」），再实现它；实现正确后，它立刻在引擎里被真实调用。

这带来两个好处：一是**认模式**被训练到了——面试里最难的一步正是从描述中识别出该用什么，而题目列表恰恰把这一步替你做掉了；二是你能看到**为什么**需要某个复杂度：比如前缀缓存的命中长度为什么要二分、EDF 队列为什么不能只用堆。

117 道题是怎么选出来的

最初的设计是把 LeetCode Top Interview 150 全部塞进引擎。逐题审视后，只有 63 道能**自然地**放进推理服务器——即引擎本来就需要这段代码。其余 87 道里，56 道属于「换壳」（算法不变，硬套一个领域故事，例如把股票买卖说成现货算力交易），23 道根本放不进去（罗马数字、N 皇后、数独），还有 8 道虽然最初看似自然，但在最终架构里找不到真实调用点（例如前缀缓存最终用 Trie 实现后，二叉树的层平均值就没有用处了）。这 87 道全部被删掉。

引擎还缺的组件，用 LeetCode 其它题来补，标准相同：必须有真实调用点。这样补了 54 道，例如 LFU Cache（淘汰策略）、Design Memory Allocator（显存分配）、Course Schedule IV（算子能否并行）、Wildcard Matching（模型路由规则）、Capacity To Ship Packages（容量规划）。最终 117 = 63（★ Top Interview 150）+ 54（其它 LeetCode）。

「真实调用点」不是一句口号，而是可检查的：`tools/callsites.py` 在参考模式下用 coverage 跑全部集成测试，检查 117 个练习的参考实现每一个都被执行到（当前 117/117）。附录 B 列出了被删掉的 87 道题及原因。

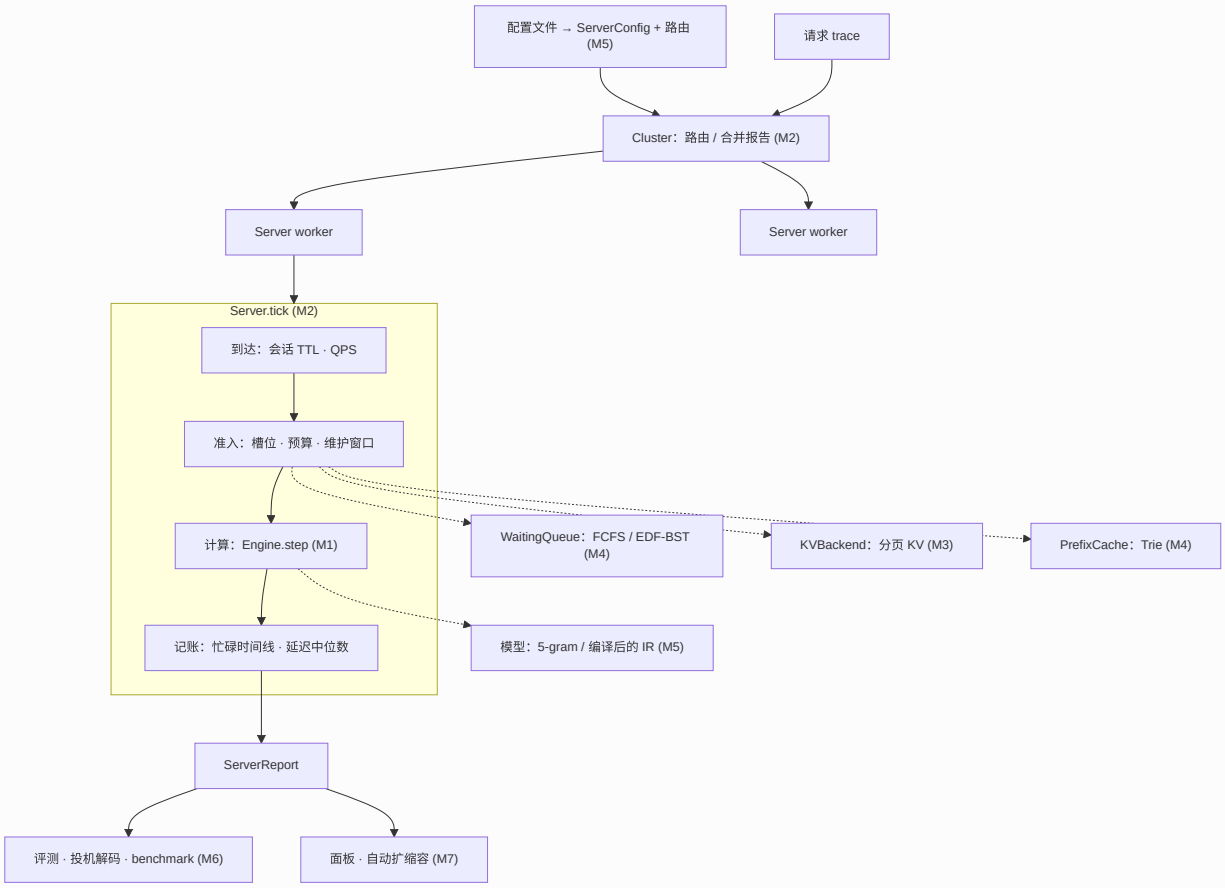


图 · nano-serve 的分层架构（括号里是里程碑）

整体架构：一个 tick 里发生了什么

时间是模拟的：1 tick = 运行中 batch 的一次 decode step。这让所有调度行为都是确定性的、可复现的、可以在测试里断言的，也让整个系统在笔记本 CPU 上秒级跑完。

每个 tick，`Server.tick()` 依次做四件事：
① **到达**：新请求进入；检查会话 token 的 TTL（`server.tokens`），统计近 3 秒 QPS（`metrics.qps`），放入等待队列。
② **准入**：只要有空闲 batch 槽位（`sched.slots`）、不在维护窗口内（`sched.maintenance`）、token 预算预留可行（`sched.capacity_ok`）、KV cache 放得下（M3），就从等待队列（FCFS 或 M4 的 EDF）取出请求，查前缀缓存（M4）缩短 prefill，创建解码状态。
③ **计算**：每个运行中的请求做一个 prefill tick 或一个 decode step（M1 的 `Engine.step`）；KV 不够时抢占最近准入的请求（重算）。
④ **记账**：忙碌时间线（`sched.insert_busy`）、队列长度、完成的请求释放槽位/KV/预留，延迟进入在线中位数（`metrics.median`）。

在 Server 之上，M2 的 `Cluster` 把多个 Server 组成集群并路由；M5 的配置系统从配置文件生成 `ServerConfig` 与路由规则；M6 在 Server 之上做评测、投机解码和 benchmark；M7 读取 `ServerReport` 渲染面板并做容量规划。

练习与框架的契约

每个练习是一个被 `@exercise("<id>")` 装饰的函数或类，学习版里函数体是 `raise TODO(...)`。框架代码（非练习）是完整、可读的，并且**只通过 stub 的模块路径**导入练习，例如 `from nanoserve.kvcache.lru import LRUBlockCache`。

装饰器根据环境变量决定用谁的实现：`NANOSERVE_REFERENCE=1` 时全部替换成 `reference/` 里的参考解（看完成后的引擎）；`NANOSERVE_REFERENCE=auto` 时只替换「还没动过」的练习（用 AST 判断函数体是否仍只是 `raise TODO`），你写过的都跑你自己的代码。后者让你可以跳着做：先写 M3 的 KV cache，让它跑在参考实现的 M1/M2 之上。

两条铁律保证了「练习是真的被用」：框架**只调用、不重写**练习的算法；框架负责建立练习规格要求的前置条件（例如严格递增的时间戳、互不相同的键）。当某个练习原题的约束与真实调用场景冲突时（例如原题只允许小写字母，而 stop string 可以是任意字符），我们放宽练习的规格而不是在框架里加一层编码转换——算法不变，测试补上。

测试策略：两层测试 + 一个调用点检查

单元测试 `tests/lc/`（672 个）：每个练习至少 4–5 个用例，包含原题示例（换成领域语言）、边界情况，以及固定种子的随机测试，与测试文件里手写的暴力解对拍。对有复杂度要求的题，部分测试会真正卡住朴素解（统计元素读取次数、超大值域、SIGALRM 限时）。

集成测试 `tests/domain/`（186 个，18 个文件）：每个里程碑一组，断言的是**行为性质**而不是具体生成文本。最核心的不变量是「**批处理不改变输出**」：任何请求在服务器里（任意 batch 大小、任意抢占、任意集群路由、开不开前缀缓存）生成的文本，必须与它单独跑 `Engine.generate` 完全一致。其他例子：EDF 在构造的截止时间 trace 上比 FCFS 多满足截止时间；KV 预算很小时会抢占但不泄漏；投机解码与贪心解码逐 token 相同；自动扩缩容推荐的集群规模达标而少一台不达标。

调用点检查 `tools/callsites.py`：在参考模式下用 coverage 跑集成测试，确认 117 个练习都被执行到。三者合起来回答三个问题：练习写对了吗？系统作为整体对吗？练习真的是系统的一部分吗？

玩具模型与真实系统：什么一样，什么不一样

模型是 numpy 写的插值 5-gram：`p = floor/V + Σ λ_k · dist_k(context[-k:])`，`logits = log p`。它生成的文本很傻，但对推理**服务器**来说，模型只是一个 `logits(context) -> 向量` 的黑盒；围绕它的一切——batching、KV 块的分配与共享、前缀复用、调度、流式输出、投机解码的验证——都与真实引擎是同一套逻辑。

不一样的地方需要心里有数：没有真实的 GPU 计算，所以「prefill 一个 chunk 需要 1 tick」「KV 块按 token 数计」都是成本模型；没有并发与异步 I/O，所以网络、内核调度、CUDA graph 之类的话题不在范围内。这是有意的取舍：把注意力集中在调度与内存管理的算法上，并让每个行为都可确定性地测试。

怎么读这份讲义

第一部分是系统设计：本章总览之后，每个里程碑一章，讲问题、架构、关键取舍（每个取舍都与替代方案对比）、框架文件导读和实测数字。第二部分是 117 道题的逐题讲解，按里程碑排列；每题包括：它在引擎里做什么、规格、核心洞见、算法步骤、复杂度、参考实现、**对比讲解**（为什么用这个而不是那个，什么时候另一种更好，与仓库里兄弟练习的区别）、易错点、模式信号。第三部分是按模式的索引：同一模式的题放在一起看，训练「认模式」。附录是完整对应表和被删题目清单。

建议的用法：先读第一部分的总览和你要做的里程碑那一章；做题时**先不要**看第二部分对应的那一页，只看仓库里 stub 的 docstring；做完或卡住时再看讲解，重点看「对比讲解」。

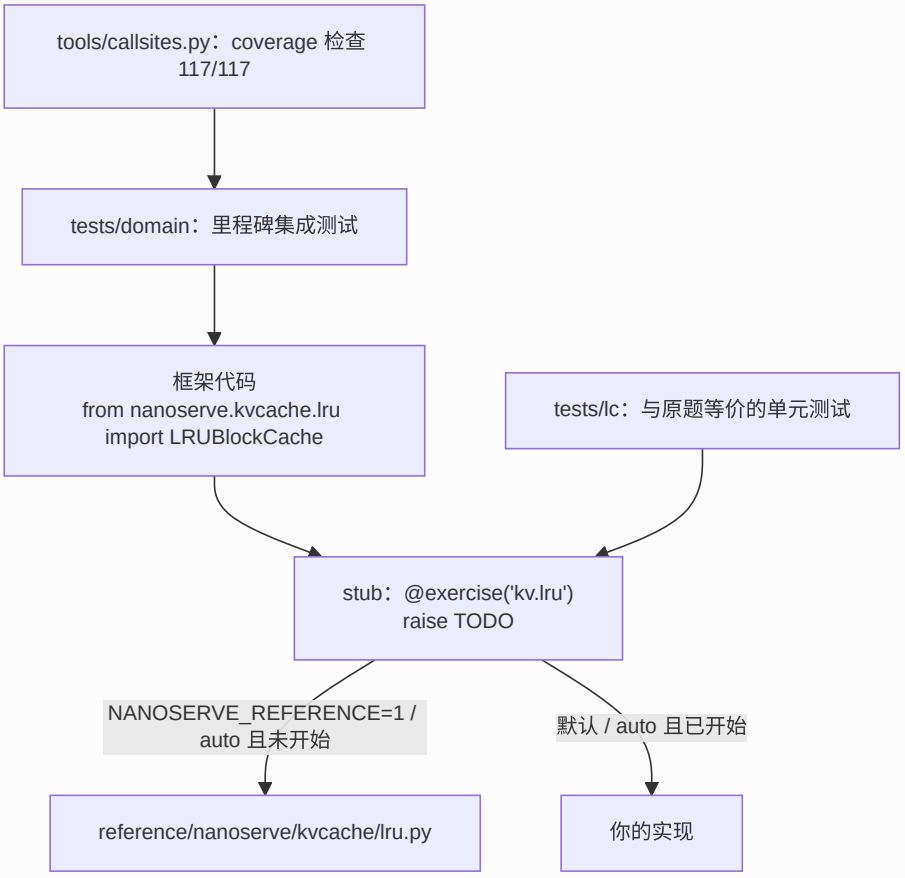


图 · 练习、参考解与框架之间的关系

关键设计取舍

模拟时间（tick）而不是真实时钟

✓ 我们的选择

1 tick = batch 的一次 decode step；到达、截止时间、TTL、维护窗口都以 tick 计。所有调度行为确定、可复现，测试可以精确断言「第几个 tick 准入了谁」。

× 真实时钟 + 线程/asyncio

更接近生产，但结果依赖机器负载，测试只能做统计性断言；调度算法的教学价值被并发噪声淹没。适合做压测，不适合做练习。

× 离散事件模拟（按事件时间跳跃）

空闲期可以跳过，长 trace 更快；但 decode 本来就是每步都有事件的稠密过程，按 tick 推进更简单直观。

依据 全部 186 个集成测试在约 9 秒内跑完；同一 trace 的任意到达顺序排列得到完全相同的输出集合（M6 的确定性测试）。

插件协议（KVBackend / PrefixCacheBackend / WaitingQueue）而不是一个大 Server

✓ 我们的选择

Server 核心只依赖三个协议；M3 的 BlockManager、M4 的 PrefixCache 与 DeadlineQueue 各自实现一个协议，由 `ServerConfig` 决定装哪个。没装时用 Null 实现。

× 在 Server 里直接写 KV/前缀/调度逻辑

代码更少，但里程碑之间强耦合：没做完 M3 就跑不了 M4；也无法在测试中替换成故意出错的实现做故障注入。

× 继承体系（KVServer(Server) …）

组合爆炸：KV × 前缀 × 调度策略需要 2×2×2 个子类；协议 + 组合是线性的。

依据 M6 的 benchmark 网格自动发现可用插件并组合测试；M7 的面板测试注入了一个人为限制 token 数的 KV 后端来制造抢占。

参考解与 stub 同路径镜像 + 装饰器切换

✓ 我们的选择

`reference/<同路径>.py` 放参考解；装饰器在导入时按环境变量替换。框架永远从 stub 路径导入。

× 把参考解写在 stub 里、用开关分支

学习版就会泄露答案；而且框架与参考解耦合。

× 依赖注入（把练习实现作为参数传给框架）

最灵活，但每个调用点都要多一个参数，框架代码变得难读；装饰器方案让框架代码看起来就像普通的 import。

依据 `NANOSERVE_REFERENCE=auto` 在一个只实现了 kv.lru 的副本上：kv.lru 用学习者代码、其余用参考解，全部集成测试通过。

放宽练习规格，而不是在框架里加编码层

✓ 我们的选择

当原题约束（只允许小写字母、时间从 1 开始、值域很小）与真实调用冲突时，放宽 stub 规格、确认参考解成立、补测试。

× 保持原题约束，框架里把输入编码成满足约束的形式

练习与原题完全一致，但框架里出现与业务无关的「字母重映射」「坐标平移」代码，恰恰是本项目想避免的不自然。

依据 这样放宽的练习约 30 个（例如 Trie 接受任意字符、merge_free 支持 10^4 规模、capacity_ok 去掉时间上限）；顺带发现并修复了两个参考解在 `$` 字符上的真实 bug。

框架文件导读

`nanoserve/_exercise.py`	练习注册与参考解切换（1 / auto 两种模式）。
`nanoserve/engine.py`	M1 生成循环：单步 API（start/step）与流式输出。

`nanoserve/server/server.py`	M2 服务器核心：每 tick 的到达、准入、计算、记账，以及插件装配。
`nanoserve/server/plugins.py`	三个插件协议及默认实现。
`nanoserve/__main__.py`	命令行入口与各里程碑 demo 的子命令分发。
`tools/progress.py`	进度面板（无剧透 / reveal）。
`tools/callsites.py`	维护者自检：每个练习都被集成测试真实执行。

实测数字

- 117 个练习：63 道 ★ Top Interview 150 + 54 道其它 LeetCode
- 测试：tests/lc 672 个 + tests/domain 186 个 = 858 个，参考模式全绿，约 15 秒
- 调用点检查：117/117 个练习被集成测试执行到
- 代码规模：框架 + stub 约 1.15 万行，参考解约 2500 行，测试约 1 万行
- 11 个命令行 demo，参考模式下每个都在 1.2 秒内完成

建议的完成顺序

按 M1 → M7 的顺序做最顺：M1 让单请求生成跑起来；M2 的服务器核心依赖 M1；M3/M4 是 M2 的插件；M5–M7 相对独立。借助 `NANOSERVE_REFERENCE=auto`，任何顺序都可以：例如对内存管理最感兴趣，就直接从 M3 开始。每个里程碑内部，建议先做被框架核心路径调用的练习（模块 docstring 的 "Exercises this file calls" 里列在前面的），再做报告/监控类的。

第 1 章 · M1

第 1 章　单请求生成：分词、采样、流式输出、stop 检测与扣留

> M1 让引擎能把一个 prompt 变成一串可流式发送的文本：分词 → 逐 token 采样 → 增量判断 stop → 只把「确定不会再变」的文本发出去，并且以可被 M2 批处理复用的逐步（step-wise）接口实现。

问题：一个请求从 prompt 到流式文本要经过什么

一次生成的外部契约很简单：输入 prompt 和 `GenerationConfig`（`max_tokens`、`temperature`、`top_k`、`seed`、`stop`、`no_repeat_window`、`word_stream`），输出一串文本块（chunk），拼起来等于最终结果，并给出 `finish_reason`（`stop` 或 `length`）。难点不在「生成」本身，而在三个流式约束：

1）**每个 chunk 一旦发出就不能撤回**。如果 stop 是 " the queue."，而输出此刻以 " the" 结尾，这 4 个字符可能是 stop 的开头，现在发出去，下一步发现 stop 成立时就收不回来了。
2）**每步的开销不能随已生成长度增长**。生成 n 个 token 若每步都重扫全文，总代价是 $O(n^2)$；引擎里所有每步执行的检查（stop 判断、扣留计算、半词扣留）都设计成只看尾部的 O(L)。
3）**可复现**。同一 seed 必须得到同一文本（`test_deterministic_with_seed`），批处理时还要与其他请求的交错顺序无关。

`nanoserve/engine.py` 用一个循环满足这三点：`start` 编码 prompt、建好每请求状态；`step` 做一次「logits → 采样 → 追加 → stop 检查 → 计算可安全发出的前缀」；`stream` / `generate` 只是把 `step` 循环到 `done`。M1 的 12 个练习全部挂在这条路径上：分词用 `tok.top_pieces`（建词表）和 `tok.segment`（编码前校验），采样用 `sample.kth_largest`（top-k 阈值）和 `sample.cdf_index`（按 CDF 抽样），流式用 `stream.multi_stop`、`stream.find_stop`、`stream.stop_overlap`、`stream.holdback`，护栏与外围用 `stream.repeat_window`、`stream.unique_run`、`stream.justify`、`tok.common_prefix`。

分词：字符 + 高频词的词表，校验后贪心最长匹配

`nanoserve/tokenizer/tokenizer.py::Tokenizer` 的词表 = 语料中出现过的全部单字符（34 个，含换行、空格、`,`、`.`）+ 语料里最常见的长度 ⩾3 的英文单词，加前导空格成为多字符 token（25 个，如 ' the'、' engine'、' queue'）。挑高频词用 `top_k_pieces`：语料切出 341 个候选片段、167 个不同片段，`n_words=48` 但只有 25 个出现次数 ⩾3（`min_count`），再把 k 往下调到频次断点上，使「前 k 个」唯一——这正是练习规格里「保证答案唯一」的来源。合并后的 token 按 (-次数, 字典序) 排序再编号，保证词表 id 每次启动都一样，模型统计和 seed 复现才有意义。

编码 `encode` 先调用 `can_segment(text, vocab)`（Word Break DP）确认文本能被词表完整拼出，否则抛 `UnknownCharacterError`；CLI 接住后用 `sanitize` 把未知字符换成空格（试试 "Hé 🚀 test"：H、é、🚀 都不在语料里）。通过校验后做贪心最长匹配：从位置 i 起，长度从 `max_len`=10 往下试，命中就吞掉。整段语料 2169 字符编码成 1451 个 token（约 1.49 字符/token）。注意前导空格的效果："The cache" 是 4 个 token（T、h、e、' cache'），而 "The scheduler" 是 13 个——行首的 The 前面没有空格，' scheduler' 也不在前 25 名里，只能逐字符切。

一个值得想清楚的细节：一般情况下「可切分」并不保证「贪心能切完」（词表 {ab, abc, cd} 切 "abcd"，DP 说可以：ab+cd；贪心先吞 abc，剩下 d 卡死）。这里之所以能在 `encode` 的 `for…else` 分支写「can_segment guarantees we never get here」，是因为每个语料字符本身都是 token，贪心永远可以退到长度 1。换成一个裁掉了稀有字符的词表，就必须改成 DP 记录回溯指针、直接求出切分，而不是先校验再贪心。

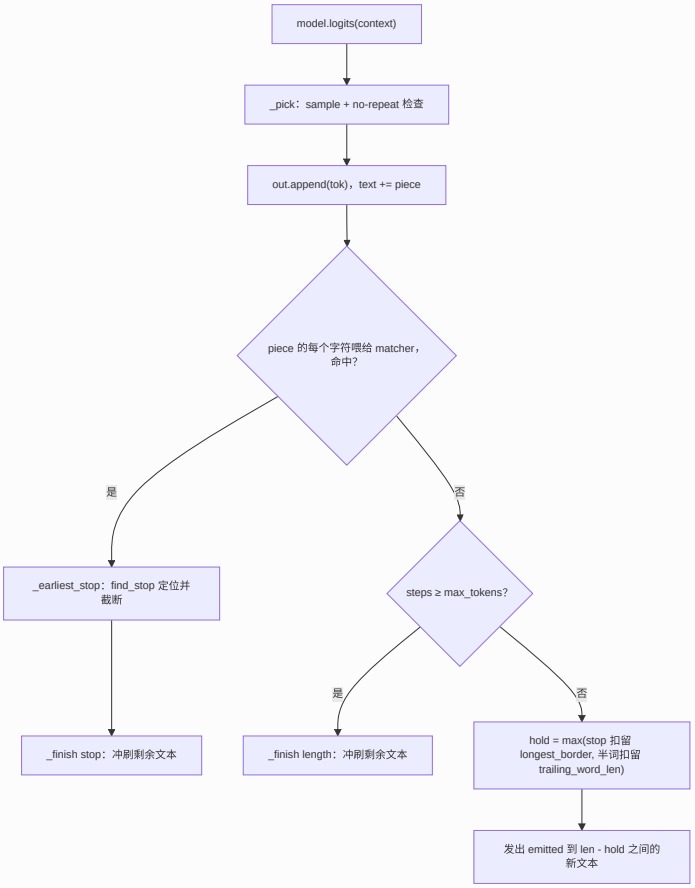


图 · Engine.step 一步内部的控制流

采样：温度 → top-k 截断 → 在 CDF 上二分

`nanoserve/sampling/sampler.py::sample` 把 logits 变成一个 token id：

1）把 `banned` 里的 token 置 −inf（no-repeat 窗口用）；全部被禁则报错。

2）`temperature <= 0` 直接 `argmax`（贪心解码，不碰两个练习）。

3）除以温度；若 `0 < top_k <` 有限值个数，用 `kth_largest` 求第 k 大的值作阈值，把 `< thr` 的置 −inf。注意是「严格小于」才删，所以与阈值相等的并列值全部保留，实际候选可能多于 k 个。

4）`p = exp(x - max(x))`（减最大值防溢出），只取 `p > 0` 的候选，对它们求前缀和得到 CDF，取 `r = rng.random() * cdf[-1]`，用 `cdf_index`（lower bound）找到第一个 `cdf[i] >= r` 的位置，`min(i, len-1)` 兜底浮点误差。

两点设计意图。其一，只对候选（而不是整个词表）建 CDF：被截掉的 token 概率是 0，若留在 CDF 里就会产生相等的相邻元素；lower bound 语义下零宽区间本来也不会被选中，但只保留候选让 CDF 严格递增、长度只有 ≈k。其二，随机源是每请求一个 `random.Random(cfg.seed)`，存在 `DecodeState.rng` 里，不用全局 `random`：M2 把很多请求交错 `step`，如果共享一个全局 RNG，某个请求的输出会取决于它和谁排在同一批——可复现性就没了。`test_greedy_matches_top1` 也检验了这条链路：`temperature=0` 与 `top_k=1` 的结果一致。

逐步 API：DecodeState 和 step

引擎没有写成一个 `for _ in range(max_tokens)` 的大函数，而是把一个在途请求需要的一切放进 `DecodeState`（`prompt_ids`、`out`、`text`、`emitted`、`rng`、`matcher`、`steps`、`done`、`finish_reason`），`Engine` 本身只持有只读的分词器和模型。`step(st)` 做恰好一次解码并返回「这一步新变得可发送」的文本（可能是空串）。

这样做的回报在 M2 才兑现：`nanoserve/server/server.py::_admit` 用 `engine.start` 为每个请求建状态，`_compute` 每个 tick 对所有在跑的槽位各调一次 `engine.step(req.state)`，可以在任意两步之间插入别的请求、做 chunked prefill、抢占（`_make_room`）或因 KV 不够把请求踢掉；`nanoserve/kvcache/workloads.py::parallel_sample` 和 `nanoserve/decode/speculative.py` 也直接驱动 `start/step`。`server/cluster.py` 的注释点明了前提：模型只读、所有每请求状态都在 `DecodeState` 里，多个副本才能共享一个 `Engine`。M1 的 `stream` 只是这个接口的最简单客户端（一个请求、循环到 done）。

stop 检测：增量判断「是否」，一次性定位「在哪」

每一步，新 token 的文本 `piece` 被逐字符喂给 `MultiStopMatcher.feed`（反向 Trie + 长度为最长 stop 的 deque），每个字符 O(L)，L 为最长 stop 长度，与已生成多少无关。代码写成 `hit = st.matcher.feed(ch) or hit`：先调用 feed 再做或运算，保证 piece 里每个字符都被喂进去，matcher 的缓冲区不会和真实文本错位。

matcher 只回答「刚才是否有某个 stop 收尾」，不回答「从哪里截」。命中后 `_earliest_stop` 对每个 stop 调一次 `find_stop`（KMP）在全文中找首次出现位置，取最小者截断，然后 `_finish(st, "stop")`。这个全文扫描每个请求只发生一次（结束时），O(|text| · stop 个数) 的代价被摊到整次生成上；而如果每步都对全文 `find`，就是 $O(n^2)$。语义上引擎截在「所有 stop 中最早出现的起点」：之前的步骤没有命中，说明所有出现都在最后这个 piece 里收尾，find_stop 不会找到更早的「漏网」位置。`test_multiple_stop_strings_cut_at_earliest` 验证了这一点：同一 seed 去掉 stop 后的输出以截断结果为前缀，且下一个字符正是某个 stop。stop 只匹配生成部分（matcher 只喂输出字符，find_stop 只扫 `st.text`），prompt 里出现 stop 不会让请求立刻结束。

扣留：什么时候文本才算「安全」可发

没有命中 stop、也没到 `max_tokens` 时，`step` 计算要扣住多少尾部字符 `hold`，发出 `text[emitted : len(text) - hold]`。扣留有两个来源：

stop 扣留（`_stop_holdback`）：对每个长度 >1 的 stop s，取尾部 `tail = text[-(len(s)-1):]`，计算 `len(longest_border(s + "\x00" + tail))`——即 tail 最长的、同时是 s 前缀的后缀长度。分隔符 \x00 不出现在 stop 和语料里，保证 border 不会跨过它；tail 只取 len(s)−1 个字符，所以每个 stop 的代价是 O(|s|)。长度为 1 的 stop 不需要扣留：它要么已经出现（已经命中），要么没有任何「半成品」。真实轨迹（stop=" the queue."，seed=5，逐字符流）里，输出以 "…watches the" 结尾时扣留 4 个字符（" the"），以 "…a t" 结尾时扣 2 个，以空格结尾时扣 1 个，其余时刻扣 0。

半词扣留（`word_stream=True`，默认）：若文本末字符不是空白，用 `trailing_word_len` 扣住最后那个还可能被下一个 token 接长的单词。它只把 ' ' 当分隔符，所以 "metrics.\n\nThe" 会被当成一个 13 字符的「词」整体扣住——偏保守（晚发），但不会发出半个词，`test_word_stream_emits_whole_words` 要求除最后一块外每个 chunk 都以空白结尾。

两个约束取 `max`：两者都是「扣住一段后缀」，并集就是更长的那段后缀。结束时 `_finish` 把剩下的全部冲刷出去（stop 情况下是截断后的文本）。`test_stop_holdback_is_exact` 在每一步断言 `len(st.text) - st.emitted` 恰好等于暴力算出的重叠长度：不多扣，也不少扣。

护栏与外围：no-repeat 窗口、退化指标、排版、共享前缀

`Engine._pick` 实现 `no_repeat_window=W`：采样一个 tok，用 `repeats_within(out[-W:] + [tok], W)` 检查是否在距离 ≤W 内重复；重复就屏蔽窗口内的全部 token、在剩下的 token 里再采样一次（只多一次采样，没有重试循环）。因为从不提交重复，窗口本身始终无重复，所以这个整窗检查的答案只取决于新 token。`result` 在结束时用 `longest_unique_run(st.text)` 给出一个退化指标（无重复字符的最长片段），出现在 `stats` 里；CLI 用 `Engine.render` → `justify` 把 prompt+输出排成固定宽度（默认 56）的两端对齐摘要框；给 CLI 多个 prompt 时，`Engine.shared_prefix` → `common_prefix` 报告它们的公共前缀（"The scheduler wakes" 与 "The scheduler

sleeps" 共享 14 个字符 'The scheduler ')。这最后一个是 M4 的种子：`nanoserve/prefix/cache.py::PrefixCache.shared_prefix` 把缓存的 prompt 排序后对每个长度为 m 的窗口调用 `common_prefix`，找出值得常驻的 system prompt。

玩具模型：n-gram 替身，服务侧什么都不变

`nanoserve/model/toy.py::NGramModel` 是一个 token 级 5-gram 插值模型：对 4/3/2/1/0 阶上下文各查一张计数表（5 张表分别有 1、58、395、924、1197 个上下文），按权重 (0.5, 0.25, 0.15, 0.06, 0.03) 混合，再加一个 `floor` 平滑，取 log 作为 logits。它对外只有一个接口 `logits(context_ids) -> np.ndarray[V]`，V=59——这正是 transformer 在服务系统眼里的样子。所以温度、top-k、CDF 采样、stop、扣留、逐步状态、以及之后的批处理、KV cache、前缀缓存，换成真模型时一行都不用改；变的只是 logits 的质量和计算成本（这里一次 `logits` ≈54 µs、一次 `sample` ≈23 µs，生成 200 个 token 约 16 ms，CLI 报告 7–13k tokens/s；真模型的一次前向是毫秒级 GPU 计算，于是批处理和 KV 管理才成为主角）。n-gram 的输出读起来像 "wakes the text more shows the queue, and it is a graph of…"，这对检验服务逻辑足够了。

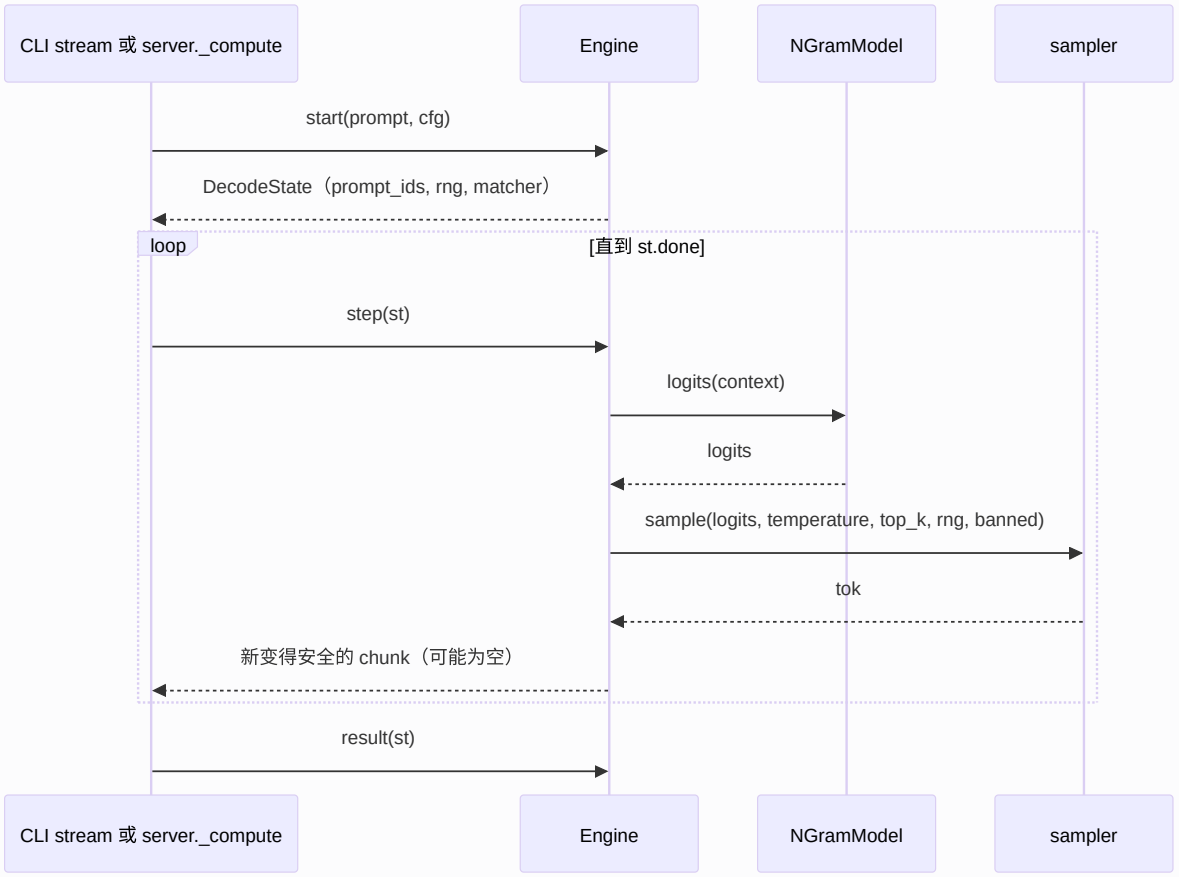


图 · 单请求 stream 与 M2 服务器都通过 start/step 驱动引擎

关键设计取舍

逐步状态机（DecodeState + step）而不是一个整体的 generate 循环

✓ 我们的选择

所有每请求可变状态放进 `DecodeState`，`Engine` 只读；`step` 推进一个 token 并返回新的可发送文本，`stream` / `generate` 只是它的循环包装。这让 M2 的服务器能在每个 tick 交错推进多个请求、插入 prefill、抢占、在请求之间共享一个 Engine。

× 单个 generate(prompt) 函数内部循环到结束

最短、最直观，单用户 CLI 或离线批量生成足够；但控制权在函数里，调度器无法在两步之间插入别的请求，只能一个请求跑完再跑下一个（静态批处理都做不到）。

× Python 生成器（yield）保存状态

`stream` 就是这么用的，写起来自然；但状态藏在生成器帧里，外部看不到 `steps`、`done`、`out`，抢占后无法检查或重建，也不方便让 KV 管理器读取上下文长度，所以生成器只作为外层薄包装。

依据 `server/server.py::_compute` 每 tick 对每个槽位调用 `engine.step(req.state)`；`kvcache/workloads.py::parallel_sample`、`decode/speculative.py` 也直接用 `start/step`；`test_stream_chunks_concatenate_to_result` 验证 chunk 拼接等于最终文本。

stop：增量多模式匹配器判断「是否」+ 结束时 find_stop 定位「在哪」，而不是每步重扫全文

✓ 我们的选择

`MultiStopMatcher`（反向 Trie + 有界 deque）每字符 O(L)；仅在命中后对每个 stop 用 KMP `find_stop` 在全文找最早出现的起点。每步成本与已生成长度无关，全文扫描每请求只做一次。

× 每步对全文 `any(s in text for s in stops)` 或正则交替

实现最简单，stop 少、输出短时完全够用；但每步 O(|text| · Σ|s|)，整次生成 $O(n^2)$，在长输出和高并发下浪费明显。

× 每步对每个 stop 做 `text.endswith(s)`

每步 O(Σ|s|)，同样与长度无关，stop 只有一两个时比 Trie 还快；但必须把 piece 的每个字符位置都检查一遍（多字符 token 可能在中间完成 stop），stop 多时代价线性增长。

× Aho–Corasick 自动机

每字符摊还 O(1)，并且自动机当前状态的深度恰好就是「尾部与某个 stop 前缀的最长重叠」，可以顺带给出扣留长度、省掉 longest_border；代价是构建 fail 指针更复杂。stop 很多或很长时它是更好的工程选择。

依据 `test_multiple_stop_strings_cut_at_earliest`（stops `,` `.` `\n`）验证截在最早位置且去掉 stop 的同 seed 输出以截断结果为前缀；`test_stop_string_truncates` 验证输出里不含 stop。

精确扣留（KMP border）而不是总扣住 len(stop)−1 个字符

✓ 我们的选择

对每个 stop 计算 `len(longest_border(stop + "\x00" + tail))`，只扣住尾部真正可能长成 stop 的那几个字符。

× 永远扣住最后 max_len−1 个字符

零计算、绝对安全；但客户端看到的文本总是落后 max_len−1 个字符，stop 越长越卡顿（长 stop 如多行分隔符时尤其明显）。对只关心最终结果、不关心首字延迟的离线调用可以接受。

× 对每个 j 暴力判断 `text.endswith(stop[:j])`

$O(|s|^2)$ 每 stop 每步，stop 只有几个字符时其实很便宜，测试里的 `_brute_overlap` 就是它；KMP 把它降到 $O(|s|)$，在 stop 较长或个数较多时才有实际差别。

依据 stop=" the queue."、逐字符流、20 个 seed 共 1545 步：精确扣留平均 0.38 个字符，而固定扣留平均 9.47 个；`test_stop_holdback_is_exact` 每步断言扣留量等于暴力重叠长度。

贪心最长匹配 + can_segment 校验，而不是 BPE

✓ 我们的选择

词表 = 全部语料字符 + 25 个高频词（`top_k_pieces`），编码前 Word Break DP 校验，再贪心最长匹配。实现几十行，结果确定，且字符级兜底保证贪心不会卡住。

× BPE（字节对合并）

真实 LLM 的选择：从字节开始按学到的合并规则逐步合并，任何输入都能编码（不需要 UnknownCharacterError），压缩率更高；代价是训练与编码都更复杂（合并优先级、正则预切分），对讲解服务逻辑没有额外价值。

× DP 求最少 token 数的最优切分

对一般词表是唯一正确的做法（贪心在 {ab, abc, cd} 切 abcd 会失败），还能最小化 token 数；但每次编码 O(n · D) 且要回溯，而本词表有字符兜底，贪心已足够。

依据 语料 2169 字符 → 1451 token；"The cache" 4 个 token、"The scheduler" 13 个；`test_unknown_character_rejected` ("emoji 🚀 here")；`test_vocab_uses_most_frequent_words`。

top-k 用大小为 k 的堆求阈值 + CDF 上二分抽样，而不是 np.random.choice / 全排序

✓ 我们的选择

`kth_largest` O(V log k) 求第 k 大作阈值，`cdf_index` 在候选的前缀和上做 lower bound，O(log k)。RNG 是每请求一个 `random.Random(seed)`。

× `np.random.choice(V, p=softmax)`

一行搞定；但用的是 numpy 的全局/独立随机流，与每请求 `random.Random` 不统一，且内部仍是 O(V) 前缀和；在 GPU 上真实系统常用 Gumbel-max（argmax(logits+Gumbel 噪声)）避免建 CDF。

× 完整排序后取前 k

O(V log V)，V=59 时毫无差别；V≈10^5（真实词表）时每步排序就是浪费，生产实现用 `torch.topk` / `np.partition` 这类选择算法。

× 别名方法（alias method）

O(V) 建表后每次抽样 O(1)，适合同一分布抽很多次；而这里每一步分布都变，建表成本无法摊销。

依据 一次 `sample` ≈23 µs（V=59、top_k=4）；`test_greedy_matches_top1` 验证 top_k=1 与温度 0 一致。

用 n-gram 玩具模型代替 transformer

✓ 我们的选择

`NGramModel.logits(context_ids)` 只有一个接口，纯 numpy、确定、毫秒内建好，让整个服务栈能在 CPU 上跑测试。

× 接一个真实的小 transformer

输出更像人话，也能真实体现 prefill/decode 的计算差异和 KV cache 的意义；但需要权重、依赖与算力，测试变慢且不确定性更多，而服务侧的每一行代码（采样、stop、扣留、调度）都不会因此改变。

依据 5 张计数表分别有 1/58/395/924/1197 个上下文；一次 `logits` ≈54 µs；生成 200 token ≈16 ms。

框架文件导读

`nanoserve/engine.py`	生成循环：`GenerationConfig` / `DecodeState` / `Engine.start/step/stream/generate`；在 `step`、`_pick`、`_earliest_stop`、`_stop_holdback`、`result`、`render`、`shared_prefix` 里调用全部 12 个 M1 练习（或经由分词器/采样器间接调用）。
`nanoserve/tokenizer/tokenizer.py`	词表构建（`top_k_pieces`，在频次断点截 k）与编码（`can_segment` 校验 + 贪心最长匹配），以及 CLI 用的 `sanitize`。
`nanoserve/sampling/sampler.py`	logits → token：禁用、温度、`kth_largest` 求 top-k 阈值、在候选前缀和上用 `cdf_index` 抽样。
`nanoserve/model/toy.py`	5-gram 插值模型，接口 `logits(context_ids)`，代替 transformer；构建时编码整段语料（会调用 `can_segment`）。

`nanoserve/model/corpus.py`	2169 字符的内置语料，决定字符集（34 个字符）、高频词和 n-gram 统计。
`nanoserve/__main__.py`	CLI：单 prompt 时流式打印并用 `render` 打出两端对齐摘要框和 stats；多 prompt 时报告 `shared_prefix`；遇到未知字符先警告再 `sanitize`。
`tests/domain/test_m1_engine.py`	M1 集成测试：可复现、chunk 拼接、整词流式、stop 截断与多 stop 最早截断、精确扣留、贪心=top1、no-repeat、未知字符、排版宽度、词表高频词。
`nanoserve/stream/multistop.py`	练习 `stream.multi_stop` 的桩：增量多 stop 检测器，`step` 每个输出字符调用一次 `feed`。
`nanoserve/stream/overlap.py`	练习 `stream.stop_overlap` 的桩：KMP 前缀函数求最长 border，`_stop_holdback` 用它精确计算扣留量。

实测数字

- 词表 V=59：34 个单字符 + 25 个带前导空格的高频词；max_len=10；不同 token 长度 8 种（1、4–10）
- 语料 2169 字符 → 1451 token（≈1.49 字符/token）；"The cache" 4 token，"The scheduler" 13 token
- top_k_pieces 输入：341 个候选片段、167 个不同；出现 ⩾3 次的恰 25 个（第 25 名 3 次、第 26 名 2 次），'the' 出现 49 次
- 精确 stop 扣留 vs 固定扣留 len−1：平均 0.38 vs 9.47 字符（stop=" the queue."，20 seed，1545 步）
- 整词流式平均扣留 2.81 字符（默认配置，20 seed）
- 一次 logits ≈54 µs、一次 sample ≈23 µs；生成 200 token ≈16 ms；CLI 显示 7–13k tokens/s
- `python3 -m nanoserve "The scheduler" --stop "."`（seed 0）跑满 60 token 未遇到 '.'，finish=length，longest_unique_run=8；同 seed max_tokens=200 时第 74 个 token 遇到 '.' 停止
- "The scheduler wakes" / "The scheduler sleeps" 共享前缀 14 字符

建议的完成顺序

按「引擎第一次需要它」的顺序做，每完成一题就能多跑通一段：先 `tok.top_pieces` 和 `tok.segment`（`Engine()` 构造时建词表、模型编码语料就要用到，不做连引擎都起不来）；再做 `sample.kth_largest` 和 `sample.cdf_index`（默认 temperature=0.6、top_k=4，第一个 token 就会调用）；然后 `stream.unique_run`（每次 `result` 都算统计）和 `stream.holdback`（默认整词流式）——到这里 `make run` 已能完整生成；接着 `stream.justify`（CLI 摘要框）；再做 stop 三件套：先 `stream.find_stop`（KMP 匹配），然后 `stream.stop_overlap`（同一个前缀函数，换成求 border），最后 `stream.multi_stop`（Trie 版的多模式增量匹配），三者连起来理解「判断—定位—扣留」；最后 `stream.repeat_window`（只在 `--no-repeat-window` 时用到）和 `tok.common_prefix`（多 prompt 才用，并在 M4 前缀缓存里复用）。

第 2 章 · M2

第 2 章　服务器：连续 batching、准入控制、集群与离线规划

> M1 的引擎一次只会生成一个请求；M2 把它包进一个按 tick 推进的服务器：连续 batching 复用解码步，分层准入（槽位、会话、维护窗口、token 预算）决定谁能进 batch，插件协议让 M3/M4 的 KV 与前缀缓存后接，多 worker 集群负责路由，传输层把多路输出可靠地送到客户端，事后分析与离线规划器则回答运维问题。

问题：从“生成一个请求”到“同时服务很多请求”

M1 的 `Engine.generate` 只处理一个 prompt。服务器要同时面对到达时间、长度各异的很多请求，于是要回答：同一时刻谁在算（batch 组成）、新请求何时进入（准入）、资源不够怎么办（拒绝、排队还是抢占）、多台机器怎么分（路由）。

`server/server.py::Server` 用**模拟的整数 tick** 推进：1 tick = 运行中 batch 的一次 decode step（或一个 `prefill_chunk=16` token 的 prefill 块）。关键建模假设是**一步耗时与 batch 大小无关**，对应 decode 受显存带宽限制的事实，batching 的收益就来自这里：同一 30 请求 trace，`--batch 1/4/8` 分别是 544/141/80 tick，p50 281.5/62/30.5。`Request` 分为调用方字段（prompt、arrival、max_tokens、seed、session、tenant、deadline）与服务器写入的运行时字段，`reset_runtime()` 清空后者，供抢占重算使用。

每个 tick 的四步循环

① **到达** `_arrivals`：每个到达调用 `QpsCounter.hit`（`metrics.qps`），框架用 `_t_ms = max(_t_ms + 1, now * tick_ms)` 保证同 tick 多个到达的毫秒戳严格递增，即由框架满足练习的前置条件；`_session_ok` 用 `TokenManager`（`server.tokens`）给老会话续期，新会话超过 `max_sessions` 就拒绝。

② **准入** `_admit_waiting`：不在维护窗口（`sched.maintenance`）时，只要有空闲 batch 位就看队头，token 预算（`fits_capacity`，`sched.capacity_ok`）与 `kv.can_admit` 都通过才接纳：`SlotAllocator.reserve()`（`sched.slots`）拿最小槽位，前缀缓存缩短 prefill，`engine.start` 建 `DecodeState`。

③ **计算** `_compute`：按槽位顺序，每个请求消耗一个 prefill tick 或 `engine.step` 一个 token；`kv.on_token` 失败则抢占。输出以 `(tick, rid, chunk)` 追加到 `events`，即多路复用的输出流。

④ **记账**：有计算就 `insert_busy(busy, [now, now+1])`（`sched.insert_busy`）；记录队列长度与 `count_unexpired`；完成的请求释放槽位、预留与 KV，延迟进入 `MedianTracker`（`metrics.median`）。

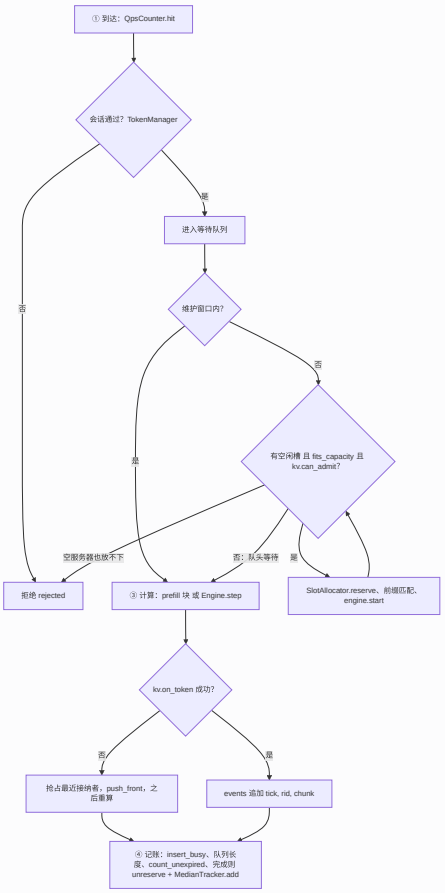


图 · Server.tick()：一个 tick 内的四步

分层准入

几道关卡各挡一种风险。**槽位**限制 batch 行数。**会话**只拒绝“开新会话”的请求，老会话永远放行（`--sessions 6 --max-sessions 4` 拒绝 10/30）。**维护窗口**只挡新准入、不打断运行中的请求（`--maintenance 20 40`：141 → 155 tick，忙碌时间线变成 2 段）。**token 预算**给每个运行中请求登记预留 `[prompt+max_tokens，开始，开始 + [未缓存 prompt / prefill_chunk] + max_tokens)`，按最坏情况记账，因此永远不会因预算抢占。

准入循环遇到放不下的队头就 `break`，不跳过它去接后面的小请求（有意的队头阻塞，见取舍）。例外是“空服务器上也放不下”的请求：等待永无尽头，直接拒绝，免得整条队列被永久堵住。

插件协议与抢占重算

核心不知道 KV 如何分块。`plugins.py` 定义 `KVBackend`、`PrefixCacheBackend`、`WaitingQueue` 三个 `Protocol`，功能关闭时用 `NullKV`、`NullPrefixCache`、`FCFSQueue`；`build_plugins` 按配置懒加载 M3 的 `BlockManager`、M4 的 `PrefixCache` 与 `DeadlineQueue`；规划器塞入自己的 `PlannedQueue`。M2 因此可以单独运行，后续里程碑也能各自接入。

KV 存不下新 token 时，`_make_room` 反复抢占 `(admitted_at, slot)` 最大的请求（最近接纳、已做工作最少），直到放得下或当前请求自己被抢占。`_preempt` 归还槽位、KV、预留，删掉它已输出的 events，`push_front` 回队头从头重算。同一 trace 下 `kv_blocks` 16/12 抢占 20/34 次，tick 增加，但输出逐字不变。

核心不变量：输出与单独运行完全一致

一个请求生成什么，与它和谁 batch、在哪个 worker、被抢占几次无关，必须等于 `Engine.generate` 单独运行它的结果。每个 `DecodeState` 自带 `random.Random(seed)`，模型只读，可变状态全在请求自己身上，所以 batch 只是

交错推进多个独立的状态机。重算正是靠这一点才安全：同 seed 重算出的 token 相同，真实系统甚至可以只抑制已发送前缀的重复输出，而不必撤回。

`test_batching_never_changes_outputs`、demo 的 `30/30`、集群三种策略、KV 抢占都用它做回归检查。真实 GPU 上这只是近似（batch 大小不同会走不同 kernel，浮点归约顺序变化，即 batch invariance 问题）；玩具模型把它变成精确不变量，任何调度优化都能用“输出字节相同”来验证。

传输、指标与分析

`transport.py`：每 tick 的事件拍平成 `[rid, chunk, …]` 用 `FrameCodec`（`stream.frame_codec`）编成一条消息，UTF-8 字节每 `packet_size` 字节切一刀，**不管字符边界**；`StreamClient` 只在缓冲区整体是合法 UTF-8（`server.utf8_ok`）时预览，消息收齐后才解帧。帧长度按字符计、切包按字节计，两层边界互不相干。

指标分两类。**在线**：`QpsCounter`（deque 头部过期）、`MedianTracker`（双堆）、`insert_busy`，只能看过去、必须增量。**事后** `analytics.py::analyze`：按完成顺序的滚动 max（单调队列）与滚动 p50（双堆 + 惰性删除）、热点租户（投票后数一遍确认 > 50%）、队列突发（对每 tick 队列增量做 Kadane）、周期 trace 的跨午夜突发（环形 Kadane）。demo：滚动 5 窗口 max 23..117，最大突发 +22，`--hot 4` 时租户 a 占 77%。

集群与离线规划

`cluster.py::Cluster`：每个 worker 是完整的 `Server`，共享只读 `Engine`，同步 tick。路由 `weighted`（`sched.weighted_route`）、`rr`（单流副本，全忙丢弃，事后 `sched.rr_busiest` 重放）、`weighted_free`（批模式，`sched.assign_workers` 选最便宜的空闲批槽位）；报告用 `sched.merge_busy` 合并时间线、`metrics.merged_median` 求全局 p50。

`planner.py::BatchPlanner` 面向事先已知的批任务，回答 SJF 顺序、静态发车数、多 LoRA 冷却、零排队通道数、SLO 上界、最热通道（m2b 的六题），并且**每个答案都用真实服务器验证**：SJF 经 `PlannedQueue` 真跑，均值 223.1 vs FCFS 264.6；24 条通道 0 人等待、23 条 2 人等待；SLO 上界 7/30，实测 FCFS 2、SJF 5。

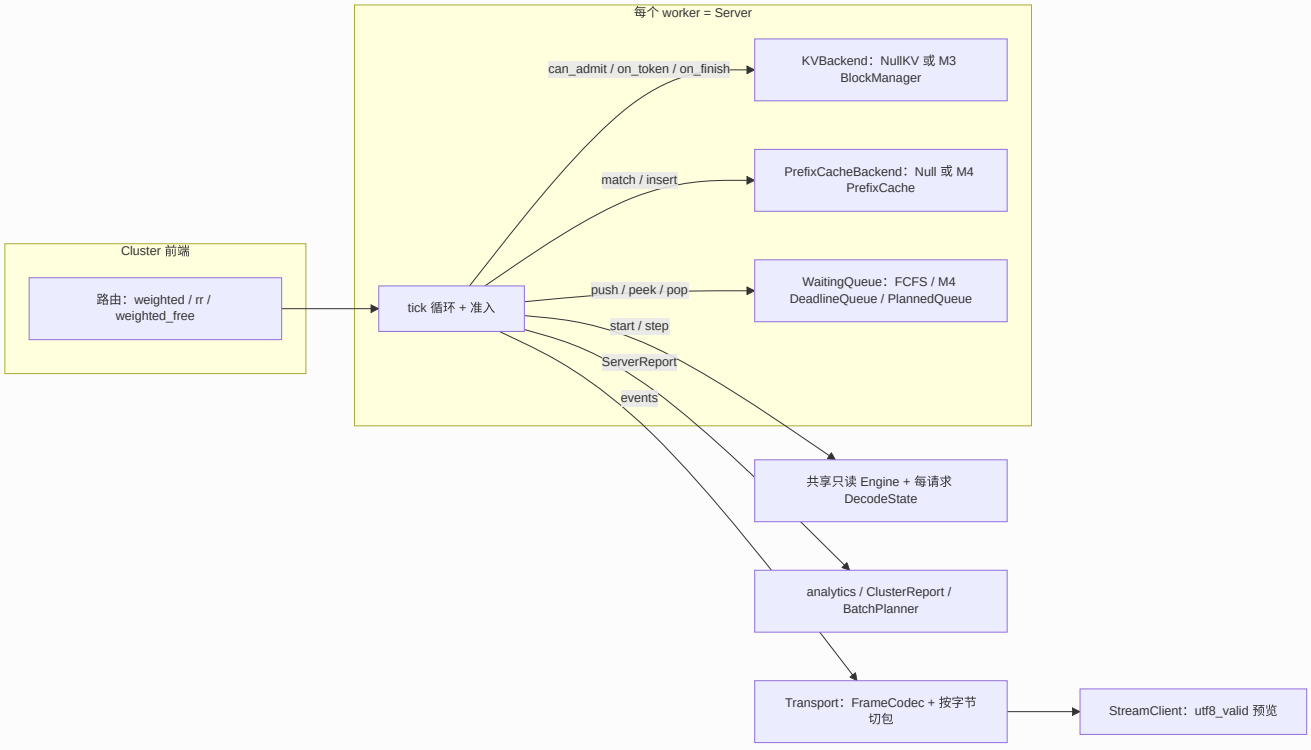


图 · 服务器核心、插件、集群与下游消费者

关键设计取舍

连续（iteration 级）batching，而不是静态 batching

✓ 我们的选择

每个 tick 结束时完成的请求立刻释放槽位，下一 tick 开头就能接纳新请求（`_admit_waiting` 每 tick 都跑）。batch 组成按 decode step 变化，短请求不必陪长请求等到整批结束，新请求也不必等下一次“发车”。

× 静态 batching：凑一批、一起跑到最长的那个结束

实现简单，batch 形状固定（适合 padding 后的大矩阵、编译期固定 shape 的加速器），离线批处理且长度相近时损失很小。代价是槽位空转：短请求结束后它的行一直空到整批结束，新请求还要等下一批。规划器里的 `sched.min_launches` 恰好就是静态 batching 的问题：给每个请求一个允许启动的时间窗，求最少发车次数。

× 请求级 FCFS、一次只跑一个（max_batch=1）

没有任何干扰，延迟可预测，rr 策略里的单流副本就是这种。但 decode 一步的成本几乎与 batch 大小无关，串行等于浪费这部分并行度：同一 trace 544 tick vs 141 tick。

依据 同一 30 请求 trace：连续 batching（max_batch=4）141 tick、p50 62、槽位占用率 93%；按“每次发车取至多 4 个已到达请求、整批占用 = 组内最长服务时间”手算的静态 batching 需 9 批、189 tick、p50 97.5、槽位占用率 70%。

模拟 tick 时钟，而不是墙钟 + 异步事件循环

✓ 我们的选择

所有时间都是整数 tick，`Server.run()` 是一个确定性的 while 循环；`tick_ms=10` 只用来给 QPS 计数器换算毫秒。好处是可复现、可断言：测试可以写 `admitted_at == 30`、`rep.p50_latency == statistics.median(lat)`，规划器可以拿“精确服务时间”与真实运行逐 tick 比对。

× 墙钟 + asyncio / 线程（真实服务器的做法）

能真实地重叠 CPU 调度、分词、网络 IO 与 GPU 计算，测出真实吞吐。但结果依赖机器负载，调度 bug 难以复现，测试只能断言区间而非精确值。nano-serve 的目标是把调度算法讲清楚，所以选了确定性。

× 按 token 数加权的 tick（step 成本随 batch 规模增长）

更接近真实 GPU（prefill 计算密集，大 batch 的 decode 也会变慢），能暴露“batch 越大越好”的极限。代价是模型更复杂、规划器的服务时间不再精确。nano-serve 只用 `prefill_chunk` 近似 chunked prefill，其余每步 1 tick。

依据 demo 每次运行数字完全相同（141 tick / p50 62.0）；`_t_ms = max(_t_ms + 1, now * tick_ms)` 把模拟时钟适配成 `QpsCounter` 需要的严格递增毫秒戳。

token 预算用“预留 + 扫描线”的悲观准入，并接受队头阻塞

✓ 我们的选择

每个运行中请求按最坏情况登记 `[prompt+max_tokens，开始，估计结束)`，新请求只有在 `fits_capacity` 证明任意时刻总量 ⩽ 预算时才能进来，因而预算永远不会被突破、也不需要因预算而抢占。队头放不下就停止准入（`break`），不越过它接后面的小请求；空服务器上都放不下的直接拒绝。

× 贪心/乐观准入：只看当前占用，超了再抢占

利用率更高（多数请求不会生成满 max_tokens），vLLM 管理 KV 就是这样做的，本仓库 M3 的 KV 插件也是。代价是要有抢占与重算机制，且需要额外保证单个请求独自装得下，否则会活锁：早期版本在 kv_blocks=9 时跑满 10 万 tick 只完成 7 个、抢占 9522 次（一个需要 10 块的请求不断抢占别人再抢占自己）。现在 KV 插件提供 `can_ever_fit`，服务器在准入前拒绝「独占整个缓存也放不下」的请求；有了这个保证，「抢占最近准入者」就能确保最老的运行请求总能完成。

× 越过队头回填（backfill / skip-ahead）

队头大请求卡住时让后面的小请求先进，提高利用率，HPC 调度器的 EASY backfilling 就是这样。但若不给队头留预留，大请求可能被一直插队、饿死；FCFS 的公平性也没了。nano-serve 的 M4 用 EDF 队列换顺序，但准入循环本身保持“只看队头”。

依据 `--budget 150`：141 → 226 tick，p50 62 → 110，任意 tick 运行中 Σ(prompt+max_tokens) ⩽ 150（测试逐 tick 断言）；`--budget 300` 无影响。

抢占时重算 + 插件协议，而不是一个包揽一切的服务器

✓ 我们的选择

核心只通过 `KVBackend / PrefixCacheBackend / WaitingQueue` 三个协议说话；KV 不够时抢占最近接纳的请求、清空其运行时状态、`push_front` 回队头，之后从头重算。重算依赖确定性解码，因此无需保存任何中间状态，也不需要插件之间互相知道。

× 换出（swap）KV 到主机内存

长 prompt 被抢占时省掉重做 prefill 的计算，适合 prompt 很长、PCIe 带宽充足的场景。但需要第二级存储与搬运逻辑，KV 插件要暴露块内容而不只是计数。nano-serve 的 KV 只做记账，重算的成本只是 tick。

× 单体服务器：KV、前缀缓存、排队策略写死在循环里

少一层间接，可做跨模块优化（例如按前缀命中率决定准入）。代价是里程碑无法独立替换：现在 M2 单独可跑（Null 实现），规划器能塞进自己的 `PlannedQueue`，M4 能把 FCFS 换成 EDF，核心代码都不用改。

依据 kv_blocks 16/12：抢占 20/34 次，tick 213/344，输出与不限 KV 逐字相同；被抢占请求已输出的 events 被删除，流式往返仍 exact。

集群路由三选一：weighted、round-robin + 丢弃、weighted_free

✓ 我们的选择

三种策略对应三种信息条件。`weighted` 无状态、事先决定，适合多个前端各自路由、只知道各 worker 相对容量的情况；`rr` 面向单流副本，宁可丢弃也不排队，被接纳的请求一到就开始（测试断言 `admitted_at == arrival`）；`weighted_free` 要求整批已知且服务时间可估，按 `(cost, worker)` 选最便宜的空闲批槽位，计划精确时没有请求会在有空槽时等待。

× 最少连接 / power-of-two-choices

按实时负载选 worker，不用事先知道容量，抗随机不均的能力强于 weighted。但需要前端看到 worker 状态（一次查询或心跳），多前端时状态会过期。本仓库的 rr 已经读取 worker 的在途数，只是只用来判断忙闲。

× 一致性哈希（按会话或前缀路由）

让同一会话或同一前缀落到同一 worker，前缀缓存命中率高（SGLang 的 cache-aware 路由思路）。代价是负载可能倾斜，需要再叠加负载阈值。M2 的集群没有前缀感知。

依据 rate 1.5：weighted 66 tick、p50 25.5、worker 完成数 8/13/9；rr 只完成 6、丢弃 24；weighted_free（costs 1 2 2）62 tick、p50 20.5，便宜的 worker0 利用率 98%、worker2 68%。rate 0.3 时 rr 完成 17/30（p50 19），weighted 30/30（p50 18.5）。权重 1 3 1 配给三个相同的 worker：96 tick，worker1 利用率 99%、p50 39.5，其余两台约 45%。

长度前缀分帧 + 无视字符边界切包，而不是换行分隔的 JSON

✓ 我们的选择

每 tick 一条消息，`len#payload` 拼接；字节层按固定大小切，客户端用 `utf8_valid` 判断缓冲区能否安全预览。分帧不扫描内容、不转义，任意字符（`#`、换行、emoji）原样通过；切包模拟真实 socket，强迫客户端正确处理半个字符。

× NDJSON / SSE（每行一个 JSON 对象）

OpenAI 等 API 的实际做法：可读、易调试、浏览器原生支持 SSE。JSON 会把换行转义成 `\n`，所以“分隔符出现在内容里”的问题由转义解决，代价是编码/解码都要逐字符转义、反转义，接收方必须扫描找分隔符。练习规格禁用 json，就是要你自己面对这个问题。

× 服务端只在字符边界切包

客户端更简单，不需要 UTF-8 校验。但中间的代理、TCP 本身都会重新分段，客户端无论如何都要能处理半个字符；把文本编码知识塞进传输层也是错误的分层。

× 二进制定长头（如 gRPC 的 5 字节前缀）

头部定长，解析时不用找 `#`，长度可直接按字节计，是生产协议更常见的选择。代价是不可读、要处理字节序，且帧边界与字符边界又是两回事。

依据 包大小 1/3/16/4096 字节下 `client.frames == report.events`；含中文与 emoji 的片段用 1/2/3/5 字节包时，`held` 分别为 8/4/1/2，没有出现半个字符的预览。

框架文件导读

`nanoserve/server/server.py`

`ServerConfig`、`Server`、`ServerReport`：tick 循环、分层准入、抢占重算、记账。调用 sched.slots、server.tokens、sched.maintenance、

	metrics.qps、sched.insert_busy、metrics.median、sched.capacity_ok。
`nanoserve/server/request.py`	`Request` 数据类：调用方字段 + 服务器写入的运行时字段；`latency`、`ttft`、`met_deadline` 属性与抢占用的 `reset_runtime()`。
`nanoserve/server/plugins.py`	`KVBackend`、`PrefixCacheBackend`、`WaitingQueue` 三个协议及其空实现 `NullKV`、`NullPrefixCache`、`FCFSQueue`，让 M3/M4 可独立接入。
`nanoserve/server/trace.py`	确定性的合成 trace：指数间隔到达、租户权重、可选 deadline 与会话；部分 prompt 共享长前缀，供 M4 前缀缓存使用。
`nanoserve/server/transport.py`	`Transport`（每 tick 一条 `FrameCodec` 消息，按字节切包）与 `StreamClient`（重组、`utf8_valid` 预览、收齐后解帧）。调用 stream.frame_codec、server.utf8_ok。
`nanoserve/server/analytics.py`	事后分析：按完成顺序的滚动 max/p50、热点租户（投票 + 确认）、队列突发与周期性跨午夜突发，`periodic_trace` 生成周期流量。调用 metrics.window_max、metrics.window_median、metrics.dominant、metrics.max_burst、metrics.max_burst_circular。
`nanoserve/server/cluster.py`	`Cluster`：多个同步 tick 的 `Server` 共享一个 `Engine`，三种路由策略，合并忙碌时间线与全局 p50。调用 sched.weighted_route、sched.rr_busiest、sched.assign_workers、sched.merge_busy、metrics.merged_median。
`nanoserve/server/planner.py`	`BatchPlanner`：离线批任务的容量问题，每个答案都用真实 `Server` 验证；`PlannedQueue` 插件按预定顺序出队。调用 sched.sjf、sched.min_launches、sched.cooldown、sched.min_workers、sched.slo_bound、sched.busiest_worker。
`nanoserve/server/demo.py`	`python -m nanoserve serve`：单服务器摘要 + “与单独运行一致”检查 + 传输往返、分析、集群表（`--workers`）与离线计划（`--plan`）。
`tests/domain/test_m2_server.py`	服务器核心集成测试：输出不变量、槽位上限、p50 与忙碌时间线、QPS 序列、token 预算（含空服务器也放不下时拒绝）、会话上限、维护窗口。
`tests/domain/test_m2_transport_analytics.py`	传输往返（1/3/16/4096 字节包）与 Unicode 不预览半个字符；滚动 max/p50、热点租户、队列突发、跨周期突发与暴力解对拍。
`tests/domain/test_m2_cluster.py`	三种路由下输出与单独运行一致；加权路由的比例与可复现性；全局 p50 与合并时间线；weighted_free 按计划放置且优于随机路由；rr 丢弃与重放。
`tests/domain/test_m2_planner.py`	离线规划器与真实 Server 比对：服务时间估计精确、SJF 计划被遵守且优于 FCFS、最少通道即峰值并发、发车数与暴力解一致、LoRA 冷却与贪心排程一致、SLO 上界从未被超过、最热通道与实测槽位一致、`PlannedQueue` 满足队列协议。

实测数字

- 默认 demo（30 请求、rate 1.5、max_batch 4）：141 tick，p50 62.0，最大延迟 117，利用率 99%，输出与单独运行一致 30/30
- max_batch 1 / 4 / 8：544 / 141 / 80 tick，p50 281.5 / 62 / 30.5

- 静态 batching（手算模拟，同一 trace、每批至多 4 个）：9 批、189 tick、p50 97.5、槽位占用率 70%；连续 batching 槽位占用率 93%
- token 预算 150 / 200 / 300：226 / 169 / 141 tick，p50 110 / 78 / 62
- 维护窗口 [20,40)：155 tick、忙碌 2 段、p50 76.5；会话上限 4（6 个会话）：拒绝 10/30，峰值活跃会话 4
- kv_blocks 不限 / 16 / 12：抢占 0 / 20 / 34 次，141 / 213 / 344 tick，输出始终逐字相同；kv_blocks 9：2 个需要 10 块的请求在准入时被拒绝，其余 28 个在 418 tick 内完成、输出不变（修复前是活锁：10 万 tick 只完成 7 个、抢占 9522 次）
- 传输：186 个事件、108 条消息、16 字节包 185 个，往返 exact；Unicode 片段 1/2/3/5 字节包 held = 8/4/1/2
- 分析：滚动 5 窗口 max 23..117、p50 20..111；最大队列突发 +22；--hot 4 时热点租户 a 占 77%
- 集群 3 worker：weighted 66 tick / p50 25.5；rr 完成 6、丢弃 24；weighted_free（costs 1 2 2）62 tick / p50 20.5；权重 1 3 1 时 96 tick
- 离线计划：SJF vs FCFS 平均延迟 223.1 vs 264.6；静态 batching ⩾ 3 次发车；零排队 24 条通道（23 条时 2 人等待）；SLO 上界 7/30，FCFS 实测 2、SJF 5；多 LoRA 冷却 2 需 34 个 prefill 槽

建议的完成顺序

先做让默认配置跑起来的四个：`sched.slots`（准入拿槽位）→ `metrics.median`（完成时记 p50）→ `sched.insert_busy`（每 tick 记忙碌）→ `metrics.qps`（每个到达都调用），此时 `python -m nanoserve serve` 的第一行就是你的代码在跑。再做三道准入选项，难度递增：`server.tokens`（`--sessions/--max-sessions`）→ `sched.maintenance`（`--maintenance`）→ `sched.capacity_ok`（`--budget`），三者都是“时间区间 + 有序结构”，先做 TTL 过期再做区间冲突、再做容量扫描线，概念逐步叠加。接着做传输的 `stream.frame_codec` 与 `server.utf8_ok`，用 `--packet 1` 检验。分析五题按依赖排：`metrics.max_burst` → `metrics.max_burst_circular`（在 Kadane 上加一个补集），`metrics.window_max`（单调队列）→ `metrics.window_median`（在 `metrics.median` 的双堆上加惰性删除），`metrics.dominant` 随时可做。最后进入 m2b 的集群（`sched.merge_busy`、`metrics.merged_median`、`sched.weighted_route`、`sched.rr_busiest`、`sched.assign_workers`）与规划器六题，它们复用这里建立的服务器语义。

第 3 章 · M3

第 3 章　分页 KV Cache：块表、共享、驱逐与自检

> 引擎从「每个请求一段连续 KV、显存满了就拒绝」变成按块分页管理 KV：前缀共享、fork 写时复制、投机解码回滚、显存不足时冷块下沉主机层或抢占重算，并且每一步都能被不变量检查器自证无泄漏、无重复释放。

问题：连续分配 KV 为什么不行

每生成一个 token，注意力都要读该序列此前所有 token 的 K/V。最朴素的做法是给每个请求预留一段连续显存，长度按 prompt + max_tokens 估计。这有两种浪费：**内部碎片**——请求实际只生成了 20 个 token，却占着 512 个位置；**外部碎片**——请求以任意顺序结束，空闲显存被切成许多小洞，总量够但没有一段连续区间放得下新请求。仓库里其实保留了一个活生生的例子：token-slot arena（`arena.py`）就是连续分段 + first-fit（kv.mem_alloc），它的 `failed_reservations` 统计的正是「总量够、连续段不够」的失败。

分页（paged attention）的做法是把 KV 显存切成 `num_blocks` 个块，每块 `block_size` 个 token 槽位。每个序列维护一张**块表**，第 p 个 token 存在槽位 `table[p // block_size] * block_size + p % block_size`。块按需分配，最多浪费最后一块的 block_size − 1 个槽位；块之间不必相邻，外部碎片从根上消失。代价是多一次间接寻址（块表查找），以及一整套「块属于谁、还有谁在用」的簿记——M3 的 30 道练习几乎都长在这套簿记上。

块的五种状态与引用计数

`BlockManager` 里每个物理块恰好处于一种状态：空闲（在 `FreePool` 的有序链或暂存批里）、使用中（`refs[b] ≥ 1`）、缓存（refs 归零但 KV 还在显存，登记在 `cached` 和去重哈希表里）、隔离（被重复释放过，永不再分配）。`check()` 会逐块验证「恰好一种状态」，并核对位图、`in_use` 计数、`refs[b] == sum(holders[b].values())`。

引用计数是共享的基础：两个请求有相同前缀时共享前缀的满块；fork 出的样本共享全部块；checkpoint 也通过给块 +1 引用来「钉住」它们。每个使用中的块还被**记账（charge）**到某一个持有者——谁把它带进来就记谁，那人放手后记账转给 `min(holders)`。`VersionedStore`（kv.versioned）按事件号保存记账历史，于是能回答「第 t 个 tick 时块 b 记在谁头上」，这是排查泄漏和重复释放时最需要的证词。

块表本身是 `RandomListNode` 链：`val` 是块号，`next` 指向下一块，`random` 被挪用为**回指前一个结点**。追加和回滚都在尾部；释放时沿回指从尾走到头，于是尾块先进入 `cached`（插入序 = 释放序），而驱逐恰好从最老的释放开始看——最不可能被别人共享的尾块最先被驱逐，这与 vLLM 的做法一致。

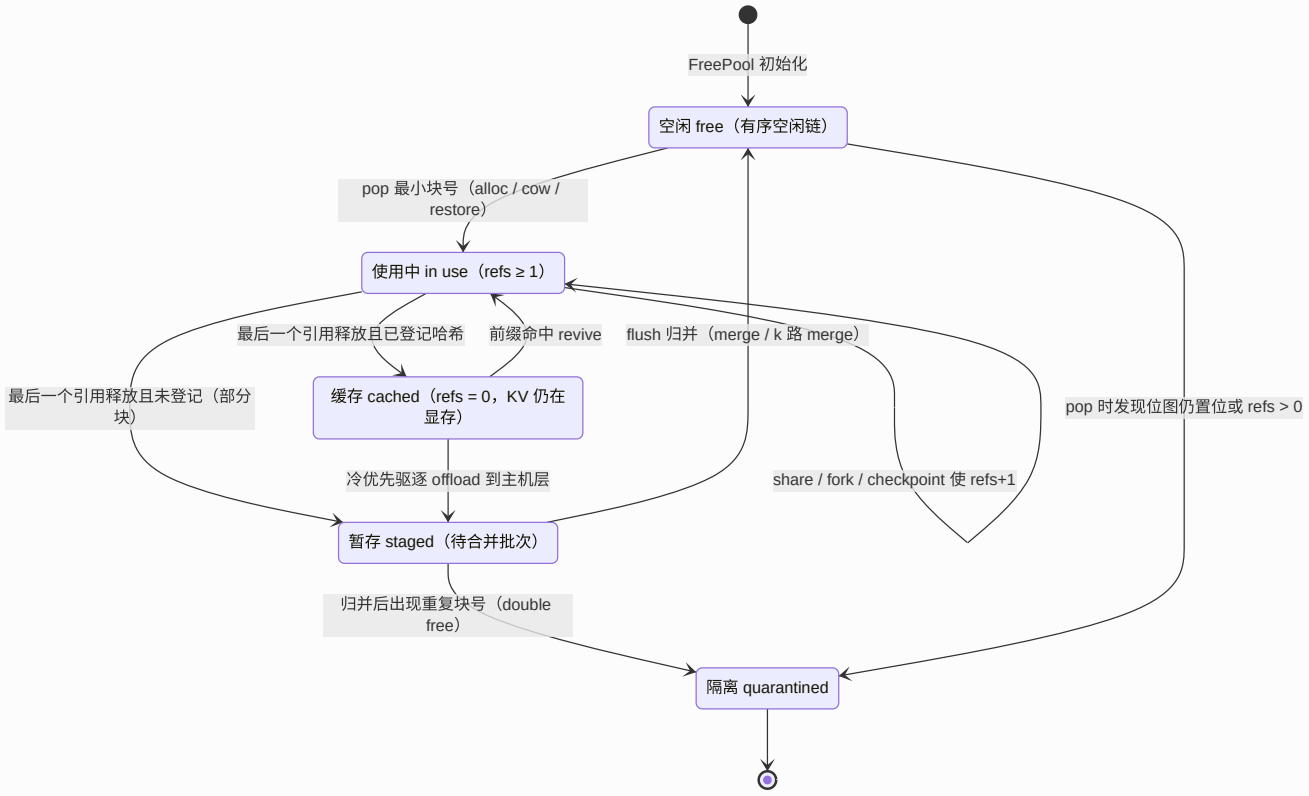


图 · 物理块的生命周期（BlockManager + FreePool）

请求的一生：admit → store → release

admit：对 prompt 的每个满块计算链式哈希 `chain_hash(parent, tokens)`（覆盖该块末尾之前的全部 token，所以相同哈希意味着相同前缀），在 `BlockHashMap`（kv.block_hash）里查找，并用块的完整 token key 复核——哈希碰撞永远不会共享错误的 KV。命中分三类：仍被别人使用 → share（refs+1）；在 cached 里 → revive；都不在但主机层存着且校验和一致 → restore（新块 + 拷回）；否则 compute。代码先把所有 GPU 命中钉住，再去分配新块，防止分配新块时触发的驱逐把刚命中的块赶走。prompt 末尾不满一块的部分总放进私有新块，从不共享。

store：每个新 token 若落在块首（off = 0）就 `_take_block()`：先从空闲链 pop 最小块号；链空则 flush 暂存批；仍空就驱逐一个 cached 块；都没有返回 False，交给服务器抢占。若尾块被共享（refs > 1），先 copy-on-write 换成私有副本再写。块写满的瞬间登记它的链式哈希，从此它可被后来者共享。

release：沿回指从尾到头逐块 unref。归零的块若已登记哈希就留作 cache（KV 不丢），否则进入本次操作的 `_freed`，操作结束时作为一个批次 stage 进空闲池。

空闲池：有序链 + 惰性批量归并 + 消毒

`FreePool` 把空闲块号存成一条**按块号升序的链**，`pop` 永远给出最小块号，于是在用块挤在池子低端（局部性好，空闲区成段）。释放不是逐个插回：一个序列释放的块先 `sort_free_list`（kv.sort_free）排好，作为一批放进容量 8 的暂存环（kv.ring_queue）。只有在空闲链真的空了、暂存环满了、或显式 `sanitize()` 时才 flush：一批用 `merge_free_lists`（kv.merge_free），多批一次 `merge_k_free_lists`（kv.merge_free_k）。

归并后的链里如果某个块号出现两次，说明它被释放了两次——两次中必有一次是错的，那个块可能还在被某个序列使用。`drop_double_freed`（kv.drop_double_free）把这种块号的**所有**副本删掉，flush 返回它们，管理器把块隔离。另一道防线在 `_take_block`：pop 出来的块若位图仍置位或 refs > 0（在用时被错误释放），同样隔离并跳过。

报告侧还会用 `merge_distinct`（kv.merge_arrays + kv.dedupe）把空闲链和暂存批合成去重有序列表，再交给 `free_summary` 输出区间串（kv.free_ranges）和最长连续空闲段（kv.longest_run）——后者就是此刻能满足的最大连续分配，一个直接的碎片指标。

显存满了：GPU 端冷优先驱逐 + 主机层策略

驱逐发生在两层。**GPU 层**（`_evict`）：取 `cached` 中最老的 `scan` 个释放（默认 32，上限 200），把它们的热度（饱和复用计数，上限 200）串成链，用 `partition_by_heat(head, hot)`（kv.partition_hot）稳定分区：热度 < hot 的冷块排前面且保持释放先后，然后从头取 k 个受害者。被复用过 hot 次以上的块因此受保护，冷块之间仍是先释放先走。受害者不直接丢弃，而是以链式哈希为键、payload 校验和为值 `put` 进**主机层**，同一 tick 内下沉的块按连续块号区间合并成一次传输（kv.free_tracker）。

主机层容量也有限，策略由 `ServerConfig.eviction` 选：LRU（kv.lru）、LFU（kv.lfu）或随机（kv.random_evict + kv.block_hash，自带种子，不扰动全局 random）。没有哪种策略处处最好，`workloads.py` 专门造了两条 trace 来证明这一点：hot_vs_scan（热 prompt 夹在越来越长的一次性扫描之间）上 LFU 赢；shifting_hot_set（早期的宠儿之后再也不来，换成新热点）上 LRU 赢。主机层足够大时（16 块）三种策略结果完全相同——策略只在容量小于工作集时才有意义。

抢占：重算而不是换出

decode 中 `store` 拿不到块时，服务器（`server.py::_make_room`）抢占**最晚入场**的请求：`on_finish` 释放它的全部块，丢掉已经流式输出的事件，`reset_runtime()` 后放回队首，之后从头重算。因为采样带种子，重算的输出逐 token 相同——demo 中 18 块预算下 14 次抢占，20 个输出与无限显存运行完全一致。

重算并不像听起来那么贵：被抢占请求的满块释放后进入 cached（或被驱逐到主机层），重新 admit 时大多以 revive / restore 命中。这也解释了一个反直觉的数字：16 块时 prompt 块命中率 0.791，反而高于 64 块时的 0.692——抢占后的重入命中了自己留下的缓存。

fork、checkpoint 与写时复制

并行采样（n > 1）时，第一个样本正常 admit，其余 `fork`：用 `fork_chain`（kv.fork）深拷贝块表结构（含回指），但物理块一个也不拷，只把每块 refs+1。第一次往共享的部分尾块写入时触发 copy-on-write。demo 中 n = 4：prompt 的 3 个块各被引用 4 次，只发生 3 次 COW，峰值 15 块，而 4 份独立拷贝需要 24 块。

投机解码用的是同一机制：`checkpoint` 把块表快照进 `SnapshotTable`（kv.snapshot，只存变化，O(changes)），并给表中每块 +1 引用钉住；草稿 token 写进被钉住（因而 refs > 1）的尾块时自然触发 COW，旧尾块原封不动；`rollback` 读回快照，精确恢复原表。demo：34 轮、34 次回滚、32 次 COW，每次回滚后块表都与 checkpoint 时完全相同。

调试：不变量、泄漏与重复释放

`debug=True` 时每个公开操作后都跑一遍 `check()`：状态唯一性、位图与计数、holders 与 refs、记账、块表无环（`has_cycle`，kv.has_cycle，必须在遍历前证明链会结束）、回指完整、每块内容与序列 token 一致、arena 中每个 token 的槽位都映射到在用页（kv.page_lookup）。集成测试 `test_small_budget_preempts_but_outputs_match_unlimited_run` 就是在 debug 模式下跑三种策略。

`audit_leaks` 找「位图说在用、却没有任何块表 / checkpoint / cache 能解释」的块：在块号区间上比较两个有序列表的计数，缺 1 个时把两个列表拼起来求 XOR（kv.leaked，其余块号恰好各出现两次），缺更多就二分区间递归。事件日志（`eventlog.py`）是定长环形缓冲，读者只靠时间戳列恢复顺序：最老事件 = `ring_min`，按事件号定位 =

`ring_search`，按时间排序的转储 = `rotate_right`。sanitizer demo 注入的四种故障——重复释放、在用时释放、3 个泄漏块、块表成环——全部被精确发现。

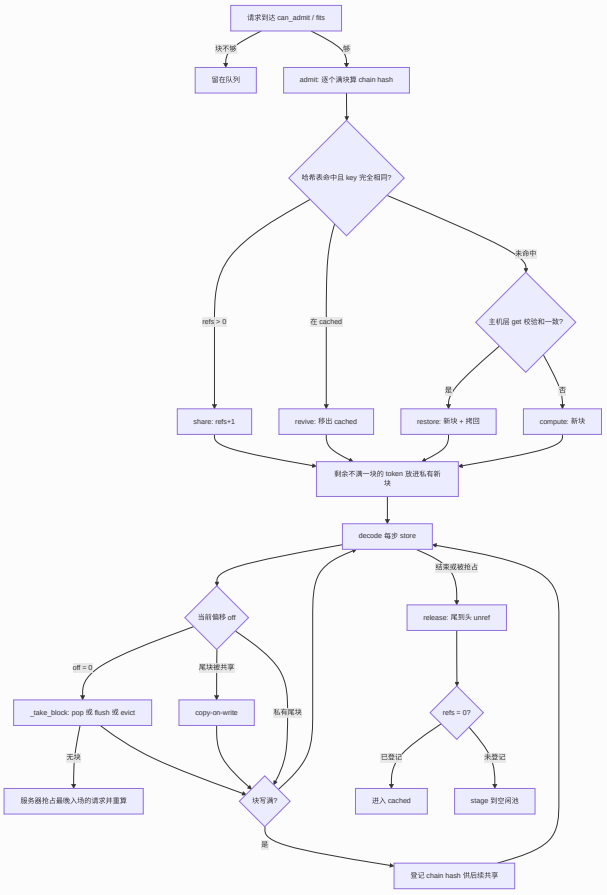


图 · 一个请求在 M3 中的路径：admit → store → release，以及块不够时的出口

关键设计取舍

分页块 vs 每请求一段连续 KV

✓ 我们的选择

KV 显存切成定长块，序列用块表间接寻址；块按需分配，最多浪费最后一块的 block_size − 1 个槽位，块之间不必相邻。共享、COW、驱逐的粒度都是块。

× 每请求按 max_tokens 预留连续区间

寻址最简单、kernel 访存连续；但内部碎片（预留远大于实际）和外部碎片（请求乱序结束留下小洞）让并发数大打折扣，也无法在请求间共享前缀。只有请求长度可预知且整齐划一（离线批处理、固定长度）时才划算。仓库里的 token arena 恰好保留了这种分配方式，用 failed_reservations 记录它的失败。

× 更大的块 / 更小的块

块越大，块表越短、哈希和簿记越便宜，但尾块浪费和 COW 拷贝量越大、前缀共享粒度越粗（只共享满块）；块越小则相反。默认 block_size = 8，上限 128。

依据 test_more_blocks_never_hurt：kv_blocks 12/16/24/40 时总 tick 单调不增；实测 12→281、14→196、16→178、24 及以上→105 ticks（与无限显存相同）。

有序空闲链 + 暂存批 + 惰性 k 路归并，而不是一个栈式 free list

✓ 我们的选择

空闲块号保持升序，pop 永远拿最小号；释放按序列成批排序后进暂存环，空闲链真空了 / 环满 / sanitize 时才一次性归并（单批二路、多批 k 路），归并后顺手把重复块号整体隔离。

× 栈（LIFO）free list：释放 push、分配 pop

两者都是 O(1)，实现最短，刚释放的块马上复用还可能对缓存友好；但块号分布越来越乱，在用块不再挤在低端，最长连续空闲段难以维持，而且重复释放会被悄悄接受——同一块号被 push 两次就会被分给两个序列。

× 每次释放立即有序插入

空闲链永远最新，但每块 O(n) 插入；成批排序后归并把代价摊成 O(b log b + n)，多批时一次 k 路归并 O(N log k)。

× 位图 + 找第一个 0 位

同样能给出最小空闲号，空间更省；但无法表达「同一块被释放两次」这一事件本身，消毒信息丢失。本仓库用位图做占用视图（kv.popcount），而不是分配结构。

依据 demo（18 块）：108 次单批归并 + 1 次 k 路归并；12 块时 139 + 2。sanitizer demo：重复释放的块 6 被隔离，test_double_free_is_quarantined_and_never_reused 验证之后再也不会分配它。

引用计数共享 + 写时复制，而不是拷贝

✓ 我们的选择

前缀满块按链式哈希去重共享；fork 只深拷贝块表结构、物理块 refs+1；checkpoint 用 +1 引用钉住块。真正的拷贝推迟到第一次写入共享的部分尾块（COW）。

× fork / checkpoint 时整份拷贝 KV

逻辑最简单、没有引用计数 bug 的风险；但 n 个样本占 n 份 prompt KV，投机解码每轮都要拷整张表。只有样本立即全面分叉、几乎不共享时，拷贝和 COW 的总量才接近。

× 按哈希共享但不复核 token key

省一次元组比较；但哈希碰撞会把别人的 KV 当成自己的，结果静默出错。本实现命中后必须 key 完全相同才共享。

依据 fork demo n=4：prompt 块 [0,1,2] 各被引用 4 次，COW 3 次，峰值 15 块 vs 独立拷贝 24 块，样本与独立 generate() 逐字相同。spec demo：34 次回滚全部精确恢复块表，COW 32 次。

抢占时重算（recompute）而不是换出（swap）

✓ 我们的选择

store 失败时服务器抢占最晚入场的请求：释放全部块、清掉已流式输出的事件、放回队首，之后从头重算；靠种子保证输出不变。释放的满块进 cached / 主机层，重入时大多命中。

× 把被抢占请求的整张块表换出到主机内存

恢复时省掉 prefill 计算，长 prompt、PCIe 带宽充足时更好；但要额外管理换出区、换入要等传输完成，而且换出的是整张表（含不可共享的尾块）。本设计把「保留有价值的 KV」交给按块的 cache + 主机层：只有登记过哈希的满块才被保存，且可以被任何请求复用，不仅仅是原请求。

依据 server demo（18 块 × 8）：14 次抢占，20 个输出与无限显存一致，prompt 块 36 share / 46 revive / 23 restore / 29 compute，命中率 0.784；16 块时命中率 0.791 反超 64 块的 0.692。

GPU 端按热度冷优先驱逐 vs 主机层 LRU / LFU / 随机

✓ 我们的选择

GPU 端只看最老的 scan 个释放，用稳定分区把热度 < hot 的冷块排前面再取受害者：近似「LRU + 热点保护」，代价 O(scan) 且不需要额外有序结构。主机层策略可配置，因为哪种更好取决于负载。

× GPU 端纯 LRU（最老释放先走）

最便宜；但一次长扫描就会把被反复命中的系统 prompt 块挤出显存。
test_gpu_eviction_offloads_cold_blocks_first 中被使用 3 次（热度 2）的热块在 12 个一次性 prompt 冲刷下仍留在 cached。

× 主机层 LFU

热点长期稳定、夹杂一次性扫描时最好（hot_vs_scan 赢）；但热点迁移后旧宠儿的高频计数会长期霸占容量（shifting_hot_set 输）。

× 主机层 LRU

热点迁移时适应最快；但扫描长度超过容量时会忘掉真正的热点。

× 主机层随机

O(1)、无状态、对抗性 trace 下不会被系统性击败；但不利用任何局部性，两条 trace 上都不是最好。

依据 host=4 块：hot_vs_scan 恢复/重算 LRU 4/51、LFU 11/44、随机 2/53；shifting_hot_set LRU 33/18、LFU 26/25、随机 33/18。host=16 时三者完全相同（11/44、33/18）。

调试期全量不变量检查 + 泄漏 / 重复释放定位

✓ 我们的选择

debug 模式下每个公开操作后跑 check()；report() 总是附带 problems 和 leaks。泄漏用「区间计数 + XOR + 二分」精确定位到块号，事件日志和版本化记账给出最后一次事件与当时的持有者。

× 只在测试结束时检查 in_use == 0

便宜，能发现「有泄漏」；但不知道是哪个块、何时、谁干的，而且中途的重复释放可能已经让两个序列写坏了同一块 KV。生产环境确实应关掉全量检查（它是 O(blocks × tokens) 级别），只保留 flush 时的重复释放消毒这种几乎免费的防线。

依据 sanitizer demo：重复释放块 6 被隔离；在用时释放的块 4 被 check() 报为 both free and in use；泄漏的 [5,7,8] 被精确找出；人为成环的块表被报告。集成测试在 debug=True 下跑完三种策略的完整 trace。

框架文件导读

`nanoserve/kvcache/manager.py`	BlockManager：服务器的 KVBackend。admit/store/release、前缀共享、fork、checkpoint/rollback、驱逐、check()/audit_leaks()/report()。直接调用 kv.fork、kv.has_cycle、kv.partition_hot、kv.block_hash、kv.refcount、kv.versioned、kv.snapshot、kv.free_tracker、kv.leaked。
`nanoserve/kvcache/pool.py`	FreePool：有序空闲链 + 暂存环 + 惰性归并 + 重复释放消毒。调用 kv.sort_free、kv.ring_queue、kv.merge_free、kv.merge_free_k、kv.drop_double_free。
`nanoserve/kvcache/occupancy.py`	占用视图：Bitmap（kv.popcount）、free_summary（kv.free_ranges、kv.longest_run）、charge_counts（kv.compact、kv.seq_range）、merge_distinct（kv.merge_arrays、kv.dedupe）、PageTable（kv.page_lookup）。供 check、report 和泄漏审计使用。
`nanoserve/kvcache/eventlog.py`	定长环形事件日志（飞行记录仪），只靠时间戳列恢复顺序：kv.ring_head 找最老事件，kv.ring_search 按事件号定位，kv.rotate_buffer 生成时间序转储。
`nanoserve/kvcache/tiers.py`	主机层：按策略构造 LRU（kv.lru）、LFU（kv.lfu）或 RandomTier（kv.random_evict + kv.block_hash），统一 get/put 接口，键是链式哈希、值是校验和。
`nanoserve/kvcache/arena.py`	token-slot arena（相当于 SGLang 的 req_to_token）：连续分段 first-fit（kv.mem_alloc）+ 越界检查（kv.address_ranges）。它是仓库里连续分配的反例，会产生外部碎片。
`nanoserve/kvcache/workloads.py`	直接驱动 BlockManager 的负载：hot_vs_scan / shifting_hot_set 两条策略对比 trace、parallel_sample（fork + COW）、speculative_generate（checkpoint + rollback）。
`nanoserve/kvcache/demo.py`	python -m nanoserve kv 的五个小节：server、policies、fork、spec、sanitizer，每节都自带正确性断言。
`tests/domain/test_m3_block_manager.py`	直接测 BlockManager：fork/COW、回滚精确性、策略差异、冷优先驱逐、重复释放与泄漏注入、报告与暴力结果一致、事件日志回绕。

`tests/domain/test_m3_server_kv.py`	接入服务器后的集成测试：小预算下抢占但输出不变（debug 全量检查）、大预算不抢占、共享前缀只存一份、块越多越快、放不下的请求被拒绝。

实测数字

- server（20 请求，max_batch 4，18 块 × 8 token，LRU）：105 → 156 ticks，14 次抢占，输出与无限显存逐字一致
- prompt 块：36 share + 46 revive + 23 restore + 29 compute，命中率 0.784；decode 新块 44，COW 0
- 下沉主机层 87 块，合并成 67 次传输；空闲链归并 108 次单批 + 1 次 k 路；arena 峰值 199/288 格，0 次分配失败
- kv_blocks 扫描：12 → 281 ticks（18 次抢占），14 → 196（15），16 → 178（15），24/40/64 → 105（0）
- 主机层 4 块，hot_vs_scan 恢复/重算：LRU 4/51、LFU 11/44、随机 2/53
- 主机层 4 块，shifting_hot_set 恢复/重算：LRU 33/18、LFU 26/25、随机 33/18；主机层 16 块时三种策略完全相同
- fork n=4：prompt 块各被引用 4 次，COW 3 次，峰值 15 块 vs 独立拷贝 24 块
- 投机解码 k=4：34 轮，133 个草稿 token 接受 14 个，34 次回滚全部精确恢复块表，COW 32 次
- sanitizer：重复释放块 6 被隔离，泄漏块 [5,7,8] 被精确找出，成环块表被报告

建议的完成顺序

先做数组三件套 kv.compact → kv.dedupe → kv.merge_arrays：它们撑起 occupancy.py 的 charge_counts 和 merge_distinct，而 check() 和 report() 每次都要用，最容易用 demo 验证。接着做链表：kv.merge_free → kv.sort_free（直接复用二路归并）→ kv.merge_free_k → kv.drop_double_free，这四题组成 FreePool，做完后分配 / 释放 / 消毒就跑通了。然后是管理器直接调用的三题 kv.has_cycle、kv.fork、kv.partition_hot，分别对应自检、并行采样和 GPU 端驱逐。环形缓冲三题 kv.ring_head → kv.ring_search → kv.rotate_buffer 一起做（同一个旋转有序数组模型，事件日志依次用到）。kv.lru 放在 kv.lfu 之前，做完用 policies 小节对比两者。kv.longest_run 最后做，它只影响报告里的 largest_free_run。其余 M3 题（block_hash、refcount、snapshot、versioned、ring_queue、mem_alloc 等）按 manager.py docstring 的列表穿插完成；全部完成后用 NANOSERVE_REFERENCE 关闭跑 python3 -m nanoserve kv 应看到 all checks passed。

第 4 章 · M4

第 4 章　前缀缓存 + EDF 截止时间队列 + checkpoint：让服务器记住前缀、按截止时间排队、能停能续

> 服务器学会三件事：复用已算过的 prompt 前缀以缩短 prefill（TTFT 下降），按最早截止时间（EDF）而不是到达顺序放行请求，并能把等待队列的索引写成 checkpoint、读回校验后原样恢复。

问题：重复的前缀和不同的紧迫程度

M2/M3 的服务器对每个请求一视同仁：prompt 从头 prefill，等待队列先来先服务（FCFS）。真实流量有两个被浪费的结构。第一，大量请求共享同一段开头——系统提示词、多轮对话的历史、few-shot 示例。`make_trace` 生成的 40 个请求里，很多都以同一句 system prompt `The engine keeps the keys and values of every token in a cache.`（37 个 token）开头，FCFS 下每次都重算这 37 个 token 的 KV。第二，请求的紧迫程度不同：聊天请求要在几十个 tick 内回答，批处理任务的截止时间宽松得多，但 FCFS 只看谁先到。

M4 用两个可插拔组件解决这两件事，二者都通过 `nanoserve/server/plugins.py` 里的协议接入，服务器核心一行不改：`PrefixCacheBackend`（`ServerConfig(prefix_cache=True)` → `PrefixCache`）和 `WaitingQueue`（`policy="edf"` → `DeadlineQueue`）。另外，等待队列是服务器最关键的易失状态，M4 还给它加上 checkpoint/restore：暂停、写盘、读回校验、换入新队列后继续跑，结果与不中断的运行逐 tick 相同（`test_restore_waiting_queue_mid_run_and_resume`）。

一个硬约束贯穿全章：这些优化只改变时间，不改变文本。demo 和测试都断言开/关前缀缓存、FCFS/EDF 两种策略下 `outputs()` 完全相同。

前缀缓存：把 token 写成字符，交给字典树

`Server._admit` 放行请求时调用 `prefix_cache.match(prompt)`，得到可复用的前缀字符数 `cached_chars`，只对剩余部分 `prompt[cached_chars:]` 计算 prefill tick；`Server._finish` 在请求完成时调用 `insert(prompt)` 记住它。

`PrefixCache` 的核心技巧是“token 即字符”：先用引擎的 tokenizer 编码，再把每个 token id t 写成单个字符 `chr(0xE000 + t)`（Unicode 私用区，不可能与真实文本混淆）。于是一条 prompt 变成“每个 token 恰好一个字符”的字符串，“前 k 个 token 是否是某条已缓存 prompt 的前缀”就成了普通的 `starts_with` 问题，直接复用练习 `prefix.trie` 的 `PrefixTrie`——这也是为什么规格要求键可以是任意 Unicode 字符、孩子必须用 dict 而不是长度 26 的数组。

这个性质对 k 是单调的（前 k 个是前缀，则前 k−1 个也是），所以 `match` 在 [0, L] 上二分最大的 k，每一步问一次 `starts_with(key[:mid])`，总代价 O(L log L)，与缓存里有多少条 prompt 无关。得到 k 后解码 `ids[:k]` 并返回其字符长度；因为 tokenizer 是从左到右的贪心匹配，这段文本恰好等于 `prompt` 的一个前缀，且剩余部分重新编码恰好是 `ids[k:]`（`test_match_is_longest_cached_token_prefix` 逐条断言这一点）。

统计部分用两份 `PrefixHitSum`（练习 `prefix.hit_sum`）：每次命中，以“被复用的文本”为键记录命中次数与节省的 token 数。`report()` 默认关注所有已缓存 prompt 的“共享前缀”——至少 `min_share` 条 prompt 共有的最长前缀，用 M1 的 `tok.common_prefix` 在排好序的 token 串的滑动窗口上求得（字典序里共享前缀的串必然相邻）。于是“有多少次命中复用了完整的 system prompt”就是一次 `sum(prefix)` 查询。

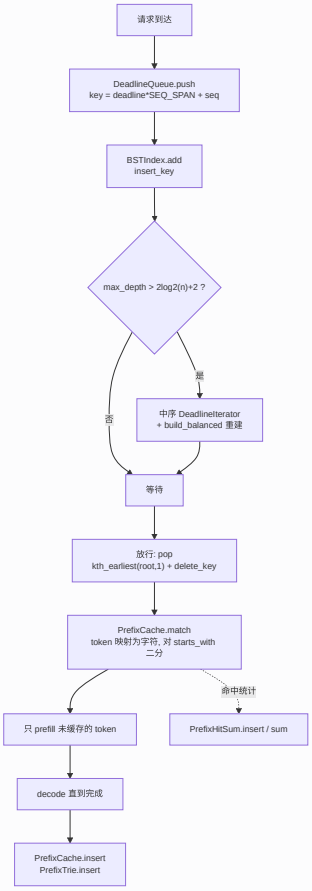


图 · 一个请求在 M4 服务器中的路径：EDF 队列决定何时放行，前缀缓存决定 prefill 多少

为什么匹配必须落在 token 边界上

token t 的 KV 依赖它之前的全部 token，所以一条缓存只能复用到两条 prompt 共享的最后一个完整 token。按字符匹配看起来更“细”，实际上是错的。用仓库的 tokenizer 跑一个具体例子：缓存了 `SYSTEM + "The cache"`，新请求是 `SYSTEM + "When ..."`。逐字符比较，两者共享 64 个字符（含 system prompt 末尾的空格）；但第 64 个字符——那个空格——在缓存里属于 token `' The'`，在新 prompt 里属于 token `' When'`。按 token 比较只共享 36 个 token = 63 个字符。

如果硬按 64 个字符切，会发生两件坏事：一是根本不存在“只含一个空格”的 KV 可复用，缓存里那个位置的 KV 是 `' The'` 的；二是剩余部分 `"When"` 被单独编码成 `W/h/e/n` 四个单字符 token，整条 prompt 从 37 个 token 变成 41 个，模型看到的序列都变了——输出文本随之改变，违反了“缓存只改时间”的硬约束。这就是为什么 `match` 返回值的语义是“字符数，但总落在 token 边界上”，以及为什么 `hits_under` 要对查询前缀做 `rstrip()`：被复用的 system prompt 往往以 `"... cache."` 结尾，末尾空格跟着下一个词的 token 走了。

EDF 队列：一个排序键，一棵 BST

`DeadlineQueue` 把等待请求放在 `BSTIndex` 里，树的中序就是排队顺序。BST 要求键互不相同，而 deadline 会重复，于是每个请求第一次入队时拿一个序号 seq，键为 `deadline · SEQ_SPAN + seq`（`SEQ_SPAN = 10^6`，无 deadline 的请求用 `NO_DEADLINE = 10^12` 代替 deadline）。一个整数同时编码了三条规则：截止时间早的先走；截止时间相同按到达顺序（FCFS 打破平局）；没有截止时间的排在所有有截止时间的后面。被抢占的请求 `push_front` 时沿用原来的键，所以它会回到原位——排在同一截止时间、后来才到的请求之前。

队列的每个操作都落在一道练习上：`push` → `BSTIndex.add` → `insert_key`（index.insert）；`peek/pop` → `first()` → `kth_earliest(root, 1)`（index.kth），再 `delete_key`（index.delete）；`__iter__` →

`DeadlineIterator`（index.iter）；`position(req)` → 在 `kth_earliest` 上二分求名次；`at(k)` → `kth_earliest(root, k)`；`check()` → `is_valid_index`（index.validate）+ 迭代计数 + `max_depth`（prefix.depth）。键→请求、请求→键用两个 dict 维护，BST 里只存整数。

第二棵 `BSTIndex` 只存“每个仍在等待的不同 deadline 一个键”，配一个 `_at_deadline` 计数（同一 deadline 第一个请求进来时插入，最后一个离开时删除）。为什么要第二棵树？`min_gap`（index.min_gap）要回答“两个不同截止时间的最小间隔”，在复合键上算出来的是 seq 的差，毫无意义；在去重后的 deadline 树上做一次中序相邻差就对了。

让 BST 保持浅：每次更新后测深度，超限就整树重建

普通 BST 插入在有序输入下退化成链表，而 deadline 恰恰大致有序到达（后来的请求截止时间更晚）。`BSTIndex` 不写旋转，而是在每次 `add/discard` 后用 `max_depth` 测量深度，一旦超过 `depth_limit(n) = 2·log2(n) + 2`，就用 `DeadlineIterator` 中序取出全部键、`build_balanced`（index.build）中点建树，整棵重建成完全平衡的形状。

这样做的好处是保证与 insert/delete 的具体实现无关：规格允许 `insert_key`、`delete_key` 返回任意合法 BST（挂叶子、用后继或前驱替换都行），平衡由外层统一兜底。代价是每次更新都 O(n) 测深度，对几百个请求的等待队列可以接受。demo 第 3 部分把 64 个请求按 deadline 递增推入：最终深度 10（上限 14.0，链表会是 64），共触发 8 次重建，历史最大深度 14；最后一层层宽 `[1, 2, 4, 8, 16, 29, 1, 1, 1, 1]` 直观地显示了“上次重建得到 60 个键的 6 层满树，之后 4 个更大的键串成右链”。可以算一下摊还：重建后最右节点在第 ⌈log2(n+1)⌉ 层，每来一个更大的键深度加 1，大约 log2 n + 2 次插入后再次超限，所以有序输入下每 Θ(log n) 次插入付一次 O(n) 的重建。

区间与统计：剪枝中序、树状数组；损坏修复：分叉点

`due_between(lo, hi)` 要列出 deadline 落在 [lo, hi] 的请求，按 EDF 顺序。把 deadline 区间换成键区间 `[lo·SEQ_SPAN, hi·SEQ_SPAN + SEQ_SPAN − 1]` 后，`BSTIndex.between` 先按键值从根往下走到区间的「分叉节点」（第一个落在 [lo, hi] 里的节点，O(h)），再从它开始做**剪枝中序**：只有节点键 > lo 才往左走、只有 < hi 才往右走。区间外的节点只会在两条边界路径上被访问，总代价 O(h + 答案个数)。

这里刻意**没有**用 prefix.fork_point：在合法的 BST 里，按值往下走就能 O(h) 找到分叉点，而通用二叉树的 LCA 要 DFS 整棵树、O(n)。通用 LCA 真正不可替代的场合恰好相反——树的**序已经坏了**、不能再相信键值比较的时候。`BSTIndex.repair` 处理的就是这种情况：一个写坏的 checkpoint 把两个键放错了节点（`restore_queue(..., repair=True)`）。中序序列里第一个和最后一个逆序的位置给出两个受损节点，它们的最近公共祖先（`fork_point`）就是包含全部损坏的最小子树；把这棵子树的键排序后按中序位置写回，其余部分一个节点都不动。单次交换一轮就修好，测试会检查修复后的出队顺序与损坏前完全一致。

`tokens_due(lo, hi)` 问“这个截止时间窗口内的请求最多还要生成多少 token”。`RangeCounter`（index.range_sum，树状数组）以 deadline tick 为桶、桶值为 Σ max_tokens，单点修改与区间求和都是 O(log n)。规格里 `update` 是赋值不是累加，所以队列做“读-改-写”：`update(d, sum_range(d, d) + delta)`；桶不够时容量翻倍、用旧桶值重建一个新的 `RangeCounter`。这些查询在每次请求到达时采样（overtakes、紧急积压、最紧 deadline 间隔），进入 `report()`。

EDF 什么时候赢，什么时候输

`deadline_trace` 是为 EDF 精心构造的场景：tick 0 到达 4 个长批处理任务（48 token，deadline 400）和 2 个无 deadline 的后台任务，之后每 tick 来一个聊天请求（8 token，deadline = 到达 + 88），`max_batch=2`。FCFS 让聊天请求排在 6 个长任务后面：截止时间满足 4/12，聊天 0/8，聊天 p50 延迟 151。EDF 让聊天请求插队（共 32 次超车，单次最多 4 个）：12/12、聊天 8/8、p50 延迟 74；代价是批处理最大延迟从 127 涨到 167，但离 400 的截止时间仍很远。

但 EDF 不是免费午餐。用 `make_trace(n=60, seed=1, deadline_slack=…)` 做均匀随机流量（deadline = 到达 + max_tokens + 均匀松弛），`max_batch=2`：负载轻时（rate=0.1，松弛 20–60）EDF 42/60 胜过 FCFS 36/60；松弛固定为 40 时，deadline 顺序几乎就是到达顺序，两者打平（53/53）；而在均匀过载下（rate=1.0，松弛 0–200）EDF 只满足 8/60，FCFS 17/60；rate=3.0 时 EDF 2/60 对 FCFS 17/60。原因是“多米诺效应”：EDF 总是先服务截止时间最早的请求，哪怕它已经注定赶不上；为它花掉的时间又让后面本来来得及的请求也错过。EDF 的最优性结论（单处理器、可抢占、负载可行时）在这里三条都不成立：我们只在放行时排序、不按 deadline 抢占运行中的请求，有多个槽位，而且负载根本不可行。过载时更合适的是准入控制（丢掉或降级已经赶不上的请求），或者以“满足的请求数最大”为目标的贪心——这正是 M2 的 `sched.slo_bound`（LC 630，Course Schedule III）的思路。

Checkpoint：前序序列 + 读回校验

`checkpoint_queue` 写下重建队列所需的一切：索引树、每个键属于哪个请求、所有序号、下一个序号。索引树有两种格式。紧凑格式 `"P:" + 前序键列表`：键互不相同的 BST 由前序序列唯一确定，`bst_from_preorder`（index.restore）用单调栈 O(n) 重建出**形状完全相同**的树。通用格式 `"T:" + TreeCodec().serialize(root)`（prefix.codec）：不假设 BST，层序加空位标记，任何二叉树都能无损往返——当索引被怀疑已损坏（不再是 BST）时，只有它能如实 dump 出来（`test_codec_dumps_a_tree_that_is_not_a_bst`）。

写完立即读回：`verify` 把 blob 重新解码，用 `same_tree`（prefix.same_snapshot）逐节点与内存中的树比较，不一致就抛错，checkpoint 不被信任。恢复时 `DeadlineQueue.from_index` 通过 `BSTIndex.adopt` 原样接管这棵树（不重新插入，形状不变），其余派生状态（`_at_deadline`、第二棵 deadline 树、树状数组）都从 owners 重新计算，最后跑一遍 `check()`。保留形状很重要：形状不同虽然顺序相同，但后续的插入/删除/重建时机会不同，测试要求的是“与不中断的运行逐 tick 相同”。demo 暂停时 12 个等待请求，紧凑格式 133 字符、通用格式 159 字符，两者读回都 exact，恢复后队列顺序 identical。

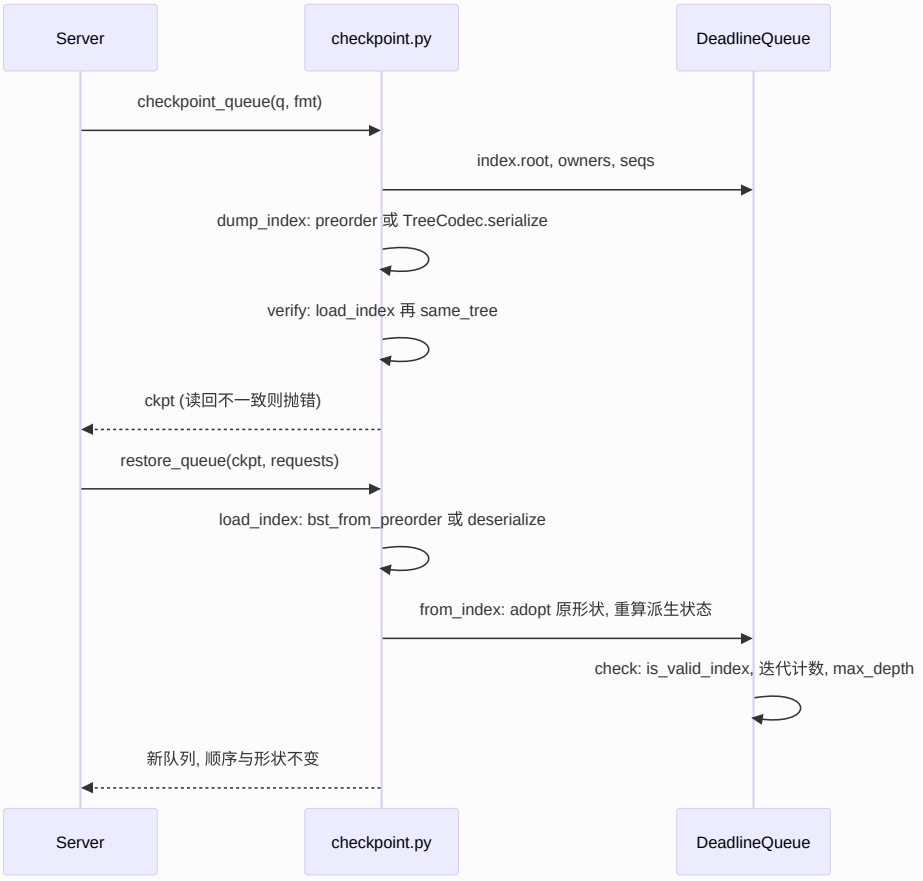


图 · checkpoint 写入、读回校验与恢复

关键设计取舍

前缀匹配的粒度：token 边界（token 当字符放进 Trie） vs 字符级 vs Radix Tree vs 块哈希

✓ 我们的选择

把每个 token 映射成一个私用区字符，在 `PrefixTrie` 上做字符串前缀查询。匹配天然只能停在 token 边界，满足“KV 只能复用到最后一个共享 token”的正确性要求；同时练习 `prefix.trie` 保持最朴素的接口（insert/search/starts_with），不用为 token 列表专门写一套。

× 字符级匹配（直接在原始 prompt 文本上建 Trie）

实现最简单，但会在 token 中间切开：在仓库 tokenizer 上，`SYSTEM+"The cache"` 与 `SYSTEM+"When"` 按字符共享 64 个字符、按 token 只共享 63 个字符；硬按 64 切会让剩余文本重新分词成 4 个单字符 token，prompt 从 37 变 41 个 token，输出随之改变。只有在 tokenizer 是逐字符的（每个字符一个 token）时它才等价。

× Radix Tree（SGLang RadixAttention）

把只有一个孩子的路径压成一条边（边上是一段 token 序列），节点数从 O(总 token 数) 降到 O(缓存的 prompt 数)，节点上直接挂 KV 页，并按 LRU 淘汰叶子、用引用计数保护正在使用的前缀。生产系统需要这些；nano-serve 的缓存不存 KV（只缩短模拟的 prefill tick）、也不淘汰，逐 token Trie 足够，而且就是 Radix Tree 的未压缩版本，概念上直接可迁移。

× 块级哈希（vLLM automatic prefix caching）

按固定大小的 KV 块计算“本块 token + 前缀哈希”的哈希，查哈希表命中整块；与分页 KV（M3）天然契合，查找 O(块数)。代价是只能按块粒度复用，最后一个不满的块永远不命中；需要逐 token 精度或没有分页时不如 Trie。

依据 `test_match_is_longest_cached_token_prefix`：对 120 次随机查询断言 match 等于“与任一缓存 prompt 的最长公共 token 前缀”换算的字符数，且 `tok.encode(prompt[got:]) == ids[best:]`。demo：40 个请求 prefill tick 335 → 95，平均 TTFT 81.2 → 50.2，输出完全相同。

求最长匹配长度：对 starts_with 二分 vs 沿 Trie 走一遍

✓ 我们的选择

`PrefixCache.match` 把 `PrefixTrie` 当黑盒谓词，在 [0, L] 上二分最大的 k 使 `starts_with(key[:mid])` 为真。利用的是谓词的单调性，代价 O(L log L)（每次切片和查询都是 O(mid)），与缓存大小无关；好处是练习接口保持 LC 208 原样，框架不依赖 Trie 的内部节点结构。

× 从根沿 Trie 一路往下走，走不动就停

O(L)，严格更快，也是生产实现（Radix Tree 的 match_prefix）的做法。需要 Trie 暴露“返回最长可走深度”的接口，即在练习之外多一个方法；当 prompt 很长（数千 token）且查询频繁时应该这样做。

× 对每条缓存 prompt 求公共前缀取最大

O(N · L)，随缓存条数线性变慢；测试里正是用它当 oracle 来验证二分结果，正确但不能上线。

依据 cache.py 中的不变量注释：前 lo 个 token 是缓存前缀，超过 hi 个不是；`lo, hi = 0, len(key)`，`mid = (lo + hi + 1) // 2` 向上取整避免死循环。

放行顺序：EDF vs FCFS

✓ 我们的选择

`policy="edf"` 时按 `deadline·SEQ_SPAN + seq` 排序，同一截止时间退化为 FCFS，无截止时间排最后；抢占回队沿用原键。在混合负载（宽松的长任务 + 紧迫的短请求）下，它让紧迫请求超车，而长任务仍在截止时间内完成。

× FCFS（默认 `FCFSQueue`，deque）

O(1) 入出队、无饥饿、无需 deadline 信息。当负载均匀过载时它反而更好：EDF 会把时间花在已经注定超时的请求上，产生多米诺效应；FCFS 至少让一部分请求按时完成。

× 最短作业优先（M2 `sched.sjf`）/ 最大化按时完成数（M2 `sched.slo_bound`）

SJF 最小化平均延迟但不看截止时间；slo_bound 的“按 deadline 排序 + 超时就丢掉最长的”贪心（Moore–Hodgson 思想）在过载时最大化满足数，需要准入控制（允许拒绝请求），本队列不做拒绝。

依据 构造 trace：deadline 满足 FCFS 4/12 → EDF 12/12，聊天 0/8 → 8/8，聊天 p50 延迟 151 → 74，批处理最大延迟 127 → 167（deadline 400）。均匀过载（make_trace n=60 seed=1，slack 0–200）：rate=1.0 时 FCFS 17/60、EDF 8/60；rate=3.0 时 17/60 对 2/60；固定 slack 40 时打平。

有序索引：不平衡 BST + 超深重建 vs 自平衡树 / 堆 / sortedcontainers

✓ 我们的选择

`BSTIndex`：普通 BST 插入删除（index.insert / index.delete），每次更新后 `max_depth`，超过 `2·log2(n)+2` 就中序取键 + `build_balanced` 整棵重建。实现只用到 LeetCode 级别的树操作，平衡保证与插删实现无关，深度有硬上限。

× 二叉堆 heapq

push/pop O(log n)、常数小，只需要“取最早”时是最佳选择。但队列还要：删除任意请求（`remove`，堆只能懒删除）、第 k 个与名次（`at` / `position`）、按区间列举（`due_between`）、有序迭代（`__iter__`）——堆做这些都要 O(n log n) 排序。

× AVL / 红黑树

每次更新 O(log n) 且不需要 O(n) 测深度，规模到上万时显著更好；代价是旋转与删除修复的实现复杂，且要求 insert/delete 维护高度/颜色，不能再让练习返回“任意合法 BST”。

× sortedcontainers.SortedList / bisect 有序列表

SortedList 的 add/remove/索引都是近似 O(log n)，第三方库、工程上最省事；纯 bisect.insort 插入 O(n) 但常数极小，几百个元素时其实很快。本仓库目的在于练习树，且 checkpoint 需要一棵真实的树形状。

依据 demo 第 3 部分：64 个递增 deadline，深度 10（上限 14.0），8 次重建，历史最大深度 14。
`test_rebalance_triggers_on_sorted_insertion`：200 个有序插入，每一步深度 ≤ depth_limit。

checkpoint 格式：前序紧凑格式 vs 通用编解码器

✓ 我们的选择

默认 `"P:"` + 前序键列表：利用“键互不相同的 BST 由前序唯一确定”，只写 n 个数，`bst_from_preorder` 单调栈 O(n) 恢复出相同形状；写完用 `same_tree` 读回比对，不一致就拒绝这个 checkpoint。

× 通用 `TreeCodec`（层序 + 空位标记，`"T:"`）

写 2n+1 个记号，不依赖 BST 性质；当索引可能已损坏（重复值、不满足 BST）时它是唯一能如实保存现场的格式，适合调试转储。

× 只存有序键列表，恢复时 build_balanced

更简单，恢复出的也是合法 BST，但形状变了：后续插删与重建的时机都会不同，做不到“恢复后与不中断运行逐 tick 一致”。

× pickle 整个队列对象

一行搞定，但格式不可读、绑定 Python 版本和类定义、无法独立校验，且把请求对象也一起序列化了；按 rid 引用请求更干净。

依据 demo 暂停在 tick 9（12 个等待请求）：preorder 133 字符、codec 159 字符，均 read-back exact、恢复后顺序 identical；`test_restore_waiting_queue_mid_run_and_resume` 要求两种格式恢复后每个请求的 admitted_at/finished_at 与不中断运行相同。

框架文件导读

`nanoserve/prefix/cache.py`	`PrefixCache`：token→私用区字符，`match` 对 `PrefixTrie.starts_with` 二分最长缓存前缀，`insert` 记住完成的 prompt；命中统计用两份 `PrefixHitSum`，共享前缀用 M1 `tok.common_prefix`。调用 prefix.trie、prefix.hit_sum。
`nanoserve/index/bst_index.py`	`BSTIndex`：互异整数键的有序集合，插删后测深度、超限重建；提供 first/kth/rank/between/min_gap/levels/check。调用 index.insert、index.delete、index.kth、index.iter、index.validate、index.build、index.min_gap、prefix.depth、prefix.level_dump、prefix.fork_point。
`nanoserve/index/queue.py`	`DeadlineQueue`：EDF 的 `WaitingQueue` 实现。复合键 deadline · SEQ_SPAN+seq，请求索引 + 去重 deadline 索引两棵 `BSTIndex`，按 deadline 桶统计 token 的 `RangeCounter`（index.range_sum），以及 checkpoint 所需的 owners/from_index。
`nanoserve/index/checkpoint.py`	两种索引格式（前序 `P:` / 通用 `T:`）的 dump/load，`verify` 读回比对，`checkpoint_queue` / `restore_queue` 整队列存取。调用 index.restore、prefix.codec、prefix.same_snapshot。
`nanoserve/index/workloads.py`	`deadline_trace`：长批处理 + 无 deadline 后台任务先到、紧迫聊天请求随后的构造 trace，EDF 在这里全部满足、FCFS 错过聊天请求。不调用练习。
`nanoserve/prefix/demo.py`	`python -m nanoserve prefix`：三部分——前缀缓存开关对比、FCFS vs EDF（中途暂停展示层序 dump、名次、窗口、checkpoint 往返）、有序插入下的深度与重建次数。

`tests/domain/test_m4_prefix_cache.py`	集成测试：match 等于最长公共 token 前缀（oracle）、剩余部分重新编码精确、命中统计与服务器实际所见一致、共享前缀即 system prompt。
`tests/domain/test_m4_deadline_index.py`	集成测试：EDF 比 FCFS 满足更多 deadline 且输出相同；单槽位下放行顺序符合 EDF oracle；随机 push/pop/remove/push_front 下的全部不变量；有序插入触发重建；区间查询正确。
`tests/domain/test_m4_checkpoint.py`	集成测试：两种格式往返形状一致、verify 能发现差异、codec 能 dump 非 BST 的损坏树、暂停-checkpoint-恢复-续跑与不中断运行逐 tick 一致。

实测数字

- 前缀缓存（make_trace 40 请求，max_batch=4，prefill_chunk=4）：prefill tick 335 → 95，平均 TTFT 81.2 → 50.2，p50 延迟 98.0 → 65.5，总 tick 255 → 192
- 命中 33/40（未命中 7，完全重复 26 次），节省 920 个 token（占 prompt token 72%），节省 1432 个字符
- 共享前缀即 system prompt（37 token）；完整复用它的命中 10 次，共 495 token
- 构造 deadline trace（max_batch=2）：满足 FCFS 4/12 → EDF 12/12，聊天 0/8 → 8/8，聊天 p50 延迟 151 → 74，批处理最大延迟 127 → 167；32 次超车（单次最多 4）
- 均匀过载（make_trace n=60 seed=1，slack 0–200，max_batch=2）：rate=1.0 时 FCFS 17/60 vs EDF 8/60；rate=3.0 时 17/60 vs 2/60；轻载 rate=0.1、slack 20–60 时 36/60 vs 42/60
- tick 9 暂停：12 个等待请求，索引深度 9（上限 9.2）；checkpoint 前序 133 字符 vs 通用 codec 159 字符，均读回一致
- 64 个递增 deadline：深度 10（上限 14.0，链表为 64），8 次重建，历史最大深度 14，层宽 [1, 2, 4, 8, 16, 29, 1, 1, 1, 1]

建议的完成顺序

先做前缀缓存线：prefix.trie → prefix.hit_sum，做完即可跑 demo 第 1 部分（另需 M1 的 tok.common_prefix）。再做二叉树基础：prefix.depth、prefix.level_dump、prefix.same_snapshot，它们是后面所有树练习的热身，也是 BSTIndex 的监控与校验工具。然后是 BST 核心：index.insert、index.delete、index.iter、index.kth、index.validate、index.build——`DeadlineQueue` 的 push/pop/迭代/自检和重建全部依赖它们，缺一个 EDF 都跑不起来；接着 index.min_gap、index.range_sum 补齐到达时的统计（`_sample_arrival` 每次到达都会调用它们），此时 demo 第 2、3 部分可以跑通。最后做 checkpoint：prefix.codec 与 index.restore，配合已完成的 same_tree 跑 test_m4_checkpoint；prefix.fork_point 用于从写坏的 checkpoint 中修复索引，也在这一组里。index.restore 放在 index.insert 之后做，正好体会“按前序逐个插入能得到同一棵树，但单调栈把 O(n · h) 降到 O(n)”。

第 5 章 · M5

第 5 章　模型 IR 编译 + 配置系统：把模型变成可分析的算子图，把部署变成可校验的配置

> M5 让引擎第一次「看得懂」自己要跑的东西：模型被 lower 成 13 个算子的依赖图，编译器在上面做校验、调度、死代码消除、按上下文长度剪枝、关键路径延迟估计和多设备放置；同时一套自定义的配置格式经过分阶段流水线（去注释 → 括号预检 → 分节 → flag 拓扑求值 → 数值/布尔/模板/路径 → glob 路由）变成多 worker 的部署。

问题：直接调用模型函数，引擎对它一无所知

M1–M4 的引擎把模型当黑盒：`NGramModel.logits(context)` 进去一个上下文，出来一个 logits 向量。黑盒能跑，但引擎回答不了任何「关于计算本身」的问题：这次调用到底要做哪些事？哪些步骤彼此独立、可以并行？上下文只有 2 个 token 时，4-gram、3-gram 那两路查表还需要做吗？放到 4 张卡上，一步要多久？

真实的推理服务（TensorRT、TVM、torch.compile、XLA）的答案都是同一个：先把模型 **lower** 成一张显式的算子图（IR, intermediate representation），然后所有这些问题都变成图上的经典算法题。M5 用最小的模型把这件事完整走了一遍：`compiler.py::lower_ngram` 把 5 阶插值 n-gram 模型 p = floor/V + Σ λ_k · dist_k(context[-k:])、logits = log p 写成 13 个算子：5 个 `lookup`（dist4…dist0，cost = 2 + k）、5 个 `scale`（term4…term0）、1 个 `floor`（base）、1 个变参 `sum`（p）和 1 个 `log`（logits）。`IRModel` 是 `NGramModel` 的 drop-in 替换：demo 在一条真实 prompt 的全部 39 个前缀上比较，两者 logits **逐位相等**（`sum` 的可选项按声明顺序累加，就是为了让浮点求和顺序与原实现一致）。

IR 的形状：张量名即边，params / feeds / optional

`model_ir.py::Op` 是一个 frozen dataclass：`out`（产出的张量名，同时是算子名）、`kind`、`inputs`（必需输入）、`optional`（变参可选输入，只有 `sum` 用）、`cost`（抽象延迟单位）、`program`（系数字节码）。图里没有显式的边对象，边是由「名字相同」隐式定义的：`ModelIR.deps()` 扫一遍，凡是 a 的输入是 b 的输出，就产生一条 [a, b]（a 依赖 b）。没有算子产出的张量分两类：**params**（权重表、vocab_size，永远在）和 **feeds**（每次调用的输入，例如 `ctx4` = 最后 4 个 token，上下文不够长时就**不存在**）。

这里有一个刻意的职责划分：`ModelIR` 的构造函数只检查**名字**（唯一、算子格式正确、字节码结构合法），图的性质——无环、没有未知张量、没有死算子——全部留给编译器。这样一张坏图仍然可以被构造、打印、再被编译器带着精确的错误信息拒绝（demo 里那个 attn ↔ proj 互相依赖的 3 算子图就是这样被拒的）。

系数不是直接存 float，而是存成 32 位整数上的后缀字节码：λ_4 = 0.5 在定点 100 下写成 [5 100 * 10 /] = 50，表示 50/100。`fold_constants` 用 `eval_rpn` 把每段字节码跑一次（常量折叠），`mass_program` 把所有段拼成一个大程序求总概率质量，demo 里结果正好是 100/100 = 1，这是对模型归一化的一个廉价自检。

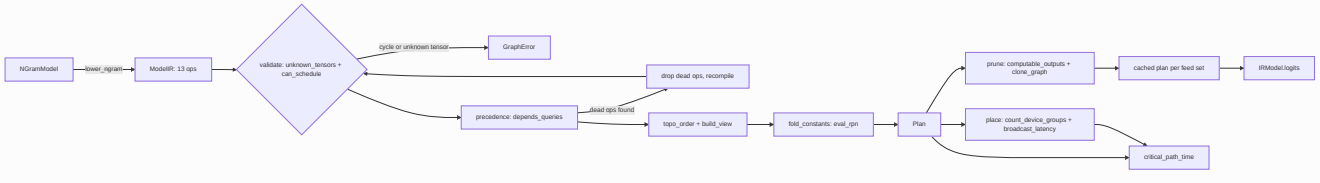


图 · IR 编译流水线：从 NGramModel 到按上下文特化的 Plan

compile_ir：五步流水线，每一步是一道题

`compiler.py::compile_ir` 的顺序是：(1) `validate`：每个输入必须是 param、feed 或某个算子的输出（`unknown_tensors`），然后 `can_schedule` 判环，有环抛 `GraphError`；(2) `precedence`：把「u 必须先于 v」对所有 n(n−1) 个有序对一次性批量问 `depends_queries`，得到 n×n 的布尔矩阵 `before`；(3) 死代码消除：不是输出、也不是输出祖先（`before[i][out]` 为假）的算子全部删掉，然后对剩下的图**递归**再编译一次（因为 before 是传递闭包，删一次就干净，递归只会多走一层）；(4) `topo_order` 给出执行顺序，`build_view` 构造无向的 `GraphNode` 视图供优化 pass 编辑；(5) `fold_constants`。

产物 `Plan` 携带 `order`、`before`、`view`、`consts`，后续所有分析都只读它：`parallel_pairs()` 就是 before 矩阵里两个方向都为假的对——13 个算子的 78 对里有 50 对可以并行，例如 dist4|dist3；dist4|term4 是直接的生产者-消费者，dist4|logits 是传递依赖，都不能并行。

顺序本身也值得看一眼：Kahn 算法给出 dist4 → dist3 → dist2 → dist1 → dist0 → base → term4 → … → p → logits，所有入度为 0 的查表排在最前面，天然是「一层一层」的波前（wavefront）顺序。

运行时特化：按上下文长度剪枝，在副本上做 pass

上下文只有 2 个 token 时，`ctx4`、`ctx3` 这两个 feed 根本不存在，dist4/term4/dist3/term3 必然算不出来。`compiler.py::prune` 把「已有的张量」= 全部 params + 实际存在的 feeds 交给 `computable_outputs`（只传**必需**输入；`sum` 的可选项不阻塞计算），得到可计算的算子集合。demo：长度 2 的上下文剪掉 4 个算子，剩下 9 个；长度 0/1/2/3/4 分别对应 5/7/9/11/13 个算子，长度 ⩾ 4 与 4 相同。

剪枝像所有优化 pass 一样**在副本上编辑**：先 `clone_graph(plan.view)`，在副本里把不可计算的节点从邻居表中摘掉，再从输出节点出发收集仍然连着的分量，最后把 `p` 的 `optional` 里已经被删的项过滤掉、重新 `compile_ir` 一遍（顺便再做一次死代码消除）。为什么必须克隆？因为 `IRModel` 只编译一次原始 plan，然后按「可用 feed 的集合」缓存每个特化 plan（`_pruned` 字典，键是 frozenset），原始 `plan.view` 要留给下一个不同长度的上下文用；测试 `test_clone_is_deep_and_pass_leaves_original_untouched` 检查原图邻接表不变、副本与原图没有共享任何节点对象。

延迟估计与多设备放置：关键路径、并查集、Dijkstra

有了依赖图和每个算子的 cost，`Plan.critical_path()` 调用 `critical_path_time` 求无限并行下的完成时间：dist4(6) → term4(1) → p(2) → logits(2) = **11**；而串行把 30 个单位全加起来是 **30**，最大的单个算子是 6。11 是任何并行执行的下界，30 是单设备的精确值，真实部署落在两者之间。

`devices.py::place` 把这个区间具体化。第一步，用 `count_device_groups`（并查集）在对称化后的链路矩阵上数「岛」：默认 6 设备拓扑有 2 个岛 [1,2,3,4] 和 [5,6]；岛成员关系甚至是用同一个练习当 oracle 求出来的——加一条 d–rep 直连边如果组数不变，说明它们本来就在一个岛里。第二步，权重要从一台 host 沿**有向**链路广播到岛内所有设备，`broadcast_latency`（Dijkstra）对每个候选 host 都算一遍：1:5、2:4、3:3、4:6 ms，选 host 3。注意并查集看的是无向（弱）连通，广播看的是有向可达，两者不一致时（例如 1→2→3 的单向链，host 3 谁也到不了）该 host 就无效。第三步，把重的 lookup 按 cost 从大到小贪心放到负载最小的设备上（LPT 思路），scale 跟随它的 lookup，floor/sum/log 留在 host。第四步，同一设备上的算子按调度顺序串行，于是把这些「设备内链」作为额外的先后约束 `extra_before` 加进依赖图，再求一次关键路径：**12**，正好落在 [11, 30] 之间（host 3 承担了 dist1、dist0 和整条尾巴，负载 12）。测试还验证了两个极端：单设备时 latency == serial，8 设备环时 latency == parallel。

配置系统：自定义格式 + 分阶段流水线

`config/loader.py` 定义了一种类 INI 的格式：`[server]`、`[model]`、`[workers]`、`[pool.<kind>]`、`[flags]`、`[routes]` 六类节，值里可以写 C 风格注释、整数表达式（`2 * 2`、`4096 - (512 + 128)`、`-(-50)`）、feature flag 条件（`&(stable, !(debug))`）、模板（`2[gpu-a,]cpu_b`）和路径（`/models/./toy/../toy/ngram-5/`）。`load_config` 是一条严格分阶段的流水线，每一阶段的输出满足下一阶段练习的**输入契约**：

(1) `clean_lines`：拒绝非可打印 ASCII，然后 `strip_comments`。练习的规格保证「块注释一定闭合」，而真实文件不保证，于是 loader 在文本后面追加两行哨兵 ["<eof>", "*/<eof>"]：块注释没开着时两行原样回来；开着时第一行被吞掉、第二行的 `*/` 把它关上、剩下的 `<eof>` 被拼到注释前的文本上，输出末尾就不可能恰好是这两行——一次调用同时完成剥离和「未闭合」检测。
(2) `split_sections`：先对整段（已去注释的）文本做 `brackets_balanced`，括号不配对直接报 unbalanced，不必进入后面的解析；再按行切成节和键值对。
(3) `resolve_flags`：flag 可以引用别的 flag，loader 用 `flag_refs` 建依赖边，交给 `topo_order`（与 IR 编译器同一个练习）求顺序，返回 [] 就是循环引用；再按顺序把名字替换成 t/f、交给 `eval_flag_expr`。server.conf 里 canary = &(stable, !(debug)) 写在 stable/debug 之后还是之前都行。
(4) 类型化取值：整数走 `eval_number`，按出现的运算符分派——有 `* / ` 走 `eval_arith`，只有 `+ - ( )` 走 `eval_expr`，**两者同时出现直接报错**；布尔走 `eval_condition`；`[workers] pool` 走 `expand_list` → `expand_template`；模型路径走 `normalize_path`。
(5) 路由：`LoadedConfig.route` 按文件顺序逐条 `glob_match`，第一条命中的规则胜出。

契约在框架，算法在练习：为什么每个求值器都能假设输入合法

所有 config 练习的规格都写着「保证输入合法」：`eval_arith` 假设没有括号和一元负号、不会除零；`eval_expr` 假设没有两个连续运算符；`eval_flag_expr` 假设语法正确、深度 ⩽ 200；`expand_template` 假设数字只作重复次数。把这些假设变成事实是 loader 的工作：`_check_arith` 检查 token 交替为「数 运算符 数 …」并拒绝 x/0；`_check_plus_minus` 用一个小状态机（want_operand + 括号深度）只允许一元负号出现在开头或运算符/左括号之后，所以 `1 + -2` 被拒；`_check_condition` 在调用前把条件语法走一遍；`expand_list` 检查每个数字后面紧跟 `[`、k ⩾ 1，并用 len × Πk ⩽ 100000 防止模板炸弹。溢出也在调用前用上界挡住：乘除式用 Π(n+1)，加减式用 Σn，保证中间值在 32 位以内——练习的「中间结果都在 32 位范围内」由此成立。

这种分工让每个练习保持 LeetCode 原题的纯净形态，同时让整个 loader 对任意输入都只抛 `ConfigError` 而不会崩溃；`to_text()` 再给出规范文本，`load_config(c.to_text()) == c` 被测试逐个示例验证（tiered.conf 的规范形式里 kv_blocks = 1024 / 8 - 2 * 4 已经变成 120，pool = 3[2[hbm,]ddr,] 展开成 9 个 worker）。

从配置到运行：Deployment 与 first-match 路由

`config/deploy.py::Deployment` 为每个 worker 建一个 `Server`，配置是 [server] 的值叠加该 kind 的 [pool.<kind>] 覆盖（`worker_config`）；所有 worker 共享一个 engine，`compiled = t` 时 engine 的模型就是上面的 `IRModel`。请求按模型名路由并打上 `req.worker`，没有任何规则命中的请求在到达 tick 被拒绝。

默认 demo（server.conf）：flags 解析为 stable=t debug=f canary=t fast=t paranoid=f，于是 compiled = fast = t，prefix_cache = &(paranoid, fast) = f；max_batch = 2*2 = 4，prefill_chunk = 64/4−8 = 8，token_budget = 3456；pool 展开为 gpu-a, gpu-a, cpu_b，cpu_b 被覆盖为 max_batch=1。路由表 toy-*-instruct → 1、toy-chat* → 0、Toy?Base → 2、* → 0：toy-7b-instruct 去 1，ToyXBase 去 2，toy-chat-v2 和 mystery 都去 0。24 个请求的 trace 跑下来，w0 处理 16 个（97 ticks），w1 处理 5 个（49 ticks），只有 max_batch=1 的 w2 处理 3 个却用了 87 ticks——配置的差异直接体现在吞吐上。first-match 的语义意味着规则顺序就是优先级：测试里 toy-chat-instruct 同时匹配前两条，取第一条 → worker 1，兜底的 * 必须放最后。

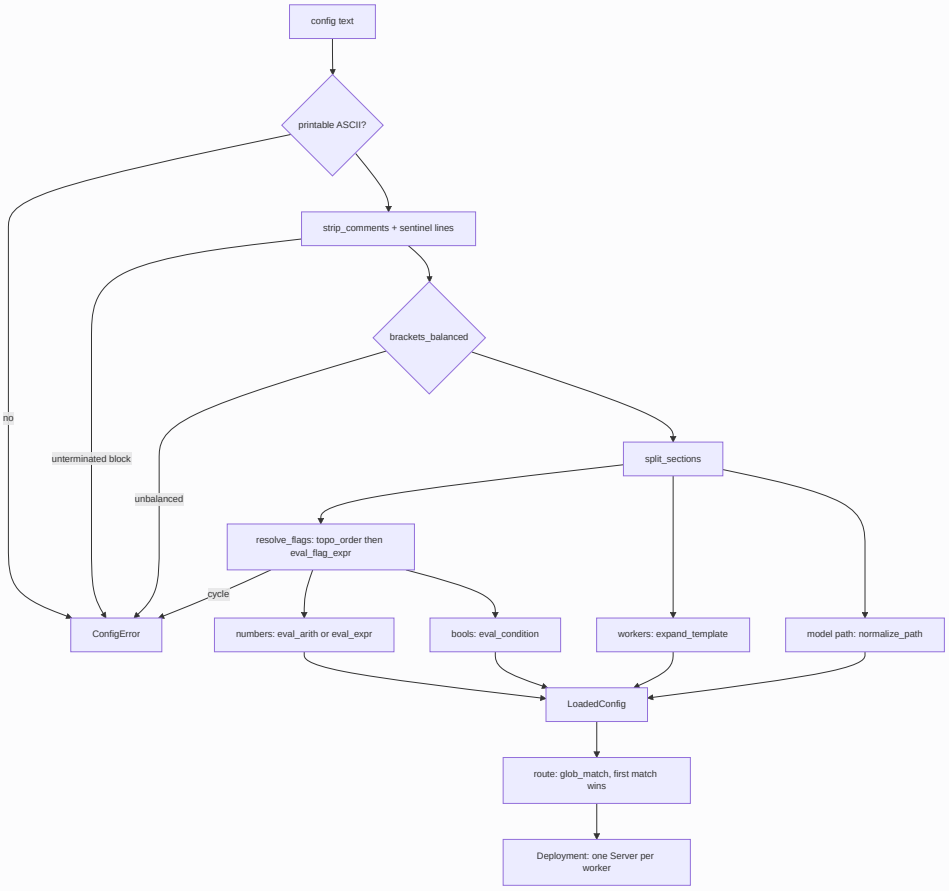


图 · 配置加载流水线：每一阶段为下一个练习建立输入契约

关键设计取舍

把模型 lower 成算子图再执行，而不是直接调用 NGramModel.logits

✓ 我们的选择

`lower_ngram` 生成 13 个算子的 `ModelIR`， `compile_ir` 产出 `Plan`， `IRModel.logits` 解释执行特化后的 plan。代价是多了一层解释器和一次编译；换来的是一组原来根本无法回答的问题都有了答案：图是否合法（判环、未知张量）、执行顺序、哪些算子可并行、对给定上下文哪些算子可以不跑、无限并行和具体放置下的延迟、权重广播时间。正确性用「逐位相等」兜底，所以这层抽象不改变任何输出。

× 直接调用模型函数（eager 执行）

零编译开销、调试最直观，模型结构经常变（研究代码、一次性脚本）时更好；但引擎拿不到结构信息，剪枝只能写死在模型里（NGramModel 里的 if d is not None），也无法做放置和延迟估计。

× 追踪（tracing）得到图，如 torch.fx / torch.compile

不需要手写 lowering，适合大模型；但追踪出的图依赖示例输入，数据相关的分支（这里「上下文够不够长」）会被固化或导致重新追踪。我们手写 lowering，把「可能缺席的 feed」显式建模，剪枝就成了图上的可计算性问题。

依据 demo：IR logits == NGramModel logits on all 39 prefixes；tests/domain/test_m5_ir.py 中 engine / server 使用 IRModel 生成的文本与原模型完全一致。

拓扑排序用 Kahn（BFS 入度）而不是 DFS 后序

✓ 我们的选择

`can_schedule`、`topo_order`（以及 `critical_path_time`、`depends_queries` 的骨架）都用 Kahn：入度为 0 的入队，弹出时给后继减入度。判环只需比较输出长度是否为 n；没有递归，不受 Python 递归深度限制；而且产出的是「波前」顺序——所有独立的 lookup 排在最前，正好对应可以同时发射的一批算子，`critical_path_time` 也能在同一遍里顺手做最长路 DP。

× DFS 三色标记 + 逆后序

同为 O(V+E)，判环靠「遇到灰色节点」；需要按依赖深度递归（或手写显式栈），输出会把每条 lookup→scale 链连在一起（深度优先），不体现并行层次。需要顺带求强连通分量、或者想要「尽早完成某个目标节点」的局部顺序时 DFS 更自然。

× 反复扫描找无依赖节点

实现最直白，O(V · (V+E))；n ⩽ 100 时也能跑，但 config 的 flag 解析和 IR 编译共用这个函数，没有理由接受平方复杂度。

依据 demo 的 schedule：dist4 -> dist3 -> dist2 -> dist1 -> dist0 -> base -> term4 -> ⋯ -> p -> logits；loader.py::resolve_flags 同样调用 topo_order，返回 [] 即报「flags refer to each other in a cycle」。

一次算出传递闭包矩阵 before[u][v]，而不是每次查询做 DFS

✓ 我们的选择

`compiler.py::precedence` 把 n(n−1) 个有序对一次性交给 `depends_queries`，参考解在 Kahn 顺序上传播位掩码：before[v] |= before[u] | (1 << u)。结果矩阵被反复使用：死代码消除查 before[i][out]，`parallel_pairs` 查两个方向，都是 O(1)。

× 每个查询从 u 出发 DFS/BFS 看能否到达 v

无预处理，查询很少（几十个以内）或图会频繁变化时更划算；但这里查询数是 n(n−1)，最多 9900，每次 O(V+E)，总量是闭包的 n 倍以上。

× Floyd–Warshall 式三重循环求可达性

$O(n^3)$，n ⩽ 100 时 10^6 次操作，代码最短且不需要先拓扑排序；位掩码版是 O((V+E) · n/w)，图稀疏时快得多，并且自然利用了 DAG 的结构。

依据 MAX_OPS = 100（model_ir.py 注释：依赖分析为小图设计）；depends_queries 规格 n ⩽ 100、len(queries) ⩽ 10^4 正是为 precedence 的批量查询量身定的。demo：parallel op pairs 50 of 78。

两种剪枝都要：从输出反向的死代码消除 + 从输入正向的可计算性剪枝

✓ 我们的选择

`compile_ir` 删「对输出没有贡献」的算子（反向：不是输出的祖先）；`prune` 删「在当前 feed 下算不出来」的算子（正向：`computable_outputs` 从 params + 现有 feeds 出发做 Kahn 式传播）。两者回答的问题不同：前者是结构性的、编译期一次；后者依赖每次调用的输入，按 feed 集合缓存。prune 之后再调用 compile_ir，等于两种剪枝串联。

× 只做死代码消除

对静态图足够（大多数编译器的 DCE 就是这个）；但它不知道「ctx4 缺席」，dist4 永远是输出的祖先，删不掉。

× 运行时动态跳过（算到 None 就跳过）

这正是 NGramModel 原本的做法，也是 execute 对 None 的处理；无需编译，但每次调用都要走一遍全部算子才知道哪些没用，也无法提前算出特化图的关键路径和放置。

依据 test_dead_ops_are_eliminated（unused 被删）；demo：len(context)=0..4 时分别 5/7/9/11/13 个算子，长度 2 剪掉 dist4 term4 dist3 term3。

延迟估计用关键路径（下界）+ 设备内串行链，而不是串行求和

✓ 我们的选择

`Plan.critical_path` 用 `critical_path_time` 求无限并行下的完成时间（11），`serial_time` 是全部 cost 之和（30）。具体放置时把同一设备上按调度顺序相邻的算子对作为额外先后约束加进图，再求一次关键路径（12）。这样一个函数同时给出下界、上界和具体放置的估计，并且有可检验的不变量 parallel ⩽ latency ⩽ serial。

× 串行求和

单设备、单流时是精确值，计算平凡；但对多设备完全没有区分度，也看不出 lookup 之间可以重叠。

× 离散事件模拟 / 列表调度（有限设备数、计入传输时间）

更接近真实（devices.py 明确说估计忽略了跨设备传输）；代价是实现复杂、结果依赖调度策略。在只需要比较放置方案优劣的阶段，带额外约束的关键路径已经够用。

依据 demo：critical path 11 vs serial 30（max single op 6）；放置后 step latency estimate 12；test_placement_latency_between_parallel_and_serial：单设备 latency == serial，8 设备环 latency == parallel。

自定义配置格式 + 分阶段流水线 + 受限文法，而不是 YAML/TOML + eval

✓ 我们的选择

每种值有一个小而封闭的文法，各由一个栈式求值器处理（eval_arith、eval_expr、eval_flag_expr、expand_template），loader 在调用前检查其输入契约；不同文法之间不混用（同一个值里不能同时出现括号和 * /）。整个加载过程不执行任何用户代码，任意输入只会得到 ConfigError，且可以规范化回文本（round-trip）。

× YAML/TOML 负责结构，表达式用 Python eval

零解析代码、生态成熟；但 eval 等于让配置文件执行任意代码（安全红线），而 flag 依赖、模板、路径规范化仍要自己写。结构简单、团队已有 YAML 工具链且值都是字面量时，YAML 更好。

× 一个完整的递归下降解析器（统一的 + - * / ( ) 文法，即 Basic Calculator III）

用户体验更好，表达力更强；但递归深度受限、错误处理更复杂。我们选择禁止混用，以复用两道经典栈题并保持每个求值器 O(n)、无递归；需要更丰富表达式时再升级为递归下降。

× 路由用最长匹配 / 最具体匹配，或 fnmatch/正则

最具体匹配对写规则的人更宽容，但「具体」难以定义（toy-*-instruct 和 toy-chat* 谁更具体？）；first-match 语义简单可预测，代价是规则顺序必须由写的人负责。fnmatch 还支持 [] 字符类、内部转成正则，语义与我们的 ? * 不同。

依据 test_number_dispatch_and_arithmetic 拒绝 2 * (3 + 1)、-2 * 3、4 / 0、1 + -2；test_round_trip_examples；test_routes_first_match_wins：toy-chat-instruct → worker 1。

框架文件导读

`nanoserve/ir/model_ir.py`	IR 数据结构：`Op`、`ModelIR`（只查名字，`deps()` 由张量名推出依赖边，`build_view` 给出无向 GraphNode 视图）、系数字节码（`coef_program`、`fold_constants`、`mass_value` 调用 ir.rpn）和解释器 `execute`。
`nanoserve/ir/compiler.py`	编译器：`lower_ngram`、`validate`（ir.order_ok）、`precedence`（ir.depends_on）、`compile_ir`（死代码消除 + ir.order）、`prune`（ir.computable + ir.clone）、`Plan.critical_path`（ir.critical_path），以及 drop-in 的 `IRModel`。
`nanoserve/ir/devices.py`	多设备放置：`Topology.islands/group_count`（ir.device_groups）、`broadcast_time`（ir.op_latency）、`choose_island`、贪心放置 `place` 与带设备内串行链的延迟估计。
`nanoserve/ir/demo.py`	`python -m nanoserve compile`：打印算子表、字节码总质量、调度、剪枝、可并行对、关键路径、放置、被拒绝的环图，并核对 IR logits 与原模型逐位相等。
`nanoserve/config/loader.py`	配置格式与加载流水线：`clean_lines`（strip_comments + 哨兵）、`split_sections`（brackets）、`resolve_flags`（ir.order + flags）、`eval_number`（arith / expr 分派与契约检查）、`expand_list`（expand）、`_model_path`（norm_path）、`LoadedConfig.route`（glob）、`to_text` 规范化。

`nanoserve/config/deploy.py`	`Deployment`：每个 worker 一个 Server（[server] + [pool.kind] 覆盖），按 glob 路由分发请求，未命中者拒绝；`compiled = t` 时 engine 使用 IRModel。
`nanoserve/config/demo.py`	`python -m nanoserve config [path]`：打印解析后的配置和几个模型名的路由结果，再用 Deployment 回放一条合成 trace；`--canonical` 输出规范文本。
`nanoserve/config/examples/server.conf`	默认示例：flag 互相引用、两种算术、模板 2[gpu-a,]cpu_b、路径规范化、[pool.cpu_b] 覆盖和 4 条路由规则。minimal.conf 是最小配置，tiered.conf 演示 M3/M4 插件参数与嵌套模板 3[2[hbm,]ddr,]。
`tests/domain/test_m5_ir.py`	IR 集成测试：logits 等价、环/未知张量/坏字节码被拒、死代码消除、可并行对、关键路径上下界、克隆不改原图、按上下文剪枝、岛与广播、放置延迟介于并行与串行之间。
`tests/domain/test_m5_config.py`	配置集成测试：示例取值、flag 依赖顺序、数值分派与非法表达式、注释处处剥离、括号预检、嵌套模板、路径、first-match 路由、round-trip、配置驱动 Server/Deployment 运行。

实测数字

- toy 模型 lower 成 13 个算子（5 lookup + 5 scale + floor + sum + log），params 6 个、feeds 4 个（ctx4…ctx1）
- 系数字节码定点 100：λ_4 = 0.5 写成 [5 100 * 10 /] = 50；总质量程序求值 100 / 100 = 1
- 按上下文长度特化：len(context) = 0/1/2/3/4(+) → 5/7/9/11/13 个算子；长度 2 剪掉 4 个（dist4 term4 dist3 term3）
- 可并行算子对 50 / 78
- 延迟估计：关键路径 11（dist4 6 + term4 1 + p 2 + logits 2）vs 串行 30；默认 6 设备拓扑放置后 12
- 默认拓扑 2 个设备岛 [1,2,3,4] 与 [5,6]；各 host 广播时间 1:5 / 2:4 / 3:3 / 4:6 ms，选 host 3
- IR logits 与 NGramModel logits 在 39 个前缀上逐位相等
- server.conf：max_batch = 2*2 = 4，prefill_chunk = 64/4−8 = 8，token_budget = 4096−(512+128) = 3456，session_ttl = −(−50) = 50
- server.conf 24 个请求：w0(gpu-a) 16 个 / 97 ticks，w1(gpu-a) 5 个 / 49 ticks，w2(cpu_b, max_batch=1) 3 个 / 87 ticks
- tiered.conf：kv_blocks = 1024/8 − 2*4 = 120；pool = 3[2[hbm,]ddr,] 展开为 9 个 worker

建议的完成顺序

先做 ir.order_ok → ir.order：两题共用 Kahn 骨架，做完 compile_ir 的校验与调度就能跑，config 的 flag 解析也依赖 topo_order。接着 ir.rpn（常量折叠，compile_ir 的最后一步），此时 `python -m nanoserve compile` 的前半段已经可用。再做 ir.depends_on（precedence 与死代码消除）和 ir.computable + ir.clone（prune，IRModel 按上下文特化必需），IRModel 至此完整，logits 等价测试可通过。然后 ir.critical_path（在 Kahn 上加最长路 DP，与 ir.order 对照着写）、ir.device_groups 与 ir.op_latency（放置）。配置部分按流水线顺序：config.strip_comments → config.brackets → config.flags → config.arith / config.expr（两道计算器对照着做，体会「有优先级无括号」与「有括号无优先级」的差别）→ config.expand → config.norm_path → config.glob（最难，留到最后，它只在路由时才被调用）。

第 6 章 · M6

第 6 章　解码与评测：推测解码、离线评测、基准网格与穷举 fuzz

> M6 让引擎能用更少的串行目标模型调用生成逐 token 相同的贪心输出，并且能给自己打分、证明多路复用流没有损坏、在配置网格和所有到达顺序上回归测试。

问题：会生成之后，怎么生成得更快、怎么知道生成得对

到 M5 为止，引擎能调度、能分页缓存、能复用前缀、能读配置。但还有两个问题没有回答。第一，贪心解码每生成一个 token 就要串行地跑一次目标模型；在真实 GPU 上，单步 decode 受内存带宽限制，一次前向多算几个位置几乎是免费的，瓶颈在串行步数。第二，我们对改动缺乏系统的验证：加了一个调度特性，输出变没变？两个请求被交错写进同一条流，有没有丢字？配置解析器的预检查有没有漏网之鱼？

M6 用三个框架模块回答：`decode/speculative.py`（Medusa 式推测解码，调用 decode.beam_pairs）、`eval/harness.py`（离线评测与流完整性，调用 eval.edit_distance、eval.rouge_l、decode.regex、eval.interleave_ok）、`bench/runner.py`（基准网格、确定性检查和括号 fuzz，调用 bench.choose、bench.orders、bench.json_fuzz，以及 M5 的 config.brackets）。三个 demo 子命令 `eval --spec`、`bench` 直接给出本章所有数字。

推测解码：两个草稿头、一棵 k 叶草稿树、一次验证

`DraftHeads` 是两个只看最后一个已提交 token 的廉价预测器，类比 Medusa 头读取最后一个隐藏状态：头 1 是 bigram 表 P(x[t+1] | x[t])，头 2 是 skip-gram 表 P(x[t+2] | x[t])，都从语料计数并做 add-α 平滑，代价是量化成整数的 −log p（单位 1/1000 nat）。假设两头独立，一个两 token 猜测 (u, v) 的联合代价就是 cost1(u) + cost2(v)。于是「最可能的 k 个两步猜测」正好是「两个升序代价列表中和最小的 k 对」——这就是 decode.beam_pairs，不必给 V×V 对全部打分。

`SpeculativeDecoder.draft` 先从每个头取前 m = min(k, V) 名（前 k 对不可能用到第 k+1 名），再把每个代价编码成 cost · W + rank（W = 2m）让值互不相同、可以从值反查 token，且两个 rank 之和仍小于 W，因此按编码值排序等价于按 (代价和, rank 和) 排序。k 对组成 `DraftTree`：根、所有不同的第一个 token、所有两 token 对。

`verify` 让目标模型给树上每个节点的上下文打分（真实引擎用 tree attention 在一次前向里完成，我们的 n-gram 模型逐节点调用 `VerifiedLogits.score` 来模拟）。然后从根开始：目标模型在当前上下文的 argmax 若是树上的子节点就接受并下探，直到走出树或到达深度 2；最后再附上目标模型在最深已接受节点处的 argmax 作为 bonus token。所以每次验证提交 1 到 3 个 token。

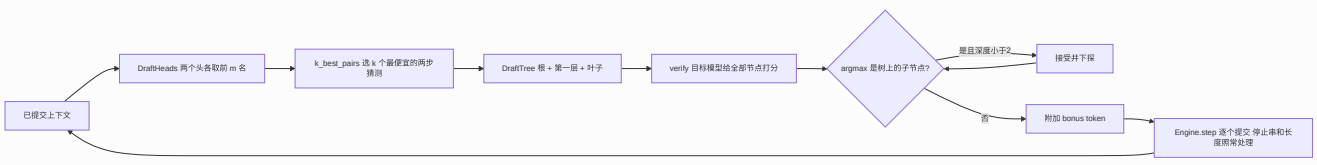


图 · 推测解码的一次迭代：草稿、验证、经 Engine.step 提交

为什么输出逐 token 相同：一个不变量加一个旁证

不变量：每个被提交的 token 都是目标模型在它那个位置、那个上下文下的 argmax。被接受的草稿 token 之所以被接受，正是因为它等于目标的 argmax；bonus token 本身就是目标的 argmax。草稿头只决定「提前给哪些上下文打分」，从不决定提交什么。因此推测解码产生的序列与 `Engine.generate(temperature=0)` 逐位相同，只是串行验证次数变少。

工程上框架没有另写一套提交逻辑：它构造一个模型为 `VerifiedLogits` 的 `Engine`，用同一个 `Engine.step` 逐个提交，`step` 从已验证的 logits 重新算出 token，若与验证得到的不同就抛 RuntimeError。停止串、max_tokens、按词流式输出和 finish_reason 因此与普通解码共用一份代码。`VerifiedLogits.misses` 统计「从未在验证中打过分的上下文」被请求的次数，测试断言它恒为 0（`unverified_commits == 0`），并在 9 个 prompt × 3 种停止配置 × k ∈ {1, 4, 16} 上比较 token_ids、text、finish_reason 全部相等。

这也解释了为什么只支持贪心：采样时要保持分布不变，需要拒绝采样式的接受规则（按 p_target / p_draft 概率接受），本仓库的 `stream` 对 temperature ≠ 0 直接抛 ValueError。

接受率与 k：树越宽，每次验证提交越多，但验证越贵

`eval --spec` 在 12 个 prompt、max_tokens = 40 上的结果：普通贪心 480 次目标调用（每次 1 个 token）；k = 1 时 360 次（1.33 tok/pass，接受率 17%）；k = 4 时 280 次（1.71，37%）；k = 8 时 242 次（1.98，51%）；k = 16 时 219 次（2.19，62%），所有配置输出都与贪心相同。接受率 = 被接受的草稿 token 数 / (2 × 验证次数)，每次验证的上限是 3 个 token（2 个草稿 + 1 个 bonus）。

代价在 scored 一列：树上要打分的位置从 480 涨到 1080、2086、3056、4952。一棵 k 叶树最多 2k + 1 个节点，真实引擎把它们放进一次前向，只要还没把算力打满，多出来的位置就几乎不增加延迟；一旦超出，宽树就开始拖慢每次验证。所以 k 不是越大越好，收益递减明显：k 从 8 翻到 16，串行次数只从 242 降到 219，打分位置却多了 62%。我们的 Python 实现没有真正的批量前向，墙钟时间反而变慢（22 ms → 60–140 ms），这里应该看串行验证次数，不看墙钟时间。

离线评测：同一批输出，三种不同的尺子

`evaluate` 把每个 `Example` 变成一个贪心 `Request`，按 spacing = 1 tick 依次到达，整批通过一个连续批处理的 `Server`（max_batch = 4）——和生产流量走同一条路。每个输出先 `normalize`（小写、去标点、压缩空白），再打三种分：字符级编辑距离（eval.edit_distance）、词级 ROUGE-L F1（eval.rouge_l，把每个不同的词映射成一个私用区字符，使字符级 LCS 在词序列上运行），以及可选的输出约束：只含 「.」 与 「*」 的正则对整段输出做完整匹配（decode.regex）。

三种尺子回答不同的问题。demo 中 sched 样例输出「milliseconds and looks at the」，比参考少一个词：编辑距离 6，ROUGE-L 0.91，正则「milliseconds .*」通过——这是一个被截断的好答案。blocks 样例输出「other」：编辑距离同样是 6，但 ROUGE-L 是 0、正则失败——这是一个错误答案。只看编辑距离，两者分不开。反过来 temp 样例编辑距离高达 39，正则「more s.*」却通过：约束只检查格式，不检查内容。12 个样例的汇总：mean edit 14.6，mean ROUGE-L F1 0.39，exact 2/12，constraints 55%（11 个带约束的样例中 6 个通过）。

流完整性：用交错 DP 证明多路复用流没有损坏

服务器把所有运行中的请求复用进一条事件流 (tick, rid, chunk)。harness 对每一对运行区间 [admitted_at, finished_at) 重叠的请求，把事件流限制到这两个 rid，得到一段原始文本（大小写、空格、换行原样保留），用 eval.interleave_ok 判断它能否拆成两组、分别按原顺序恰好构成两个请求的输出。没有和任何人重叠的请求与空串配对，等价于逐字相等。

为什么成对检查就够：任何一个请求内部的丢字、重复或乱序，都会出现在包含该请求的每一对的限制流中。demo：30 个并发对、0 个独立请求、65 个事件全部通过；`corrupt_events` 在 share 请求上分别制造 swap（两个可区分的 chunk 交换顺序）、drop（一个 chunk 丢一个字符）、dup（一个 chunk 发两次），三种破坏都被涉及 share 的 3 个检查抓到，且测试断言只有涉及受损请求的检查失败。若一次性检查 n 个并发请求，状态空间是各输出长度之积，指数级；两两检查是多项式的。

基准网格、到达顺序与生成式 fuzz：小规模穷举就是证明

`run_grid` 把 n 个特性开关展开成「恰好 k 个打开」的配置（bench.choose），在同一条 16 请求的 trace 上逐个运行，报告 makespan、p50、利用率、槽位占用、截止时间命中，以及每个请求的输出是否与 baseline 完全相同（每请求独立 seed，批处理/准入/缓存都不应改变输出）。demo 6 个开关、k ∈ {1, 2} 共 22 行，全部 same-out = yes：batch8 把 p50 从 27.5 降到 16.0，budget 与 prefill4 把 makespan 从 78 拉长到 116 和 111，prefix 降到 69，batch8+prefix 的 p50 最低（15.0），batch8+edf 命中 16/16 截止时间。

`check_arrival_orders` 让 m 个请求在同一 tick 到达、只给 2 个槽位，枚举全部到达顺序（bench.orders）重放：4 个请求 24 种顺序得到 24 种不同调度、但只有 1 种输出集合。`fuzz_brackets` 用 bench.json_fuzz 生成 n 对括号的全部合法结构，按嵌套深度着色成 ()[]{}（3 种偏移），再生成每个串的全部单字符替换/删除/插入，喂给 M5 的 `brackets_balanced`。n = 6：132 个结构、396 个合法串、52668 个变体，0 误拒 0 误收；测试还证明一个只数括号个数的弱检查器会被抓住。

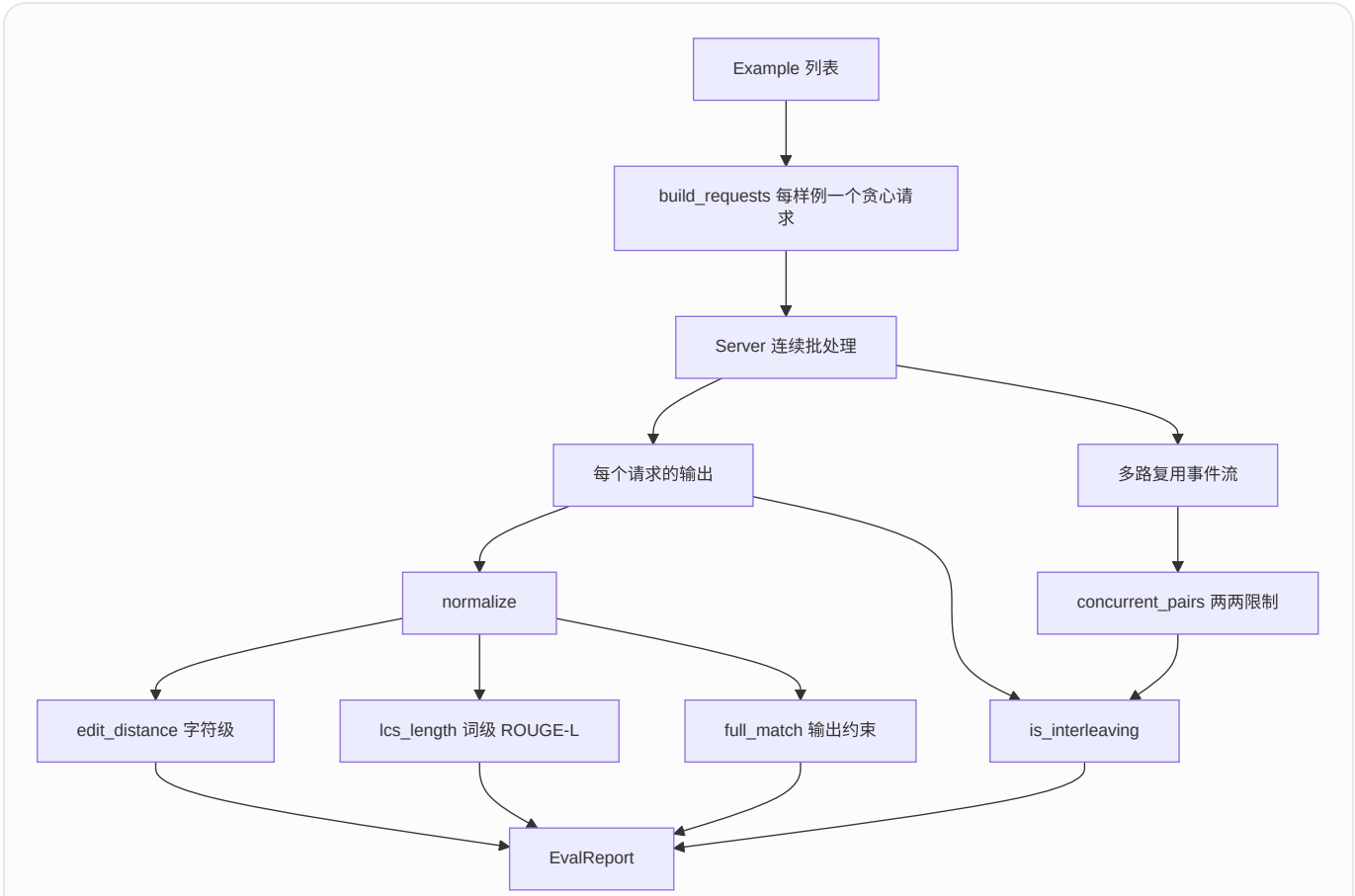


图 · 离线评测流水线：同一批输出的三种打分与流完整性检查

关键设计取舍

Medusa 式 k 叶草稿树 vs 单链草稿模型推测解码 vs 普通贪心

✓ 我们的选择

两个只看最后一个 token 的草稿头 + k_best_pairs 选出 k 个最便宜的两步猜测，组成深度 2 的树，一次验证全部节点，接受最长匹配前缀加一个 bonus token。草稿几乎零成本（查表），树覆盖了第一步猜错时的备选，输出与贪心逐位相同。

× 单链草稿模型（经典 speculative decoding）

用一个小模型串行生成 γ 个 token 的一条链，目标模型一次验证。草稿模型更准（看完整上下文），γ 可以更长，能获得更高的 tokens/pass；但草稿本身需要 γ 次小模型串行调用，且只要第一个 token 猜错整条链就作废。我们的 k = 1 正是长度 2 的单链：1.33 tok/pass，而 k = 16 的树达到 2.19。草稿模型很强、目标模型很大时单链更划算；没有好的小模型、只能挂轻量头时，树更好。

× 普通贪心

每 token 一次目标调用，480 次；无额外打分（scored = 480），实现最简单。计算受限（大 batch、算力已满）时推测解码多打的分不再免费，普通贪心反而吞吐更高。

依据 eval --spec：plain 480 passes；k=1/4/8/16 → 360/280/242/219 passes，接受率 17%/37%/51%/62%，scored 1080/2086/3056/4952，identical 全为 yes；测试 test_output_identical_to_plain_greedy 与 test_draft_tree_is_the_k_cheapest_pairs（V×V 暴力核对）。

评测指标：字符编辑距离 + 词级 ROUGE-L + 正则约束，而不是单一指标

✓ 我们的选择

三者都报。编辑距离最严格（=0 当且仅当 exact match），ROUGE-L 衡量按顺序的词级重合、容忍截断，正则只检查格式。全部在 normalize 之后计算，避免大小写和标点噪声。

× 只用 exact match

最容易解释，适合有唯一正确答案的任务（分类、算术）。对自由文本几乎总是 0（demo 只有 2/12），无法区分「差一个词」与「完全错误」。

× 只用编辑距离

对每个字符敏感，适合拼写/代码等形式严格的输出；但截断的好答案和错误的短答案可能同分（sched 与 blocks 都是 6）。

× BLEU / ROUGE-N 等 n-gram 重叠

线性时间、衡量局部流畅度，是机器翻译的惯例；不看全局顺序。LCS 天然要求顺序一致，且不需要选择 n。

依据 eval demo：sched edit 6 / ROUGE-L 0.91 / ok；blocks edit 6 / 0.00 / FAIL；temp edit 39 / 0.18 / ok。测试：rouge_l(「queue model」，「model queue」) 的 F1 = 0.5。

流完整性：成对交错检查 vs 按 rid 重组

✓ 我们的选择

对每一对并发请求，把事件流限制到两者后检查原始文本是否为两路输出的交错（O(|a|·|b|) DP），独立请求与空串配对。它验证的是「同一连接上的混合文本」这一客户端视角，并且不依赖 chunk 边界。

× 按 rid 分组拼接后与输出逐字比较

O(总长度)，更便宜也更强：rid 标签可信时还能发现两请求之间的错标（交错检查对这对请求之间互换字符是看不见的）。若事件携带可靠的 rid，生产监控应该优先用它；交错 DP 的用武之地是标签丢失或不可信、只剩混合字节流的时候。

× 对全部并发请求做一次多维交错 DP

一次检查就覆盖所有请求，但状态数是各输出长度之积，指数级；成对检查已能定位任何单请求内部的损坏。

依据 eval demo：30 个并发对 + 0 个独立请求、65 个事件全部通过；swap/drop/dup 三种破坏各被涉及 share 的 3 个检查抓到；test_corrupted_stream_is_detected 断言只有涉及受损请求的检查失败。

穷举网格与全部到达顺序 vs 随机抽样配置

✓ 我们的选择

规模小时全部枚举：k ∈ {1,2} 的全部开关组合（bench.choose），m ⩽ 6 个同 tick 请求的全部 m! 种顺序（bench.orders）。每一种都断言输出与 baseline 相同，所以这是对该规模的证明，而不是统计信心。

× 随机抽样配置 / 随机打乱到达顺序

开关多（几十个）或请求多时唯一可行的办法，配合固定 seed 可复现，也可用 pairwise 覆盖设计压缩组合数。代价是只能发现高概率的问题；「对所有顺序输出相同」这类全称命题，抽样无法证明。

× 位掩码枚举全部 2^n 个子集

无递归，一个整数就是一个配置；但 n = 20 时有约 100 万个子集，而 k ⩽ 2 只需要 1 + 20 + 190 个。

依据 bench demo：22 个配置 same-out 全为 yes；24 种到达顺序 → 24 种调度、1 种输出集合；test_grid_is_k_of_n_combinations 断言组合数为 1 + n + C(n,2)，且改写同一字段的开关不会被组合。

生成式 fuzz（全部合法括号 + 单字符变异）vs 随机 fuzz

✓ 我们的选择

枚举 n 对括号的全部合法结构（Catalan 数个），按深度着色成三种括号，再生成全部单字符替换/删除/插入。合法串必须接受，变异必须拒绝——单字符变异必然破坏平衡，所以期望答案已知，不需要第二个检查器当 oracle。

× 随机 fuzz

实现简单、可无限运行、能发现意料之外的输入（例如超长、奇怪字符）。但随机长 12 的六字符串中合法的比例约 132 · 3^6 / 6^12 ≈ 4.4×10^-5，几乎从不触发「接受」路径，也很难恰好命中「只差一个字符」的边界情况；还需要一个可信的 oracle 判断对错。

× 手写若干单元测试

可读性最好、定位快，但只能覆盖作者想到的情况；一个只数括号个数的检查器很可能通过手写测试。

依据 bench demo n=6：132 个结构 → 396 个合法串、52668 个变体，valid rejected 0、mutants accepted 0；test_config_precheck_fuzz 验证 n=1..5 的结构数为 Catalan 数，并证明只数括号个数的弱检查器被抓到。

框架文件导读

`nanoserve/decode/speculative.py`	Medusa 式推测解码：DraftHeads（bigram 与 skip-gram 代价表）、DraftTree、VerifiedLogits（验证时缓存 logits、统计 misses）、SpeculativeDecoder（draft 调用 decode.beam_pairs，verify 接受最长前缀加 bonus，经 Engine.step 提交）。
`nanoserve/eval/harness.py`	离线评测：evaluate 通过 Server 生成，score_example 调用 eval.edit_distance、eval.rouge_l、decode.regex；check_streams 对并发对调用 eval.interleave_ok；corrupt_events 制造 swap/drop/dup 破坏。
`nanoserve/eval/demo.py`	python -m nanoserve eval：打印评测表、流完整性与三种破坏的检测结果；--spec 追加推测解码与普通贪心的对比（k 默认 1,4,8,16）。
`nanoserve/bench/runner.py`	grid/run_grid 用 bench.choose 展开开关组合并比较输出；check_arrival_orders 用 bench.orders 重放全部到达顺序；fuzz_brackets 用 bench.json_fuzz 生成结构、typed 着色、mutations 变异，测试 M5 的 brackets_balanced。
`nanoserve/bench/demo.py`	python -m nanoserve bench：打印基准网格表、确定性检查与括号 fuzz 结果，任一检查失败返回 1。
`tests/domain/test_m6_speculative.py`	推测解码与贪心逐 token 相同、unverified_commits == 0、串行次数少于 token 数、k=16 少于 k=1、草稿树等于 V×V 暴力的前 k 小。
`tests/domain/test_m6_eval.py`	评测指标的已知样例、输出约束、多路复用流是交错、三种破坏都被抓到且只影响涉及受损请求的检查。
`tests/domain/test_m6_bench.py`	网格大小为 1 + n + C(n,2)、输出与 baseline 相同、24 种到达顺序输出唯一、Catalan 数个结构与弱检查器被抓到。

实测数字

- 推测解码（12 个 prompt，max_tokens 40）：目标调用 480 → 360 / 280 / 242 / 219（k = 1 / 4 / 8 / 16），tok/pass 1.33 / 1.71 / 1.98 / 2.19，输出全部与贪心相同
- 接受率 17% / 37% / 51% / 62%；打分位置 1080 / 2086 / 3056 / 4952（普通贪心 480）
- 评测 12 个样例经 Server（68 tick，p50 29）：mean edit 14.6，mean ROUGE-L F1 0.39，exact 2/12，约束通过 55%
- 流完整性：30 个并发对、65 个事件全部是交错；swap / drop / dup 三种破坏各被 3 个检查抓到
- 基准网格 22 个配置全部 same-out：baseline p50 27.5 → batch8 16.0 → batch8+prefix 15.0；budget makespan 78 → 116
- 确定性：4 个同 tick 请求 24 种顺序 → 24 种调度、1 种输出集合
- 括号 fuzz n = 6：132 个结构 → 396 个合法串、52668 个变体，0 误拒 0 误收

建议的完成顺序

先做评测的 DP 阶梯：eval.edit_distance（最基础的双序列 DP）→ eval.rouge_l（同一张表，换成相似度并练一维滚动）→ eval.interleave_ok（第三个串的下标由 i+j 隐含）→ decode.regex（最难，后缀 DP 加 「*」 的两种转移）；四题完成后 `eval` 子命令就能完整运行。然后做 decode.beam_pairs（堆，多路归并），解锁 `eval --spec`。最后是回溯三连：bench.choose（组合，有序化去重 + 剪枝）→ bench.orders（排列，交换法）→ bench.json_fuzz（带计数约束的生成），三者完成后 `bench` 子命令可运行（它还需要 M5 的 config.brackets）。

第 7 章 · M7

第 7 章　可观测性与自动扩缩容：文本面板、健康感知路由与闭环容量规划

> M7 让引擎能看见自己（占用热力图、最慢 k 个、可修正的吞吐序列、集群利用率、健康矩阵），并能回答「至少需要多少资源」，再在真实集群上验证这个答案恰好够用。

问题：从「能跑」到「能运维」

前六个里程碑之后，服务器和集群已经能处理真实的 trace，但运维还需要两类能力。第一是可观测性：哪些槽位在什么时候被占用？哪些请求最慢、门槛是多少？吞吐的峰值和谷值是多少——而且当服务器抢占请求、丢弃它已经流出的输出时，历史数据会被事后修正？三个 worker 的忙碌时段重叠时，集群实际有多少时间在干活？某个模型副本坏了，应该屏蔽哪些资源？第二是容量规划：一波突发流量需要几个 worker？多 LoRA 每次 launch 要多大的 batch？有序批任务每波要多大的 token 预算？

M7 的框架分两块：`dashboard/panel.py` 与 `dashboard/health_router.py`（调用 dash.region_sum、metrics.slowest_k、dash.fuzzy、metrics.corrected、metrics.busy_time、dash.mask_failed），以及 `autoscale/autoscaler.py` 与 `autoscale/planned.py`（调用 autoscale.min_time、autoscale.min_rate、autoscale.min_throughput）。这一章的共同主题是：面板数据要么冻结、要么流式、要么可被修正，每种形态决定了一种数据结构；而容量问题都是单调的，答案用二分找、用真实运行验证。

占用热力图：冻结的矩阵，频繁的矩形查询

`Occupancy` 把一次运行画成 0/1 矩阵：每行一个批处理槽位（集群时是 (worker, slot)），每列一个 tick，请求占用期间 admitted_at ⩽ t < finished_at 为 1。这张矩阵在一次刷新内不变，却被查询很多次：`cells` 把它切成宽度为 width 的粗格子（demo：4 槽 × 119 tick，每列 3 tick；集群按 worker 分组，12 槽 × 66 tick，每列 2 tick），`heatmap` 还要算每行的总占用率，`worker_busy_slots` 查某个 worker 在某个时段的忙碌槽数，运维框选也是矩形查询。

于是构造时一次性建二维前缀和（dash.region_sum），之后每个矩形 4 次查表。`Occupancy.region` 负责把越界矩形裁剪到矩阵内，`RegionSum` 只处理合法输入。格子的明暗用 SHADES 字符按「已用槽 tick / 容量」分 10 档画出。测试用 300 个随机矩形与暴力求和对拍，并检查粗格子恰好铺满矩阵、某 worker 某 tick 非零当且仅当它在忙。

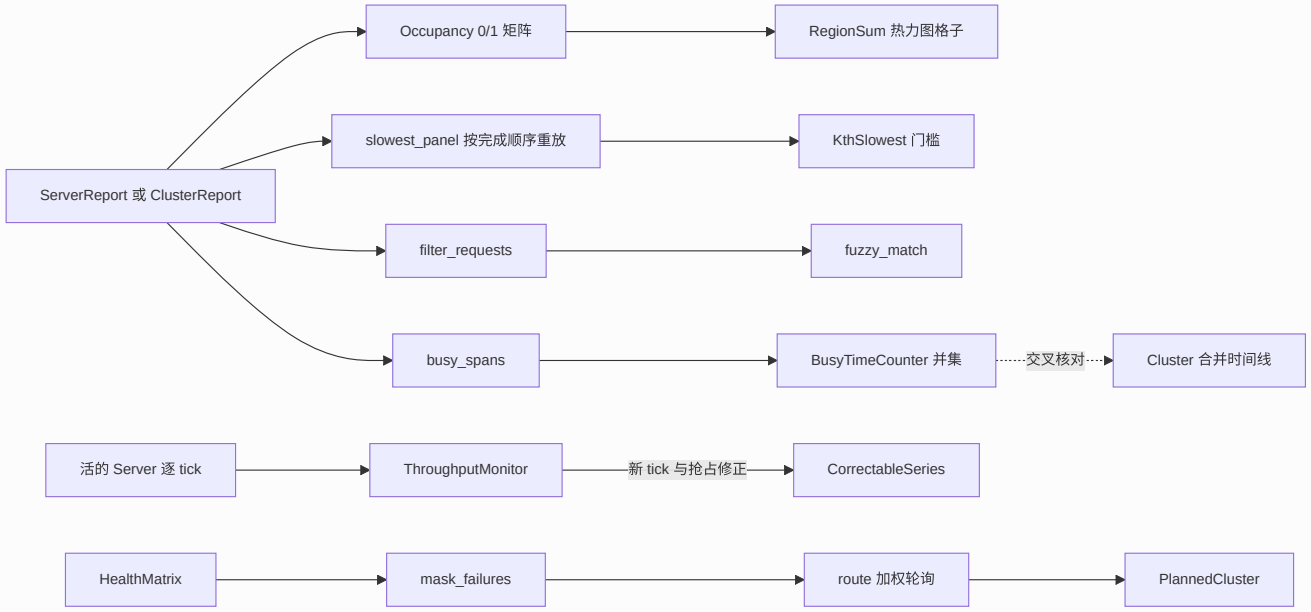


图 · M7 面板的数据来源与对应练习

流式面板：最慢 k 个与模糊过滤

`slowest_panel` 按完成顺序 (finished_at, slot, rid) 重放请求，用 metrics.slowest_k 在线维护第 k 慢的延迟——新请求进入「最慢 k 个」面板的门槛。前置条件要求每次 add 之后至少有 k 个值，所以框架用前 k−1 个延迟做初始值，从第 k 个开始每完成一个就 add 一次。demo：单服务器 k = 5，门槛在 20 次完成中从 12 升到 76 tick；集群 3 个 worker 时从 10 升到 29，最慢的请求延迟从 99 降到 46。测试在每一步都与「把已完成的延迟排序取第 k 个」对拍。

请求列表的过滤是 fzf 式的：`row_text` 把 rid、租户、模型名、prompt 拼成一行，`filter_requests` 用 dash.fuzzy 判断查询串是否是它的子序列。面板每输入一个字符就要过滤全部请求，线性双指针足够快；测试用正则「.*」.join 作独立 oracle 对拍 70 多个查询。

可修正的吞吐序列与集群利用率

`ThroughputMonitor` 逐 tick 推进一个活的 `Server`，每个 tick 后从事件流重算各 tick 输出的字符数。新 tick 上报一次；但服务器在 KV 压力下会抢占请求并丢弃它已流出的事件（之后重算），于是更早 tick 的总数事后变小，monitor 把变化的 tick 作为修正重新上报。metrics.corrected 用哈希表保存每个 tick 的当前值、用两个懒删除堆回答峰值与谷值，current 是最大 tick 的值。demo（20 个 KV 块、block_size 4）：24 个请求 251 tick、18 次抢占、18 次迟到修正，例如「在 tick 5 把 tick 3 修正为 1 个字符」，最终 current 10、peak 14、min 0。测试逐 tick 断言 (current, max, min) 等于对当前上报值的暴力计算。

集群利用率问的是「至少一个 worker 在计算」的 tick 数。各 worker 的忙碌区间逐个 worker 到达、彼此重叠，metrics.busy_time 维护有序不相交区间与总长度，count 为 O(1)。它与 `Cluster` 用 sched.merge_busy 合并出的时间线交叉核对：demo 中 3 个 worker 分别忙 35、44、65 tick，各自之和 144，并集 66/66，与合并时间线一致（ok）。

健康矩阵：故障的爆炸半径与健康感知路由

`HealthMatrix` 的格子 (w, m) 是模型 m 在 worker w 上副本的路由权重，0 表示故障。一个故障同时让我们不再信任整个 worker（共享 GPU、驱动、主机）和整个模型（坏 checkpoint 或配置发布），所以 `masked()` 在矩阵的深拷贝

上调用 dash.mask_failed，把故障所在行列整体置 0；屏蔽只依据故障快照，不能级联，否则一个故障就会 drain 整个集群。全零的行是被 drain 的 worker，全零的列是被暂停的模型。

`route` 只用屏蔽后的格子：模型 m 的请求按权重在其仍为正的 worker 之间加权轮询，被暂停模型的请求得到 None，由 `PlannedCluster` 在前端直接拒绝（相当于 503）。demo：fail(2, toy-code) → drained [2]、paused [toy-code]，健康感知运行 14 个完成、10 个被拒、drain 掉的 worker 上 0 个请求。测试还覆盖第 0 行第 0 列这个角落——O(1) 空间算法恰好拿它们当标记。

容量规划：三个单调问题，一个二分骨架

`Autoscaler` 回答三个「至少需要多少」的问题，服务时长模型是 ⌈prompt_tokens / prefill_chunk⌉ + max_tokens tick（这些 trace 没有停止串和前缀缓存，所以是精确的）。

突发流量的 worker 池（autoscale.min_time）：30 个相同请求在 tick 0 到达异构单流 worker 池，gpu-a / gpu-b / cpu(4) / cpu(2) 每个请求分别需要 5 / 6 / 8 / 11 tick。按从快到慢加入 worker，每加一个就二分一次最小完成时间 T（满足 Σ⌊T/t_i⌋ ⩾ 30 的最小 T），直到 T ⩽ 目标 40：推荐 5 个 worker，T = 36。多 LoRA 的 batch 大小（autoscale.min_rate）：一次 launch 只能服务一个租户的 adapter，积压 {7, 3, 11, 5}、40 tick 内、每次 launch 5 tick → 8 次 launch，最小 max_batch 是满足 Σ⌈q_i/b⌉ ⩽ 8 的 b = 4。有序批任务的每波预算（autoscale.min_throughput）：12 个请求最多 4 波，每波一段连续请求，`token_budget` 限制一波的 Σ(prompt + max_tokens)，最小预算 C = 108，波次 [3, 3, 2, 4]。

三题的可行性函数不同：前两个是各单元独立的求和（向下取整 / 向上取整），第三个任务有序、必须连续分段，只能用贪心打包模拟。但它们都关于资源单调，答案范围巨大（规格中可达 10^9 到 10^14），所以都是「二分答案 + O(n) 可行性检查」。

闭环验证：估计值够用，少一个单位就不够

容量规划最怕的是模型与系统脱节：公式说 36 tick，真实调度器跑出 41。M7 的每个推荐都在真实的 M2 服务器上运行两次。worker 池：`plan_burst` 让 worker i 承担 ⌊T/t_i⌋ 个请求，在 `PlannedCluster`（按预先计算的路由表分发、每个 worker 可以有自己的 ServerConfig）上运行，5 个 worker 实测正好 36 tick（每 worker 7, 7, 6, 6, 4 个），4 个 worker 估计与实测都是 42，未达标；起始的 1 个 worker 需要 150 tick。多 LoRA：`AdapterBatchQueue` 是一个只允许单租户 batch 的等待队列，batch 运行期间对服务器报告长度 0 以免混装 adapter；max_batch = 4 时 8 次 launch、40 tick 清空，3 时需要 50 tick。每波预算：第 k 波在 k · L tick 放行（L = 12，最长请求的服务时长），预算 108 时没有任何请求在其波次放行后等待，107 时有 1 个请求卡住。

测试把「刚好够」做成断言：估计值等于实测值；少一个单位一定不达标；并用暴力（逐个 T 计数、枚举 worker 子集、逐个 batch 大小、逐个预算）证明没有更小的答案。demo 最后打印 all estimates validated: True。

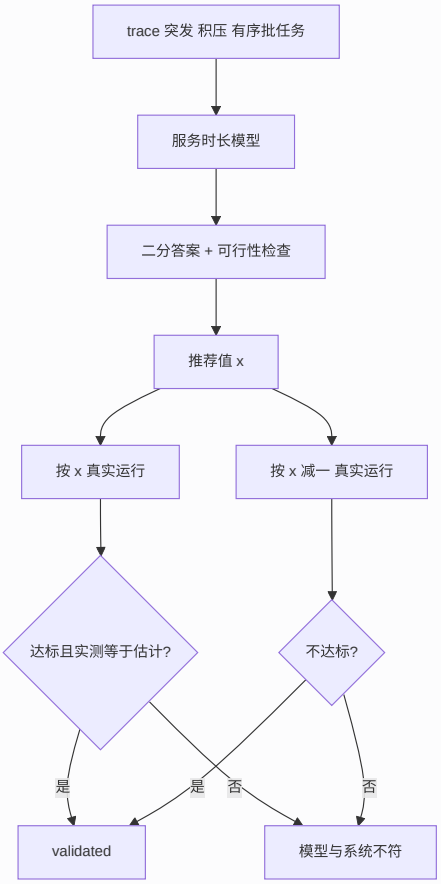


图 · 容量规划的闭环：二分得到推荐值，再在真实集群上验证

关键设计取舍

热力图格子：二维前缀和 vs 每次重新累加

✓ 我们的选择

Occupancy 构造时一次性建 RegionSum（O(槽数 × tick)），之后每个矩形查询 O(1)：粗格子、每行占用率、每个 worker 每个时段的忙碌槽数、运维框选都走同一个接口。

× 每次查询直接累加矩形

无预处理、零额外内存。若只画一次热力图，粗格子恰好铺满矩阵，总代价也是 O(槽数 × tick)，与前缀和持平；查询重叠且频繁时（逐 tick 的 worker_busy_slots、反复框选）就会重复扫描同一片格子。

× 二维 Fenwick 树

支持单点更新 O(log m · log n)，适合实时面板每个 tick 增量修改矩阵；我们的矩阵在一次刷新内冻结，静态前缀和查询更快、实现更短。

依据 test_heatmap_region_sums_match_brute_force：300 个随机矩形与暴力求和一致；粗格子之和等于全部占用；demo 4 槽 × 119 tick 与 12 槽 × 66 tick 两张热力图。

最慢 k 个的门槛：大小为 k 的最小堆 vs 每次排序

✓ 我们的选择

KthSlowest 只保留目前最大的 k 个延迟，堆顶就是门槛，每次 add O(log k)、内存 O(k)，与请求总数无关。

× 保存全部延迟，每次排序取第 k 个

实现直观，可以随时换 k；但请求流无界，每完成一个请求 O(n log n)。

× SortedList 保存全部延迟

每次 O(log n)，能回答任意 k 和任意分位数、也支持删除；需要保存全部历史和第三方库。面板若要支持切换 k 或撤回请求，它更合适。

依据 test_kth_slowest_matches_sorted_latencies_at_every_step 在 k = 1, 3, 5 时逐步与排序结果对拍；demo 门槛 12 → 76（单服务器）、10 → 29（集群）。

可修正的吞吐序列：懒删除双堆 vs 每次重算

✓ 我们的选择

哈希表 ts → 当前值保存真相，最大/最小堆只追加，查询时弹掉与哈希表不一致的过期堆顶。抢占导致的修正只是普通的 update，均摊 O(log n)。

× 每次查询对全部值求 max / min

最简单，数据点很少时最快；面板每个 tick 都查，序列越长越慢。

× SortedList / 平衡树保存值的多重集合，修正时删旧插新

严格 O(log n)，内存只与不同 ts 数有关，不会积累过期条目；同一 ts 被频繁修正时更好。代价是依赖和常数。

× 单调队列（metrics.window_max）

只适用于按到达顺序删除的滑动窗口；修正可以作用于任意历史 tick，不适用。

依据 dashboard demo：18 次抢占产生 18 次迟到修正，最终 current 10、peak 14、min 0；test_throughput_series_takes_late_corrections_from_preemption 逐 tick 与暴力计算一致，且所有修正都作用于更早的 tick。

集群忙碌时间：有序不相交区间计数器 vs 时刻集合

✓ 我们的选择

BusyTimeCounter 用两张平行的有序数组保存不相交区间，二分定位重叠段并合并，维护总长度，count O(1)；结果再与 Cluster 的合并时间线交叉核对。

× 把每个忙碌 tick 放进 set

测试里当 oracle 很方便（tick 只有几十个）；时间戳可达 10^9 时内存和插入都不可行。

× 每次查询排序合并全部区间（sched.merge_busy）

离线只查一次时最简单，Cluster 正是这样算合并时间线；在线反复查询时每次 O(n log n)。两者并存恰好构成交叉核对。

× 动态开点线段树

还支持删除区间、区间内查询；节点多、常数大，只增不删时没必要。

依据 dashboard demo：per worker [35, 44, 65]，并集 66/66，merged timeline 66，ok；test_cluster_busy_time_is_the_union_of_worker_spans 断言并集 < 各 worker 之和、< 总 tick 数。

容量问题：二分答案 + 贪心/求和可行性检查 vs 公式或线性搜索

✓ 我们的选择

三个 sizing 函数都对资源量二分，可行性检查 O(n)：Σ⌊T/t_i⌋ ⩾ n、Σ⌈q_i/b⌉ ⩽ launches、顺序贪心打包的波数 ⩽ D。复杂度只随答案范围对数增长。

× 连续近似公式

O(n) 一步给出，适合粗估和设定搜索下界；但忽略了取整和不可拆分。T ≈ 30 / (2/5 + 2/6 + 1/8) ≈ 35.0，真实 36；⌈Σq/H⌉ 在 [30,11,23,4,20]、H = 5 时给出 18，真实 30。

× 线性搜索

demo 的规模下（几十 tick、几个 batch）也能跑，代码直观；规格中的答案可达 10^9–10^14，线性不可行。

× DP（例如顺序分段的 dp[i][d]）

可处理非求和型代价，但 $O(n^2 \cdot D)$；只要可行性能贪心判断，二分答案更好。

依据 autoscale demo：推荐 5 个 worker 估计 36；max_batch 4；token budget 108。test_m7_autoscale 用暴力搜索确认这三个都是最小值。

闭环验证：在真实集群上运行推荐值与少一个单位 vs 只信解析模型

✓ 我们的选择

每个推荐都用 PlannedCluster / AdapterBatchQueue / 波次放行在 M2 服务器上运行：推荐值必须达标且估计值等于实测值，少一个单位必须不达标。demo 的退出码取决于全部验证是否通过。

× 只信公式 / 二分结果

快、不需要模拟器；但服务时长模型一旦与调度器行为不符（停止串、前缀缓存、准入策略），推荐就会悄悄失效。闭环验证把模型假设变成可执行的断言。

× 在线反馈式自动扩缩容（按队列长度或延迟逐步加减 worker）

不需要准确的服务模型，能适应负载变化，是生产里的常见做法；但会滞后、可能振荡，也无法事先保证「刚好够」。离线规划适合已知的突发和批任务，两者常常结合使用。

依据 autoscale demo：5 个 worker 实测 36 tick（4 个为 42，起始 1 个为 150）；max_batch 4 → 40 tick，3 → 50 tick；预算 108 → 0 个等待，107 → 1 个卡住；all estimates validated: True。

框架文件导读

`nanoserve/dashboard/panel.py`	文本面板：Occupancy（RegionSum 热力图）、slowest_panel（KthSlowest）、filter_requests（fuzzy_match）、ThroughputMonitor（CorrectableSeries 吸收抢占修正）、cluster_utilization（BusyTimeCounter 与合并时间线交叉核对）、Dashboard.render。
`nanoserve/dashboard/health_router.py`	HealthMatrix：fail 注入故障，masked 在深拷贝上调用 dash.mask_failed，drained_workers / paused_models，route 按屏蔽后权重加权轮询。
`nanoserve/dashboard/demo.py`	python -m nanoserve dashboard：单服务器面板、KV 压力下的吞吐修正面板；--workers > 1 时追加集群面板、健康矩阵与健康感知运行。
`nanoserve/autoscale/autoscaler.py`	Autoscaler：size_pool（min_time，按快到慢加 worker）、size_batch（min_rate，配 AdapterBatchQueue 单租户 batch）、size_waves（min_throughput，pack_waves 贪心分波），每个都运行推荐值与少一个单位做验证。
`nanoserve/autoscale/planned.py`	PlannedCluster：按预先计算的路由表分发请求（None 即前端拒绝），每个 worker 可有独立 ServerConfig；被自动扩缩容和健康感知路由共用。
`nanoserve/autoscale/demo.py`	python -m nanoserve autoscale：打印三个推荐及其验证运行，全部通过时输出 all estimates validated: True，否则退出码 1。
`tests/domain/test_m7_dashboard.py`	热力图矩形对拍、逐步第 k 慢对拍、模糊过滤与正则 oracle 对拍、抢占修正逐 tick 对拍、并集与合并时间线一致、故障爆炸半径与路由避让（含角落故障）。
`tests/domain/test_m7_autoscale.py`	推荐值实测达标且等于估计、少一个单位不达标；用逐个 T、枚举 worker 子集、逐个 batch 大小、逐个预算的暴力证明最小性；不可行的 horizon 报错。

实测数字

- 单服务器面板：24 个请求 119 tick、p50 53.5；热力图 4 槽 × 119 tick；最慢 5 个的门槛 12 → 76 tick

- KV 压力（20 个块）：251 tick、18 次抢占、18 次迟到修正；吞吐 current 10、peak 14、min 0 字符/tick
- 集群 3 个 worker：66 tick、p50 22.5；各 worker 忙 35 / 44 / 65 tick，并集 66/66，与合并时间线一致
- 健康矩阵 fail(2, toy-code)：drained [2]、paused [toy-code]；14 个完成、10 个被拒、drain 掉的 worker 上 0 个请求
- 突发 30 个请求、目标 40：1 个 worker 150 tick → 推荐 5 个（估计 36，实测 36）；4 个为 42
- 多 LoRA 积压 {7,3,11,5}、40 tick（8 次 launch）：max_batch 4 → 40 tick；3 → 50 tick
- 有序批任务 12 个请求、4 波：预算 108（波次 [3,3,2,4]）0 个等待；107 时 1 个卡住

建议的完成顺序

先做三个独立的面板小题热身：dash.fuzzy（双指针）→ dash.region_sum（二维前缀和）→ dash.mask_failed（原地标记，先写集合版再改 O(1)）。然后是三个在线数据结构，难度递增：metrics.slowest_k（单堆）→ metrics.corrected（哈希表 + 懒删除双堆）→ metrics.busy_time（有序区间 + 二分 + 合并），做完这六题 `dashboard` 子命令就完整了。最后是二分答案三连，建议按可行性函数从简单到复杂：autoscale.min_time（Σ⌊T/t⌋）→ autoscale.min_rate（Σ⌈q/r⌉，注意取整与比较方向相反）→ autoscale.min_throughput（需要贪心模拟、下界必须是 max(work)），三题完成后运行 `autoscale` 看闭环验证全部通过。

第二部分

117 道题逐题讲解

按里程碑排列。每道题都是 nano-serve 里一个真实被调用的函数或类。

M1　单请求生成

M1

分词、采样、流式输出：单请求生成

12 题 · ★ Top 150 9 · 其它 3

	练习	LC	LeetCode 题目	模式
★	`tok.segment`	139	Word Break	一维 DP
★	`tok.common_prefix`	14	Longest Common Prefix	双指针
★	`stream.find_stop`	28	Find the Index of the First Occurrence in a String	字符串匹配（KMP）
★	`stream.justify`	68	Text Justification	贪心
★	`stream.holdback`	58	Length of Last Word	双指针
★	`stream.unique_run`	3	Longest Substring Without Repeating Characters	滑动窗口
★	`stream.repeat_window`	219	Contains Duplicate II	滑动窗口
★	`sample.cdf_index`	35	Search Insert Position	二分查找
★	`sample.kth_largest`	215	Kth Largest Element in an Array	堆
	`tok.top_pieces`	347	Top K Frequent Elements	哈希表
	`stream.multi_stop`	1032	Stream of Characters	Trie
	`stream.stop_overlap`	1392	Longest Happy Prefix	字符串匹配（KMP）

`tok.segment`

`tok.segment` `can_segment`

★ Top Interview 150　LC 139 · Word Break　一维 DP

`nanoserve/tokenizer/segment.py` · `can_segment(text: str, vocab: list[str]) -> bool`

引擎里的角色

`nanoserve/tokenizer/tokenizer.py::Tokenizer.encode` 在贪心最长匹配之前先调用 `can_segment(text, self.vocab)`，返回 False 就抛 `UnknownCharacterError`（CLI `__main__.py::generate` 接住后 `sanitize`）。encode 会在 `NGramModel.__init__` 编码整段语料（2169 字符）、每个请求的 `Engine.start`、M2 `server.py::_admit` 中被调用，所以必须是多项式且便宜。

被调用于：`nanoserve/tokenizer/tokenizer.py`

题目

给定 text 和词表 vocab（互不相同的非空字符串，可含空格标点），判断 text 能否被切成若干词表 token 首尾相接（token 可重复使用、可不用），恰好覆盖整个 text。空串返回 True。len(text) 至多数千、词表至多上千，要求多项式时间。

核心洞见

ok[i] 表示前缀 text[:i] 可切分；ok[i] 为真当且仅当存在 token t 使 ok[i−|t|] 为真且 text 在 i 处以 t 结尾。只需枚举词表中出现过的不同长度 L（本仓库仅 8 种：1、4–10），配合哈希集合查子串。

算法

1. words = set(vocab)，lengths = 词表中不同长度升序；ok = [False]*(n+1)，ok[0] = True。
2. 对 i = 1..n：遍历 L in lengths，L > i 时 break（升序保证后面更长）。
3. 若 ok[i−L] 且 text[i−L:i] in words，则 ok[i] = True 并 break（已可切分，无需再试）。
4. 返回 ok[n]。
5. 例：text="catsandog"，vocab={cats,dog,sand,and,cat}：ok[3]（cat）、ok[4]（cats）、ok[7]（cat+sand / cats+and）为真，但以 9 结尾只能是 dog（需 ok[6]）或 og 等，ok[6] 为假 → False。

时间 O(n · D · Lmax)：D 为不同 token 长度数，切片与哈希各 O(L)　**空间** O(n + Σ|t|)（dp 数组 + 词表集合）

参考实现

```python
def can_segment(text: str, vocab: list[str]) -> bool:
    words = set(vocab)
    lengths = sorted({len(w) for w in words})
    n = len(text)
    ok = [False] * (n + 1)
    ok[0] = True
    for i in range(1, n + 1):
        for L in lengths:
            if L > i:
                break
            if ok[i - L] and text[i - L : i] in words:
                ok[i] = True
                break
    return ok[n]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
直接贪心最长匹配（encode 本身）	O(n · Lmax)	一般词表上会错判：{ab, abc, cd} 切 "abcd"，贪心先吞 abc 剩 d 卡死，DP 却能找到 ab+cd。本仓库所有语料字符都是单字符 token，所以「可切分」时贪心一定成功，校验实际上等价于「所有字符都在词表里」；若词表去掉了单字符兜底，就该用 DP 记录回溯指针直接求切分。
朴素递归 / 回溯枚举切法	最坏指数级	"aaaa⋯ab" 配词表 {a, aa, aaa} 时分支爆炸；加上按起点记忆化就退化为同一个 DP，因此直接写自底向上 DP 更清楚。
Trie + 前向 DP	O(n · Lmax)	从每个 ok 位置沿 Trie 前进，省掉切片与哈希；词表大、长度种类多（真实分词器几万 token）时更快。本词表长度只有 8 种，集合查询已够。可与 `prefix.trie` 对照。
按切点 j 枚举（ok[j] 且 text[j:i] in words）	$O(n^2 \cdot L)$	LeetCode 常见写法，但 j 要遍历全部 i 个切点；按 token 长度枚举把内层循环从 i 降到 D，当 token 都很短时差别明显。

易错点

- ok[0] 必须为 True（空前缀可切分），否则所有位置都推不出来；空 text 直接返回 True。
- token 可以含空格（如 ' the'），不能先按空白 split 再查词。
- 找到一种切法就 break，否则白做后续长度；lengths 升序后遇到 L > i 也要 break 而不是 continue。

模式信号

「能否拆成若干个可重复使用的词」+「多项式时间，不能枚举切法」→ 前缀可达性 DP。

相关练习： `tok.top_pieces` `prefix.trie` `decode.regex` `config.glob`

---

tok.common_prefix

tok.common_prefix common_prefix

★ Top Interview 150 | LC 14 · Longest Common Prefix | 双指针

`nanoserve/tokenizer/prefix.py · common_prefix(prompts: list[str]) -> str`

引擎里的角色

`nanoserve/engine.py::Engine.shared_prefix` 直接调用它，CLI 给多个 prompt 时打印「N prompts share a prefix of k chars」（"The scheduler wakes"/"The scheduler sleeps" → 'The scheduler '，14 字符）。M4 的 `nanoserve/prefix/cache.py::PrefixCache.shared_prefix` 把缓存的 prompt 排序后，对每个长度为 m 的连续窗口调用它，找出至少 m 个请求共享的 system prompt。

被调用于： `nanoserve/engine.py` ， `nanoserve/prefix/cache.py`

题目

给定字符串列表 prompts（可含空串、任意字符），返回所有字符串共同的最长前缀（逐字符、区分大小写）。空列表或无公共前缀返回 ""；只有一个元素返回它本身。

核心洞见

答案不会长于最短的字符串。以最短串为基准纵向扫描：第 i 列只要有一个串不同，答案就是 shortest[:i]。

算法

1. prompts 为空返回 ""；shortest = min(prompts, key=len)。
2. 对 i, ch 枚举 shortest 的每个字符：检查所有 p 的 p[i] 是否等于 ch。
3. 第一次不等时返回 shortest[:i]。
4. 全部列都相同则返回 shortest。
5. 例：["flower","flow","flight"]，shortest="flow"，列 0 'f'、列 1 'l' 全同，列 2 'o' vs "flight" 的 'i' 不同 → "fl"。

时间 O(S)，S ≤ n · min_len（最坏比较到最短串末尾的全部列）　空间 O(1) 额外空间（返回切片 O(答案长度)）

参考实现

```python
def common_prefix(prompts: list[str]) -> str:
    if not prompts:
        return ""
    shortest = min(prompts, key=len)
    for i, ch in enumerate(shortest):
        for p in prompts:
            if p[i] != ch:
```

```
                return shortest[:i]
    return shortest
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
排序后只比较首尾两个串	O(n · L · log n) 排序 + O(L)	字典序下首尾差异最大，二者的公共前缀就是全体的；单次调用比纵向扫描慢。但 M4 的 `PrefixCache.shared_prefix` 恰好需要排序：它找的是「至少 m 个」共享的前缀，排序后共享前缀的串相邻，只需在大小为 m 的滑动窗口上调用本函数。
横向两两合并：prefix = lcp(prefix, p)	O(S)	复杂度相同，但会先把第一个串当作候选，若最短串在最后才出现，前面会做很多无效比较；纵向扫描能在第一处分歧立刻停止。
建 Trie（`prefix.trie`）沿单分支走	O(总字符数) 时间与空间	一次性查询不值得建树；但 prompt 持续到达、需要反复查询「谁和谁共享多少」时，Trie/基数树正是 M4 前缀缓存的做法。
二分答案长度 + 校验	O(S · log L)	在分布式或只能整块比较（如哈希比较前缀）的场景有用；内存里逐字符比较时纯属多此一举。

易错点

- 空列表要先判，否则 min() 抛 ValueError。
- 以最短串为基准才不会越界；若以 prompts[0] 为基准，必须额外检查 i < len(p)。
- 包含空串时答案必为 ""，基准选最短串可自然处理。

模式信号

「一批字符串的最长公共前缀」→ 以最短串为界纵向逐列比较。

相关练习：`prefix.trie` `prefix.hit_sum` `tok.segment`

`stream.find_stop`

stream.find_stop `find_stop`

★ Top Interview 150 · LC 28 · Find the Index of the First Occurrence in a String · 字符串匹配（KMP）

`nanoserve/stream/stop.py` · `find_stop(text: str, stop: str) -> int`

引擎里的角色

`nanoserve/engine.py::Engine._earliest_stop` 在 `MultiStopMatcher` 报告命中后，对每个 stop 调一次 `find_stop(st.text, s)`，取最小起点截断文本，然后 `_finish(st, "stop")`。它每个请求只在结束时运行一次，所以全文扫描的 O(n+m) 被摊到整次生成上；每步的判断交给增量匹配器。

被调用于：`nanoserve/engine.py`

题目

返回 stop 在 text 中首次出现的起始下标（0-based），不存在返回 −1。stop 非空、text 可空，均可含任意字符。不得使用 str.find/index/re 等现成查找。

核心洞见

KMP：预处理 stop 的前缀函数 fail[i]（stop[:i+1] 的最长真 border 长度），失配时模式指针 k 跳到 fail[k−1]，文本指针从不回退，总共 O(n+m)。

算法

1. 构造 fail：k=0；对 i=1..m−1，当 k>0 且 stop[i]≠stop[k] 时 k=fail[k−1]；相等则 k+=1；fail[i]=k。
2. 扫描 text：k=0；对每个字符 ch，当 k>0 且 ch≠stop[k] 时 k=fail[k−1]。
3. 若 ch==stop[k]，k+=1；k==m 时返回 i−m+1（第一次完整匹配即最小下标）。
4. 扫描结束返回 −1。
5. 例：stop="aab"，fail=[0,1,0]；text="aaab"：读 a,a 后 k=2；第三个 a 与 'b' 失配 → k=fail[1]=1，再匹配 a → k=2；读 b → k=3，返回 1。

时间 O(n + m)　**空间** O(m)（前缀函数）

参考实现

```python
def find_stop(text: str, stop: str) -> int:
    m = len(stop)
    fail = [0] * m
    k = 0
    for i in range(1, m):
        while k and stop[i] != stop[k]:
            k = fail[k - 1]
        if stop[i] == stop[k]:
            k += 1
        fail[i] = k
    k = 0
    for i, ch in enumerate(text):
        while k and ch != stop[k]:
            k = fail[k - 1]
        if ch == stop[k]:
            k += 1
            if k == m:
                return i - m + 1
    return -1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
朴素逐位置比较	最坏 O(n · m)	stop 通常只有 1–10 个字符，朴素法在 Python 里常数更小，实际不慢；最坏情况出现在 text="aaaa…"、stop="aaab" 这种高度自相似的输入。题目要求自写匹配，KMP 也是理解 `stream.stop_overlap` 的前置。
Rabin–Karp 滚动哈希	期望 O(n+m)	便于一次比较多个等长模式或做 2D 匹配；需要处理哈希碰撞（命中后再逐字符确认），单模式时没有优势。
str.find（CPython 的 fastsearch / two-way）	实践中接近线性	生产代码直接用它；本题禁用是为了让你实现前缀函数。
兄弟练习 `stream.multi_stop` / `stream.stop_overlap`	—	信号区别：已知全文、找「第一次出现在哪」→ find_stop；字符逐个到来、问「现在是否以某 stop 结尾」→ multi_stop；问「尾部与 stop 开头重叠多少」→ 同一个前缀函数的最后一项，即 stop_overlap。

易错点

- fail 的循环从 i=1 开始（fail[0]=0），且失配回退要用 while 而不是 if。
- 匹配成功时返回 i−m+1 而不是 i；要求最小下标，所以第一次 k==m 立即返回。
- k==m 时立即返回，所以永远不会访问 stop[m] 越界；text 为空时循环不执行，自然返回 −1。

模式信号

「子串首次出现位置」+「禁用内置查找」→ KMP 前缀函数。

相关练习：`stream.stop_overlap` `stream.multi_stop`

---

stream.justify justify

stream.justify

★ Top Interview 150 | LC 68 · Text Justification | 贪心

`nanoserve/stream/render.py` · `justify(words: list[str], width: int) -> list[str]`

引擎里的角色

`nanoserve/engine.py::Engine.render(text, width)` 把文本 `split()` 后交给 `justify`；`__main__.py::generate` 在生成结束后用它把 prompt+输出排成宽 `--width` （默认 56）的两端对齐摘要框。`test_render_respects_width` 要求每行长度恰好等于 width。它不在热路径上，正确性比速度重要。

被调用于： `nanoserve/engine.py`

题目

words 非空、每个词非空无空格且长度 ⩽ width。贪心分行（每行尽量多放词，词间至少 1 空格）。除最后一行外两端对齐：空格尽量均分到词间空隙，不能均分时多出的从左往右分；只有一个词的行左对齐右补空格。最后一行词间单空格、左对齐、右补空格。每行长度恰为 width。

核心洞见

两步分离：先贪心确定每行包含哪些词（只依赖长度和），再对非末行用 divmod(总空格, 空隙数) 得到 base 与 extra，前 extra 个空隙多放 1 个空格。

算法

1. i 指向当前行首词；j 从 i+1 起，只要 length + 1 + len(words[j]) ⩽ width 就加入，length 记录「词长和 + 最少空格」。
2. gaps = j−i−1。若 j==n（最后一行）或 gaps==0（单词行）：' '.join 后右补空格到 width。
3. 否则 spaces = width − 词长和，base, extra = divmod(spaces, gaps)；第 g 个空隙放 base + (g < extra) 个空格。
4. i = j，继续下一行。
5. 例：width=16，["example","of","text"] 词长和 13，spaces=3，gaps=2 → base=1, extra=1 → "example of text"。

时间 O(总字符数)：每个词放入一次、每个字符输出一次　空间 O(总字符数)（输出）

参考实现

```python
def justify(words: list[str], width: int) -> list[str]:
    lines: list[str] = []
    i, n = 0, len(words)
```

```
    while i < n:
        j, length = i + 1, len(words[i])
        while j < n and length + 1 + len(words[j]) <= width:
            length += 1 + len(words[j])
            j += 1
        line_words = words[i:j]
        gaps = j - i - 1
        if j == n or gaps == 0:
            line = " ".join(line_words)
            lines.append(line + " " * (width - len(line)))
        else:
            spaces = width - sum(len(w) for w in line_words)
            base, extra = divmod(spaces, gaps)
            parts: list[str] = []
            for g, word in enumerate(line_words[:-1]):
                parts.append(word + " " * (base + (1 if g < extra else 0)))
            parts.append(line_words[-1])
            lines.append("".join(parts))
        i = j
    return lines
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
最小参差度 DP（Knuth–Plass 风格）	$O(n^2)$ 或更优	全局最小化各行空白平方和，排版更好看（TeX 的做法）；但题面明确规定贪心分行，DP 会产出不同的分行，测试必挂。只有在排版质量本身是目标时才值得。
`textwrap.fill`	O(总字符数)	标准库只做贪心换行、不做两端对齐填充，且会处理长词折断等额外逻辑；想要固定宽度对齐框必须自己分配空格。
先算行再逐个空格轮流插入	O(width) 每行	结果相同，但循环插空格容易在「多出的空格靠左」上出错；divmod 一次算出 base/extra 更不易错。

易错点

- 最后一行和单词行走同一条左对齐分支，单词行不能做除以 gaps=0 的分配。
- 多出的空格放在左侧空隙（g < extra），不是右侧。
- 引擎侧没有检查前置条件：若 `--width` 小于某个词的长度，规格外的输入会产生超宽行。

模式信号

「每行尽可能多放」+「空格均分、余数靠左」→ 贪心分行 + divmod 分配。

相关练习：`stream.holdback`

stream.holdback

stream.holdback trailing_word_len

★ Top Interview 150 | LC 58 · Length of Last Word | 双指针

`nanoserve/stream/detok.py` · `trailing_word_len(s: str) -> int`

引擎里的角色

`nanoserve/engine.py::Engine.step` 在 `word_stream=True`（默认）且文本末字符不是空白时，令 `hold = max(hold, trailing_word_len(st.text))`，扣住可能被下一个 token 接长的半个单词。每一步都在不断增长的 `st.text` 上调用，所以必须只从尾部扫描；默认配置下平均每步扣留 2.81 字符。

`test_word_stream_emits_whole_words` 要求除最后一块外每个 chunk 都以空白结尾。

被调用于：`nanoserve/engine.py`

题目

s 由单词和空格 ' ' 组成，返回忽略尾部空格后最后一个单词的长度；空串或全空格返回 0。复杂度 O(最后一个单词长度 + 尾部空格数)。

核心洞见

从尾部两段扫描：先跳过空格得到单词末尾 end，再跳过非空格到单词前一位 i，长度 = end − i。代价与整个字符串长度无关。

算法

1. i = len(s) − 1；当 i ⩾ 0 且 s[i]==' ' 时 i −= 1（跳过尾部空格）。
2. end = i（最后一个单词的末字符下标；若全是空格则 end = −1）。
3. 当 i ⩾ 0 且 s[i] != ' ' 时 i −= 1。
4. 返回 end − i。
5. 例：" fly me to the moon "：跳过 2 个空格到 'n'（end），向左越过 "moon" 停在空格，长度 4。引擎例："… the queue, and it is a gra" → 扣住 3 个字符 "gra"。

时间 O(最后单词长度 + 尾部空格数)　空间 O(1)

参考实现

```python
def trailing_word_len(s: str) -> int:
    i = len(s) - 1
    while i >= 0 and s[i] == " ":
        i -= 1
    end = i
    while i >= 0 and s[i] != " ":
        i -= 1
    return end - i
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
`len(s.split()[-1])` 或 `s.rstrip().split(' ')`	O(|s|) 每次 + 分配列表	一行就写完，单次调用没问题；但引擎每步都对增长中的全文调用，n 步总共 $O(n^2)$。从尾部扫描让每步代价只与最后一个词有关，这是热路径里该有的写法。
正则 `\S+\s*$`	可能 O(|s|)	从左往右尝试匹配，最坏也要扫全文；可读性好但同样不适合每步调用。
兄弟练习 `stream.stop_overlap` / `server.utf8_ok`	—	三者都是「尾部还没定型，先扣住」：半个词（本题）、半个 stop（KMP border）、半个 UTF-8 字符（字节级，M2 客户端）。引擎对前两者取 max，因为两者都是后缀，并集就是更长的那个。

易错点

- 全空格时 end=−1、i=−1，返回 0；不要额外特判出错。
- 规格只把 ' ' 当分隔符：引擎里 "metrics.\n\nThe" 会被算成 13 字符的一个「词」整体扣住（偏保守、不出错）；引擎只在末字符非空白时调用，所以不会因尾部换行误扣。
- 必须从尾部扫，不要先 strip 全串（O(n) 且复制）。

模式信号

「最后一个单词」+「只需从尾部扫描」→ 反向两段扫描。

相关练习：`stream.stop_overlap` `server.utf8_ok` `stream.justify`

`stream.unique_run`

stream.unique_run longest_unique_run

★ Top Interview 150 · LC 3 · Longest Substring Without Repeating Characters · 滑动窗口

`nanoserve/stream/repetition.py` · `longest_unique_run(s: str) -> int`

引擎里的角色

`nanoserve/engine.py::Engine.result` 在请求结束时计算 `longest_unique_run(st.text)`，作为退化（复读）指标写入 `stats`（CLI 末行如 `longest_unique_run=8`）；`test_stats_and_shared_prefix` 要求它 ⩾1。每个请求调用一次，在 `generate` / `stream` 结束路径上，所以没实现时任何生成都跑不完。

被调用于：`nanoserve/engine.py`

题目

返回 s 中不含重复字符（空格也算）的最长连续子串长度；s 可含任意 Unicode，可空（返回 0）。要求 O(len(s))。

核心洞见

可变滑动窗口：右端逐个扩展，用哈希表记录每个字符最后出现的下标；新字符若在窗口内出现过，左端直接跳到它上次位置 +1。

算法

1. last = {}，left = best = 0。
2. 对 right, ch：若 last.get(ch, −1) ⩾ left，令 left = last[ch] + 1。
3. last[ch] = right；best = max(best, right − left + 1)。
4. 返回 best。
5. 例："abba"：right=2 的 b 上次在 1 ⩾ left → left=2；right=3 的 a 上次在 0 < left=2，不能把 left 往回拉 → 窗口 "ba"，答案 2。

时间 O(n) **空间** O(min(n, 字符集大小))

参考实现

```python
def longest_unique_run(s: str) -> int:
    last: dict[str, int] = {}
    best = left = 0
    for right, ch in enumerate(s):
        if last.get(ch, -1) >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
集合 + while 收缩左端	O(n)（每个字符进出各一次，约 2n 步）	同样线性，逻辑更直观；跳跃版只做一次比较就移动左端。两者都对，题目只要 O(n)。
长度 128/256 的数组代替哈希表	O(n)，常数更小	只适用于 ASCII/字节；规格允许任意 Unicode 字符（输出可能含 "。"、emoji），必须用 dict。
枚举所有子串检查	$O(n^2)$~$O(n^3)$	仅用于对拍。
兄弟练习 `stream.repeat_window`	—	信号差别：这里窗口长度由「无重复」约束决定、问最长能多长（可变窗口）；repeat_window 的窗口长度固定为 k、问窗口内是否有重复（定长窗口 + 集合）。

易错点

- 必须检查 last[ch] ⩾ left：窗口左侧之外的旧下标不能让 left 后退（"abba"）。
- 空串返回 0；每步都要更新 best，而不是只在遇到重复时更新。

模式信号

「最长的连续片段」+「片段内不能重复」+ O(n) → 可变滑动窗口 + 哈希表记录最后位置。

相关练习：`stream.repeat_window` `metrics.window_max`

stream.repeat_window

stream.repeat_window repeats_within

★ Top Interview 150 | LC 219 · Contains Duplicate II | 滑动窗口

`nanoserve/stream/repetition.py` · `repeats_within(tokens: list[int], k: int) -> bool`

引擎里的角色

`nanoserve/engine.py::Engine._pick` 实现 `no_repeat_window=W`：采样 tok 后检查 `repeats_within(out[-W:] + [tok], W)`，为真就屏蔽窗口内所有 token 再 `sample` 一次（窗口因此始终保持无重复）。`test_no_repeat_window` 在 (W, top_k) = (3, 20)、(8, 3)、(20, 3) 三种组合下断言每个 id 都不在前 W 个 id 中——后两种的窗口比 top-k 还宽，单纯「拒绝—重采样」会反复抽到同几个 token。

被调用于：`nanoserve/engine.py`

题目

给定 token 列表（整数，可负，可空）与 k ⩾ 0，判断是否存在 i≠j 使 tokens[i]==tokens[j] 且 |i−j| ⩽ k。k==0 必为 False。要求 O(n) 时间，O(min(n,k)) 额外空间更佳。

核心洞见

维护「最近 k 个元素」的集合：新元素若已在集合中，就找到了距离 ⩽k 的重复；否则加入，集合超过 k 个时移除 tokens[i−k]。

算法

1. k ⩽ 0 直接返回 False；window = set()。
2. 对 i, tok：若 tok in window，返回 True。
3. window.add(tok)；若 len(window) > k，移除 tokens[i−k]（窗口只保留下标 i−k+1..i）。
4. 遍历完返回 False。
5. 引擎例：W=3，out[-3:]=[5, 9, 2]，候选 tok=5 → [5, 9, 2, 5] 中两个 5 距离 3 ⩽ 3 → True，ban 5 后重采样。

时间 O(n) 空间 O(min(n, k))

参考实现

```python
def repeats_within(tokens: list[int], k: int) -> bool:
    if k <= 0:
        return False
    window: set[int] = set()
    for i, tok in enumerate(tokens):
        if tok in window:
            return True
        window.add(tok)
        if len(window) > k:
            window.remove(tokens[i - k])
    return False
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
dict 记录每个值最后出现的下标	O(n) 时间，O(n) 空间	写法同样简单，且能顺带给出最近重复的距离；但空间随不同值个数增长，不满足 O(min(n,k)) 的加分要求。
对每个 i 检查后面 k 个元素	O(n · k)	k 很小（如 3）时其实不慢，但不满足规格。
引擎里更直接的 `tok in out[-W:]`	O(W)	引擎每次只需问「新 token 是否在最近 W 个里」，这个判断就够了。整窗检查要能回答「新 token 是否重复」，前提是窗口里原本没有重复。框架靠「一旦重复就屏蔽整个窗口再采样」维持这个不变量；早期版本用最多 16 次的拒绝重采样，放弃后窗口会残留一对重复，导致之后每步都白白重试 16 次——这正是不变量被打破的后果，现已修复。通用函数的价值在于审计整段序列（测试就是这么用的）。
logits 预先屏蔽 set(out[-W:])	O(W) 一次采样	一次采样就满足约束、无需重试；与「拒绝—重采样」相比 top-k 候选集也不同（预屏蔽是在剩余 token 里取 top-k）。生产系统（如 no_repeat_ngram）通常这样做。

易错点

- 先判断再加入；移除的是 tokens[i−k]，此时窗口里恰好有 k+1 个元素（集合中无重复，大小即元素数）。
- k==0 按规格直接返回 False（同一下标不算重复，距离为 0 的「重复」不存在）。
- 距离是「⩽ k」，窗口保留 k 个历史元素再加上当前元素。

模式信号

「距离 ⩽ k 内是否有相同值」+ O(n)/O(k) 空间 → 定长窗口集合。

相关练习： `stream.unique_run` `metrics.qps`

`sample.cdf_index`

sample.cdf_index　cdf_index

★ Top Interview 150　LC 35 · Search Insert Position　二分查找

`nanoserve/sampling/search.py` · `cdf_index(cdf: list[float], r: float) -> int`

引擎里的角色

`nanoserve/sampling/sampler.py::sample` 对 top-k 后 p>0 的候选求前缀和 `cdf`，取 `r = rng.random() * cdf[-1]`，`i = cdf_index(cdf, r)`，返回 `cand[min(i, len(cand)-1)]`。每个解码步（temperature>0）调用一次，`rng` 是每请求的 `random.Random(seed)`。

被调用于： `nanoserve/sampling/sampler.py`

题目

cdf 严格递增（int 或 float，长度 ⩾1），返回 r 的位置或插入点：满足 cdf[i] ⩾ r 的最小下标 i；r 大于全部元素返回 len(cdf)。O(log n)，不得用 bisect。

核心洞见

这是 lower bound。把第 i 个候选对应到区间 (cdf[i−1], cdf[i]]，r 落在哪个区间就选哪个；「第一个 ⩾ r」恰好实现这种左开右闭的划分，各区间长度等于概率。

算法

1. lo, hi = 0, len(cdf)（半开区间 [lo, hi)，hi 可取到 len 表示「插在末尾」）。
2. 当 lo < hi：mid = (lo + hi) // 2。
3. 若 cdf[mid] < r，答案在右边：lo = mid + 1；否则 cdf[mid] ⩾ r，mid 可能是答案：hi = mid。
4. 返回 lo。
5. 例：p=[0.5, 0.3, 0.2]，cdf=[0.5, 0.8, 1.0]；r=0.65 → 1；r=0.5 恰在边界 → 0（属于左边区间）；r=0 → 0。

时间 O(log n) **空间** O(1)

参考实现

```python
def cdf_index(cdf: list[float], r: float) -> int:
    lo, hi = 0, len(cdf)
    while lo < hi:
        mid = (lo + hi) // 2
        if cdf[mid] < r:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
线性扫描找第一个 ⩾ r	O(n)	引擎里 top_k=4，候选只有约 4 个，线性扫描同样快；二分的价值在 top_k=0（全词表，真实模型 V≈10^5）或 top-p 采样时才体现。前缀和本身已是 O(n)，所以二分只省下查找那一半。
`np.searchsorted(cdf, r)` / `bisect_left`	O(log n)	语义完全相同（side='left'），生产代码直接用；本题禁用 bisect 是要你写对边界。
Gumbel-max：argmax(logits + Gumbel 噪声)	O(V)，不需要 CDF	GPU 上易并行、无需前缀和与归一化，是很多推理框架的做法；但需要 V 个随机数，与「每步一个 random()」的可复现方案不同。
兄弟练习 `sched.weighted_route`（LC 528）	O(log n)	同一个想法的整数版：权重前缀和 + randint(1, total) + bisect_left。那里权重固定、建一次前缀和反复抽；这里每步分布都变，每次都重建 CDF。

易错点

- hi 初值是 len(cdf) 而不是 len−1，否则 r 大于所有元素时无法返回 len。
- 条件写成 cdf[mid] < r（lower bound）；写成 ⩽ 会变成 upper bound，r 恰好等于边界时选到下一个候选。
- 调用方用 min(i, len−1) 兜底：浮点乘法极少数情况下可能让 r 不小于 cdf[−1]。

模式信号

「严格递增序列中的位置或插入点」+ O(log n) → lower bound 二分。

相关练习：`sample.kth_largest` `sched.weighted_route`

---

sample.kth_largest

sample.kth_largest　kth_largest

★ Top Interview 150　LC 215 · Kth Largest Element in an Array　堆

`nanoserve/sampling/topk.py` · `kth_largest(logits: list[float], k: int) -> float`

引擎里的角色

`nanoserve/sampling/sampler.py::sample` 在除以温度后、若 `0 < top_k <` 有限值个数，调用 `kth_largest(x[finite].tolist(), top_k)` 得阈值 thr，再 `x[x < thr] = -inf`。每个解码步调用一次，输入长度 = 未被禁的词表大小（V=59），k = top_k（默认 4）。

被调用于：`nanoserve/sampling/sampler.py`

题目

logits 非空、可负、允许重复，1 ⩽ k ⩽ len。返回降序排序后下标 k−1 的元素（重复值各占一个名次，不是第 k 个不同值）。不得修改传入列表，复杂度优于全排序：平均 O(n) 或 O(n log k)。

核心洞见

维护大小为 k 的小顶堆，堆里始终是「目前见过的最大 k 个」，堆顶就是其中最小的——即第 k 大。

算法

1. heap = list(logits[:k])（切片产生副本，不改输入），heapify。
2. 对剩余的 x：若 x > heap[0]，heapreplace(heap, x)（弹出最小、压入 x，一次 O(log k)）。
3. 返回 heap[0]。
4. 例：[2.0, 1.5, 1.5, 0.3]，k=2 → 堆最终为 {1.5, 2.0}，返回 1.5。采样器保留所有 ⩾1.5 的值，于是 2.0、1.5、1.5 三个候选都留下——阈值语义下并列值会让候选数多于 k。

时间 O(n log k)　空间 O(k)

参考实现

```python
def kth_largest(logits: list[float], k: int) -> float:
    heap = list(logits[:k])
    heapq.heapify(heap)
    for x in logits[k:]:
        if x > heap[0]:
            heapq.heapreplace(heap, x)
    return heap[0]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
快速选择（随机 pivot）	平均 O(n)，最坏 $O(n^2)$；需要 O(n) 副本	渐近更优，k 接近 n/2 时优势明显；但要复制数组再原地划分，大量重复值时需三路划分。引擎里 k=4 远小于 V，O(n log k) 已接近线性，且堆写法不会有退化。
`sorted(logits, reverse=True)[k-1]`	O(n log n)	V=59 时根本测不出差别，但违反规格；真实词表 V≈10^5 每步排序就是浪费。
`np.partition` / `torch.topk`	O(n)（introselect）	生产实现的选择，向量化、在 GPU 上并行；本题要求自己实现选择逻辑。
兄弟练习 `metrics.slowest_k`（LC 703）/ `tok.top_pieces`	—	数据一个个流入、要随时回答第 k 大 → 只能用大小为 k 的堆（slowest_k）；值是小范围整数计数 → 桶排序可做到 O(n)（top_pieces）；一次性数组 → 堆或快速选择皆可（本题）。

易错点

- 用小顶堆而不是大顶堆：大顶堆要把全部 n 个元素入堆再弹 k 次，空间 O(n)。
- 只有 x > heap[0] 才替换；用 ⩾ 也对，但多做无用的堆操作。
- 不要对传入列表 heapify（会修改调用方数据），先切片复制。

模式信号

「第 k 大」+「优于全排序」+ k ≪ n → 大小为 k 的小顶堆（或快速选择）。

相关练习：`metrics.slowest_k` `tok.top_pieces` `sample.cdf_index`

`tok.top_pieces`

tok.top_pieces　top_k_pieces

其它 LeetCode　LC 347 · Top K Frequent Elements　哈希表

`nanoserve/tokenizer/train.py` · `top_k_pieces(pieces: list[str], k: int) -> list[str]`

引擎里的角色

`nanoserve/tokenizer/tokenizer.py::Tokenizer.__init__` 从语料切出所有长度 ⩾3 的英文片段（341 个，167 种），先把 k 定为 min(48, 出现 ⩾3 次的个数)=25，并在频次相等处往下调使答案唯一，再调用 `top_k_pieces(pieces, k)`，结果按 (−次数, 字典序) 排序后加前导空格并入词表（' the' 出现 49 次居首）。只在引擎启动时调用一次。

被调用于：`nanoserve/tokenizer/tokenizer.py`

题目

pieces 为可重复的片段序列（len ⩾1），1 ⩽ k ⩽ 不同片段数，返回出现次数最多的 k 个不同片段（顺序任意）。保证第 k 名与第 k+1 名次数不同（答案唯一）。复杂度优于 O(n log n)。

核心洞见

出现次数是 1..n 的整数，可以用次数当下标做桶排序：buckets[c] 存所有恰好出现 c 次的片段，从高到低收集 k 个即可，O(n)。

算法

1. counts = Counter(pieces)（O(n)）。
2. buckets = n+1 个空列表；对每个 (piece, c) 放入 buckets[c]。
3. c 从 n 递减到 1，依次把 buckets[c] 里的片段加入 out，凑满 k 个立即返回。
4. 例：[a,a,a,b,b,c]，k=2 → buckets[3]=[a]、buckets[2]=[b] → [a, b]。

时间 O(n) **空间** O(n)（计数表 + n+1 个桶）

参考实现

```
def top_k_pieces(pieces: list[str], k: int) -> list[str]:
    counts = Counter(pieces)
    buckets: list[list[str]] = [[] for _ in range(len(pieces) + 1)]
    for piece, c in counts.items():
        buckets[c].append(piece)
    out: list[str] = []
    for c in range(len(buckets) - 1, 0, -1):
        for piece in buckets[c]:
            out.append(piece)
            if len(out) == k:
                return out
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
大小为 k 的小顶堆（按次数）	O(n + m log k)，m 为不同片段数	满足规格，也是 LeetCode 最常见解法；当次数范围很大或数据流式到达、无法预知 n 时比桶更通用。这里一次性给出全部片段，桶排序更简单且严格线性。
`Counter.most_common(k)`	O(n + m log k)（内部用 heapq.nlargest）	生产代码一行就够；并列时按首次出现顺序返回，答案不唯一时会悄悄截断——这正是 Tokenizer 先把 k 调到频次断点的原因。
按次数全排序取前 k	O(n + m log m)	最直接，但在 m 接近 n 时退化为 O(n log n)，不满足规格。
兄弟练习 `sample.kth_largest`	—	那里值是任意浮点 logit，无法开桶，只能用堆/快速选择；这里值是有界整数计数，桶排序把 log 去掉。

易错点

- 桶要开 len(pieces)+1 个（某片段可能出现 n 次），不是不同片段数个。
- 返回顺序任意，但 Tokenizer 会重新排序以保证 token id 每次启动都一致；自己的实现不要依赖返回顺序。
- 凑满 k 个就返回；「答案唯一」保证第 k 个恰好落在某个桶的末尾，不会在一个桶中途截断并列片段——若去掉这个保证，就得自己定义并列规则。

模式信号

「出现次数最多的 k 个」+「优于 O(n log n)」+ 次数为有界整数 → 计数 + 桶排序（或堆）。

相关练习：`sample.kth_largest` `tok.segment` `metrics.dominant`

stream.multi_stop

stream.multi_stop MultiStopMatcher

其它 LeetCode　LC 1032 · Stream of Characters　Trie

`nanoserve/stream/multistop.py` · `class MultiStopMatcher(stops: list[str]): feed(ch: str) -> bool`

引擎里的角色

`nanoserve/engine.py::Engine.start` 在 cfg.stop 非空时构造 `MultiStopMatcher(stops)` 存进 `DecodeState.matcher`； `Engine.step` 把每个新 token 的 piece 逐字符 `feed` （`hit = st.matcher.feed(ch) or hit`），命中后交给 `find_stop` 定位截断。每个输出字符调用一次，代价必须与已生成长度无关。

被调用于： `nanoserve/engine.py`

题目

构造时给出非空 stop 列表（每个非空，可重复，可含任意字符如 "<|eot|>"、"\n\n"、"\x00"、emoji）。feed(ch) 追加一个字符，若某个 stop 是迄今全部输入的后缀返回 True。返回 True 不重置状态。每次 feed O(L)，L 为最长 stop 长度。

核心洞见

「是否以某个模式结尾」= 从最新字符往回读是否走出某个模式的倒序。把所有 stop 倒序插入 Trie，只保留最近 L 个字符，每次 feed 从新到旧沿 Trie 走，遇到结束标记即命中。

算法

1. 构造：对每个 stop，从最后一个字符向前插入 Trie（dict 嵌套），末节点放结束标记；用 None 作键（不是 str，不会与任何字符冲突）。
2. buf = deque(maxlen=最长 stop 长度)，自动丢弃更早的字符。
3. feed(ch)：buf.append(ch)；node = root；对 c in reversed(buf)：node = node.get(c)，为 None 则返回 False；若结束标记在 node 中返回 True。
4. 走完 buf 仍未命中返回 False。
5. 例：stops=["cd","f","kl"]，Trie：d→c→END、f→END、l→k→END，maxlen=2；喂 "abcd" 时读 d→c 命中；喂到 f 直接命中；喂 "kl" 命中。

时间　构造 O(Σ|s|)；每次 feed O(L)　空间　O(Σ|s| + L)

参考实现

```python
_END = None


class MultiStopMatcher:
    def __init__(self, stops: list[str]) -> None:
        # Trie over the *reversed* stop strings, so a suffix check walks back from the newest char.
        self.root: dict = {}
        for s in stops:
            node = self.root
            for ch in reversed(s):
                node = node.setdefault(ch, {})
            node[_END] = True
        self.buf: deque[str] = deque(maxlen=max(len(s) for s in stops))

    def feed(self, ch: str) -> bool:
```

```
        self.buf.append(ch)
        node = self.root
        for c in reversed(self.buf):
            node = node.get(c)
            if node is None:
                return False
            if _END in node:
                return True
        return False
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每次 feed 对每个 stop 检查 endswith	$O(\Sigma\|s\|)$ 每字符	stop 只有一两个时更简单甚至更快；stop 多且有公共后缀（如 "</s>"、"<s>"）时 Trie 共享路径、在第一个不匹配字符就退出。
Aho–Corasick 自动机	摊还 $O(1)$ 每字符	正向 Trie + fail 指针，严格更快，且当前状态深度就是「尾部与某 stop 前缀的最长重叠」，能顺带替代 `stream.stop_overlap` 的扣留计算；构建复杂得多。stop 很多、很长时值得。
保存全文，每次对全文做正则/子串查找	$O(\|text\|)$ 每字符，总 $O(n^2)$	违反「与已喂入总长无关」的要求；这正是规格强调增量的原因。
兄弟练习 `prefix.trie`	—	同为 Trie，方向相反：前缀缓存问「哪个已存串是查询的前缀」，正向插入；这里问「哪个 stop 是当前输入的后缀」，所以倒序插入、从最新字符往回走。

易错点

- 结束标记不能用 "$"、"#" 这类字符：stop 本身可能包含它们（规格特别提醒），用 None 或独立对象。
- buf 的 maxlen 取最长 stop 长度，不能取最短；也不要无限增长保存全文。
- 命中后不重置：之后继续 feed 依然基于全部历史判断。
- 引擎写成 `feed(ch) or hit` 而非 `hit or feed(ch)`：后者短路会跳过喂字符，让缓冲区与真实文本错位。

模式信号

「字符流逐个到来」+「是否以若干模式之一结尾」+ 每次 O(最长模式) → 倒序 Trie + 有界缓冲。

相关练习：`stream.find_stop` `stream.stop_overlap` `prefix.trie`

stream.stop_overlap

stream.stop_overlap longest_border

其它 LeetCode　LC 1392 · Longest Happy Prefix　字符串匹配（KMP）

`nanoserve/stream/overlap.py` · `longest_border(s: str) -> str`

引擎里的角色

`nanoserve/engine.py::Engine._stop_holdback` 每一步对每个长度 >1 的 stop s 取 `tail = text[-(len(s)-1):]`，计算 `len(longest_border(s + "\x00" + tail))`，所有 stop 取最大作为扣留字符数。例：stop=" the queue."，输出以 "…watches the" 结尾时扣 4 个（" the"）。20 个 seed 共 1545 步平均只扣 0.38 字符，而固定扣 len−1 要 9.47； `test_stop_holdback_is_exact` 每步与暴力结果比对。

被调用于： `nanoserve/engine.py`

题目

返回 s 最长的「既是真前缀又是后缀」的子串（border），不存在返回 ""。s 含任意字符（包括 "\x00"），len ≥1；前后缀允许重叠（"ababab" → "abab"）。要求 O(n)。

核心洞见

KMP 前缀函数 pi[i] = s[:i+1] 的最长真 border 长度，答案就是 pi[n−1]。引擎把 stop 和尾部用不在 stop 中的分隔符拼起来，border 不能跨过分隔符，于是它的长度恰好是「tail 的最长后缀同时是 stop 前缀」。

算法

1. pi = [0]*n，k = 0。
2. 对 i = 1..n−1：当 k > 0 且 s[i] ≠ s[k] 时 k = pi[k−1]（退到次长 border）。
3. 若 s[i] == s[k]，k += 1；pi[i] = k。
4. 返回 s[:pi[−1]]。
5. 例："</s>\x00ok </"：遇到末尾的 '<' 时 k 从 0 变 1，'/' 时变 2 → 返回 "</"，引擎扣住 2 个字符等待下一个 token。

时间　O(n)（k 的总增量 ⩽ n，回退总次数也 ⩽ n）　空间　O(n)

参考实现

```python
def longest_border(s: str) -> str:
    # KMP prefix function: pi[i] = length of the longest proper border of s[:i + 1].
    pi = [0] * len(s)
    k = 0
    for i in range(1, len(s)):
        while k and s[i] != s[k]:
            k = pi[k - 1]
        if s[i] == s[k]:
            k += 1
        pi[i] = k
    return s[:pi[-1]] if s else ""
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
对每个 k 从大到小比较 s[:k] 与 s[n−k:]	$O(n^2)$	规格明确不接受；不过在引擎里 n ≈ 2|s|，stop 只有几个字符时它其实很便宜——测试里的 `_brute_overlap` 就是它。KMP 在 stop 长或多时才有实际意义。
多项式滚动哈希比较前后缀	期望 O(n)	一次扫描就能比较所有 k，但有碰撞风险（需双哈希或校验）；前缀函数确定性 O(n)，且可复用 `stream.find_stop` 的代码。
Z 函数	O(n)	z[i] = n−i 的最小 i 给出最长 border，同样线性；与前缀函数等价，看个人熟悉程度。
不算重叠，固定扣住 len(stop)−1 个字符	O(1)	绝对安全但让流式输出一直落后近 10 个字符；精确扣留几乎总是 0，只有尾部真的像 stop 开头时才扣。

易错点

- 答案是 pi[−1]，不是 max(pi)（中间位置的 border 不是整个串的 border）。
- 失配要用 while 反复回退到 pi[k−1]，用 if 只退一次会漏掉较短的 border。
- 引擎依赖分隔符 "\x00" 不出现在 stop 中，否则 border 可能跨过分隔符、扣留量偏大。
- 真前缀：k < n，"aaaa" 的答案是 "aaa" 而不是自身。

模式信号

「既是前缀又是后缀的最长子串」+ O(n) → KMP 前缀函数最后一项。

相关练习：`stream.find_stop` `stream.multi_stop` `stream.holdback`

M2　服务器与调度

M2

服务器：连续 batching、准入、多 worker、传输、离线批任务规划

25 题 · ★ Top 150 8 · 其它 17

	练习	LC	LeetCode 题目	模式
	`stream.frame_codec`	271	Encode and Decode Strings	字符串编码（长度前缀）
★	`metrics.median`	295	Find Median from Data Stream	双堆
★	`metrics.merged_median`	4	Median of Two Sorted Arrays	二分查找
★	`sched.merge_busy`	56	Merge Intervals	区间合并
★	`sched.insert_busy`	57	Insert Interval	区间合并
★	`sched.min_launches`	452	Minimum Number of Arrows to Burst Balloons	贪心
★	`metrics.dominant`	169	Majority Element	投票（Boyer–Moore）
★	`metrics.max_burst`	53	Maximum Subarray	一维 DP
★	`metrics.max_burst_circular`	918	Maximum Sum Circular Subarray	一维 DP
	`sched.weighted_route`	528	Random Pick with Weight	前缀和
	`sched.capacity_ok`	1094	Car Pooling	差分/扫描线
	`sched.sjf`	1834	Single-Threaded CPU	堆 + 模拟
	`sched.assign_workers`	1882	Process Tasks Using Servers	堆 + 模拟
	`sched.busiest_worker`	2402	Meeting Rooms III	堆 + 模拟
	`sched.cooldown`	621	Task Scheduler	贪心
	`metrics.qps`	933	Number of Recent Calls	滑动窗口
	`metrics.window_max`	239	Sliding Window Maximum	单调队列
	`metrics.window_median`	480	Sliding Window Median	双堆
	`server.utf8_ok`	393	UTF-8 Validation	位运算
	`server.tokens`	1797	Design Authentication Manager	设计（数据结构）
	`sched.slots`	1845	Seat Reservation Manager	堆
	`sched.maintenance`	729	My Calendar I	二分查找
	`sched.min_workers`	2406	Divide Intervals Into Minimum Number of Groups	差分/扫描线
	`sched.slo_bound`	630	Course Schedule III	堆
	`sched.rr_busiest`	1606	Find Servers That Handled Most Number of Requests	堆 + 模拟

stream.frame_codec

stream.frame_codec FrameCodec

其它 LeetCode | LC 271 · Encode and Decode Strings | 字符串编码（长度前缀）

`nanoserve/stream/protocol.py · class FrameCodec: encode(chunks: list[str]) -> str; decode(data: str) -> list[str]`

引擎里的角色

`transport.py::Transport.message` 把一个 tick 的事件 `[(rid, chunk), …]` 拍平成 `[rid, chunk, rid, chunk, …]` 后 `encode`，每 tick 一条消息；`StreamClient._complete` 在整条消息的字节收齐后 `decode`，再 `zip(flat[0::2], flat[1::2])` 还原成帧。片段是模型输出，可能含数字、`#`、换行、中文、emoji，也可能是空串。实测 `[("r0", "3#4#"), ("r1", "")]` 编成 `2#r04#3#4#2#r10#`。测试用 1/3/16/4096 字节包往返真实服务器的事件流，要求 `client.frames == report.events`。

被调用于：`nanoserve/server/transport.py`

题目

`encode(list[str]) -> str`、`decode(str) -> list[str]`，对任意字符串列表（含空列表、空串、任意 Unicode）满足 `decode(encode(x)) == x`；禁止 pickle/json/eval；无实例状态；O(总字符数)。

核心洞见

任何字符都可能出现在内容里，所以“选一个分隔符”注定失败，除非转义。改成每段前写长度：`长度#内容`。解码时读到 `#` 就知道后面恰好取几个字符，内容里有多少 `#` 都不影响。

算法

1. encode：对每个 s 输出 `f"{len(s)}#{s}"` 并拼接；空列表得到空串。
2. decode：i 从 0 开始；`j = data.index('#', i)` 找到长度结束处（从 i 往后找，长度部分只有数字）。
3. `n = int(data[i:j])`，取 `data[j+1 : j+1+n]` 作为一段，`i = j + 1 + n`。
4. 直到 i 到达末尾。例：`2#r04#3#4#` 先读 `2#` 得 `r0`，再读 `4#` 取 4 个字符得 `3#4#`。

时间 encode / decode 均 O(总字符数)　空间 O(总字符数)

参考实现

```python
class FrameCodec:
    def encode(self, chunks: list[str]) -> str:
        return "".join(f"{len(s)}#{s}" for s in chunks)

    def decode(self, data: str) -> list[str]:
        out: list[str] = []
        i = 0
        while i < len(data):
            j = data.index("#", i)
            n = int(data[i:j])
            out.append(data[j + 1 : j + 1 + n])
            i = j + 1 + n
        return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
分隔符 + 转义（例如把 `#` 写成 `##`，用 `#`，分段）	O(n)，输出膨胀	也正确，是 CSV、NDJSON 走的路；但编码和解码都要逐字符处理转义，且无法预知一段有多长。长度前缀可以直接切片、跳过不关心的帧。
换行分隔的 JSON（NDJSON / SSE）	O(n)	真实 API 的流式输出多用 SSE + JSON，可读性好、浏览器原生支持；JSON 的转义解决了分隔符冲突。本题禁用 json，就是要你直面这个问题。
定长二进制头（4 字节大端长度，gRPC/HTTP2 风格）	O(n)	不必扫描 `#`，头部大小固定，生产协议更常见；但输出不是 str，且长度通常按字节计，与本题按字符计不同。
兄弟练习 prefix.codec	O(n)	同样是“格式自定、只要能无损往返”，但序列化的是树结构；思路相通：要么写长度/结构标记，要么约定空节点标记，绝不靠猜内容。

易错点

- 解码时必须从当前位置 i 往后找 `#`，不能 `split('#')`，也不能从串首找。
- 空串编码成 `0#`，空列表编码成 `""`，两者解码结果不同（`['']` vs `[]`），都要对。
- 长度单位要一致：Python 的 `len(str)` 是码点数，不是 UTF-8 字节数；本题全程按字符，传输层再整体转字节。

模式信号

“把任意字符串列表编码成一个串并无损还原，内容可能包含任何字符”：长度前缀分帧。

相关练习：`server.utf8_ok` `prefix.codec` `stream.holdback`

`metrics.median`

metrics.median MedianTracker

★ Top Interview 150 · LC 295 · Find Median from Data Stream · 双堆

`nanoserve/metrics/latency.py` · `class MedianTracker: add(x: int) -> None; median() -> float`

引擎里的角色

`Server.latency = MedianTracker()`； `server.py::Server._finish` 对每个完成的请求 `latency.add(req.latency)`（latency = finished_at - arrival，单位 tick）； `report()` 在至少有一个完成时取 `latency.median()` 作为 `ServerReport.p50_latency`。测试断言它等于 `statistics.median`。demo 的 p50 62.0 就来自这里。集群的全局 p50 不用它，而是用兄弟练习 `metrics.merged_median` 合并各 worker 已排序的列表。

被调用于： `nanoserve/server/server.py`

题目

`add(x)` 加入整数样本（可重复、可负）； `median()` 返回当前全部样本的中位数（float，偶数个取中间两数平均）。add O(log n)，median O(1)。

核心洞见

把样本分成较小一半和较大一半：较小一半放最大堆、较大一半放最小堆，并保持 `len(low) == len(high)` 或多 1。中位数只取决于两个堆顶。

算法

1. `low` 是取负的最大堆，`high` 是最小堆。
2. `add(x)`：先推入 low，再把 low 的顶移到 high，保证 low 的每个元素 ⩽ high 的每个元素。
3. 若 `len(high) > len(low)`，把 high 的顶移回 low，恢复大小约束。
4. `median`：low 较大时返回 `-low[0]`，否则返回 `(-low[0] + high[0]) / 2`。例：加入 1、2 → low=[1]、high=[2] → 1.5；再加 3 → low=[2,1]、high=[3] → 2.0。

时间 add O(log n)，median O(1)　**空间** O(n)

参考实现

```python
class MedianTracker:
    def __init__(self) -> None:
        self._low: list[int] = []   # max-heap (negated), holds the smaller half
        self._high: list[int] = []  # min-heap, holds the larger half

    def add(self, x: int) -> None:
        heapq.heappush(self._low, -x)
        heapq.heappush(self._high, -heapq.heappop(self._low))
        if len(self._high) > len(self._low):
            heapq.heappush(self._low, -heapq.heappop(self._high))

    def median(self) -> float:
        if len(self._low) > len(self._high):
            return float(-self._low[0])
        return (-self._low[0] + self._high[0]) / 2.0
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
有序列表 + bisect.insort	add O(n)（memmove），median O(1)	几千个样本以内其实很快且代码只有两行；样本量大、add 频繁时才必须用双堆。
每次查询时快速选择	median O(n)	查询远少于插入时可行（例如只在报告末尾算一次）；本服务器只在报告时查询，但规格要求 median O(1)，双堆是能同时满足两边的方案。
兄弟练习 metrics.window_median / metrics.merged_median	—	window_median 要支持删除出窗口的元素，双堆加惰性删除；merged_median 的两份数据已经有序且是离线的，用二分划分 O(log(m+n))，连堆都不需要。“只增不删、随时查询”才是本题。
流式分位数草图（t-digest、P^2 算法）	O(1) 内存，近似	生产监控要的是 p99 且样本无界，精确双堆的 O(n) 内存不可接受，于是用近似草图；需要精确 p50 时才用本题。

易错点

- Python 的 heapq 只有最小堆，最大堆要存负数，取顶时记得取反。
- 偶数个样本返回两数平均，必须是 float（`/` 而非 `//`）。
- 只按大小平衡而不经过“先进 low 再移到 high”，会破坏 low ⩽ high 的有序性。

模式信号

“数据流在线中位数 / 分位数”，只增不删：最大堆 + 最小堆各存一半。

相关练习：`metrics.window_median` `metrics.merged_median` `metrics.slowest_k`

`metrics.merged_median`

metrics.merged_median merged_median

★ Top Interview 150 · LC 4 · Median of Two Sorted Arrays · 二分查找

`nanoserve/metrics/latency.py · merged_median(a: list[int], b: list[int]) -> float`

引擎里的角色

`nanoserve/server/cluster.py::cluster_p50` 计算集群级 p50：每个 worker 的延迟列表先各自排序，按下标对半折成两组，每组用 `heapq.merge` 归并成一个有序列表，最后由 `merged_median(left, right)` 在不合并两半的前提下取中位数。`Cluster.report` 把结果写进 `ClusterReport.p50_latency`；`tests/domain/test_m2_cluster.py::test_cluster_p50_and_merged_busy_timeline` 断言它与 `statistics.median`(全部延迟) 完全相等，并覆盖了空 worker、只有 1 个 worker（右半为空）等折叠情形。说实话，这里两半各自的 `heapq.merge` 已是 O(N log k)，`merged_median` 只省掉最后一次合并；它的价值在于把“两个已排序分片求全局分位数”这个接口固定下来（分片在不同机器上时，二分只需 O(log) 次随机访问，而不用传输整份数据）。

被调用于：`nanoserve/server/cluster.py`

题目

输入两个非降序列表 a（长 m）、b（长 n），0 ⩽ m, n 且 m + n ⩾ 1（允许其中一个为空）。返回合并后的中位数（float）：总数为奇数取正中间，为偶数取中间两个数的算术平均。要求 O(log(m+n))，不能真正合并。例：([1,3],[2]) → 2.0；([1,2],[3,4]) → 2.5。

核心洞见

中位数等价于一条“划分线”：从 a 取前 i 个、从 b 取前 j = half − i 个组成左半（half = ⌈(m+n)/2⌉），只要 a[i−1] ⩽ b[j] 且 b[j−1] ⩽ a[i]，左半就恰好是最小的 half 个数。i 的合法性关于 i 单调，所以对较短数组的 i 二分。

算法

1. 若 len(a) > len(b) 交换，保证在短数组上二分（这样 j = half − i 一定落在 [0, n] 内）。
2. half = (m + n + 1) // 2，在 i ∈ [0, m] 上二分；j = half − i。
3. 取四个边界值 a_left = a[i−1]、a_right = a[i]、b_left = b[j−1]、b_right = b[j]，越界用 −∞ / +∞ 代替。
4. 若 a_left ⩽ b_right 且 b_left ⩽ a_right：奇数返回 max(a_left, b_left)，偶数返回 (max(a_left,b_left) + min(a_right,b_right)) / 2。
5. 若 a_left > b_right，说明 a 取多了，hi = i − 1；否则 lo = i + 1。
6. 走一遍 ([1,3],[2])：交换后 a=[2], b=[1,3]，half=2。i=0,j=2：b_left=3 > a_right=2 → lo=1；i=1,j=1：2 ⩽ 3 且 1 ⩽ +∞，成立，奇数返回 max(2,1)=2.0。

时间 O(log min(m, n))　**空间** O(1)

参考实现

```python
def merged_median(a: list[int], b: list[int]) -> float:
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    half = (m + n + 1) // 2
    lo, hi = 0, m
    neg, pos = float("-inf"), float("inf")
    while lo <= hi:
        i = (lo + hi) // 2  # elements taken from a into the left half
        j = half - i
        a_left = a[i - 1] if i > 0 else neg
        a_right = a[i] if i < m else pos
        b_left = b[j - 1] if j > 0 else neg
        b_right = b[j] if j < n else pos
        if a_left <= b_right and b_left <= a_right:
            left_max = max(a_left, b_left)
            if (m + n) % 2:
                return float(left_max)
            return (left_max + min(a_right, b_right)) / 2.0
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1
    raise ValueError("inputs must be sorted")
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
先 heapq.merge / 双指针走到第 (m+n)/2 个	O(m+n)	最直观，也是 `cluster_p50` 里两半内部实际做的事；当数据已在同一进程内存、规模几千个样本时完全够用。规格明确禁止，是因为这道题考的是“只看边界就能确定全局位置”的划分思想。
第 k 小递归：每轮比较 a[k/2−1] 与 b[k/2−1]，丢掉较小一侧的 k/2 个	O(log(m+n))	同样满足要求，且自然推广到任意分位数（p90/p99 只要换 k）。代价是递归边界多（某个数组被丢空、k=1），偶数情形要调用两次。划分法一次二分同时拿到中间两个数，更适合求中位数。
兄弟练习 metrics.median（双堆 MedianTracker）	add O(log n)，查询 O(1)	信号差别：数据是“流式逐个到达、要随时查询”就用双堆；数据是“已经排好序的两个静态分片、只查一次”就用划分二分。前者的不变量是两堆大小差 ⩽ 1，后者的不变量是划分线两侧交叉有序。

易错点

- 必须在较短数组上二分；否则 j = half − i 可能为负或超过 n，越界。
- half 用 (m+n+1)//2，让奇数时左半多一个，这样奇数答案统一是 max(左边界)。
- 空数组是合法输入：m = 0 时循环只试 i = 0，靠 ±∞ 哨兵处理，不能写 a[0]。
- 偶数情形返回 float（/ 2.0），并且右侧取 min(a_right, b_right) 而不是 a_right。

模式信号

两个已排序数组 + 要求 O(log) + 只关心某个秩（中位数/第 k 小）→ 对划分位置二分。

相关练习：`metrics.median` `sched.merge_busy` `metrics.window_median`

---

sched.merge_busy `merge_busy`

`sched.merge_busy`

★ Top Interview 150 · LC 56 · Merge Intervals · 区间合并

`nanoserve/scheduler/timeline.py` · `merge_busy(intervals: list[list[int]]) -> list[list[int]]`

引擎里的角色

`nanoserve/server/cluster.py::Cluster.report` 把所有 worker 的忙碌时间线（每个 `ServerReport.busy` 是若干 [s, e) 段）拼成一个列表交给 `merge_busy`，得到“至少有一个 worker 在算”的并集时间线 `ClusterReport.busy`，再由 `busy_ticks = Σ(e − s)` 算出集群忙碌 tick 数。测试 `test_cluster_p50_and_merged_busy_timeline` 用逐 tick 展开的集合做对照：并集覆盖的 tick 必须等于各 worker 忙碌 tick 的并集，且结果区间两两严格分离。demo（weighted，3 workers）输出 any-worker-busy 84/85 ticks，而三个 worker 各自利用率只有 47%/93%/64%：并集和求和回答的是不同问题。

被调用于：`nanoserve/server/cluster.py`

题目

输入非空区间列表 [start, end]（0 ⩽ start ⩽ end，顺序任意）。有公共点的区间要合并，端点相接也算（[1,4] 与 [4,5] → [1,5]）。返回按 start 升序、两两不相交的合并结果。例：[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]。

核心洞见

按起点排序后，能与当前区间合并的只可能是结果列表的最后一个区间：后来者起点不更小，若它与最后一段不相交，就不会与更早的任何一段相交。

算法

1. 按 (start, end) 排序（sorted 返回新列表，不改输入）。
2. 遍历每个 [s, e]：若 out 非空且 s ⩽ out[-1][1]，把 out[-1][1] 更新为 max(out[-1][1], e)。
3. 否则 append 一个新列表 [s, e]。
4. 例：排序后 [1,3],[2,6],[8,10],[15,18]：2 ⩽ 3 合并为 [1,6]；8 > 6 新开；15 > 10 新开。

时间 O(n log n)（排序主导，扫描 O(n)） 空间 O(n) 输出

参考实现

```python
def merge_busy(intervals: list[list[int]]) -> list[list[int]]:
    out: list[list[int]] = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 sched.min_launches（按终点排序贪心）	O(n log n)	两题都“排序 + 一次扫描”，但问题不同：merge_busy 求并集的连通块，min_launches 求最少刺穿点。[[1,2],[2,3],[3,4],[4,5]] 合并成 1 块，却需要 2 次发射。信号：要“覆盖了哪些时间”→ 按起点合并；要“最少几个时刻点能碰到每个区间”→ 按终点贪心。
兄弟练习 sched.min_workers（扫描线 / 终点小顶堆）	O(n log n)	同样按起点排序，但 min_workers 关心同一时刻叠了几层（峰值深度），merge_busy 只关心有没有被覆盖（深度 ⩾ 1）。合并会丢掉深度信息，所以不能用合并结果算最少 worker 数。（区间/堆家族信号表：要并集/利用率 → 按起点排序合并 merge_busy；一次动作覆盖所有包含 t 的区间、求最少动作 → 按终点排序贪心 min_launches；只问峰值重叠数/最少资源数 → 扫描线或终点小顶堆 min_workers；要知道任务落在哪个具体资源上、资源有优先级、任务会排队/延后/丢弃 → 空闲堆 + 忙碌堆的事件模拟 sjf / assign_workers / busiest_worker / rr_busiest；可放弃任务、按截止期最大化完成数 → 交换论证 + 大顶堆 slo_bound；顺序完全自由只求一个数 → 计数公式 cooldown）
按时间开布尔数组 / 差分数组标记	O(n + T)，T 为时间跨度	tick 数很小时更简单，测试里的对照就是这样展开的；但时间是 10^9 量级或浮点时不可行。区间已排好序、只插入一个新区间时用 sched.insert_busy 的 O(n) 三段扫描；需要在线反复插入并查询覆盖总长时用 metrics.busy_time（LC 2276）。

易错点

- 端点相接要合并：条件是 s ⩽ last_end 而不是 <。
- 合并时取 max(last_end, e)：被包含的区间（[1,10] 后跟 [2,3]）不能把终点缩短。
- append 时新建列表 [s, e]，不要 append 输入里的原列表对象，否则后续原地改 out[-1][1] 会篡改调用方的数据（框架传入的就是 worker 报告里的 busy）。

模式信号

“合并重叠区间”“计算并集/利用率/覆盖时长” → 按起点排序，只和最后一段比较。

相关练习：`sched.insert_busy` `sched.min_launches` `sched.min_workers` `metrics.busy_time`

sched.insert_busy

sched.insert_busy insert_busy

★ Top Interview 150 | LC 57 · Insert Interval | 区间合并

`nanoserve/scheduler/timeline.py` · `insert_busy(intervals: list[list[int]], new: list[int]) -> list[list[int]]`

引擎里的角色

`server.py::Server.tick` 在本 tick 有计算时执行 `self.busy = insert_busy(self.busy, [now, now + 1])`。相邻 tick 的 [t, t+1] 与 [t+1, t+2] 端点相接而合并，所以一段连续忙碌最后是一个区间 [a, b]，`busy_ticks = Σ(e - s)`，利用率 = busy_ticks / ticks。集群再用 `sched.merge_busy` 合并各 worker 的时间线。demo：默认运行 140/141 tick 忙碌、1 段；维护窗口让它变成 2 段。测试（稀疏 trace，服务器中途会空闲）断言各段有序、互不相接，且覆盖的 tick 集合恰好等于有请求在运行的 tick 集合。

被调用于：`nanoserve/server/server.py`

题目

intervals 按 start 升序、两两不相交（可空），插入 new=[s, e]，与其有公共点（含端点相接）的区间合并，返回仍有序不相交的新列表。O(n)。

核心洞见

列表已经有序，所以一遍扫描就能分成三段：完全在 new 左边的（end < s）、与 new 相交或相接的（start ≤ e）、完全在右边的。中间那段与 new 合并成一个区间。

算法

1. 复制所有 `end < s` 的区间到输出。
2. 对所有 `start <= e` 的区间：`s = min(s, start)`、`e = max(e, end)`，吞并它们。
3. 输出合并后的 [s, e]。
4. 复制剩余区间。例：[[1,2],[3,5],[6,7],[8,10],[12,16]] 插入 [4,8]：[1,2] 原样；[3,5]、[6,7]、[8,10] 被吞并为 [3,10]；[12,16] 原样。

时间 O(n)　空间 O(n)（返回新列表）

参考实现

```python
def insert_busy(intervals: list[list[int]], new: list[int]) -> list[list[int]]:
    s, e = new
    out: list[list[int]] = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < s:
        out.append(list(intervals[i]))
        i += 1
    while i < n and intervals[i][0] <= e:
        s = min(s, intervals[i][0])
        e = max(e, intervals[i][1])
        i += 1
    out.append([s, e])
    out.extend(list(iv) for iv in intervals[i:])
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
append 后调用 sched.merge_busy（排序 + 合并）	O(n log n)	最省事的写法，但浪费了“输入已有序”这一条件；只有在一批无序区间要整理时（集群合并各 worker 时间线）才该用 merge_busy。
bisect 找两端位置再切片	查找 O(log n)，切片仍 O(n)	比较次数少，但重建列表本身就是 O(n)，渐近不变；若要真正 O(log n) 插入需要平衡树，或 metrics.busy_time 那样的区间结构。
利用“新区间总在末尾”的特化：只扩展最后一段	O(1)	服务器里 now 单调，新区间总落在末尾，特化版本 O(1)。框架选择通用的 insert_busy，是因为时间线通常只有几段，O(段数) 可以忽略，且接口不依赖调用顺序。

易错点

- 端点相接也要合并：左段条件是 `end < s`（严格），中段条件是 `start <= e`。
- 不要原地修改输入里的子列表；参考解对保留的区间做了 `list(iv)` 拷贝，调用方的旧时间线不会被意外改动。
- 空列表：直接返回 [new]。

模式信号

“已排序、不重叠的区间列表，插入一个新区间并合并”：一遍扫描分左、中、右三段。

相关练习：`sched.merge_busy` `sched.maintenance` `metrics.busy_time`

---

sched.min_launches

sched.min_launches `min_launches`

★ Top Interview 150 · LC 452 · Minimum Number of Arrows to Burst Balloons · 贪心

`nanoserve/scheduler/batching.py` · `min_launches(windows: list[list[int]]) -> int`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.launches` 回答静态批处理（static batching）问题：至少发射几次 batch，才能让每个请求在自己的可开始窗口内被启动。`launch_windows` 为每个请求算出 [arrival, deadline − service]（最晚开始时刻；无 deadline 用 `OPEN_END = 2^31 − 1`），开始时刻已晚于 arrival 的请求单独列为 infeasible，其余窗口交给 `min_launches`。测试 `test_launch_count_is_minimal_by_brute_force` 在所有窗口终点的组合上暴力验证：答案个点能覆盖、少一个就不行。demo（30 个请求）：至少 3 次发射。

被调用于：`nanoserve/server/planner.py`

题目

输入非空闭区间列表 [earliest, latest]，取值可为负（−2^31 ⩽ earliest ⩽ latest ⩽ 2^31 − 1），顺序任意。在整数时刻 t 发射一次 batch 会服务所有 earliest ⩽ t ⩽ latest 的请求。求让每个请求至少被服务一次的最少发射次数。例：[[10,16],[2,8],[1,6],[7,12]] → 2；[[1,2],[2,3],[3,4],[4,5]] → 2。

核心洞见

终点最早的那个窗口必须被某次发射刺穿，而把这次发射推到它的终点最划算：不会错过它，又尽可能多地覆盖起点更晚的窗口（交换论证）。于是按终点排序，每遇到一个没被上次发射覆盖的窗口，就在它的终点发射。

算法

1. 按 latest 升序排序。
2. last = None（上一次发射时刻），count = 0。
3. 遍历 [s, e]：若 last 为 None 或 s > last（上次发射已在窗口开始之前），count += 1，last = e。
4. 否则 s ⩽ last ⩽ e（last 是更早终点，必 ⩽ e），已被覆盖，跳过。
5. 例：排序后 [1,6],[2,8],[7,12],[10,16]：在 6 发射覆盖前两个；7 > 6，在 12 发射；10 ⩽ 12 已覆盖。答案 2。

时间 O(n log n)　**空间** O(n)（排序副本）

参考实现

```python
def min_launches(windows: list[list[int]]) -> int:
    count = 0
    last = None
    for s, e in sorted(windows, key=lambda w: w[1]):
        if last is None or s > last:
            count += 1
            last = e
    return count
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按起点排序，维护当前组的最小终点	O(n log n)	也正确：遇到 s > 当前最小终点就新开一组，否则 cur_end = min(cur_end, e)。比按终点排序多一个 min 的维护，忘了取 min（被嵌套的短窗口）就错。按终点排序时“最小终点”天然就是 last，因此更不易写错。
兄弟练习 sched.merge_busy 的合并思路（数连通块）	O(n log n)	错误：连通只要求相邻两两相交，刺穿要求一组内全体有公共点。[[1,2],[2,3],[3,4],[4,5]] 只有 1 个连通块，但 [1,2] 与 [3,4] 无公共点，必须 2 次。
兄弟练习 sched.min_workers / LC 435	O(n log n)	min_launches = 最多两两不相交窗口数（终点贪心选出的窗口互不相交，恰好是对偶），与 LC 435 无重叠区间是同一个贪心。min_workers 求的则是最大重叠深度，用扫描线或终点堆。信号：“一个动作同时服务所有包含 t 的对象，动作越少越好”→ 终点贪心；“对象互斥占用资源，资源越少越好”→ 峰值深度。

易错点

- 闭区间：只有 s > last 才需新发射；写成 s ⩾ last 会把端点相接的窗口（[1,2],[2,3]）多算一次。
- 按终点排序，不是按起点；按起点排序直接在终点发射会被嵌套窗口坑：[1,10],[2,3] 在 10 发射漏掉 [2,3]。
- 起点可以为负，哨兵不能用 0 或 −1；用 None 或 −∞。框架里 OPEN_END = 2^31 − 1，在其他语言里比较前做加减会溢出。

模式信号

“一次操作命中所有包含某点的区间，求最少操作次数”（射气球、最少发射）→ 按右端点排序，在右端点出手。

相关练习：`sched.merge_busy` `sched.min_workers` `sched.slo_bound`

metrics.dominant

metrics.dominant dominant_tenant

★ Top Interview 150 | LC 169 · Majority Element | 投票（Boyer–Moore）

`nanoserve/metrics/tenants.py` · `dominant_tenant(requests: list[int] | list[str]) -> int | str`

引擎里的角色

`analytics.py::hot_tenant`： `cand = dominant_tenant(list(tenants))` 得到唯一可能的过半租户，再数一遍得 `share`， `share > 0.5` 才报告为热点，否则返回 None。 `analyze` 对 `report.requests` 的全部租户调用（含被拒绝的请求）。demo 默认权重下无热点； `--hot 4` 时租户 a 占 77% 被标出；测试覆盖“六个请求三个租户各两次 → None”与 `["b", "a", "b"] → ("b", 2/3)`。

被调用于： `nanoserve/server/analytics.py`

题目

非空 id 列表（整数可负，或字符串，只用 == 比较），若有出现次数 > n/2 的 id 必须返回它；否则返回列表中任意一个出现过的 id。O(n) 时间，O(1) 额外空间。

核心洞见

Boyer–Moore 投票：每次把两个不同的 id 互相抵消，过半元素不可能被抵消完，最后剩下的候选必然是它（若存在）。不存在多数时候选只是某个元素，所以调用方必须再数一遍确认。

算法

1. `candidate = requests[0]`， `count = 0`。
2. 遍历每个 r：若 count == 0，令 candidate = r。
3. r == candidate 则 count += 1，否则 count -= 1。
4. 返回 candidate。例：[2,2,1,1,1,2,2]：计数依次 1,2,1,0 → 候选换成 1 → 1,0 → 候选换成 2 → 1，最终 2。

时间 O(n) 空间 O(1)

参考实现

```python
def dominant_tenant(requests: list[int] | list[str]) -> int | str:
    candidate = requests[0]
    count = 0
    for r in requests:
        if count == 0:
            candidate = r
        count += 1 if r == candidate else -1
    return candidate
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
Counter / 哈希表计数	O(n) 时间，O(不同 id 数) 空间	最直观，而且能顺手给出每个租户的份额；租户数有限时完全合理。题面要求 O(1) 额外空间（日志很长、租户很多），才需要投票。
排序后取中间元素	O(n log n)	若存在多数，它一定占据排序后的中点；但要排序（复制或修改输入），也慢一个 log。
按位投票（每一位统计 1 的个数）	O(32n)	只适用于整数 id；这里 id 可以是字符串，只能用 == 比较，这一条信号直接排除了位运算解法。
Misra–Gries（k-1 个计数器）	O(nk) 时间，O(k) 空间	投票的推广：找出所有出现次数 > n/k 的候选。若要标记“占比超过 10% 的租户”，就该用它，同样需要第二遍确认。

易错点

- 无多数时也必须返回列表里出现过的值，不能返回 None 或抛异常；从 `requests[0]` 初始化候选可以保证这一点。
- 过半是严格的 > n/2：恰好一半不算，`hot_tenant(["a", "b"])` 份额 0.5 应返回 None。
- 只信投票结果而不做确认计数，会把“没有多数时的随便一个候选”误报成热点。

模式信号

“找出现次数超过一半的元素，O(1) 额外空间”：Boyer–Moore 投票 + 一遍确认。

相关练习：`metrics.max_burst` `metrics.window_max`

`metrics.max_burst`

metrics.max_burst　max_burst

★ Top Interview 150　LC 53 · Maximum Subarray　一维 DP

`nanoserve/metrics/burst.py` · `max_burst(deltas: list[int]) -> int`

引擎里的角色

`analytics.py::analyze`： `deltas = queue_deltas(report.queue_len)`，即每个 tick 末等待队列长度的变化（第一个 delta 就是 queue_len[0]，因为 tick 0 之前队列为空）， `queue_burst = max_burst(deltas)`。它回答“队列在某一段连续时间里最多涨了多少”，用来评估峰值压力。demo 最大突发 +22；测试与 $O(n^2)$ 暴力解比对，并验证 `sum(deltas) == queue_len[-1]`。

被调用于： `nanoserve/server/analytics.py`

题目

非空整数列表，返回所有非空连续子段和的最大值；全为负时返回最大的那个元素。O(n)。

核心洞见

以位置 i 结尾的最佳子段，要么只有 a[i]，要么是“以 i-1 结尾的最佳子段 + a[i]”，取较大者（Kadane）。换个角度：deltas 是队列长度 Q 的差分，最大子段和 = max over i<j 的 (Q[j] - Q[i])，就是“买卖股票”式的最大涨幅。

算法

1. `best = cur = deltas[0]`。
2. 对后续每个 x：`cur = max(x, cur + x)`（前面累计为负就重新开始）。
3. `best = max(best, cur)`。
4. 返回 best。例：[-2,1,-3,4,-1,2,1,-5,4]：cur 依次 -2,1,-2,4,3,5,6,1,5，best = 6（子段 [4,-1,2,1]）。

时间 O(n) **空间** O(1)

参考实现

```python
def max_burst(deltas: list[int]) -> int:
    best = cur = deltas[0]
    for x in deltas[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
前缀和 + 维护最小前缀	O(n)	与 Kadane 等价：答案 = max(P[j] - min_{i<j} P[i])。在本调用点前缀和就是队列长度本身，这个视角更直观：“最低点到之后最高点的差”。
分治（跨中点的最大子段）	O(n log n)	教学价值高，也能扩展成线段树支持区间查询与单点修改；只求一次全局答案时没必要。
兄弟练习 metrics.max_burst_circular	O(n)	数据若是按天循环的周期序列，高峰可能跨过午夜，线性 Kadane 会把它切成两半，需要环形版本。

易错点

- 子段必须非空：best 初始化为 deltas[0] 而不是 0，否则全负输入会错误地返回 0。
- `cur = max(x, cur + x)` 与“cur < 0 就清零”两种写法初值要对应，混用容易漏掉全负情况。

模式信号

“连续一段的累计增长最大是多少”，元素有正有负：Kadane / 最大子段和。

相关练习：`metrics.max_burst_circular` `metrics.window_max`

`metrics.max_burst_circular`

metrics.max_burst_circular max_burst_circular

★ Top Interview 150 · LC 918 · Maximum Sum Circular Subarray · 一维 DP

`nanoserve/metrics/burst.py` · `max_burst_circular(deltas: list[int]) -> int`

引擎里的角色

`analytics.py::analyze(…, period=P)`： `period_slice` 取一个完整周期的 deltas（运行至少有两个周期时取第二个，因为第一个是从空队列起步的预热期）， `period_burst = max_burst(one)` 、 `period_burst_wrap = max_burst_circular(one)` 。测试用 `periodic_trace` 构造周期 60、高峰从 54 持续到下一周期 5 的流量：单周期内的线性突发小于环形突发，而环形结果恰好等于整次连续运行测得的 `queue_burst` 。

被调用于： `nanoserve/server/analytics.py`

题目

非空整数列表视为首尾相接的环，子段可跨过末尾接回开头，每个元素至多取一次（长度 1..n），返回最大子段和；全为负时返回最大元素。O(n)。

> **核心洞见**
> 环形子段要么不跨界（普通 Kadane），要么跨界；跨界子段的补集是中间一段连续子段，所以跨界最大值 = 总和 - 最小子段和。同时跑最大和最小两个 Kadane 即可。

算法

1. 一遍扫描同时维护 `cur_max/best_max`（最大子段和）、`cur_min/best_min`（最小子段和）与 `total`。
2. 若 `best_max < 0`（全为负），直接返回 best_max：此时 total - best_min 对应的是空子段，不合法。
3. 否则返回 `max(best_max, total - best_min)`。
4. 例：[5,-3,5]：best_max = 7，total = 7，best_min = -3，跨界 = 10（取 5 和开头的 5）；[-3,-2,-3]：best_max = -2 < 0，返回 -2。

时间 O(n) **空间** O(1)

参考实现

```python
def max_burst_circular(deltas: list[int]) -> int:
    total = 0
    cur_max = cur_min = 0
    best_max = best_min = deltas[0]
    for x in deltas:
        cur_max = max(x, cur_max + x)
        best_max = max(best_max, cur_max)
        cur_min = min(x, cur_min + x)
        best_min = min(best_min, cur_min)
        total += x
    if best_max < 0:  # all negative: wrap-around would select an empty segment
        return best_max
    return max(best_max, total - best_min)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
数组复制一遍 + 前缀和 + 长度 ≤ n 的单调队列	O(n) 时间，O(n) 空间	更通用：若题目改成“最多跨 L 个 tick 的突发”，补集技巧失效，而前缀和 + 滑动窗口最小值仍然可用（兄弟练习 metrics.window_max 的单调队列）。
以每个起点做一次 Kadane	$O(n^2)$	测试里的暴力对拍就是这个思路（枚举起点与长度）；周期很长（例如一天 86400 秒）时不可行。
兄弟练习 metrics.max_burst	O(n)	线性序列只需要一个 Kadane；看到“按天循环”“首尾相接”才需要额外的最小子段和。

易错点

- 全负的特判：此时 total - best_min = 0 对应空子段，必须返回 best_max。
- best_min 等于整个数组的和时，跨界情况退化为空子段；只要 best_max ⩾ 0，取 max 后结果仍然正确。
- 不能把数组复制一遍后直接跑 Kadane，那样同一个元素可能被取两次。

模式信号

“循环/周期序列上的最大子段和，可跨过末尾”：max(Kadane 最大, 总和 - Kadane 最小)，注意全负。

相关练习：`metrics.max_burst` `metrics.window_max`

---

sched.weighted_route

sched.weighted_route WeightedRouter

其它 LeetCode | LC 528 · Random Pick with Weight | 前缀和

`nanoserve/scheduler/routing.py · class WeightedRouter(weights: list[int]): pick() -> int`

引擎里的角色

`nanoserve/server/cluster.py::Cluster._route_weighted`（policy=“weighted”）在仿真开始前为每个请求调用一次 `WeightedRouter(self.weights).pick()`，决定它进入哪个 worker 的队列。路由器从全局 `random` 取随机数，所以框架先 `random.seed(self.seed)`、跑完再 `random.setstate` 恢复，保证同一 seed 的两次运行路由完全一致。测试 `test_weighted_routing_is_proportional_and_reproducible` 用 weights=[1,3] 路由 240 个请求，断言 worker1 份额 ≈ 0.75（±0.08）且两次结果逐个相同。demo（3 workers、weights 1 2 1、30 个请求）得到 6/15/9 的分配，期望是 7.5/15/7.5。每个请求都要 pick 一次，所以要求构造 O(n)、pick O(log n)。

被调用于： `nanoserve/server/cluster.py`

题目

构造参数 weights 为非空正整数列表。`pick()` 独立地随机返回下标 i，概率恰为 w[i]/Σw。必须使用标准库 `random` 的全局随机源（以便 `random.seed` 复现）。复杂度：构造 O(n)，pick O(log n)。

核心洞见

把每个 worker 看作数轴上长度为 w[i] 的一段，前缀和 prefix 给出各段右端点；在 [1, Σw] 上均匀取整数 target，落在哪一段就返回哪一段，即“第一个 prefix[i] ⩾ target 的 i”，用二分查找。

算法

1. 构造：prefix = accumulate(weights)，例如 [1,3] → [1,4]。
2. pick：target = random.randint(1, prefix[-1])（两端都包含）。
3. 返回 bisect_left(prefix, target)：下标 i 恰好覆盖 target ∈ (prefix[i−1], prefix[i]]，共 w[i] 个整数。
4. 例：prefix=[1,4]，target=1 → 0；target ∈ {2,3,4} → 1，概率 1/4 与 3/4。

时间 构造 O(n)，每次 pick O(log n)　空间 O(n)

参考实现

```python
class WeightedRouter:
    def __init__(self, weights: list[int]) -> None:
        self.prefix = list(itertools.accumulate(weights))

    def pick(self) -> int:
        target = random.randint(1, self.prefix[-1])
        return bisect.bisect_left(self.prefix, target)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每次 pick 线性扫描累加直到超过 target	pick O(n)	worker 数只有个位数时其实更快（无需额外数组），但规格要求 O(log n)，而且权重表大（例如按 adapter 或租户路由）时线性扫描会成为热点。
random.choices(range(n), weights=w)	每次调用重建累计权重 O(n)；传 cum_weights 才是 O(log n)	库函数内部就是前缀和 + bisect，用 random.random() 浮点取样。生产代码直接用它（配合 cum_weights）即可；练习手写是为了掌握 randint(1,S)+bisect_left 与 random()*S+bisect_right 两种边界写法。
别名法 Alias method（Vose）	构造 O(n)，pick O(1)	权重静态、pick 次数极多时最优；但实现复杂、浮点误差需要处理。若权重会动态变化（例如按实时负载调整），两者都不合适，应换 Fenwick 树：更新与按前缀和二分都是 O(log n)。

易错点

- randint 两端都包含：取 randint(1, S) 配 bisect_left；若取 randint(0, S−1) 则要配 bisect_right，混用会让第 0 个 worker 概率偏移一个单位。
- 必须用全局 random 模块而不是自建 random.Random()，否则框架的 seed/恢复机制失效，路由不可复现。
- 前缀和在构造时算一次；在 pick 里重算就退化成 O(n)。

模式信号

“按权重比例随机选”“概率为 w[i]/Σw” → 前缀和 + 二分查找落点。

相关练习：`sched.assign_workers` `sched.rr_busiest`

sched.capacity_ok

sched.capacity_ok　fits_capacity

其它 LeetCode　LC 1094 · Car Pooling　差分/扫描线

`nanoserve/scheduler/capacity.py` · `fits_capacity(requests: list[list[int]], capacity: int) -> bool`

引擎里的角色

`server.py::Server._budget_ok`（设置 `token_budget` 时）：候选预留 `[prompt_tokens + max_tokens, now, now + _estimate_ticks]` 与所有现有预留 `_reservations.values()` 一起交给 `fits_capacity(…, token_budget)`；通过后 `_admit` 登记预留，`_finish` / `_preempt` 删除。这里的“块数”实际是 token 数。由于现有预留都从 ≤ now 开始、候选从 now 开始，峰值就在 now，扫描线在这个调用点等价于对仍在进行的预留求和；保留通用实现，是为了也能检查未来时间窗的预留（例如事先排好的批任务）。demo `--budget 150`：141 → 226 tick；测试逐 tick 断言预算从未被突破。

被调用于：`nanoserve/server/server.py`

题目

requests[i] = [blocks, start, end)：在 [start, end) 占用 blocks；判断任意时刻总占用 ≤ capacity。时刻可达 10^9（不能按时刻开数组），O(n log n)，空输入返回 True。

核心洞见

总占用只在区间端点变化。把每个请求拆成 (start, +blocks) 和 (end, -blocks) 两个事件，按时间排序后累加，只需在事件点检查。半开区间意味着同一时刻要先释放再占用。

算法

1. 生成 2n 个事件 `(start, +b)`、`(end, -b)`。
2. 按元组排序：同一时刻负的 delta 排在正的前面，恰好就是“先释放后占用”。
3. 顺序累加 `used`，一旦 `used > capacity` 返回 False。
4. 扫描完返回 True。例：[[2,1,5],[3,3,7]]：事件 (1,+2)(3,+3)(5,-2)(7,-3)，在 t=3 时 used=5，容量 4 时 False、5 时 True；[[2,1,5],[3,5,7]] 在 t=5 先 -2 再 +3，峰值 3。

时间　O(n log n)（排序）　空间　O(n)

参考实现

```python
def fits_capacity(requests: list[list[int]], capacity: int) -> bool:
    events: list[tuple[int, int]] = []
    for blocks, start, end in requests:
        events.append((start, blocks))
        events.append((end, -blocks))
    events.sort()  # at equal time, releases (negative) come before acquisitions
    used = 0
    for _, delta in events:
        used += delta
        if used > capacity:
            return False
    return True
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按时刻开差分数组（LeetCode 1094 的经典 O(n + T) 解）	O(n + T)	原题时刻 ⩽ 1000 时最快；这里时刻没有上限（服务器跑得越久越大，可达 10^9），规格明确禁止，必须改成事件排序或坐标压缩。
按 start 排序 + 按 end 的最小堆	O(n log n)	会议室 II 的写法，同样正确；它在需要“分配到哪个资源”时更自然（兄弟练习 sched.min_workers），只判断容量时事件扫描更简单。
兄弟练习 sched.maintenance	每次 O(log n) 查找	相当于容量 1、在线逐个预约的特例：互斥时只需检查两个邻居；一旦允许重叠到 k，就必须看所有覆盖同一时刻的区间，回到扫描线。

易错点

- 同时刻顺序：必须先释放后占用，否则 [2,1,5] 与 [3,5,7] 在容量 3 下会被误判为超限。
- 比较是 `> capacity` 才失败，等于容量是允许的。
- 不要用 `max(end)` 开数组；也不要忘了空输入返回 True。

模式信号

“每个请求在 [start, end) 占用 w 个单位，任意时刻总量是否 ⩽ C”，时间范围很大：端点事件排序 + 累加。

相关练习： `sched.maintenance` `sched.min_workers` `sched.insert_busy`

---

sched.sjf `sjf_order`

sched.sjf

其它 LeetCode · LC 1834 · Single-Threaded CPU · 堆 + 模拟

`nanoserve/scheduler/simulate.py · sjf_order(tasks: list[list[int]]) -> list[int]`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.sjf` 把每个请求转成 [arrival, service_ticks]（service = [prompt_tokens / prefill_chunk] + max_tokens，与服务器准入用的算术一致），调用 `sjf_order` 得到单 worker 上的最短作业优先执行顺序； `BatchPlanner.run` 再用 `PlannedQueue` 插件让真实的 max_batch=1 服务器按这个顺序准入，并与 FCFS 对比。测试 `test_sjf_plan_is_followed_and_beats_fcfs` ：服务器的实际准入顺序与计划逐个相同、平均延迟更低、总 tick 数相同（同样的工作量，只换顺序）。demo（30 个请求）：平均延迟 SJF 223.1 vs FCFS 264.6 ticks，server followed the plan: True。

被调用于： `nanoserve/server/planner.py`

题目

tasks[i] = [arrive, duration]（0 ⩽ arrive，1 ⩽ duration）。单个 worker，非抢占。空闲且无已到达任务时等待下一个到达；空闲且有已到达任务时选 duration 最短的，相同选下标最小。任务完成的同一时刻即可开始下一个，时刻 t 到达的任务在 t 即可被选。返回处理顺序的下标列表，O(n log n)。例：[[1,2],[2,4],[3,2],[4,1]] → [0,2,3,1]。

核心洞见

这是一个离散事件模拟：时钟只在“任务完成”或“空闲时跳到下一次到达”两类事件上前进。按到达时间排序的指针负责把已到达的任务送入就绪集合，就绪集合按 (duration, index) 取最小 → 小顶堆。

算法

1. order = 按 arrive 排序的下标；t = 0，指针 i = 0，ready = 小顶堆。
2. 若 ready 为空且下一个任务尚未到达，把 t 直接跳到它的 arrive（不逐 tick 空转）。
3. 把所有 arrive ⩽ t 的任务以 (duration, idx) 压入 ready。
4. 弹出堆顶 (dur, idx)，记录 idx，t += dur；重复直到全部输出。
5. 例：t=1 只有 0 → 做 0，t=3；就绪 {1(4), 2(2)} → 做 2，t=5；就绪 {1(4), 3(1)} → 做 3，t=6；最后做 1。

时间 O(n log n) **空间** O(n)

参考实现

```python
def sjf_order(tasks: list[list[int]]) -> list[int]:
    order = sorted(range(len(tasks)), key=lambda i: tasks[i][0])
    ready: list[tuple[int, int]] = []
    out: list[int] = []
    t = 0
    i = 0
    while len(out) < len(tasks):
        if not ready and i < len(order) and t < tasks[order[i]][0]:
            t = tasks[order[i]][0]
        while i < len(order) and tasks[order[i]][0] <= t:
            idx = order[i]
            heapq.heappush(ready, (tasks[idx][1], idx))
            i += 1
        dur, idx = heapq.heappop(ready)
        out.append(idx)
        t += dur
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
直接按 duration 全局排序	O(n log n)	错误：忽略到达时间，会选中还没到的任务。测试里正是这个陷阱：长任务 long 在 t=0 独自到达，非抢占 SJF 必须先做它（order[0] == long）。
每次在已到达任务中线性找最短	$O(n^2)$	逻辑相同，n 只有几十个请求时也能跑；堆把每次选择降到 O(log n)，n = 10^5 时差别是不到一秒与数小时。
抢占式 SRPT（最短剩余时间优先）	O(n log n)	允许抢占时 SRPT 是平均完成时间的最优策略，会把 long 打断让短任务先走。但规划器对照的 max_batch=1 服务器一旦准入就把请求跑到完成，PlannedQueue 只能决定下一个准入谁，所以模拟的是非抢占版本。
兄弟练习 sched.assign_workers	O((n+m) log n)	结构对偶：sjf 是一个 worker、用堆选任务（任务优先级 = 时长）；assign_workers 是任务顺序固定（FIFO）、用堆选 worker（worker 优先级 = 权重）。题面里“谁在等待被挑选”决定堆里放什么。

易错点

- 只有 ready 为空时才能跳时钟；ready 非空时跳会让 worker 无故空闲。
- 到达判断用 arrive ⩽ t（同时刻到达可选），否则 [[0,3],[0,1],[0,2]] 会错。
- 堆元素必须是 (duration, idx) 元组，靠 idx 打破平局；只放 duration 会丢失下标。

模式信号

单服务者、任务按时间到达、每次从已到达者中按某个键挑一个 → 按到达排序的指针 + 就绪小顶堆 + 时钟跳跃。

相关练习：`sched.assign_workers` `sched.busiest_worker` `sched.slo_bound`

`sched.assign_workers`

sched.assign_workers assign_workers

其它 LeetCode | LC 1882 · Process Tasks Using Servers | 堆 + 模拟

`nanoserve/scheduler/simulate.py · assign_workers(weights: list[int], tasks: list[int]) -> list[int]`

引擎里的角色

`nanoserve/server/cluster.py::Cluster._place_free_slots`（policy="weighted_free"）先用 `release_one_per_tick` 把批任务改写成第 j 个请求在第 start + j tick 到达，正好是 LC 1882"任务 j 在第 j 秒到达"的模型；再把每个 batch slot 当作一个 worker（`slot_costs = costs` 每项重复 max_batch 次），以 `service_ticks` 为时长调用 `assign_workers`，slot // max_batch 就是 worker 编号。服务时长精确，所以计划就是实际：测试 `test_free_slot_placement_follows_the_plan_and_beats_random_routing` 断言实际落点 == planned、costs=[2,1,3]、每 worker 2 slot 时前三个请求落在 [1,1,0]、没有"有空 slot 时还有人在等"的 tick，且平均延迟低于随机路由。demo（同一 30 请求 trace）：weighted_free 62 ticks、p50 20.5，weighted 85 ticks、p50 23.5（后者仍用原始到达时间，前者已按 1/tick 重新释放，仅作量级参考）。

被调用于：`nanoserve/server/cluster.py`

题目

weights[i] 为 worker i 的权重（越小越优先），tasks[j] 为任务 j 的时长；任务 j 在第 j 秒进入 FIFO 队列。只要有空闲 worker 且队列非空，就把队首分给空闲者中 (weight, index) 最小的那个；没有空闲就等待。worker 在 t 开始任务 j，则 t + tasks[j] 重新空闲并可立即接任务；同一时刻多个空闲按队列顺序依次分配。返回每个任务的 worker 下标，O((n+m) log n)。例：([3,3,2],[1,2,3,2,1,2]) → [2,2,0,2,1,2]。

核心洞见

维护两个堆：free 按 (weight, index)，busy 按 (free_at, weight, index)。任务 j 的开始时刻 t = max(上一个任务的开始时刻, j)，若此刻无空闲就把 t 跳到 busy 堆顶的 free_at——FIFO 保证开始时刻单调不减，所以一个单调时钟就够。

算法

1. free = heapify([(w, i)])，busy = []，t = 0。
2. 对每个任务 j：t = max(t, j)；若 free 为空，t = max(t, busy[0][0])（跳到最早空闲时刻）。
3. 把 busy 中所有 free_at ⩽ t 的 worker 移回 free（一次可能放回多个，它们之间再按权重比较）。
4. 弹出 free 堆顶 (w, i)，ans[j] = i，压入 busy (t + tasks[j], w, i)。
5. 例：t=0 选权重 2 的 worker2（忙到 1）；t=1 worker2 已回来再选它（忙到 3）；t=2 选 (3,0)；t=3 worker2 回来选它；t=4 选 (3,1)；t=5 三个同时回来，又选 worker2。

时间 O((n + m) log n) **空间** O(n + m)

参考实现

```python
def assign_workers(weights: list[int], tasks: list[int]) -> list[int]:
    free = [(w, i) for i, w in enumerate(weights)]
    heapq.heapify(free)
    busy: list[tuple[int, int, int]] = []  # (free_at, weight, index)
    ans: list[int] = []
    t = 0
    for j, dur in enumerate(tasks):
        t = max(t, j)
        if not free:
            t = max(t, busy[0][0])
        while busy and busy[0][0] <= t:
            _, w, i = heapq.heappop(busy)
            heapq.heappush(free, (w, i))
        w, i = heapq.heappop(free)
        ans.append(i)
        heapq.heappush(busy, (t + dur, w, i))
    return ans
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
逐秒推进的朴素模拟	O(T · n)，T 可达 Σtasks	全忙时要空转大量秒数；事件驱动的“跳到 busy 堆顶”把空转压成 O(1)。只要题面里时长可以很大，就要跳时钟。
兄弟练习 sched.busiest_worker	O(m log m + m log n)	全忙时的处理不同：busiest_worker 直接把延后任务交给最早结束的那个 worker（从 busy 弹出、按 end + 时长压回），因为它的 worker 优先级就是编号，busy 堆的 (end, index) 已经给出正确的平局规则；assign_workers 必须先跳时钟、把同一时刻空闲的全部倒回 free，再按权重挑，因为同时空闲的 worker 中权重小的优先，不一定是 busy 堆顶那个。
兄弟练习 sched.weighted_route（随机按权重路由）	每请求 O(log n)	随机路由不看 worker 状态，可能把请求排在忙 worker 后面而别处有空 slot（测试断言 waiting_while_idle(rand) 非空）；assign_workers 是知道服务时长后的全局 FIFO 计划，代价是需要准确的时长估计。时长不可知的在线场景只能用随机或 rr。（区间/堆家族信号表：要并集/利用率 → 按起点排序合并 merge_busy；一次动作覆盖所有包含 t 的区间、求最少动作 → 按终点排序贪心 min_launches；只问峰值重叠数/最少资源数 → 扫描线或终点小顶堆 min_workers；要知道任务落在哪个具体资源上、资源有优先级、任务会排队/延后/丢弃 → 空闲堆 + 忙碌堆的事件模拟 sjf / assign_workers / busiest_worker / rr_busiest；可放弃任务、按截止期最大化完成数 → 交换论证 + 大顶堆 slo_bound；顺序完全自由只求一个数 → 计数公式 cooldown）

易错点

- t 必须单调：写成 t = j 会让排队的任务在过去开始。
- 只在 free 为空时才跳到 busy[0][0]；有空闲时跳会推迟任务。
- 释放用 free_at ⩽ t（结束时刻即可复用）；一次要把所有已结束的都放回，再选最小权重，否则同时空闲的 worker 之间平局判错。

模式信号

“多台服务器 + 优先级挑空闲服务器 + 任务排队等待 + 输出每个任务去了哪” → 空闲堆 + 忙碌堆的事件模拟。

相关练习：`sched.busiest_worker` `sched.rr_busiest` `sched.sjf` `sched.weighted_route`

---

sched.busiest_worker

sched.busiest_worker busiest_worker

其它 LeetCode | LC 2402 · Meeting Rooms III | 堆 + 模拟

`nanoserve/scheduler/simulate.py · busiest_worker(n: int, jobs: list[list[int]]) -> int`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.hottest_lane`：批任务按每 tick 释放一个（`release_one_per_tick`），每个请求原计划占用 [j, j + service) 的 lane（batch slot）；`busiest_worker(lanes, ...)` 预测哪条 lane 承接请求最多。服务器的 `SlotAllocator`（sched.slots）总是分配编号最小的空闲 slot、等待队列 FIFO，正好对应本题“最小编号空闲 worker + 原定开始早者优先”的规则，因此预测可以用真实运行验证：`test_hottest_lane_matches_the_server` 在 2/3/4 条 lane 上把预测与服务器 slot 使用计数（取并列最小编号）比对。demo：4 lanes，hottest lane 0，实测 [8, 7, 8, 7]（并列取 0）。

被调用于：`nanoserve/server/planner.py`

题目

worker 编号 0..n−1。jobs[i] = [start, end]，左闭右开，start 互不相同。每个任务分给当前空闲 worker 中编号最小的；没有空闲就延后到有 worker 空闲，时长 end − start 不变；有 worker 空闲时原定 start 早的任务优先；时刻 t 结束的 worker 可接时刻 t 开始的任务。返回承接任务最多的 worker，并列取最小编号，O(m log m + m log n)。例：n=2，[[0,10],[1,5],[2,7],[3,4]] → 0。

核心洞见

按 start 排序后逐个处理任务就天然满足“早者优先”。free 小顶堆按编号，busy 小顶堆按 (结束时刻, 编号)。全忙时，最早结束（并列取小编号）的 worker 就是该任务会拿到的那个，直接把它的结束时刻推后一个时长。

算法

1. free = [0..n−1]，busy = []，count = [0] * n；jobs 按 start 排序。
2. 对 [start, end]：把 busy 中结束时刻 ⩽ start 的全部弹回 free。
3. 若 free 非空：弹最小编号 w，压入 busy (end, w)。
4. 否则：弹 busy 堆顶 (t, w)，压回 (t + end − start, w)——任务从 t 开始。
5. count[w] += 1；最后返回 count 最大且编号最小者。
6. 例：job0→w0(到10)，job1→w1(到5)；job2 全忙，取 (5,w1) → 到 10；job3 全忙，取 (10,w0) → 到 11。计数 2:2，返回 0。

时间 O(m log m + m log n)　空间 O(n)

参考实现

```python
def busiest_worker(n: int, jobs: list[list[int]]) -> int:
    free = list(range(n))
    busy: list[tuple[int, int]] = []  # (end_time, worker)
    count = [0] * n
    for start, end in sorted(jobs):
        while busy and busy[0][0] <= start:
            _, w = heapq.heappop(busy)
            heapq.heappush(free, w)
        if free:
            w = heapq.heappop(free)
            heapq.heappush(busy, (end, w))
        else:
            t, w = heapq.heappop(busy)
            heapq.heappush(busy, (t + end - start, w))
        count[w] += 1
    return max(range(n), key=lambda w: (count[w], -w))
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 sched.min_workers（只算最少 worker 数）	O(n log n)	min_workers 的 worker 数不限、只问需要几个，任务不会延后，扫描线就够；busiest_worker 的 worker 数固定、任务会延后、还要知道每个任务落在哪个编号上，所以必须带编号做事件模拟。信号：题目问“一个数”且无排队 → 扫描线；问“具体是谁”或有延后 → 堆模拟。
兄弟练习 sched.rr_busiest	O(n log k)	两题都数“谁最忙”，但规则不同：这里全忙就延后，空闲时选全局最小编号（普通小顶堆）；rr_busiest 全忙就丢弃，空闲时选 ⩾ i % k 的第一个（循环），需要虚拟编号技巧。
每个任务线性扫描所有 worker 的结束时刻	O(m · n)	n 很小（框架里 lanes = max_batch，常为 2–8）时完全可行且好写；n 到 100 而 m 到 10^5 时堆版本快一个数量级。

易错点

- 左闭右开：释放条件是 结束时刻 ⩽ start；用 < 会让刚空闲的 worker 被错过。
- 延后任务的新结束时刻是 t + (end − start)，不是 end，也不是 t + end。
- 延后的结束时刻可能远超原 end，在 C++/Java 中要用 64 位；Python 无此问题。
- 并列取最小编号：max(range(n), key=lambda w: (count[w], −w))，或从左往右找第一个最大值。

模式信号

“固定数量的房间/服务器 + 编号优先 + 全满就延后保持时长 + 问谁最忙” → 空闲编号堆 + (结束, 编号) 堆。

相关练习：`sched.assign_workers` `sched.rr_busiest` `sched.min_workers` `sched.slots`

sched.cooldown

sched.cooldown　min_slots_with_cooldown

其它 LeetCode　LC 621 · Task Scheduler　贪心

`nanoserve/scheduler/cooldown.py` · `min_slots_with_cooldown(adapters: list[str], n: int) -> int`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.lora_slots`：多 LoRA 场景下每个租户（`r.tenant`）一个 adapter，同一 adapter 两次加载之间需要冷却 `cooldown` 个时间片，规划器用 `min_slots_with_cooldown` 算启动全部请求需要多少个 prefill 时间片（含插入的空闲片）。测试 `test_lora_cooldown_slots_match_greedy_schedule` 用“每片挑剩余最多且已冷却的 adapter”的贪心模拟作为 oracle，在 cooldown ∈ {0,1,2,4}、租户权重 6:2:1:1 的 20 请求 trace 上逐一比对，并断言 cooldown=0 时等于请求数。demo（cooldown 2）：30 个请求需要 34 个 prefill 片（4 个空闲）。

被调用于：`nanoserve/server/planner.py`

题目

adapters[i] 是任务 i 所需的 adapter 名；0 ⩽ n。每个时间片执行恰好一个任务或空闲，顺序任意。同一 adapter 的两个任务若在第 a、b 片执行（a < b），须 b − a > n。返回最少时间片数，O(len)。例：([A,A,A,B,B,B], 2) → 8（A B _ A B _ A B）；n=3 → 10；([A,C,A,B,D,B], 1) → 6。

核心洞见

出现次数最多的 adapter（次数 top）决定骨架：top − 1 个长度为 n + 1 的“帧”，再加最后一帧里所有次数也为 top 的 adapter（n_top 个）。其余任务填进帧里的空位；若空位不够填，说明根本不需要空闲片，答案就是任务总数。

算法

1. counts = Counter(adapters)，top = 最大次数，n_top = 次数等于 top 的 adapter 个数。
2. 骨架长度 = (top − 1) · (n + 1) + n_top。
3. 答案 = max(len(adapters), 骨架长度)。
4. 例：AAABBB, n=2：top=3, n_top=2 → 2 · 3 + 2 = 8 > 6，答案 8。[A,C,A,B,D,B], n=1：top=2, n_top=2 → 1 · 2 + 2 = 4 < 6，答案 6（无空闲）。

时间　O(len(adapters))　空间　O(不同 adapter 数)

参考实现

```python
def min_slots_with_cooldown(adapters: list[str], n: int) -> int:
    counts = Counter(adapters)
    top = max(counts.values())
    n_top = sum(1 for c in counts.values() if c == top)
    return max(len(adapters), (top - 1) * (n + 1) + n_top)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
大顶堆 + 冷却队列的逐片模拟	O(T log k)，T 为答案	能输出具体排程（哪一片跑哪个 adapter），测试 oracle 就是这种贪心。若引擎真要按此顺序执行就需要它；规划器只要一个数，公式 O(len) 足够。
兄弟练习 sched.sjf / assign_workers / busiest_worker（堆模拟）	O(n log n)	那些题的顺序被到达时间约束，只能模拟；这里顺序完全自由、所有任务时长都是 1，结构对称到能直接数出答案。信号：“任务顺序可任意安排” + “只求最少总时长” → 先想计数公式。
兄弟练习 sched.slo_bound（交换论证 + 大顶堆）	O(n log n)	两者都是可自由排序的离线单机问题，但 slo_bound 有每个任务各自的截止期和不同时长，没有对称性可利用，只能用“按截止期扫描 + 超时丢最长”的贪心。

易错点

- n_top 要数所有并列最多的 adapter，不是 1：AAABBB 里最后一帧有 A 和 B 两个。
- 别忘了 max(len, …)：不同 adapter 很多时骨架会被撑满，空闲片为 0，公式值反而小于任务数。
- 间隔定义是 b − a > n，所以帧长是 n + 1；n = 0 时答案就是任务数。

模式信号

“同类任务之间至少间隔 n、顺序任意、求最少总时长” → 按最高频任务建骨架的计数公式。

相关练习：`sched.slo_bound` `sched.sjf` `metrics.dominant`

`metrics.qps` QpsCounter

`metrics.qps`

其它 LeetCode · LC 933 · Number of Recent Calls · 滑动窗口

`nanoserve/metrics/window.py` · `class QpsCounter: hit(t: int) -> int`

引擎里的角色

`server.py::Server._arrivals` 对每个到达请求调用 `qps.hit(_t_ms)`，结果以 `(tick, 计数)` 存入 `qps_series`。框架用 `_t_ms = max(_t_ms + 1, now * tick_ms)`（`tick_ms=10`）把模拟时钟换成严格递增的毫秒戳，满足本题前置条件，即使同一 tick 有多个到达。3000ms = 300 tick，demo 的 30 个请求都在 25 tick 内到达，所以峰值是 30 req / 3 s；测试在 rate 3 时断言峰值 > 30，并逐个与暴力计数比对。

被调用于：`nanoserve/server/server.py`

题目

`hit(t)` 记录时刻 t（毫秒）的一次请求，返回 [t-3000, t]（两端都含）内的请求数，含本次。t 严格递增且 ⩾ 1。均摊 O(1)。

核心洞见

时间单调，所以落出窗口的永远是最老的那些：队列尾部进、头部出，队列长度就是答案。

算法

1. `q = deque()`。
2. `hit(t)`：`q.append(t)`。
3. 当 `q[0] < t - 3000` 时 `popleft()`。
4. 返回 `len(q)`。例：hit 1、100、3001、3002 → 1、2、3（1 仍在 [1,3001] 内）、3（1 被弹出）。

时间 hit 均摊 O(1)（每个时间戳进出各一次）　**空间** O(窗口内请求数) ≤ 3001（t 为严格递增整数）

参考实现

```python
class QpsCounter:
    def __init__(self) -> None:
        self.q: deque[int] = deque()

    def hit(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
列表 + bisect_left(t-3000)	O(log n)，内存 O(全部请求)	不删除的话内存随运行时间无界增长；适合需要回答任意历史窗口的场景（离线），在线计数用 deque 更好。
固定 3000 个按毫秒分桶的环形计数器	最坏 O(1)，内存 O(窗口长度)	高 QPS（同一毫秒很多请求）时内存与请求数无关，也不要求严格递增；代价是分辨率固定、需要清理过期桶。生产里的限流器多是这一类（滑动窗口计数）。
兄弟练习 metrics.window_max / server.tokens	O(1) 均摊	window_max 也用 deque，但还要按值保持单调，因为要回答“窗口最大值”而不只是计数；server.tokens 同样从头部按时间过期，但要支持按 id 续期，所以换成 OrderedDict。

易错点

- 窗口两端都包含：弹出条件是 `< t - 3000`，写成 `<=` 会少计边界上的请求。
- 先 append 再弹出，本次请求必须计入。
- 在服务器里直接用 `now * tick_ms` 当时间戳，同一 tick 多个到达会违反“严格递增”；框架的 `_t_ms + 1` 正是为此。

模式信号

“最近 T 时间内的计数”且时间戳单调：队列，头部过期。

相关练习：`metrics.window_max` `server.tokens` `kv.ring_queue`

metrics.window_max

metrics.window_max window_max

其它 LeetCode | LC 239 · Sliding Window Maximum | 单调队列

`nanoserve/metrics/window.py` · `window_max(latencies: list[int], k: int) -> list[int]`

引擎里的角色

`analytics.py::analyze`：把完成的请求按 `finish_order`（完成 tick，同 tick 按槽位）排好取延迟序列，`k = max(1, min(window, len(lat)))`，`rolling_max = window_max(lat, k)`。延迟尖峰在滚动最大值里表现为一段平台，比单个 p50 更容易看出“某段时间内最差的请求有多差”。demo 窗口 5 的滚动最大值范围 23..117；测试与 `[max(w) for w in windows]` 暴力解逐项比对。

被调用于：`nanoserve/server/analytics.py`

题目

整数列表（可负）与 1 ⩽ k ⩽ n，返回每个长度为 k 的连续窗口的最大值，共 n-k+1 个。O(n)。

核心洞见

若 j < i 且 a[j] ⩽ a[i]，那么只要 i 还在窗口里，j 永远不可能是最大值，可以立即丢弃。剩下的下标对应的值严格递减，队头就是当前窗口最大值。

算法

1. deque 存下标，保持对应值从头到尾递减。
2. 处理 i：从队尾弹出所有 `a[dq[-1]] <= a[i]` 的下标，再把 i 入队尾。
3. 若队头下标 `<= i - k`，已滑出窗口，从队头弹出。
4. 当 `i >= k - 1` 时输出 `a[dq[0]]`。例：[1,3,-1,-3,5,3,6,7]，k=3：前三个后队列为 [3,-1]（下标 1,2），输出 3；读到 5 时清空整个队列，之后输出 5、5、6、7。

时间 O(n)（每个下标进出队列各一次） 空间 O(k)

参考实现

```python
def window_max(latencies: list[int], k: int) -> list[int]:
    dq: deque[int] = deque()  # indices, values decreasing
    out: list[int] = []
    for i, x in enumerate(latencies):
        while dq and latencies[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            out.append(latencies[dq[0]])
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
最大堆 (-值, 下标) + 惰性删除	O(n log n)	思路更直接，堆顶下标过期就弹掉；但多一个 log，且堆可能积压过期元素。能写单调队列时没理由用它。
稀疏表（sparse table）	预处理 O(n log n)，任意区间查询 O(1)	当查询的是任意 [l, r] 而不是固定长度的滑动窗口时（例如面板上让用户随意框选时间段），稀疏表或线段树才是对的工具。
兄弟练习 metrics.window_median	O(n log n)	最大值可以“丢掉被支配的元素”，中位数不行：任何一个元素都可能在后来成为中位数，所以只能用双堆保留全部窗口元素。题面问 max/min 用单调队列，问中位数或第 k 大用堆。

易错点

- 队列里存下标而不是值，否则无法判断队头是否滑出窗口。
- 过期判断是 `dq[0] <= i - k`；输出从 `i = k - 1` 开始，不是从 i = k。
- 弹出条件 `<=` 或 `<` 都正确，但 `<=` 让重复值只保留最新的一个，队列更短。

模式信号

“每个固定长度滑动窗口的最大/最小值，要求 O(n)”：单调递减（递增）双端队列。

相关练习： `metrics.window_median` `metrics.qps` `metrics.max_burst`

`metrics.window_median`

metrics.window_median `window_median`

其它 LeetCode · LC 480 · Sliding Window Median · 双堆

`nanoserve/metrics/window.py · window_median(latencies: list[int], k: int) -> list[float]`

引擎里的角色

`analytics.py::analyze： rolling_p50 = window_median(lat, k)`，再取 p50 最大的窗口（并列取最早的）作为 `worst_window`，报告该窗口在完成顺序中的第一个与最后一个请求 id。demo 窗口 5 的滚动 p50 范围 20..111，最差窗口是按完成顺序排列的 r027..r029 那一段；测试与 `statistics.median` 暴力解逐项比对。滚动 p50 比滚动 max 更抗单个离群值，两者一起看能区分“偶发慢请求”和“整体变慢”。

被调用于： `nanoserve/server/analytics.py`

题目

整数列表（绝对值可达 2^31）与 1 ⩽ k ⩽ n，返回每个长度为 k 的窗口的中位数（float；k 为偶数取中间两数平均）。O(n log k)。

核心洞见

在 metrics.median 的双堆上加上“删除滑出窗口的元素”。堆不支持删除任意元素，于是用惰性删除：先在 `pending` 计数表里记一笔，等它浮到堆顶再真正弹出；同时用一个 balance 变量按有效元素数而不是堆的实际大小来平衡两边。

算法

1. 用前 k 个元素建 lo（最大堆，较小一半，k 为奇数时多一个）与 hi（最小堆）。
2. 每步滑入 x、滑出 y：y ≤ lo 顶则视为从 lo 删除（balance -= 1），否则从 hi 删除（balance += 1），并 `pending[y] += 1`。
3. x ≤ lo 顶则推入 lo（balance += 1），否则推入 hi（balance -= 1）。
4. balance > 0 把 lo 顶移到 hi；balance < 0 把 hi 顶移到 lo。
5. 清理两个堆顶上待删除的元素（`prune`），再按 k 奇偶取中位数。

时间 O(n log n) 最坏（惰性删除可能让堆里积压过期元素）；SortedList 版严格 O(n log k) **空间** O(n) 最坏（含待删除元素），通常接近 O(k)

参考实现

```python
def window_median(latencies: list[int], k: int) -> list[float]:
    lo: list[int] = []  # max-heap (negated), holds the smaller half (+1 if k odd)
    hi: list[int] = []  # min-heap, holds the larger half
    pending: Counter[int] = Counter()

    def prune(heap: list[int], sign: int) -> None:
        while heap and pending[sign * heap[0]]:
            pending[sign * heap[0]] -= 1
            heapq.heappop(heap)

    def median() -> float:
        if k % 2:
            return float(-lo[0])
        return (-lo[0] + hi[0]) / 2

    for x in latencies[:k]:
        heapq.heappush(lo, -x)
    for _ in range(k // 2):
        heapq.heappush(hi, -heapq.heappop(lo))
    out = [median()]

    for i in range(k, len(latencies)):
        x, y = latencies[i], latencies[i - k]
        # balance > 0 means lo has too many valid elements, < 0 means hi does
        balance = 0
        if y <= -lo[0]:
            balance -= 1
        else:
            balance += 1
        pending[y] += 1
        if lo and x <= -lo[0]:
            heapq.heappush(lo, -x)
            balance += 1
        else:
            heapq.heappush(hi, x)
            balance -= 1
        if balance > 0:
            heapq.heappush(hi, -heapq.heappop(lo))
        elif balance < 0:
            heapq.heappush(lo, -heapq.heappop(hi))
        prune(lo, -1)
        prune(hi, 1)
        out.append(median())
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
sortedcontainers.SortedList：add / remove / 按下标取中位数	O(n log k)	代码最短、复杂度严格，工程里首选；但不在标准库里，面试通常不让用。
长度为 k 的有序列表 + bisect.insort / 删除	O(nk)（memmove）	本仓库的调用点窗口只有 5，这个写法实际上最快也最不容易出错；双堆版是为原题 k 可达 10^5 的约束准备的。
兄弟练习 metrics.median	add O(log n)	只增不删，所以不需要 pending 与 balance。本题的全部额外复杂度都来自“删除”。
兄弟练习 metrics.window_max（单调队列）	O(n)	最大值可以丢弃被支配的元素，中位数不行，所以不能套用单调队列。

易错点

- 删除归属的判断规则必须和插入一致（≤ lo 顶算 lo）；值相同时删掉哪一份副本无所谓，因为值相等。
- 必须在再平衡之后对两个堆都执行 prune，否则堆顶可能是已删除的元素，中位数就错了。
- balance 衡量的是有效元素数的变化，不能直接比较 `len(lo)` 与 `len(hi)`，因为里面混着待删除元素。

模式信号

“滑动窗口的中位数/分位数”：双堆 + 惰性删除（或有序容器）。

相关练习：`metrics.median` `metrics.window_max` `metrics.merged_median`

---

server.utf8_ok `utf8_valid`

server.utf8_ok

其它 LeetCode | LC 393 · UTF-8 Validation | 位运算

`nanoserve/server/utf8.py` · `utf8_valid(data: list[int]) -> bool`

引擎里的角色

`transport.py::StreamClient.receive`：每收到一个包就把 payload 追加到该消息的缓冲区，然后调用 `utf8_valid(list(box.buf))`；合法就 decode 出预览，不合法（包切在多字节字符中间）就 `held += 1`、等下一个包。服务端切包完全不看字符边界，所以这一步是客户端不解出半个字符的唯一保证。测试用含 `café`、中文、🚀、😀 的片段配 1/2/3/5 字节包，断言每个预览都是消息字节前缀的干净解码且 `held > 0`；实测 held 分别为 8/4/1/2。

被调用于：`nanoserve/server/transport.py`

题目

给定字节列表（0..255，长度 ≥ 1），判断它是否恰好由若干完整的 1–4 字节 UTF-8 字符组成：首字节高位决定长度（0xxxxxxx / 110xxxxx / 1110xxxx / 11110xxx），后面跟恰好 长度-1 个 10xxxxxx；10xxxxxx 不能做首字节，11111xxx 任何位置都非法；末尾截断非法。不检查过长编码与码点范围（[0xC0, 0x80] 合法）。O(n)。

核心洞见

这是一个只有一个计数器的状态机：`need` = 当前字符还缺几个后续字节。need > 0 时只接受 10xxxxxx，need == 0 时读首字节设定 need。结束时 need 必须为 0。

算法

1. `need = 0`。
2. 遍历每个字节 b：若 need > 0，要求 `b >> 6 == 0b10`，否则返回 False，然后 need -= 1。
3. 否则按首字节分类：`b >> 7 == 0` 单字节；`b >> 5 == 0b110` 置 need=1；`b >> 4 == 0b1110` 置 2；`b >> 3 == 0b11110` 置 3；其余（10xxxxxx 或 11111xxx）返回 False。
4. 结束返回 `need == 0`。例：[0xE4, 0xBD]（“你”的前两个字节）：E4 置 need=2，BD 合法、need=1，结束时 need≠0，返回 False，客户端继续等包。

时间 O(n) **空间** O(1)

参考实现

```python
def utf8_valid(data: list[int]) -> bool:
    need = 0  # continuation bytes still expected for the current character
    for byte in data:
        if need:
            if byte >> 6 != 0b10:
                return False
            need -= 1
        elif byte >> 7 == 0:
            continue
        elif byte >> 5 == 0b110:
            need = 1
        elif byte >> 4 == 0b1110:
            need = 2
        elif byte >> 3 == 0b11110:
            need = 3
        else:  # 10xxxxxx as a leading byte, or 11111xxx
            return False
    return need == 0
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
`bytes(data).decode('utf-8')` + try/except	O(n)	Python 的解码器更严格：拒绝过长编码 C0 80、代理区与超过 U+10FFFF 的码点，所以与本规格的判定不一致；用异常做控制流也不适合每个包都调用。
增量解码器（`codecs.getincrementaldecoder('utf-8')`）	总 O(L)	把 need 状态跨包保存，每个字节只看一次。框架每个包都对整个缓冲区重新校验，一条消息总代价约 $O(L^2/P)$；消息很短（每 tick 一条）所以无所谓，长响应时增量状态机才是正解，算法就是本题的循环把 need 留在对象里。
兄弟练习 stream.holdback（M1）	O(n)	同样是“不要把不完整的东西发给用户”，但在文本层：扣住可能还没写完的半个单词。本题在字节层扣住半个字符，两层各管各的边界。

易错点

- 移位位数要配对：判 4 字节首字节用 `>> 3 == 0b11110`，用 `>> 4 == 0b1111` 会把 11111xxx 也放进来。
- 单独出现的 10xxxxxx（need == 0 时）必须返回 False。
- 末尾截断：循环结束后要检查 `need == 0`，否则 [0xE4, 0xBD] 会被误判为合法。

模式信号

“按首字节高位判断长度、后续字节固定前缀”：位运算 + 计数器状态机。

相关练习：`stream.frame_codec` `stream.holdback`

---

server.tokens TokenManager

server.tokens

其它 LeetCode | LC 1797 · Design Authentication Manager | 设计（数据结构）

`nanoserve/server/auth_tokens.py · class TokenManager(ttl: int): generate(token_id: str, now: int); renew(token_id: str, now: int); count_unexpired(now: int) -> int`

引擎里的角色

`Server.sessions = TokenManager(session_ttl)`。`server.py::Server._session_ok` 在每个带 session 的请求到达时调用：已见过的会话 `renew`（续期）并放行；新会话若 `count_unexpired(now) >= max_sessions` 则拒绝，否则 `generate`。每个 tick 结尾若出现过会话，就把 `count_unexpired(now)` 记入 `active_sessions`。注意框架对见过的会话一律放行，过期会话的 `renew` 按规格被忽略，它不再计入活跃数。demo `--sessions 6 --max-sessions 4` 拒绝 10/30，峰值活跃 4。

被调用于：`nanoserve/server/server.py`

题目

每个 token 过期时刻 = 签发或最近续期时刻 + ttl。`generate(id, now)` 签发新 token；`renew(id, now)` 仅对存在且未过期的 token 生效；`count_unexpired(now)` 统计未过期数。过期时刻为 t 的 token 在 t 已过期（过期先于同时刻其它操作）。所有调用的 now 非降序，同一时刻可多次调用。

核心洞见

ttl 固定且时间单调，所以每次写入的过期时刻 `now + ttl` 都 ≥ 已有的任何过期时刻：按写入（含续期）顺序排列的 token 就是按过期时刻排序的。用 OrderedDict 保持这个顺序，过期永远发生在头部，可以惰性地从头弹出。

算法

1. `expiry: OrderedDict[id → 过期时刻]`；`_evict(now)`：头部过期时刻 ≤ now 就 `popitem(last=False)`，直到头部未过期。
2. 每个操作先 `_evict(now)`，这样“过期先于同时刻操作”自然成立。
3. `generate`：`expiry[id] = now + ttl`（新键在尾部）。
4. `renew`：若 id 仍在表中，更新过期时刻并 `move_to_end(id)`；已被清掉的不会复活。
5. `count_unexpired`：清理后返回 `len(expiry)`。

时间 每个操作均摊 O(1)（每个 token 至多被弹出一次） 空间 O(未过期 token 数)

参考实现

```python
class TokenManager:
    def __init__(self, ttl: int) -> None:
        self.ttl = ttl
        # token_id -> expiry. Since `now` strictly increases, every new expiry is the largest so far,
        # so keeping insertion order (with move_to_end on renew) keeps tokens sorted by expiry.
        self.expiry: OrderedDict[str, int] = OrderedDict()

    def _evict(self, now: int) -> None:
        while self.expiry and next(iter(self.expiry.values())) <= now:
            self.expiry.popitem(last=False)

    def generate(self, token_id: str, now: int) -> None:
        self._evict(now)
        self.expiry[token_id] = now + self.ttl

    def renew(self, token_id: str, now: int) -> None:
        self._evict(now)
        if token_id in self.expiry:
            self.expiry[token_id] = now + self.ttl
            self.expiry.move_to_end(token_id)

    def count_unexpired(self, now: int) -> int:
        self._evict(now)
        return len(self.expiry)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
普通 dict，count 时全表扫描	count O(n)	LeetCode 1797 最常见的写法，n 小时完全够用；但这里每个 tick 都调用 count，会话多时线性扫描每 tick 都要付。
最小堆 (expiry, id) + 惰性删除	O(log n)	当每个 token 的 ttl 不同（过期顺序不等于写入顺序）时必须用堆或有序结构；续期要靠“堆中旧项作废”的惰性删除。本题 ttl 统一，OrderedDict 更简单也更快。
兄弟练习 kv.lru / metrics.qps	O(1)	kv.lru 也用“哈希 + 有序链表 + move_to_end”，但淘汰触发条件是容量；metrics.qps 也从头部按时间过期，但不需要按 id 续期，所以只用 deque。题面里出现“按 id 刷新”就需要哈希表。

易错点

- 边界：过期条件是 `expiry <= now`，写成 `<` 会让 token 在过期时刻还能续期、被计数。
- `renew` 必须先清理再判断存在性，否则会复活刚好在 now 过期的 token。
- now 只是非降序（同一 tick 多个请求），不要假设严格递增；相等的过期时刻在 OrderedDict 中依然有序，不影响正确性。

模式信号

“固定 TTL + 续期 + 统计未过期数”，且时间单调：按写入顺序即按过期顺序，用有序哈希表从头过期。

相关练习：`metrics.qps` `kv.lru` `sched.maintenance`

sched.slots

sched.slots SlotAllocator

其它 LeetCode　LC 1845 · Seat Reservation Manager　堆

`nanoserve/scheduler/batch_slots.py` · `class SlotAllocator(n: int): reserve() -> int; unreserve(slot: int) -> None`

引擎里的角色

`server.py::Server.__init__` 建 `SlotAllocator(max_batch)`；`Server._admit` 调 `reserve()` 给新请求分槽位，`_finish` 与 `_preempt` 调 `unreserve()` 归还。槽位号是 `self.running` 字典的键，`_compute` 按 `sorted(self.running)` 的顺序推进各请求，所以槽位号同时决定 batch 行号与 tick 内的计算顺序；规划器用 `r.slot - 1` 统计每条通道的使用次数，分析模块按 slot 给同 tick 完成的请求排序。测试对 max_batch 1/2/4/8 逐 tick 断言运行中请求的槽位两两不同且在 1..b 内。准入和完成每个 tick 都会发生，所以要 O(log n)。

被调用于：`nanoserve/server/server.py`

题目

槽位编号 1..n，初始全空闲。`reserve()` 占用并返回编号最小的空闲槽（保证有空闲）；`unreserve(slot)` 释放一个当前被占用的槽。每次操作 O(log n)。

核心洞见

空闲槽分两类：从未发出过的（编号 ≥ `next_fresh`，天然连续，用一个计数器表示即可）和被归还的（都 < `next_fresh`，放进最小堆）。只要堆非空，堆顶就一定比任何未发出的编号小，所以它就是答案。

算法

1. 初始化 `next_fresh = 1`、`returned = []`，O(1)，不必预先建一个 n 元素的堆。
2. `reserve`：若 `returned` 非空，`heappop` 返回堆顶；否则返回 `next_fresh` 并加 1。
3. `unreserve(slot)`：`heappush(returned, slot)`。
4. 例：n=5，reserve→1、reserve→2，unreserve(2) 使堆为 [2]，接下来 reserve 依次得 2（来自堆）、3、4、5（来自计数器）。

时间　reserve / unreserve 均 O(log n)（堆大小不超过已发出数）　空间　O(n)

参考实现

```python
class SlotAllocator:
    def __init__(self, n: int) -> None:
        # Slots >= next_fresh were never handed out; returned slots (all < next_fresh) sit in a
        # min-heap, so whenever the heap is non-empty its top is the smallest free slot.
        self.next_fresh = 1
        self.returned: list[int] = []

    def reserve(self) -> int:
        if self.returned:
            return heapq.heappop(self.returned)
        slot = self.next_fresh
        self.next_fresh += 1
        return slot
```

```
    def unreserve(self, slot: int) -> None:
        heapq.heappush(self.returned, slot)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
一开始把 1..n 全部放进最小堆	初始化 O(n)，操作 O(log n)	正确且直观，LeetCode 1845 的常见写法。计数器版本把从未用过的槽延迟到需要时才发，n 很大但同时运行的请求很少时更省。
布尔数组 + 线性找第一个空闲	reserve O(n)	本服务器 `max_batch` 只有 4–8，线性扫描其实毫无压力，常数还更小；当槽位数是成千上万的 KV 块或连接 id 时才必须用堆。
兄弟练习 sched.assign_workers / sched.busiest_worker	O((n+m) log n)	它们同样用“空闲集合的最小堆”，但多了一个按结束时间排序的忙碌堆，因为那里要模拟时间推进、自动释放；这里释放是外部显式调用的，一个堆就够了。

易错点

- 编号从 1 开始，不是 0；规划器用 `slot - 1` 做下标。
- 用 set 保存空闲槽再 `min()`，每次 O(n)，违反复杂度要求。
- 若想在归还 `next_fresh - 1` 时回退计数器做优化，必须同时保证堆里没有 ⩾ 新 `next_fresh` 的元素，否则同一编号会被重复发出；不做这个优化最安全。

模式信号

“总是分配编号最小的空闲 X”，同时有归还操作：最小堆（可加一个未发出计数器）。

相关练习：`sched.assign_workers` `sched.busiest_worker` `kv.free_tracker`

`sched.maintenance` MaintenanceCalendar

sched.maintenance

其它 LeetCode · LC 729 · My Calendar I · 二分查找

`nanoserve/scheduler/calendar.py` · `class MaintenanceCalendar: book(start: int, end: int) -> bool`

引擎里的角色

`server.py::Server.schedule_maintenance(start, end)` 调 `calendar.book`，成功才把窗口追加到 `self.maintenance`；`tick()` 用 `in_maintenance(now)` 判断，窗口内整段跳过 `_admit_waiting`，已在运行的请求照常计算。测试：[10,25) 接受、[20,30) 拒绝、[25,30) 因左闭右开而接受；[10,30) 内无任何准入，30 时积压立即开始，p50 高于无窗口的基线。demo `--maintenance 20 40`：141 → 155 tick，忙碌时间线变成 2 段。

被调用于：`nanoserve/server/server.py`

题目

`book(start, end)` 预约半开区间 [start, end)，与任一已接受预约有公共时刻则返回 False 且不记录，否则记录并返回 True。[10,20) 与 [20,30) 不冲突。

核心洞见

已接受的预约两两不相交，按 start 排序后 end 也有序。新区间只可能与“start 不超过它的最后一个”或“start 在它之后的第一个”冲突，所以二分找到插入位置后只需检查两个邻居。

算法

1. 维护有序的 `starts` 与对应的 `ends` 两个列表。
2. `i = bisect_right(starts, start)`：i-1 是前驱，i 是后继。
3. 前驱冲突：`i > 0 and ends[i-1] > start`。
4. 后继冲突：`i < len and starts[i] < end`。
5. 都不冲突则在 i 处插入两表，返回 True。例：已有 [10,20)，book(15,25)：i=1，ends[0]=20 > 15，拒绝；book(20,30)：20 > 20 不成立，接受。

时间 查找 O(log n)，列表插入 O(n)（memmove，常数很小） **空间** O(n)

参考实现

```python
class MaintenanceCalendar:
    def __init__(self) -> None:
        self.starts: list[int] = []  # sorted, disjoint bookings
        self.ends: list[int] = []

    def book(self, start: int, end: int) -> bool:
        i = bisect.bisect_right(self.starts, start)
        if i > 0 and self.ends[i - 1] > start:      # previous booking runs into us
            return False
        if i < len(self.starts) and self.starts[i] < end:  # next booking starts before we end
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
暴力：与每个已接受预约比较 `max(s1,s2) < min(e1,e2)`	每次 O(n)	大 O 与列表插入相同，但是 n 次 Python 级比较，远慢于一次 memmove；维护窗口很少时完全可以接受。
平衡 BST / SortedList（floor/ceiling）	每次 O(log n)	Java 的 TreeMap.floorKey 就是这个思路；预约量很大且频繁时它严格更优，但 Python 标准库没有，需要 sortedcontainers。
兄弟练习 sched.insert_busy / sched.capacity_ok	—	insert_busy 遇到重叠是**合并**（忙碌时段可以叠加），这里是**拒绝**（窗口互斥）；capacity_ok 允许重叠到容量上限，相当于 My Calendar 的“容量 k”版本，需要扫描线而非两个邻居。

易错点

- 半开区间：比较必须是严格的 `>` / `<`，写成 `>=` 会错误拒绝首尾相接的预约。
- 被拒绝的预约不能写入任何结构。
- 只检查前驱而忘了后继（或反之）；start 相同时 bisect_left / bisect_right 都能正确拒绝，但要想清楚是哪一侧的检查起作用。

模式信号

“新区间不得与已有任何区间重叠，冲突就拒绝”：有序存储 + 只查前驱和后继。

相关练习：`sched.insert_busy` `sched.capacity_ok` `sched.merge_busy`

`sched.min_workers`

sched.min_workers `min_workers`

其它 LeetCode · LC 2406 · Divide Intervals Into Minimum Number of Groups · 差分/扫描线

`nanoserve/scheduler/planning.py · min_workers(intervals: list[list[int]]) -> int`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.min_lanes`：把每个请求在“到达即准入”时占用 batch slot 的 tick 区间写成闭区间 [arrival, arrival + service − 1]（`occupancy`），`min_workers` 给出零排队所需的最少 lane 数 W。`plan()` 随后真跑两次服务器验证：max_batch = W 时没有任何请求等待，W − 1 时一定有人等。测试 `test_min_lanes_is_the_peak_concurrency` 还逐 tick 计算峰值并发，断言二者相等。demo（30 个请求）：24 lanes，0 waited；23 lanes 时 2 个请求等待。

被调用于：`nanoserve/server/planner.py`

题目

intervals[i] = [start, end] 闭区间（0 ⩽ start ⩽ end ⩽ 10^6）。每个区间分给恰好一个 worker，同一 worker 上的区间两两不相交——两端都包含，[1,5] 与 [5,8] 在 5 相交。求最少 worker 数，O(n log n)。例：[[5,10],[6,8],[1,5],[2,3],[1,10]] → 3；[[0,0],[0,4],[5,9]] → 2。

核心洞见

答案等于任一时刻同时覆盖的区间数的最大值（区间图的最大团 = 色数）。按起点排序后用小顶堆保存各 worker 的最后结束时刻：堆顶最早结束，若它严格早于新起点就复用这个 worker，否则开新 worker；堆大小即答案。

算法

1. 按 (start, end) 排序；ends = 小顶堆。
2. 对每个 [s, e]：若 ends 非空且 ends[0] < s（闭区间，严格小于），heapreplace(ends, e) 复用。
3. 否则 heappush(ends, e) 开新 worker。
4. 返回 len(ends)。
5. 例：排序 [1,5],[1,10],[2,3],[5,10],[6,8]：压 5；5<1 否压 10；压 3；3<5 复用 → {5,10,10}；5<6 复用 → {8,10,10}，答案 3。注意 [5,10] 不能复用结束于 5 的 worker。

时间 O(n log n) 空间 O(n)

参考实现

```python
def min_workers(intervals: list[list[int]]) -> int:
    ends: list[int] = []  # min-heap of each worker's last occupied time
    for start, end in sorted(intervals):
        if ends and ends[0] < start:  # closed intervals: must end strictly before we start
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
扫描线：事件 (s, +1) 与 (e + 1, −1) 排序，求前缀和最大值	O(n log n)；或计数数组 O(n + 10^6)	同样正确且常数小，因为题目只问“一个数”。闭区间的关键是在 e + 1 处 −1（或同一时刻先 +1 后 −1）。坐标 ⩽ 10^6 时差分数组甚至可以免排序。堆版本的优势是顺带给出一种具体分配（谁复用谁）。sched.capacity_ok（LC 1094 拼车）就是同一扫描线加容量判断。
兄弟练习 sched.merge_busy / sched.min_launches	O(n log n)	三题都先排序区间，问的却是三种量：并集（深度 ⩾ 1 的范围）→ 按起点合并；最少刺穿点（= 最多不相交区间）→ 按终点贪心；最大深度（= 最少互斥资源）→ 扫描线或终点堆。看题面动词：“合并/覆盖”、“一次命中多个”、“互不重叠地分组”。
兄弟练习 sched.busiest_worker / assign_workers	O(m log n)	那两题 worker 数固定、可能排队或延后、需要每个任务的具体去向，所以要带编号的空闲堆 + 忙碌堆模拟；这里 worker 不限量、只求个数，一个终点堆或一次扫描就够。

易错点

- 闭区间：复用条件是 ends[0] < s，不是 ⩽；[1,5] 与 [5,8] 必须两个 worker。框架里 occupancy 已减 1 转成闭区间，正是为此。
- 答案是堆的最终大小，不是 push 的次数（heapreplace 不增大堆）。
- 扫描线写法里同一时刻事件的顺序决定开闭语义，最容易出错。

模式信号

“区间互斥占用资源，求最少资源/最少分组/最少会议室” → 峰值重叠深度：扫描线或按起点排序 + 终点小顶堆。

相关练习：`sched.merge_busy` `sched.min_launches` `sched.busiest_worker` `sched.capacity_ok`

sched.slo_bound

sched.slo_bound max_on_time

其它 LeetCode | LC 630 · Course Schedule III | 堆

`nanoserve/scheduler/planning.py` · `max_on_time(jobs: list[list[int]]) -> int`

引擎里的角色

`nanoserve/server/planner.py::BatchPlanner.slo_bound`：对带 deadline 的请求取 [service_ticks, deadline]（`slo_jobs`，deadline < 1 的永远赶不上，排除），`max_on_time` 给出单个串行 worker 最多能满足多少个 SLO——任何调度都不可能超过的上界。测试 `test_slo_bound_brute_force_and_never_beaten` 对 7 个任务枚举全部排列验证最优性，并断言真实服务器的 FCFS 与 SJF 运行都不超过它。demo（30 个请求）：上界 7/30，FCFS 实际 2、SJF 实际 5，量化出调度策略还有多少空间。

被调用于：`nanoserve/server/planner.py`

题目

jobs[i] = [duration, deadline]（1 ⩽ duration, deadline ⩽ 10^4）。worker 从时刻 0 起串行处理，不可抢占，顺序任意，可以放弃任何请求；从 t 开始的请求须 t + duration ⩽ deadline。返回最多能按时完成的个数，O(n log n)。例：[[100,200],[200,1300],[1000,1250],[2000,3200]] → 3；[[3,2],[4,3]] → 0。

核心洞见

两个交换论证：(1) 一组任务若能全部按时完成，按截止期升序（EDF）执行也一定能；(2) 按截止期扫描时，一旦总时长超过当前截止期，丢掉已选中最长的那个，个数只少一个且剩余总时长最小，对后面最有利。用大顶堆维护已选任务的时长。

算法

1. 按 deadline 升序排序；taken = 大顶堆（Python 存负数），t = 0。
2. 对每个 (d, dl)：压入 −d，t += d。
3. 若 t > dl：t += heappop(taken)（弹出的是 −最长时长，即减去最长者），丢一个就够。
4. 返回 len(taken)。
5. 例：排序后 [100,200],[1000,1250],[200,1300],[2000,3200]：t=100、1100、1300 都按时；加 2000 后 t=3300 > 3200，丢 2000，t=1300。答案 3。

时间 O(n log n)　空间 O(n)

参考实现

```python
def max_on_time(jobs: list[list[int]]) -> int:
    taken: list[int] = []  # max-heap (negated) of durations of the jobs we keep
    t = 0
    for duration, deadline in sorted(jobs, key=lambda j: j[1]):
        heapq.heappush(taken, -duration)
        t += duration
        if t > deadline:  # drop the longest job so far; never hurts, frees the most time
            t += heapq.heappop(taken)
    return len(taken)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按时长排序贪心（SJF：能放就放）	O(n log n)	错误：[[2,2],[1,3]] 按时长先做 [1,3]（t=1），[2,2] 就赶不上，得 1；EDF 先做 [2,2] 再做 [1,3]，得 2。短作业优先优化的是平均延迟（sched.sjf），不是按时完成数——demo 里 SJF 只满足 5 个，上界是 7。
按截止期排序的 0/1 背包 DP：dp[t] = 截止期内用时 t 的最多完成数	O(n · D)，D = max deadline ⩽ 10^4	LC 原题 n ⩽ 10^4 时约 10^8 次操作，Python 过慢，但思路可推广：若每个请求有不同价值（加权按时完成数），贪心失效，DP 才正确。
兄弟练习 sched.min_launches 与 sched.cooldown	—	min_launches 的贪心只需排序后“选或不选”，选过的决定永不撤销；slo_bound 后面的任务会推翻前面的选择（反悔贪心），所以需要一个能随时取出最长者的大顶堆。cooldown 则是顺序自由且对称，直接计数。信号：“可以放弃任务 + 截止期 + 最大化完成数” → EDF 扫描 + 大顶堆反悔。（区间/堆家族信号表：要并集/利用率 → 按起点排序合并 merge_busy；一次动作覆盖所有包含 t 的区间、求最少动作 → 按终点排序贪心 min_launches；只问峰值重叠数/最少资源数 → 扫描线或终点小顶堆 min_workers；要知道任务落在哪个具体资源上、资源有优先级、任务会排队/延后/丢弃 → 空闲堆 + 忙碌堆的事件模拟 sjf / assign_workers / busiest_worker / rr_busiest；可放弃任务、按截止期最大化完成数 → 交换论证 + 大顶堆 slo_bound；顺序完全自由只求一个数 → 计数公式 cooldown）

易错点

- 超时只需弹出一次：弹出的最长者 ⩾ 刚加入的任务，弹后 t ⩽ 上一步的 t ⩽ 上一个截止期 ⩽ 当前截止期。
- 被弹出的可能正是刚加入的任务，这完全正确，不需要特判。
- heapq 是小顶堆，存 −duration；弹出后 t += 弹出值（负数），别写成 t −= 。
- 条件是 t > deadline 才丢，恰好等于 deadline 算按时。

模式信号

“每个任务有时长和截止期、单机串行、可放弃、最大化按时完成数” → 按截止期排序 + 大顶堆，超时丢最长（反悔贪心）。

相关练习：`sched.sjf` `sched.min_launches` `sched.cooldown`

sched.rr_busiest

sched.rr_busiest busiest_servers

其它 LeetCode | LC 1606 · Find Servers That Handled Most Number of Requests | 堆 + 模拟

`nanoserve/scheduler/lb.py` · `busiest_servers(k: int, arrival: list[int], load: list[int]) -> list[int]`

引擎里的角色

`nanoserve/server/cluster.py::Cluster.report` 在 policy="rr" 时调用：单流副本（max_batch 被强制为 1）的前端负载均衡器在线运行 `_dispatch_rr`（第 i 个请求从 i % k 开始绕一圈找空闲 worker，全忙就 503 丢弃）；运行结束后，把请求日志（arrival 与精确的 `service_ticks`）交给 `busiest_servers` 离线回放，得到 `hot_workers`。测试 `test_round_robin_drops_when_all_busy_and_replay_names_hot_workers` 断言回放结果与各 worker 实际完成数中的最大者完全一致，且 5 个副本比 3 个丢得少。demo（3 workers，30 请求）：6 完成、24 被丢弃，回放给出最忙 worker [0, 1, 2]（三者各 2 个，并列）。

被调用于： `nanoserve/server/cluster.py`

题目

k 个 worker，每个同时只处理一个请求。请求 i 在 arrival[i]（非降序）到达，占用 [arrival[i], arrival[i] + load[i])，结束时刻即可接新请求；同一时刻多个请求按下标依次分配。从 i % k 开始按 i%k, i%k+1, …, k−1, 0, 1, … 找第一个空闲 worker；全忙则丢弃。返回处理数最多的所有 worker（任意顺序），O(n log k)。例：k=3，arrival [1,2,3,4,5]，load [5,2,3,3,3] → [1]。

核心洞见

"从 i % k 起循环找第一个空闲" = 在空闲集合里找 ≥ i % k 的最小者，找不到再绕回最小者。虚拟编号技巧把它变成普通小顶堆：worker w 在第 i 步以 v = i + (w − i) % k 存放，v ∈ [i, i + k) 且 v % k = w，于是堆最小值恰是循环意义下第一个空闲 worker。

算法

1. free = [0..k−1]（第 0 步时虚拟编号就是本身），busy = 小顶堆 (free_at, w)，count = [0] * k。
2. 第 i 步：把 busy 中 free_at ≤ arrival[i] 的 worker 弹出，以 v = i + (w − i) % k 压入 free。
3. free 为空则丢弃（continue）；否则 w = heappop(free) % k，count[w] += 1，压入 busy (arrival[i] + load[i], w)。
4. 不变量：每步都弹出最小值，所以留在堆中的 v 都 ≥ i + 1，且新压入的都 < i + 1 + k，下一步仍在窗口内，无需重新编号。
5. 例：前三个请求占满 w0(到6)、w1(到4)、w2(到6)。i=3 在 t=4 释放 w1，v = 3 + (1−3)%3 = 4，弹出 4 → w1。i=4 全忙丢弃。count = [1,2,1]，返回 [1]。

时间 O(n log k) 空间 O(k)

参考实现

```python
def busiest_servers(k: int, arrival: list[int], load: list[int]) -> list[int]:
    # Free workers are stored as "virtual ids" v with v % k == worker and i <= v < i + k at step i,
    # so the heap minimum is exactly the first free worker at or after i % k (wrapping around).
    # Invariant: the minimum is popped every step, so no stored v ever falls below the current i.
    free = list(range(k))
    busy: list[tuple[int, int]] = []  # (free_at, worker)
    count = [0] * k
    for i, (t, dur) in enumerate(zip(arrival, load)):
```

```
        while busy and busy[0][0] <= t:
            _, w = heapq.heappop(busy)
            heapq.heappush(free, i + (w - i) % k)
        if not free:
            continue
        w = heapq.heappop(free) % k
        count[w] += 1
        heapq.heappush(busy, (t + dur, w))
    best = max(count)
    return [w for w in range(k) if count[w] == best]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
有序集合（sortedcontainers.SortedList）存空闲编号 + bisect_left(i % k)，越界取第 0 个	O(n log k)	最直接表达“≥ x 的最小者”，C++ 的 std::set::lower_bound 也是这个思路，可读性最好；Python 标准库没有平衡树，list + bisect.insort 插入是 O(k)。虚拟编号把需求变成只需 heapq。
每个请求从 i % k 线性绕一圈	O(n · k)	就是框架 `_dispatch_rr` 在线做的事——那里 k 只有 3–5 个，又要对照 worker 的实时状态，线性最清楚；离线回放 10^5 个请求、k = 10^5 时必须 O(log k)。
兄弟练习 sched.busiest_worker 与 sched.assign_workers	—	三个“谁接这个任务”的堆模拟，差别全在题面的两个信号上：空闲者怎么挑（全局最小编号 → 普通堆；(权重, 编号) → 元组堆；从 i%k 起循环 → 虚拟编号堆）和全忙怎么办（延后保持时长 → 抢最早结束者；FIFO 排队 → 跳时钟；直接丢弃 → continue）。

易错点

- 释放要在分配之前，并用 free_at ⩽ arrival（左闭右开）。
- 压回 free 时必须按当前步 i 重新计算虚拟编号，不能压原编号 w，否则 ⩾ i%k 的约束失效。
- 丢弃的请求不计数；返回所有并列最大的 worker，而不是只返回一个。

模式信号

“从 i % k 号开始循环找第一个空闲服务器，全忙丢弃，问谁最忙” → 忙碌堆 + 支持“循环后继”的空闲集合（虚拟编号小顶堆或有序集合）。

相关练习：`sched.busiest_worker` `sched.assign_workers` `sched.weighted_route`

M3　分页 KV Cache

KV Cache 块管理器：分页分配、空闲表、淘汰策略、快照与泄漏检测

30 题 · ★ Top 150 21 · 其它 9

	练习	LC	LeetCode 题目	模式
★	`kv.lru`	146	LRU Cache	哈希表 + 双向链表
★	`kv.fork`	138	Copy List with Random Pointer	哈希表
★	`kv.has_cycle`	141	Linked List Cycle	快慢指针
★	`kv.partition_hot`	86	Partition List	链表操作
★	`kv.merge_free`	21	Merge Two Sorted Lists	链表操作
★	`kv.merge_free_k`	23	Merge k Sorted Lists	堆
★	`kv.sort_free`	148	Sort List	分治/归并
★	`kv.drop_double_free`	82	Remove Duplicates from Sorted List II	链表操作
★	`kv.compact`	27	Remove Element	双指针
★	`kv.dedupe`	26	Remove Duplicates from Sorted Array	双指针
★	`kv.merge_arrays`	88	Merge Sorted Array	双指针
★	`kv.rotate_buffer`	189	Rotate Array	双指针
★	`kv.ring_search`	33	Search in Rotated Sorted Array	二分查找
★	`kv.ring_head`	153	Find Minimum in Rotated Sorted Array	二分查找
★	`kv.longest_run`	128	Longest Consecutive Sequence	哈希表
★	`kv.free_ranges`	228	Summary Ranges	双指针
★	`kv.seq_range`	34	Find First and Last Position of Element in Sorted Array	二分查找
★	`kv.page_lookup`	74	Search a 2D Matrix	二分查找
★	`kv.popcount`	191	Number of 1 Bits	位运算
★	`kv.leaked`	136	Single Number	位运算
★	`kv.random_evict`	380	Insert Delete GetRandom O(1)	哈希表
	`kv.lfu`	460	LFU Cache	设计（数据结构）
	`kv.ring_queue`	622	Design Circular Queue	设计（数据结构）
	`kv.snapshot`	1146	Snapshot Array	二分查找
	`kv.versioned`	981	Time Based Key-Value Store	二分查找
	`kv.mem_alloc`	2502	Design Memory Allocator	设计（数据结构）

练习	LC	LeetCode 题目	模式
`kv.refcount`	432	All O(1) Data Structure	设计（数据结构）
`kv.block_hash`	706	Design HashMap	哈希表
`kv.free_tracker`	352	Data Stream as Disjoint Intervals	区间合并
`kv.address_ranges`	715	Range Module	区间合并

kv.lru

kv.lru LRUBlockCache

★ Top Interview 150 · LC 146 · LRU Cache · 哈希表 + 双向链表

`nanoserve/kvcache/lru.py` · `class LRUBlockCache(capacity: int): get(key: int) -> int; put(key: int, value: int) -> None`

引擎里的角色

主机层的 LRU 策略：`tiers.py::make_host_tier("lru", capacity)` 直接返回 `LRUBlockCache`。GPU 端驱逐时 `manager.py::BlockManager._evict` 调 `host.put(chain_hash, checksum)` 把块的 KV（以校验和代表）存下；之后 `admit` 对每个未命中的满块调 `host.get(h)`，返回值等于期望校验和就算 restore，否则 compute。主机层在每个被驱逐块和每个 prompt 满块上都要被访问一次，所以必须 O(1)；容量满时它自己淘汰，无需通知管理器。

被调用于：`nanoserve/kvcache/tiers.py`

题目

实现容量为 capacity（≥ 1）的缓存。get(key)：存在返回值并标记为最近使用，否则返回 -1 且不改变任何状态。put(key, value)：已存在则更新值并标记最近使用；否则插入，若插入后键数 > capacity，淘汰最久未使用的键。键、值为非负整数，两操作均摊 O(1)，建议不用 OrderedDict。

核心洞见

需要两种 O(1) 能力：按键找到条目（哈希表），以及把任意条目移到「最近」端、从「最久」端取出（双向链表）。哈希表存键 → 链表结点，结点里同时存 key，淘汰时才能从表中删掉它。

算法

1. 用 head、tail 两个哨兵结点：head.next 是最近使用，tail.prev 是最久未用；哨兵让插入删除不需要判空。
2. get：查表，未命中返回 -1；命中则把结点摘下再插到 head 后面，返回 val。
3. put 已有键：改 val，摘下再插到头部；直接返回，不触发淘汰。
4. put 新键：建结点、入表、插到头部；若 len(map) > capacity，取 tail.prev 摘下，并用它保存的 key 从表中删除。
5. 例：容量 2，put(1)、put(2)、get(1) 后顺序是 1,2；put(3) 淘汰 2；put(4) 淘汰 1。

时间 get / put 均 O(1)　空间 O(capacity)

参考实现

```python
class _Node:
    __slots__ = ("key", "val", "prev", "next")

    def __init__(self, key: int = 0, val: int = 0) -> None:
        self.key = key
```

```
        self.val = val
        self.prev: _Node | None = None
        self.next: _Node | None = None


class LRUBlockCache:
    def __init__(self, capacity: int) -> None:
        self.capacity = capacity
        self.map: dict[int, _Node] = {}
        self.head = _Node()  # sentinel: head.next is most recently used
        self.tail = _Node()  # sentinel: tail.prev is least recently used
        self.head.next = self.tail
        self.tail.prev = self.head

    def _unlink(self, node: _Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev

    def _push_front(self, node: _Node) -> None:
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key: int) -> int:
        node = self.map.get(key)
        if node is None:
            return -1
        self._unlink(node)
        self._push_front(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        node = self.map.get(key)
        if node is not None:
            node.val = value
            self._unlink(node)
            self._push_front(node)
            return
        node = _Node(key, value)
        self.map[key] = node
        self._push_front(node)
        if len(self.map) > self.capacity:
            lru = self.tail.prev
            self._unlink(lru)
            del self.map[lru.key]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
collections.OrderedDict + move_to_end / popitem(last=False)	O(1)	Python 里这是工程上的正确答案，C 实现还更快。题目要求手写是为了掌握结构本身；同一套哈希表 + 双向链表正是 kv.lfu 的「每个频率一条 LRU 链」的积木，OrderedDict 在那里就不够顺手了。
数组 + 时间戳，淘汰时线性扫描最小时间戳	淘汰 O(n)	写起来最短，容量很小（几十）时完全够用；主机层容量默认等于 GPU 块数（最多 4096），每次驱逐线性扫描就不划算了。
兄弟练习 kv.lfu	O(1)	信号差别在于「热点是否稳定」。hot_vs_scan（热 prompt 夹在越来越长的一次性扫描间，主机层 4 块）上 LRU 只恢复 4 块、LFU 恢复 11 块；shifting_hot_set（旧宠儿之后热点迁移）上 LRU 恢复 33、LFU 只有 26。另注意细节：LRU 先插入再淘汰，LFU 规格是满了先淘汰再插入，容量为 1 时两者行为都要对。
兄弟练习 kv.random_evict（RandomTier）	O(1)	随机不维护任何访问顺序，对抗性 trace 下不会被系统性击败，但也不利用局部性；两条 trace 上分别是 2/53 和 33/18。

易错点

- get 命中后忘了移到头部，退化成 FIFO。
- put 已有键时既要更新值也要移到头部，并且不能走淘汰分支（否则容量未超也会误删）。
- 结点里不存 key，淘汰 tail.prev 时无法从哈希表中删除，表与链表失去同步。
- 淘汰条件写成 len > capacity 之前就淘汰，或用 >=，容量 1 时会把刚插入的键淘汰掉。

模式信号

「O(1) 的 get/put」+「淘汰最久未使用」：按键随机访问加按时间顺序增删，必然是哈希表配有序链。

相关练习：`kv.lfu` `kv.random_evict` `kv.block_hash`

---

kv.fork `fork_chain`

kv.fork

★ Top Interview 150 | LC 138 · Copy List with Random Pointer | 哈希表

`nanoserve/kvcache/block_table.py` · `fork_chain(head: RandomListNode | None) -> RandomListNode | None`

引擎里的角色

并行采样 / beam 分叉： `manager.py::BlockManager.fork` 执行 `seq.head = fork_chain(src.head)` ，为子序列得到一张独立的块表，然后遍历新链给每个物理块 refs+1。块表结点的 `random` 被挪用为**回指前一个结点**，release 和 check() 都依赖它，所以副本的回指必须指向副本自己的结点。注意拷贝的只是表结构，KV 一个字节都不拷，真正的复制推迟到写入共享尾块时的 copy-on-write。

被调用于： `nanoserve/kvcache/manager.py`

题目

给定带 next 和 random 的链表（random 指向同一链中任意结点或 None），返回深拷贝：全部是新结点，val 相同，next/random 只指向新链中对应位置的结点。返回后原链结构与值不变。head 为 None 返回 None，链长 0 ~ 10^4。

核心洞见

难点是 random 可能指向尚未拷贝的后面结点。分两遍：第一遍只建「旧结点 → 新结点」的映射，第二遍再连指针，此时任何目标的副本都已存在。映射必须按结点身份（id 或对象本身），不能按 val。

算法

1. 第一遍遍历原链，为每个结点创建 RandomListNode(cur.val)，存入 clones[id(cur)]。
2. 第二遍遍历：c = clones[id(cur)]；c.next = clones[id(cur.next)]（cur.next 为 None 则 None）；random 同理。
3. 返回 clones[id(head)]，head 为 None 时直接返回 None。
4. 例：[[7,None],[13,0],[11,4],[10,2],[1,0]] 中第 2 个结点的 random 指向第 4 个（后面的结点），第一遍已建好它的副本，第二遍可直接连上。

时间 O(n)　**空间** O(n) 映射表（不计输出）

参考实现

```python
def fork_chain(head: RandomListNode | None) -> RandomListNode | None:
    clones: dict[int, RandomListNode] = {}
    cur = head
    while cur is not None:
        clones[id(cur)] = RandomListNode(cur.val)
        cur = cur.next
    cur = head
    while cur is not None:
        c = clones[id(cur)]
        c.next = clones[id(cur.next)] if cur.next is not None else None
        c.random = clones[id(cur.random)] if cur.random is not None else None
        cur = cur.next
    return clones[id(head)] if head is not None else None
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
交织法：A→A'→B→B'，再令 A'.random = A.random.next，最后拆开	O(n) 时间、O(1) 额外空间	LeetCode 进阶解法，省掉映射表。代价是函数执行期间原链被临时改写：如果别的代码（例如并发读块表的检查器）此时遍历原链会看到错乱结构，任何中途异常都会留下损坏的块表。引擎里用映射表更稳妥；内存极紧、无并发读者时交织法更好。
copy.deepcopy(head)	O(n)，但递归	能正确处理共享引用（memo），但它沿 next 递归，链长 10^4 远超 Python 默认递归深度 1000，直接 RecursionError。
系统层对比：fork 时连 KV 一起拷贝	O(表长 × block_size) 显存	样本立刻全面分叉时与 COW 差不多；但通常样本共享整个 prompt。demo 中 n = 4：COW 只发生 3 次，峰值 15 块，整份拷贝需要 24 块。

易错点

- 用 val 作映射键：不同结点可以有相同 val，random 会连错。
- random 为 None 时去查 clones[id(None)]，KeyError 或连到错误结点。
- RandomListNode 有 __slots__，不能给原结点临时挂 .clone 属性当映射。
- 交织法拆分时忘了恢复原链的 next，违反「原链保持原样」。

模式信号

「深拷贝」+「指针可指向任意（包括后面的）结点」：需要旧→新映射，两遍处理前向引用。

相关练习：`kv.has_cycle` `kv.snapshot` `kv.refcount`

---

kv.has_cycle

kv.has_cycle has_cycle

★ Top Interview 150 | LC 141 · Linked List Cycle | 快慢指针

`nanoserve/kvcache/block_table.py` · `has_cycle(head: ListNode | None) -> bool`

引擎里的角色

块表损坏检测： `manager.py::BlockManager.check` 和 `audit_leaks` 在遍历任何序列的块表之前都先调 `has_cycle(seq.head)` 。检查器的职责是验证它不信任的状态：如果某个结点的 next 被错误地指回前面，直接 `while node` 遍历会死循环，检查器自己就挂了。成环的表在 check() 里报告为「block table has a cycle」，audit_leaks 则跳过它。sanitizer demo 人为制造 `tail.next = head` 来验证这一点。

被调用于： `nanoserve/kvcache/manager.py`

题目

判断单链表沿 next 走是否会回到已经过的结点。不得修改链表，额外空间 O(1)（不能用 set），链长 0 ~ 10^4，head 为 None 返回 False。

核心洞见

Floyd 快慢指针：慢指针每次一步、快指针每次两步。无环时快指针先走到 None；有环时两者都进环后，快指针每步相对慢指针缩近 1，必然在环内相遇。

算法

1. slow = fast = head。
2. 当 fast 和 fast.next 都不为 None：slow 走一步，fast 走两步。
3. 移动之后若 slow is fast，返回 True。
4. 循环退出（快指针到达链尾）返回 False。
5. 例：3→2→0→-4→(回到 2)：第 1 步 slow=2,fast=0；第 2 步 slow=0,fast=2；第 3 步 slow=-4,fast=-4，相遇。

时间 O(n) 空间 O(1)

参考实现

```python
def has_cycle(head: ListNode | None) -> bool:
    slow = fast = head
    while fast is not None and fast.next is not None:
```

```
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
set 记录访问过的结点	O(n) 时间、O(n) 空间	最直观，还能顺便给出入环点；但规格要求 O(1) 空间。检查器对每个序列都要跑，O(n) 额外分配在调试模式的每个操作后都会发生。
最多走 seq.length + 1 步，超了就判环	O(n) 时间、O(1) 空间	管理器确实记着 seq.length，看似更简单；但检查器正是要验证 length 等簿记是否一致（check() 单独报告 tail/length out of sync），用待检查的状态去检查自己是循环论证。Floyd 只依赖链表本身。
Brent 算法（步长翻倍的传送指针）	O(n)、O(1)	常数更小、next 调用更少，适合环很长的场景；链长上限 10^4 时差别可以忽略，Floyd 更好写对。

易错点

- 循环条件只判 fast 不判 fast.next，fast.next.next 抛 AttributeError。
- 用 slow.val == fast.val 比较：val 相同不代表是同一个结点（无环链里也可以有两个块号相同的结点），必须用 is 比身份。
- 在移动之前比较 slow is fast：开始时两者都等于 head，会立刻误报 True。

模式信号

「链表是否成环」+「O(1) 额外空间、不得修改」：快慢指针是唯一同时满足两者的标准解。

相关练习：`kv.fork` `kv.partition_hot` `kv.sort_free`

---

kv.partition_hot

kv.partition_hot　partition_by_heat

★ Top Interview 150　LC 86 · Partition List　链表操作

`nanoserve/kvcache/block_table.py` · `partition_by_heat(head: ListNode | None, x: int) -> ListNode | None`

引擎里的角色

GPU 端驱逐的选择器： `manager.py::BlockManager._evict` 从 `cached` （插入序 = 释放序）取最老的 scan 个块，把它们的热度（饱和复用计数）做成 ListNode 链，调 `partition_by_heat(nodes[0], self.hot)` ，再从结果头部取 k 个作为受害者。冷块（热度 < hot）排前面，且因为分区稳定，冷块之间仍是「先释放先驱逐」；复用过 hot 次以上的块被保护。管理器用 `block_of[id(node)]` 把结点映射回块号，所以函数必须重新串接原结点。 `MAX_SCAN = 200` 、 `HEAT_CAP = 200` 正是为了满足本题「链长 ≤ 200、值域 [-200, 200]」的约束。

被调用于： `nanoserve/kvcache/manager.py`

题目

链表每个结点的 val 是热度。把 val < x 的结点排到前面、val ≥ x 的排到后面，两部分内部都保持原相对顺序（稳定分区），返回新链头。链长 0 ~ 200，val 与 x 都在 [-200, 200]。

核心洞见

一次遍历分拣到两条子链（各带哑结点），最后把「小」链尾接到「大」链头。稳定性来自始终在尾部追加。

算法

1. 建 lo_dummy、hi_dummy 两个哑结点，lo、hi 指向各自尾部。
2. 遍历原链：val < x 接到 lo 后面，否则接到 hi 后面，并推进对应尾指针。
3. 遍历结束令 hi.next = None，切断可能残留的旧 next。
4. lo.next = hi_dummy.next，返回 lo_dummy.next。
5. 例：1→4→3→2→5→2，x=3：小链 1→2→2，大链 4→3→5，结果 1→2→2→4→3→5。

时间 O(n) **空间** O(1)（只重连原结点）

参考实现

```
def partition_by_heat(head: ListNode | None, x: int) -> ListNode | None:
    lo_dummy, hi_dummy = ListNode(), ListNode()
    lo, hi = lo_dummy, hi_dummy
    while head is not None:
        if head.val < x:
            lo.next = head
            lo = head
        else:
            hi.next = head
            hi = head
        head = head.next
    hi.next = None
    lo.next = hi_dummy.next
    return lo_dummy.next
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按热度稳定排序（sorted(key=heat)）后取前 k	O(n log n)	会把热度 0 排在热度 1 前面，即使热度 1 的块释放得更早；驱逐只关心「冷 / 热」两类，同为冷块时更早释放的更该走。二分类信号决定用分区而不是排序；需要多级优先级时才用排序或堆。
数组上的原地分区（Lomuto / Hoare）	O(n)、O(1)	快速排序里的分区不稳定，会打乱冷块之间的释放顺序。链表上稳定分区天然 O(1) 空间，数组上要稳定则需要 O(n) 辅助数组。
堆按 (热度, 释放序) 取前 k	O(n + k log n)	支持多级热度且只取 k 个时更省；本场景 scan ≤ 200 且 k 通常为 1，分区足够也更简单。

易错点

- 忘了 hi.next = None：大链最后一个结点可能还指向某个小链结点，形成环。
- 新建结点而不是重连原结点：管理器靠 id(node) 找回块号，会 KeyError。
- 比较写成 <= x：题目是严格小于，热度恰好等于 hot 的块应受保护。

模式信号

「按阈值分两组」+「组内保持原相对顺序」：稳定分区，链表上用双哑结点拼接。

相关练习：`kv.lru` `kv.lfu` `kv.has_cycle`

kv.merge_free

kv.merge_free　merge_free_lists

★ Top Interview 150　LC 21 · Merge Two Sorted Lists　链表操作

`nanoserve/kvcache/free_list.py` · `merge_free_lists(a: ListNode | None, b: ListNode | None) -> ListNode | None`

引擎里的角色

空闲池单批归并： `pool.py::FreePool.flush` 若暂存环里只有一批，执行 `self.head = merge_free_lists(self.head, chains[0])` ，把一个序列释放的（已排序）块号并回有序空闲链。参考解的 `sort_free_list` 也以它为归并步骤。重复块号必须全部保留：紧接着的 `drop_double_freed` 要靠它们发现重复释放。demo（18 块）中 109 次 flush 有 108 次走这条路径。

被调用于： `nanoserve/kvcache/pool.py`

题目

合并两条按块号非递减的链表，返回包含全部结点、同样非递减的链表，重复保留；可以直接重新拼接输入结点。两条链各 0 ~ 10^4 长，可为空，块号任意整数。

核心洞见

二路归并：每步取两条链头较小者接到结果尾部。哑结点消除「结果头是谁」的特判；一条链耗尽后，另一条剩余部分整体接上。

算法

1. dummy = ListNode()，tail = dummy。
2. 当 a、b 都非空：若 a.val <= b.val，接 a 并前进 a，否则接 b 并前进 b；tail 前进。
3. tail.next = a 或 b 中非空的那条（都空则 None）。
4. 返回 dummy.next。例：7→4096 与 -3→512 → -3→7→512→4096。

时间 O(m + n)　空间 O(1)

参考实现

```python
def merge_free_lists(a: ListNode | None, b: ListNode | None) -> ListNode | None:
    dummy = ListNode()
    tail = dummy
    while a is not None and b is not None:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a if a is not None else b
    return dummy.next
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归归并 merge(a.next, b) …	O(m + n) 时间、O(m + n) 栈	代码最短，但链长 10^4 时递归深度可达 2×10^4，远超 Python 默认上限 1000。Python 里链表算法一律优先迭代。
两条链转成列表、拼接、sorted、再建链	O((m+n) log(m+n))	Timsort 识别出两段有序 run 后其实接近线性，Python 里常常更快；但要分配新结点，而空闲池希望复用同一批结点、不在热路径上分配。
兄弟练习 kv.merge_arrays	O(m + n)	同一个二路归并，数组版要在 a 的尾部原地写，所以从后往前；链表版改指针即可，从前往后最自然。

易错点

- 用 < 而不是 <=：结果依然有序，但相等元素的来源顺序被交换（失去稳定性）；在归并排序里这会让排序不稳定。
- 循环结束后忘了接上剩余链，丢结点。
- 任一链为空时的处理：哑结点写法自然覆盖，不要额外特判出错。

模式信号

「两条已排序」+「合成一条有序」：二路归并，哑结点 + 尾指针。

相关练习：`kv.merge_free_k` `kv.sort_free` `kv.merge_arrays`

`kv.merge_free_k`

kv.merge_free_k merge_k_free_lists

★ Top Interview 150 · LC 23 · Merge k Sorted Lists · 堆

`nanoserve/kvcache/free_list.py · merge_k_free_lists(lists: list[ListNode | None]) -> ListNode | None`

引擎里的角色

空闲池多批归并： `pool.py::FreePool.flush` 暂存环里有 ⩾ 2 批时执行 `merge_k_free_lists([self.head, *chains])`，空闲链和所有暂存批一次归并（self.head 可能为 None，所以列表里会出现空链）。暂存环容量为 8，因此 k ⩽ 9。管理器惰性 flush（空闲链真空、环满或 sanitize 时），几个已结束序列的释放通常会攒在一起走这里；报告里的 `pool_k_way_merges` 统计次数。

被调用于： `nanoserve/kvcache/pool.py`

题目

合并 k 条非递减链表成一条，重复保留。lists 长度 0 ~ 10^4，元素可为 None，结点总数 ⩽ 10^4，块号在 [-10^4, 10^4]。全空返回 None。期望 O(N log k)。

核心洞见

每一步只需知道 k 个链头中的最小者：用大小为 k 的最小堆维护「每条链的当前头」，弹出最小接到结果上，再把它的后继压回堆。

算法

1. 把每条非空链的头以 (val, i, node) 入堆并 heapify；i 是链下标，用来打破平局，避免比较 ListNode。
2. 循环：弹出 (_, i, node)，接到 tail 后面；若 node.next 非空，压入 (node.next.val, i, node.next)。
3. 堆空后 tail.next = None，返回 dummy.next。
4. 例：[1→4→5, 1→3→4, 2→6]：堆初始 {1a,1b,2}，依次弹出 1,1,2,3,4,4,5,6。

时间 O(N log k) **空间** O(k) 堆

参考实现

```python
def merge_k_free_lists(lists: list[ListNode | None]) -> ListNode | None:
    heap = [(node.val, i, node) for i, node in enumerate(lists) if node is not None]
    heapq.heapify(heap)
    dummy = ListNode()
    tail = dummy
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next is not None:
            heapq.heappush(heap, (node.next.val, i, node.next))
    tail.next = None
    return dummy.next
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
逐条两两归并：res = merge(res, lists[i])	O(kN)	前面的结点被反复扫描。k ⩽ 9 时（本引擎的实际情况）与堆差别很小，但规格的 k 可达 10^4。
分治两两归并（锦标赛式）	O(N log k)、O(log k) 递归或 O(1) 迭代	复杂度相同且不需要堆，直接复用 kv.merge_free；当 k 很大而每条链很短时堆的常数更大，分治更好。
全部取出排序再建链	O(N log N)	最省脑子；N ⩽ 10^4 时实际也够快，但丢掉了「各条已排序」这一信息，且需要额外 O(N) 列表。

易错点

- 堆元组只放 (val, node)：val 相同时 Python 会去比较 ListNode，抛 TypeError；必须插入唯一的平局键 i。
- 把 None 链头放进堆。
- lists 为空或全为 None 时返回 None，而不是哑结点。

模式信号

「k 条已排序序列合并」+「O(N log k)」：k 路归并，最小堆维护各链当前头（或分治两两归并）。

相关练习：`kv.merge_free` `kv.sort_free` `kv.ring_queue`

kv.sort_free

kv.sort_free　sort_free_list

★ Top Interview 150　LC 148 · Sort List　分治/归并

`nanoserve/kvcache/free_list.py` · `sort_free_list(head: ListNode | None) -> ListNode | None`

引擎里的角色

释放批次的排序： `pool.py::FreePool.stage` 把一个序列释放的块号建成链后调 `sort_free_list` ，排好序的批次放进暂存环，等 flush 时才能做线性归并。release 沿回指从尾到头释放，块号进来时大致是逆序或乱序。参考解的归并步骤复用 kv.merge_free。

被调用于： `nanoserve/kvcache/pool.py`

题目

把无序链表按块号非递减排序，重复保留，返回新链头。必须 O(n log n)，链长 0 ~ 5×10^4，块号在 [-10^5, 10^5]。进阶：O(1) 额外空间。

核心洞见

链表不能随机访问，快排选枢轴和堆都不方便，归并排序却很自然：快慢指针找中点断开，递归排两半，再线性归并，归并时只改指针。

算法

1. 空链或单结点直接返回。
2. slow = head，fast = head.next；fast 每次两步、slow 一步，结束时 slow 停在前半段末尾。
3. mid = slow.next；slow.next = None 断开。
4. 返回 merge_free_lists(sort(head), sort(mid))。
5. 例：4→2→1→3 → 拆成 4→2 与 1→3 → 各自排成 2→4、1→3 → 归并得 1→2→3→4。

时间　O(n log n)　空间　O(log n) 递归栈（自顶向下）

参考实现

```python
def merge_free_lists(a: ListNode | None, b: ListNode | None) -> ListNode | None:
    dummy = ListNode()
    tail = dummy
    while a is not None and b is not None:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a if a is not None else b
    return dummy.next


def sort_free_list(head: ListNode | None) -> ListNode | None:
    if head is None or head.next is None:
        return head
    slow, fast = head, head.next
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
    mid = slow.next
```

```
    slow.next = None
    return merge_free_lists(sort_free_list(head), sort_free_list(mid))
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
自底向上归并排序（子段长度 1,2,4…）	O(n log n)、O(1)	满足进阶的 O(1) 空间，没有递归；代价是切段、拼接的指针簿记容易错。自顶向下递归深度只有 $\log_2(5\times10^4) \approx 16$，Python 里不存在栈溢出问题。
拷到列表 sorted() 再写回 val 或重建链	O(n log n)、O(n)	Python 里通常最快（Timsort 是 C 实现，且对部分有序输入近线性）；但写回 val 会改变结点身份与块号的对应关系，重建链则要分配新结点。
插入排序（LC 147）	$O(n^2)$，最好 O(n)	一般情况太慢；但若输入几乎有序（或近乎逆序且每次插到表头），每次插入 O(1)，反而线性。信号是「输入是否基本有序」，规格不保证，所以选归并。

易错点

- fast 从 head 开始：两个结点时 slow 不动，拆分得到 2+0，无限递归。必须 fast = head.next。
- 找到中点后忘了 slow.next = None，前半段仍连着后半段。
- 在链表上做快排：枢轴选得不好时退化为 $O(n^2)$，且不稳定。

模式信号

「链表排序」+「O(n log n)」：链表上归并排序，快慢指针找中点。

相关练习：`kv.merge_free` `kv.merge_free_k` `kv.has_cycle`

---

kv.drop_double_free drop_double_freed

kv.drop_double_free

★ Top Interview 150 · LC 82 · Remove Duplicates from Sorted List II · 链表操作

`nanoserve/kvcache/free_list.py · drop_double_freed(head: ListNode | None) -> ListNode | None`

引擎里的角色

重复释放消毒： `pool.py::FreePool.flush` 归并完成后执行 `self.head = drop_double_freed(self.head)` ，比较前后两份列表，把消失的块号返回；管理器 `_flush → _quarantine` 将它们加入隔离区，永不再分配。sanitizer demo 中向池子再 stage 一次已空闲的块 6， `sanitize()` 返回 [6]；测试 `test_double_free_is_quarantined_and_never_reused` 验证之后连续 10 个新请求都拿不到它。

被调用于： `nanoserve/kvcache/pool.py`

题目

给定非递减链表，凡出现超过一次的值，其所有结点全部删除，只出现一次的保留，返回仍有序的链头（可能为 None）。链长 0 ~ 10^4，值任意整数。

核心洞见

一个块号出现两次，说明两次释放中至少有一次是错误的，它可能仍被某个序列使用；无法判断该信哪一次，就一份都不留。实现上用哑结点 + prev 指针，prev 只在确认当前值唯一时才前进。

算法

1. dummy.next = head，prev = dummy，cur = head。
2. 若 cur.next 存在且与 cur 值相同：记下 v，cur 跳过所有值为 v 的结点，令 prev.next = cur（prev 不动）。
3. 否则 cur 是唯一值：prev = cur，cur 前进。
4. 返回 dummy.next。例：1→1→1→2→3：跳过三个 1，prev.next = 2，结果 2→3（头被删，靠哑结点处理）。

时间 O(n)　空间 O(1)

参考实现

```python
def drop_double_freed(head: ListNode | None) -> ListNode | None:
    dummy = ListNode(0, head)
    prev = dummy
    cur = head
    while cur is not None:
        if cur.next is not None and cur.next.val == cur.val:
            v = cur.val
            while cur is not None and cur.val == v:
                cur = cur.next
            prev.next = cur
        else:
            prev = cur
            cur = cur.next
    return dummy.next
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
LC 83 每个值保留一份	O(n)	这在空闲池里是严重的错误修复：块号 b 留在空闲链上，下次 pop 给新序列，而原持有者仍在往 b 写 KV，两个序列互相覆盖，且没有任何报错。隔离（全部删除）宁可少用一块，也不冒数据损坏的风险。
Counter 计数后过滤	O(n) 时间、O(n) 空间	对无序输入也成立；而输入已有序，相同值必然相邻，一次扫描 O(1) 空间就够。
兄弟练习 kv.dedupe	O(n)	同为「有序 + 去重」，信号差在题面：dedupe 是「每个块号只保留一份」（集合语义），本题是「出现多次的一个不留」（隔离语义）。

易错点

- 删除一段重复后 prev 也前进：可能跳过紧接着的另一段重复（如 3→3→4→4）。
- 不用哑结点：表头本身就是重复值时（1→1→2）无法正确返回新头。
- 只删掉多余的副本、保留一份——那是另一道题。

模式信号

「有序链表」+「凡重复的全部删除」：哑结点 + prev 只在确认唯一时前进。

相关练习：`kv.dedupe`　`kv.merge_free`　`kv.leaked`

kv.compact

kv.compact　remove_freed

★ Top Interview 150　LC 27 · Remove Element　双指针

`nanoserve/kvcache/compact.py` · `remove_freed(slots: list[int], freed: int) -> int`

引擎里的角色

按序列统计记账块数： `occupancy.py::charge_counts` 拷贝 `charge` 数组（下标是块号，值是持有者 sid，0 表示未使用）， `remove_freed(slots, NOT_IN_USE)` 把 0 挤出去， `sorted(slots[:k])` 得到按持有者排序的表，再对每个 sid 用 `owner_range` （kv.seq_range）二分出它的片段长度。report() 的 `charged` 字段和 check() 的「charged blocks != blocks in use」都来自这里。

被调用于： `nanoserve/kvcache/occupancy.py`

题目

原地删除数组中所有等于 freed 的元素：返回剩余个数 k，使 slots[:k] 恰为这些元素（顺序任意），slots[k:] 不作要求，不改变长度。长度 0 ~ 10^4。

核心洞见

读写双指针：读指针扫描全部元素，写指针 k 只在遇到保留元素时写入并前进。因为 k ≤ 读位置，写入永远不会覆盖还没读到的元素。

算法

1. k = 0。
2. 对每个 v：若 v != freed，slots[k] = v，k += 1。
3. 返回 k。
4. 例：[0,1,2,2,3,0,4,2]，freed=2：依次写入 0,1,3,0,4，返回 5。

时间 O(n)　空间 O(1)

参考实现

```python
def remove_freed(slots: list[int], freed: int) -> int:
    k = 0
    for v in slots:
        if v != freed:
            slots[k] = v
            k += 1
    return k
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
列表推导 [v for v in slots if v != freed]	O(n)、O(n)	Python 里最地道，也最快；但会生成新列表。题目约定原地，是在模拟热路径上不分配的定长数组。charge_counts 自己已经拷贝了一份，所以在那里两种写法都可以。
与末尾交换（遇到 freed 就用最后一个元素覆盖，n -= 1）	O(n)	写入次数等于被删元素个数，而读写双指针的写入次数等于保留元素个数；要删的很少时交换法写得少。题目允许顺序任意，两种都合法；需要保持相对顺序时只能用双指针。
反复 slots.remove(freed)	$O(n^2)$	每次 remove 线性查找并左移，0 很多时（大部分块空闲）平方级。

易错点

- 返回新列表或修改长度（del / pop），违反「不改变长度」。
- 边遍历边 remove：迭代器跳过元素。

模式信号

「原地删除」+「返回新长度、尾部内容不作要求」：读写快慢双指针。

相关练习：`kv.dedupe` `kv.seq_range` `kv.merge_arrays`

---

kv.dedupe

kv.dedupe `dedupe_sorted`

★ Top Interview 150 · LC 26 · Remove Duplicates from Sorted Array · 双指针

`nanoserve/kvcache/compact.py` · `dedupe_sorted(ids: list[int]) -> int`

引擎里的角色

`occupancy.py::merge_distinct` 的去重步骤：每并入一条有序列表后执行 `del out[dedupe_sorted(out):]`，让累积结果保持去重且短小。merge_distinct 用于 check()（空闲链 + 各暂存批，长度差即未消毒的重复释放数）、audit_leaks()（所有块表、checkpoint 和 cache 持有的块，共享块会在多张表里出现）和 report()。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

非递减数组原地去重：返回不同值个数 k，使 ids[:k] 为这些值且保持升序，ids[k:] 不作要求，不改变长度。保证长度 ⩾ 1（⩽ 3×10^4）。

核心洞见

有序意味着相同值相邻，只需把每个值和「最后写入的值」比较：不同才写入。写指针 k 之前的前缀就是去重结果。

算法

1. k = 0。
2. 对每个 v：若 k == 0 或 ids[k-1] != v，则 ids[k] = v，k += 1。
3. 返回 k。
4. 例：[0,0,1,1,1,2,2,3,3,4] → 写入 0,1,2,3,4，返回 5。

时间 O(n)　空间 O(1)

参考实现

```
def dedupe_sorted(ids: list[int]) -> int:
    k = 0
    for v in ids:
        if k == 0 or ids[k - 1] != v:
            ids[k] = v
            k += 1
    return k
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
sorted(set(ids))	O(n log n)、O(n)	无序输入时的一行解；但丢掉了「已经有序」的信息，多付排序和新列表的代价。
LC 80（每个值最多保留两份）	O(n)	把比较对象换成 ids[k-2] 即可。这正是要和「最后写入的」比较、而不是和 ids[i-1] 比较的原因：前者能推广到「最多 m 份」。
兄弟练习 kv.drop_double_free	O(n)	本题是集合语义（多张块表共享同一块，算一次）；那题是隔离语义（重复释放的块号一份不留）。同是有序去重，题面决定是否保留一份。

易错点

- 边遍历边 del 或 pop，O(n^2) 且容易越界。
- 返回的是 k 而不是去重后的列表；调用方靠 del out[k:] 截断。

模式信号

「有序数组」+「原地去重、返回长度」：写指针与最后写入值比较。

相关练习：`kv.merge_arrays` `kv.drop_double_free` `kv.compact`

---

kv.merge_arrays

kv.merge_arrays　merge_into

★ Top Interview 150　LC 88 · Merge Sorted Array　双指针

`nanoserve/kvcache/compact.py · merge_into(a: list[int], m: int, b: list[int], n: int) -> None`

引擎里的角色

`occupancy.py::merge_distinct` 的归并步骤：对每条有序列表 b，先 `out.extend([0] * n)` 预留尾部，再 `merge_into(out, m, b, n)` 原地并入，然后用 kv.dedupe 截断。这样多张块表的块号能在一个不断增长的列表里合并，而不必每次新建列表。结果交给 check()、audit_leaks()（再做区间计数与 XOR 定位泄漏）和 report() 的 free_summary。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

a 长 m + n，前 m 个有效且非递减，后 n 个是占位 0；b 长 n 且非递减。原地把 b 并入 a，使 a 成为全部 m + n 个数的非递减排列；不能换列表对象、不能改长度。0 ⩽ m, n ⩽ 10^4，m + n ⩾ 1。

> **核心洞见**
> 从前往后写会覆盖 a 中还没读的元素；从后往前写，空位恰好在尾部：写指针 w = m+n−1 永远 ⩾ i，不会踩到未读元素。

算法

1. i = m−1，j = n−1，w = m+n−1。
2. 当 j ⩾ 0：若 i ⩾ 0 且 a[i] > b[j]，a[w] = a[i]，i 减 1；否则 a[w] = b[j]，j 减 1。w 减 1。
3. j < 0 时结束：a 剩下的前缀本来就在正确位置。
4. 例：a=[0,4,9,0,0], m=3, b=[4,5]：写 9、5、4(b)、4(a)，得 [0,4,4,5,9]；第一个 0 是真实块号，不是占位。

时间 O(m + n) **空间** O(1)

参考实现

```python
def merge_into(a: list[int], m: int, b: list[int], n: int) -> None:
    i, j, w = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and a[i] > b[j]:
            a[w] = a[i]
            i -= 1
        else:
            a[w] = b[j]
            j -= 1
        w -= 1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
a[:] = sorted(a[:m] + b)	O((m+n) log(m+n))、O(m+n)	切片赋值保留了列表对象身份，满足「不换对象」，在 Python 里也很快；但有额外分配。题目考察的是无额外空间的写法。
从前往后归并	O(m+n)、O(m)	需要先拷贝 a 的前 m 个，否则写入会覆盖未读元素。尾部有空位这一信号就是在提示从后往前。
兄弟练习 kv.merge_free	O(m+n)	链表归并只改指针，没有覆盖问题，从前往后即可；数组原地归并必须从空位一侧开始写。

> **易错点**
> - 循环条件写成 i ⩾ 0 and j ⩾ 0，然后只处理 i 的剩余：b 的剩余会丢失。正确做法是只要 j ⩾ 0 就继续。
> - 把 a 里的 0 当占位跳过：只有下标 ⩾ m 的才是占位，真实块号也可能是 0。
> - m = 0 时 i = −1，要靠 i ⩾ 0 的判断短路。

> **模式信号**
> 「原地合并两个有序数组」+「目标数组尾部预留空位」：从后往前三指针。
>
> 相关练习：`kv.dedupe` `kv.merge_free` `kv.compact`

kv.rotate_buffer

kv.rotate_buffer rotate_right

★ Top Interview 150 | LC 189 · Rotate Array | 双指针

`nanoserve/kvcache/ring.py` · `rotate_right(buf: list[int], k: int) -> None`

引擎里的角色

事件日志的时间序转储：`eventlog.py::EventLog.chronological` 复制五个列（ts/tick/kind/block/sid），用 `start = ring_search(ts, ring_min(ts))` 找到最老事件的位置，然后对每个列执行 `rotate_right(col, (n - start) % n)`，最老的事件就转到下标 0。report() 的 recent_events 与测试 test_event_log_wraps_and_stays_searchable 都依赖它。

被调用于：`nanoserve/kvcache/eventlog.py`

题目

把数组原地向右循环移动 k 位（每移一位，末元素到最前），O(1) 额外空间，不换对象、不改长度。1 ⩽ n ⩽ 10^5，0 ⩽ k ⩽ 10^5，k 可能大于 n。

核心洞见

右移 k 位等于把数组看成 [A | B]（B 是最后 k 个）变成 [B | A]。三次反转：整体反转得 [B^r | A^r]，再分别反转前 k 个和后 n−k 个，得 [B | A]。

算法

1. k %= n。
2. 反转整个数组 [0, n−1]。
3. 反转前 k 个 [0, k−1]。
4. 反转剩余 [k, n−1]。
5. 例：[1..7]，k=3 → 7654321 → 567|4321 → 5671234。

时间 O(n) 空间 O(1)

参考实现

```python
def _reverse(buf: list[int], i: int, j: int) -> None:
    while i < j:
        buf[i], buf[j] = buf[j], buf[i]
        i += 1
        j -= 1


def rotate_right(buf: list[int], k: int) -> None:
    n = len(buf)
    k %= n
    _reverse(buf, 0, n - 1)
    _reverse(buf, 0, k - 1)
    _reverse(buf, k, n - 1)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
buf[:] = buf[-k:] + buf[:-k]	O(n)、O(n)	Python 一行，C 速度；切片赋值也保持了对象身份。只是不满足 O(1) 额外空间。注意它同样要先 k %= n：k ⩾ n 时 buf[-k:] 是整个数组、buf[:-k] 为空，结果原样不动，旋转量就错了。
环状替换（juggling，按 gcd(n,k) 条环逐个搬）	O(n)、O(1)	每个元素恰好写一次（n 次写），三次反转约 2n 次交换；写入代价高时更好，但环的计数与终止条件容易写错。
collections.deque.rotate(k)	O(k)	如果日志本来就用 deque 存，直接旋转最省事；这里列是普通 list，转成 deque 再转回要 O(n) 额外空间。

易错点

- 忘了 k %= n：k > n 时反转区间越界。
- 方向弄反：先反转前 n−k 个再整体反转得到的是左移。
- 返回新列表，调用方手里的列对象没变。

模式信号

「数组循环移位」+「O(1) 额外空间」：三次反转。

相关练习：`kv.ring_head` `kv.ring_search` `kv.ring_queue`

---

kv.ring_search

kv.ring_search `ring_search`

★ Top Interview 150 · LC 33 · Search in Rotated Sorted Array · 二分查找

`nanoserve/kvcache/ring.py` · `ring_search(buf: list[int], target: int) -> int`

引擎里的角色

按事件号查日志：`eventlog.py::EventLog.find(ts)` 调 `ring_search(self._stamps(), ts)`；管理器 `describe(b)` 用它取出块 b 的最后一次事件（leak 和 double-free 诊断里那条 last_event），`chronological()` 用它定位最老事件的下标。时间戳严格递增、写指针绕圈覆盖，所以缓冲回绕后时间戳列就是一个被旋转过的有序数组；未回绕时 `_stamps` 返回有序前缀（旋转 0 位）。被覆盖的事件查不到，返回 None。

被调用于：`nanoserve/kvcache/eventlog.py`

题目

严格升序数组在未知位置旋转后（也可能旋转 0 位），查找 target，返回下标或 −1。元素互不相同，必须 O(log n)，1 ⩽ n ⩽ 5000。

核心洞见

任取 mid，[lo, mid] 和 [mid, hi] 中至少有一半是有序的（不跨越断点）。比较 buf[lo] ⩽ buf[mid] 判断哪半有序，再看 target 是否落在有序那半的值域里，决定丢掉哪一半。

算法

1. lo = 0，hi = n−1，当 lo ⩽ hi：mid = (lo+hi)//2，命中即返回。
2. 若 buf[lo] ⩽ buf[mid]（左半有序）：target 在 [buf[lo], buf[mid]) 内则 hi = mid−1，否则 lo = mid+1。
3. 否则右半有序：target 在 (buf[mid], buf[hi]] 内则 lo = mid+1，否则 hi = mid−1。
4. 循环结束返回 −1。
5. 例：[4,5,6,7,0,1,2] 找 0：mid=3（7），左半 4..7 有序但 0 不在其中 → lo=4；mid=5（1），左半 [0,1] 有序且 0 在 [0,1) → hi=4；mid=4 命中。

时间 O(log n)　**空间** O(1)

参考实现

```python
def ring_search(buf: list[int], target: int) -> int:
    lo, hi = 0, len(buf) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if buf[mid] == target:
            return mid
        if buf[lo] <= buf[mid]:
            if buf[lo] <= target < buf[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:
            if buf[mid] < target <= buf[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
先用 kv.ring_head 找断点，再在对应有序段 bisect	两次 O(log n)	每一步都是标准二分，更难写错，还能直接用 bisect 模块；只是多一次二分。chronological() 实际上就是这样「先找最小值再定位」的。
已知写指针时直接算下标	O(1)	时间戳是连续整数，若知道 cursor 与 total，下标 = (cursor − (total − ts) − 1) mod capacity。日志的设计前提是「读者只看时间戳列就能恢复顺序」（例如拿到一份没有表头的转储），所以走二分；有元数据时直接算更好。
线性扫描 / list.index	O(n)	容量最多 5000，实际也不慢，但违反 O(log n) 规格；describe 在泄漏诊断里对每个块都调用。

易错点

- 写成 buf[lo] < buf[mid]：lo == mid 时左半只有一个元素也算有序，必须用 ⩽，否则走错分支。
- 区间端点开闭搞错：左半判断要 buf[lo] ⩽ target < buf[mid]，右半要 buf[mid] < target ⩽ buf[hi]。
- 有重复元素时此法失效（LC 81 需要额外收缩）；本题保证互不相同，时间戳严格递增正好满足。

模式信号

「旋转过的有序数组」+「O(log n) 查找」：每次判断哪半有序，再看目标在不在那半。

相关练习：`kv.ring_head`　`kv.rotate_buffer`　`kv.seq_range`

kv.ring_head

kv.ring_head ring_min

★ Top Interview 150 | LC 153 · Find Minimum in Rotated Sorted Array | 二分查找

`nanoserve/kvcache/ring.py` · `ring_min(buf: list[int]) -> int`

引擎里的角色

找最老事件： `eventlog.py::EventLog.oldest()` 返回 `ring_min(stamps)` ，即缓冲里仍保留的最早事件号（report() 的 oldest_event，demo 中为 #232，共 487 个事件、容量 256）。 `chronological()` 也先求最小值，再用 ring_search 找它的下标，作为 rotate_right 的位移量。

被调用于： `nanoserve/kvcache/eventlog.py`

题目

严格升序数组旋转若干位（可能 0 位或 n 位）后，返回最小值本身（不是下标）。元素互不相同，1 ⩽ n ⩽ 5000，必须 O(log n)。

核心洞见

把 buf[mid] 和 buf[hi] 比较：buf[mid] > buf[hi] 说明断点（最小值）在 mid 右侧；否则 [mid, hi] 有序，最小值在 mid 或其左侧。区间不断收缩到一个点。

算法

1. lo = 0，hi = n−1，当 lo < hi：mid = (lo+hi)//2。
2. 若 buf[mid] > buf[hi]，lo = mid+1（mid 不可能是最小值）。
3. 否则 hi = mid（mid 可能就是最小值，不能排除）。
4. 返回 buf[lo]。
5. 例：[3,4,5,1,2]：mid=2（5 > 2）→ lo=3；mid=3（1 ⩽ 2）→ hi=3；返回 1。

时间 O(log n) 空间 O(1)

参考实现

```python
def ring_min(buf: list[int]) -> int:
    lo, hi = 0, len(buf) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if buf[mid] > buf[hi]:
            lo = mid + 1
        else:
            hi = mid
    return buf[lo]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
min(buf)	O(n)	最直接，n ⩽ 5000 时也很快，但没有利用旋转有序结构，违反规格。
改为与 buf[lo] 比较	O(log n)	未旋转时会出错：[11,13,15,17] 中 buf[mid] ⩾ buf[lo] 总是成立，往右走会错过下标 0 的最小值。与 hi 比较对旋转 0 位和 n 位都成立，这正是规格特意给出 [11,13,15,17] 示例的原因。
返回下标而不是值	O(log n)	chronological() 真正需要的是最老事件的下标，现在要先 ring_min 再 ring_search，两次二分；若返回下标一次就够。题目沿用 LC 153 的接口返回值。

易错点

- else 分支写成 hi = mid−1：mid 可能就是最小值，会被跳过。
- 循环条件写成 lo ⩽ hi 搭配 hi = mid，lo == hi 时死循环。
- 有重复元素时 buf[mid] == buf[hi] 无法判断方向（LC 154 需 hi −= 1）；本题互不相同。

模式信号

「旋转有序数组」+「找最小值 / 逻辑起点」：mid 与右端比较，收缩到断点。

相关练习：`kv.ring_search` `kv.rotate_buffer` `kv.ring_queue`

`kv.longest_run`

kv.longest_run longest_free_run

★ Top Interview 150 · LC 128 · Longest Consecutive Sequence · 哈希表

`nanoserve/kvcache/allocator.py · longest_free_run(ids: list[int]) -> int`

引擎里的角色

碎片指标： `occupancy.py::free_summary(free_ids)` 返回 `(free_ranges(ids), longest_free_run(ids))` ，进入 report() 的 `largest_free_run` ，表示池子此刻能满足的最大一次连续块分配。demo（18 块）结束时只剩块 15 空闲，largest_free_run = 1。测试 test_report_agrees_with_brute_force 用暴力结果核对它。

被调用于： `nanoserve/kvcache/occupancy.py`

题目

无序块号中（可重复），求值连续的最长一段的长度，与位置无关，重复只算一次。空返回 0，n ⩽ 10^5，值在 [−10^9, 10^9]，必须 O(n)（不能排序）。

核心洞见

放进集合后，只从「段的起点」（v−1 不在集合中）开始向上数。每个元素至多在一次向上计数中被访问，总共 O(n)。

算法

1. s = set(ids)，best = 0。
2. 遍历 s 中每个 v：若 v−1 在 s 中，跳过（不是起点）。
3. 否则从 v 向上数：end 一直加 1 直到 end+1 不在 s 中，best = max(best, end − v + 1)。
4. 返回 best。
5. 例：[100,4,200,1,3,2]：起点只有 100、200、1；从 1 数到 4，长度 4。

时间 O(n) 平均（哈希）　空间 O(n)

参考实现

```python
def longest_free_run(ids: list[int]) -> int:
    s = set(ids)
    best = 0
    for v in s:
        if v - 1 not in s:
            end = v
            while end + 1 in s:
                end += 1
            best = max(best, end - v + 1)
    return best
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
排序后线性扫描（跳过重复）	O(n log n)	违反规格；但在引擎的实际调用点，free_ids 来自 merge_distinct，已经严格升序且去重，此时一次线性扫描就是 O(n)，比建集合更省。题面的信号是「无序」，调用点的信号是「已排序」，这就是 kv.free_ranges 能直接线性扫描的原因。
并查集：把 v 与 v±1 合并，取最大集合	O(n α(n))	能支持增量插入（空闲块一个个回来时实时维护最长段），一次性查询时比集合法复杂。
对每个元素都向上数（不判断起点）	$O(n^2)$	全连续输入时每个元素都数一遍整段，平方级。

易错点

- 遍历原列表而不是集合：大量重复的起点会让同一段被反复数，退化成 $O(n^2)$。
- 忘了「v−1 不在集合中」的起点判断。
- 空输入返回 0；max() 用在空序列上会抛异常。

模式信号

「无序」+「最长连续值段」+「O(n)、不能排序」：哈希集合，只从段起点向上扩展。

相关练习：`kv.free_ranges`　`kv.dedupe`　`kv.merge_arrays`

kv.free_ranges

kv.free_ranges free_ranges

★ Top Interview 150 | LC 228 · Summary Ranges | 双指针

`nanoserve/kvcache/allocator.py` · `free_ranges(ids: list[int]) -> list[str]`

引擎里的角色

`occupancy.py::free_summary` 把空闲块号压缩成区间串，由 `manager.py::BlockManager.report` 调用：先用 `merge_distinct(self.pool.contents())` 得到空闲链表 + 暂存批次的有序去重块号，再交给 `free_ranges`，结果写进报告的 `free_ranges` 字段，和 `largest_free_run`（kv.longest_run）一起作为碎片化指标。`nanoserve kv demo` 结束时打印 `free ranges ['15'] largest free run 1`：18 个块里只有块 15 空闲，其余 17 个都留作 GPU 缓存。报告每次只调用一次，O(n) 足够。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

输入严格升序、互不相同的整数列表 `ids`（长度 0~10^4，值在 [-2^31, 2^31-1]）。把它切成若干段极大连续段（相邻差 1 归为一段），按升序输出：a≠b 写 `"a->b"`，a==b 写 `"a"`，负数照常带负号。空输入返回 []。例：[0,1,2,4,5,7] → ["0->2","4->5","7"]。

核心洞见

输入已经有序，所以“连续段”就是数组里的极大子段：段内满足 `ids[j+1] == ids[j] + 1`。用双指针一次扫完：i 定段首，j 尽量往右延伸，然后输出这一段，令 i = j+1。

算法

1. i = 0；当 i < n 时，令 j = i。
2. 当 j+1 < n 且 ids[j+1] == ids[j] + 1 时 j += 1（把段向右延伸）。
3. i == j 时输出 str(ids[i])，否则输出 f"{ids[i]}->{ids[j]}"。
4. i = j + 1，进入下一段。例：[0,2,3,4,6,8,9] 依次切出 0 | 2..4 | 6 | 8..9。

时间 O(n)：每个元素只被 j 越过一次　空间 O(1) 额外空间（不计输出）

参考实现

```python
def free_ranges(ids: list[int]) -> list[str]:
    out: list[str] = []
    i, n = 0, len(ids)
    while i < n:
        j = i
        while j + 1 < n and ids[j + 1] == ids[j] + 1:
            j += 1
        out.append(str(ids[i]) if i == j else f"{ids[i]}->{ids[j]}")
        i = j + 1
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.longest_run（LC 128）	O(n)，用哈希集合	两题共用同一份空闲块号，信号在于输入是否有序：longest_run 的输入无序、可能重复，而且只要最长段的长度，于是用集合并只从段首（v-1 不在集合里）开始数；free_ranges 的输入已严格升序，相邻比较就够了，不需要集合。如果拿到的是无序输入还要输出区间串，就先排序（O(n log n)）再套本题。
兄弟练习 kv.free_tracker（LC 352，在线版）	add O(log k) 查找 + O(k) 插入，ranges O(k)	块号一个个到达、而且到达过程中要多次读区间时用 tracker；一次性拿到整份有序快照（report 的情形）时，本题一遍扫描最简单、常数最小。
itertools.groupby(enumerate(ids), key=lambda p: p[1]-p[0])	O(n)	连续段里 值-下标 是常数，可以用它分组，写法短，但不如双指针直观，空段和单元素段的格式化也还得自己处理。

易错点

- 用 `ids[j+1] - ids[j] == 1` 判断相邻：Python 里没问题，换到 Java/C++，-2^31 与 2^31-1 相减会溢出，要写成 `ids[j+1] == ids[j] + 1`，或者先转成 long。
- 单元素段要输出 `"7"`，不能输出 `"7->7"`。
- 空列表返回 []；循环条件里先写 j+1 < n，再访问 ids[j+1]。

模式信号

“有序、不重复”加上“把连续的值压缩成区间描述”：输入有序，相邻比较就能分段。

相关练习：`kv.longest_run` `kv.free_tracker` `kv.address_ranges`

`kv.seq_range`

kv.seq_range owner_range

★ Top Interview 150 · LC 34 · Find First and Last Position of Element in Sorted Array · 二分查找

`nanoserve/kvcache/allocator.py` · `owner_range(owners: list[int], seq_id: int) -> list[int]`

引擎里的角色

`occupancy.py::charge_counts` 统计每个序列被计费（charge）的在用块数：先复制 `charge` 数组，用 `remove_freed`（kv.compact）原地挤掉值为 0 的“未使用”槽，再排序得到按 owner 非递减的表，然后用 `owner_range(table, table[i])` 找出该 owner 的首尾下标，计数 = last-first+1，再令 i = last+1 跳到下一位 owner。调用方是 `manager.py::BlockManager.check`（检查 Σ计数 == in_use）和 `BlockManager.report` 的 `charged` 字段。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

`owners` 非递减（长度 0~10^5），求值 `seq_id` 第一次和最后一次出现的下标 `[first, last]`，不存在返回 [-1,-1]。要求 O(log n)。例：[5,7,7,8,8,10], 8 → [3,4]；6 → [-1,-1]；[] → [-1,-1]。

核心洞见

两个边界都能用 lower_bound 求：first = lower_bound(x)，即第一个 ⩾x 的位置；last = lower_bound(x+1) - 1，即第一个 ⩾x+1 的位置往左退一格。只要写一个二分函数，调用两次。

算法

1. lower_bound(x)：lo=0, hi=len（半开区间 [lo,hi)）；当 lo<hi 时取 mid，若 owners[mid] < x 则 lo=mid+1，否则 hi=mid；返回 lo。
2. first = lower_bound(seq_id)；若 first == len 或 owners[first] != seq_id，返回 [-1,-1]。
3. last = lower_bound(seq_id + 1) - 1，返回 [first, last]。
4. 走一遍：[5,7,7,8,8,10] 找 8，lower_bound(8)=3，lower_bound(9)=5，结果 [3,4]。

时间 O(log n)（两次二分） **空间** O(1)

参考实现

```python
def owner_range(owners: list[int], seq_id: int) -> list[int]:
    def lower_bound(x: int) -> int:
        lo, hi = 0, len(owners)
        while lo < hi:
            mid = (lo + hi) // 2
            if owners[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        return lo

    first = lower_bound(seq_id)
    if first == len(owners) or owners[first] != seq_id:
        return [-1, -1]
    return [first, lower_bound(seq_id + 1) - 1]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
二分找到任意一个 x 后向两边线性扩展	最坏 O(n)	当一个序列占了整张表（长 prompt 独占大部分块）时会退化成线性扫描。题目要求 O(log n)，就是为了逼你把两个边界都二分出来。
bisect.bisect_left / bisect_right	O(log n)	first = bisect_left(a,x)，last = bisect_right(a,x)-1，和本解完全等价；manager.py 自己就大量使用 bisect（ `_bisect_leaks` 、 `owner_at` ）。手写一遍是为了吃透半开区间不变量。
直接对 charge 用 Counter 计数	O(k)	单看引擎需求，Counter 更快（charge_counts 里的排序已经要 O(k log k)）；它用排序 + 二分，是想让“按 owner 排序的块表上的切片”这个练习有真实的调用点。如果要反复查询同一张排好序的表，二分更合适；只统计一次的话，哈希计数更合适。

易错点

- 找到 first 后必须检查 owners[first] == seq_id，否则会把“插入位置”误当成命中位置。
- 开区间写法和闭区间写法混用（hi=len 配 lo<=hi）会死循环或越界，选一种不变量写到底。
- last 别再写一个“找最后一个 ≤x”的二分，复用 lower_bound(x+1)-1 更不容易错（值是整数，所以 x+1 成立）。

模式信号

“有序数组”“首尾位置”“O(log n)”：用 lower_bound 和 upper_bound 两次二分。

相关练习：`kv.page_lookup` `kv.versioned` `kv.snapshot`

---

kv.page_lookup

kv.page_lookup page_exists

★ Top Interview 150 · LC 74 · Search a 2D Matrix · 二分查找

`nanoserve/kvcache/page_table.py · page_exists(table: list[list[int]], addr: int) -> bool`

引擎里的角色

`occupancy.py::PageTable` 为每个在用块建一行 `[b*bs, …, b*bs+bs-1]`，行按块号升序排列，所以整张表逐行展开后全局有序。`manager.py::BlockManager.check` 对每个运行中序列的每个已存 token，先从 token arena 读出 slot，再要求 `pages.mapped(slot)` 为真，也就是“注意力内核将要读的 KV 槽位确实映射到某个在用块”。`mapped` 内部调用 `page_exists`，表为空时由调用方用 `bool(self.rows)` 挡掉。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

m×n 矩阵（m,n ≥ 1，m · n ≤ 10^6），每行非递减，且每行首元素大于上一行末元素。判断 `addr` 是否存在，要求 O(log(m · n))。例：[[1,3,5,7],[10,11,16,20],[23,30,34,60]] 查 3 → True，查 13 → False。

核心洞见

这两个条件合起来，就是一个被折成 m 行的一维有序数组。在虚拟下标 [0, m · n-1] 上做一次普通二分，取元素时把下标 k 映射成 table[k // n][k % n]。

算法

1. m = len(table)，n = len(table[0])；lo = 0，hi = m · n - 1（闭区间）。
2. 当 lo ≤ hi 时：mid = (lo+hi)//2，v = table[mid // n][mid % n]。
3. v == addr 返回 True；v < addr 则 lo = mid+1，否则 hi = mid-1。
4. 循环结束返回 False。例：上表查 13，依次取 k=5(11)、k=8(23)、k=6(16)，区间变空，返回 False。

时间 O(log(m · n)) 空间 O(1)

参考实现

```python
def page_exists(table: list[list[int]], addr: int) -> bool:
    m, n = len(table), len(table[0])
    lo, hi = 0, m * n - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        v = table[mid // n][mid % n]
```

```
        if v == addr:
            return True
        if v < addr:
            lo = mid + 1
        else:
            hi = mid - 1
    return False
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
两段二分：先在行首二分定位行，再在行内二分	O(log m + log n) = O(log mn)	复杂度相同，但要写两个二分，还要处理 addr 小于第一行首元素的边界。虚拟下标法只有一个循环，更不容易错。如果各行长度不同（锯齿页表），就只能用两段二分。
从右上角走楼梯（LC 240 的解法）	O(m + n)	LC 240 只保证“行有序、列有序”，没有“下一行整体更大”这一条，所以不能展平，只能走楼梯。本题条件更强，O(m+n) 是浪费。题面里出现“下一行首元素大于上一行末元素”，就说明可以展平。
引擎内的 O(1) 捷径：b = slot // bs，看 refs[b] > 0	O(1)	在 nano-serve 里，slot 的结构已知，直接除以块大小就能判断。PageTable 是故意做成的独立交叉校验：它只依赖“在用块集合”，不依赖 slot 的编码方式，arena 写错了地址时它会报出来。

易错点

- 列数是 len(table[0])，不是 len(table)。
- 下标换算用 mid // n 和 mid % n（除以列数），不是除以行数。
- hi 取 m · n - 1 配 lo ⩽ hi；取 m · n 就要配 lo < hi，两种写法别混用。

模式信号

二维矩阵“逐行展开后整体有序”加上 O(log(mn)) 的要求：展平成一维再二分。

相关练习：`kv.seq_range` `kv.versioned`

---

kv.popcount

kv.popcount popcount

★ Top Interview 150 · LC 191 · Number of 1 Bits · 位运算

`nanoserve/kvcache/bitmap.py · popcount(n: int) -> int`

引擎里的角色

`occupancy.py::Bitmap` 每块 1 位，打包成 31 位的字（`WORD = 31`），`count()` 返回 `sum(popcount(w) for w in self.words if w)`。`manager.py::BlockManager.check` 要求 `bitmap.count() == in_use + len(cached)`，`report` 的 `used` 字段也来自它。选 31 位而不是 32 位，是为了让每个非零字都落在 popcount 的合法输入范围 [1, 2^31-1] 内；调用方再用 `if w` 跳过零字，满足 n ⩾ 1。

被调用于：`nanoserve/kvcache/occupancy.py`

题目

给定 1 ⩽ n ⩽ 2^31-1，返回其二进制中 1 的个数。禁止使用 `bin(n).count('1')` 和 `int.bit_count`，要用位运算实现。例：11 → 3，128 → 1，2147483645 → 30。

核心洞见

`n & (n-1)` 会清掉 n 最低位的那个 1（Brian Kernighan 技巧）：n-1 把最低位的 1 变成 0，并把它右边的 0 全变成 1，再和 n 相与，这些位都归零。所以循环次数正好等于 1 的个数。

算法

1. count = 0。
2. 当 n ≠ 0 时：n &= n - 1，count += 1。
3. 返回 count。例：11=0b1011 → 0b1010 → 0b1000 → 0，共 3 次。

时间 O(popcount(n)) ⩽ 31 次迭代　**空间** O(1)

参考实现

```python
def popcount(n: int) -> int:
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
逐位移位测试：count += n & 1；n >>= 1	O(位宽) = 31 次	永远跑满位宽。位图很稀疏时（每个字只有一两个块在用），Kernighan 只跑一两次。字很满时两者差不多。
查表：预计算 256 项的字节 popcount，每个字查 4 次	O(1)，4 次查表 + 256 项表	需要批量统计大量字时最快（硬件没有 POPCNT 指令的平台常用）。对只有几十个字的块池来说，建表本身就不划算。
SWAR 并行计数（0x55555555、0x33333333… 掩码逐层相加）	O(1)，约 12 条位运算	无分支、常数时间，是 C/CUDA 里 __popc 的软件替代。Python 里每条运算都是大整数对象操作，不比 Kernighan 快，可读性也差。
兄弟练习 kv.leaked	—	同为位运算题：popcount 利用 n&(n-1) 消掉最低位的 1，leaked 利用 a^a=0 抵消成对元素。两者都是把“集合运算”压进整数位里。

易错点

- Python 的负整数是无限长补码，`n &= n-1` 对负数永远不会归零。这也是位图用 31 位字、保证输入为正的原因之一。
- 从 Java 移植时要用无符号右移 `>>>`，否则符号位会被复制，导致死循环。
- 调用方必须跳过 0 字（规格要求 n ⩾ 1）；实现本身对 0 也会正确返回 0。

模式信号

“二进制中 1 的个数”或“统计位图中被占用的块”，并且禁止库函数：用 n & (n-1)。

相关练习：`kv.leaked`

kv.leaked

kv.leaked find_leaked

★ Top Interview 150 | LC 136 · Single Number | 位运算

`nanoserve/kvcache/leak.py` · `find_leaked(events: list[int]) -> int`

引擎里的角色

`manager.py::BlockManager.audit_leaks` 比较两份有序块号：`marked`（位图里置位的块）和 `held`（从块表、checkpoint、缓存能到达的块，经 `merge_distinct` 合并后，再按位图过滤，所以 held ⊆ marked）。`_bisect_leaks(lo, hi)` 先用 bisect 数出区间 [lo,hi) 里缺了几块：缺 0 块直接返回；缺 1 块时，把 `marked[i:j] + held[k:m]` 拼起来，正常块各出现两次、泄漏块只出现一次，交给 `find_leaked` 异或出来；缺 2 块以上就对半二分。demo 的 sanitizer 段注入泄漏 [5,7,8]，审计结果正是 [5,7,8]。

被调用于：`nanoserve/kvcache/manager.py`

题目

数组里恰有一个数出现一次，其余每个数恰好出现两次（长度 1~3 • 10^4，值在 [-3 • 10^4, 3 • 10^4]），找出出现一次的那个。时间 O(n)，额外空间 O(1)，不能用 dict/set/Counter。例：[4,1,2,1,2] → 4。

核心洞见

异或满足交换律和结合律，且 a^a = 0、a^0 = a。把所有数异或起来，成对的数两两抵消，剩下的就是落单的那个。

算法

1. x = 0。
2. 对每个 v：x ^= v。
3. 返回 x。例：4^1^2^1^2 = 4^(1^1)^(2^2) = 4。

时间 O(n) 空间 O(1)

参考实现

```python
def find_leaked(events: list[int]) -> int:
    x = 0
    for v in events:
        x ^= v
    return x
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
哈希集合：出现就加入、再出现就删除，最后剩下的就是答案	O(n) 时间，O(n) 空间	直观，但违反 O(1) 空间约束。允许额外空间、且要找的是“出现奇数次的所有元素”时，它反而更通用。
求和之差：sum(marked) - sum(held)	O(n)，O(1) 空间	在引擎的这个场景里也行，因为两份列表是分开给的，且 held ⊆ marked。但在定长整数语言里会溢出，而且要求事先知道哪份是“全集”。异或作用在混合后的单一列表上，对顺序、来源都不敏感，也不存在溢出问题。
缺 ⩾2 块时：LC 260 按 a^b 的某一位分组	O(n)	两块泄漏时异或结果是 a^b，可以按其最低位的 1 把元素分成两组，各组再异或一次。引擎改用按块号区间对半二分，因为它能推广到任意个泄漏块，而 bisect 计数本来就要做。
LC 137（其余出现三次）	O(32n)	成对抵消不再成立，要对每一位按出现次数模 3 计数，或者用 ones/twos 两个变量的状态机。信号是“其余元素出现几次”：两次就直接异或。

易错点

- x 的初值必须是 0（异或的单位元），不能用 events[0] 初始化后再从下标 0 开始循环。
- 前提是“其余恰好两次”：引擎只在 missing == 1 时调用；缺 2 块直接异或会得到 a^b 这样的垃圾值。
- held 里如果有位图没置位的块（悬空引用），就会破坏成对假设，所以 audit_leaks 先用 bitmap.test 过滤 held。

模式信号

“其余元素都恰好出现两次”加上 O(1) 额外空间：把所有元素异或起来。

相关练习：`kv.popcount` `kv.free_ranges`

---

kv.random_evict

kv.random_evict RandomEvictSet

★ Top Interview 150 · LC 380 · Insert Delete GetRandom O(1) · 哈希表

`nanoserve/kvcache/evict.py` · `class RandomEvictSet: insert(x) -> bool; remove(x) -> bool; pick() -> int`

引擎里的角色

`tiers.py::RandomTier` 是 `eviction="random"` 时的主机层： `capacity` 个槽位， `slot_of`（BlockHashMap，kv.block_hash）记录“哈希 → 槽”， `occupied`（RandomEvictSet）存放被占用的槽号。 `put` 新键而没有空槽时， `_pick()` 等概率选一个受害槽，再依次 `slot_of.remove` 、 `occupied.remove` ，把槽还给 spare 后复用。 `_pick` 在调用前后切换全局 `random` 的状态，换成自己带种子的状态，所以结果可复现，也不扰动别人（见 `test_random_eviction_is_seeded_and_leaves_global_random_alone` ）。

被调用于： `nanoserve/kvcache/tiers.py`

题目

维护一个元素互不相同的集合，支持 `insert(x)`（新加入返回 True，已存在返回 False）、`remove(x)`（存在则删除并返回 True，否则返回 False）、`pick()`（等概率返回一个元素，不删除，保证集合非空，必须用全局 `random` 模块），三个操作均摊 O(1)。

> **核心洞见**
> 等概率随机要求能 O(1) 按下标访问，所以需要数组；O(1) 删除要求能 O(1) 找到位置，所以需要哈希表记录“值 → 下标”。删除时把末尾元素搬到被删元素的位置上，再 pop 末尾，数组保持紧凑，没有空洞。

算法

1. 数据：items 列表，pos 字典（值 → 下标）。
2. insert(x)：x 已在 pos 中则返回 False；否则 pos[x] = len(items)，items.append(x)。
3. remove(x)：i = pos.pop(x)，不存在则返回 False；last = items.pop()；若 i < len(items)，说明删的不是末尾，令 items[i] = last、pos[last] = i。
4. pick()：返回 items[random.randrange(len(items))]。
5. 例：items=[1,2,3]，remove(1)：last=3 搬到下标 0，变成 [3,2]，pos={3:0, 2:1}。

时间 insert/remove 均摊 O(1)，pick O(1)　**空间** O(n)

参考实现

```python
class RandomEvictSet:
    def __init__(self) -> None:
        self.items: list[int] = []
        self.pos: dict[int, int] = {}

    def insert(self, x: int) -> bool:
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True

    def remove(self, x: int) -> bool:
        i = self.pos.pop(x, None)
        if i is None:
            return False
        last = self.items.pop()
        if i < len(self.items):
            self.items[i] = last
            self.pos[last] = i
        return True

    def pick(self) -> int:
        return self.items[random.randrange(len(self.items))]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
只用 set，pick 时 random.choice(list(s))	pick O(n)	集合没有下标，每次 pick 都要物化成列表。淘汰频繁时就成了热路径上的 O(n)。
兄弟练习 kv.lru（哈希表 + 双向链表）	get/put O(1)	LRU 每次命中都要改链表（写操作）；random 命中时是纯读，没有任何元数据更新，所以并发友好，常数也最小。代价是不利用访问规律：demo 的 hot vs scan 轨迹里，random 只恢复了 2 块（LRU 4 块，LFU 11 块）；shifting hot set 里 random 和 LRU 同为 33 块。
兄弟练习 kv.lfu	get/put O(1)，常数大	LFU 要维护频率桶，适合“长期热点 + 偶发扫描”的负载；random 的优势是对抗性的访问模式（例如循环扫描刚好比容量大 1 时，LRU 每次都命中失败）打不败它。选择信号：负载有稳定热点用 LFU，最近性强用 LRU，没有规律或要求最便宜用 random。

易错点

- 删的恰好是末尾元素时，pop 之后 i == len(items)，不能再写 items[i]（越界），也不能把 pos[last] 写回去，那会让已删除的键复活。
- 先 pos.pop(x) 再搬运 last。顺序反了，当 x 就是 last 时，会把刚删掉的键重新写回 pos。
- 题目要求用全局 random（引擎靠切换全局状态来注入种子），自己 new 一个 Random() 会让 RandomTier 的可复现性失效。

模式信号

“O(1) 插入、删除，并且等概率随机取一个”：用数组 + 值到下标的哈希，删除时和末尾交换。

相关练习：`kv.lfu` `kv.lru` `kv.block_hash`

---

kv.lfu

kv.lfu LFUBlockCache

其它 LeetCode · LC 460 · LFU Cache · 设计（数据结构）

`nanoserve/kvcache/lfu.py` · `class LFUBlockCache(capacity: int): get(key: int) -> int; put(key: int, value: int) -> None`

引擎里的角色

`tiers.py::make_host_tier("lfu", capacity)` 返回 `LFUBlockCache`，用作被逐出 GPU 的块在主机内存里的那一层。`manager.py::BlockManager._evict` 执行 `self.host.put(hash_of[b], checksum(key[b]))`，之后 prompt 在 GPU 上没命中时，用 `host.get(h)` 恢复。测试 `test_eviction_policies_differ_on_crafted_reuse_traces` 和 demo 的 policies 段（主机容量 4 块）给出对比：hot vs scan 轨迹里 LFU 恢复 11 块，LRU 只有 4 块；shifting hot set 里反过来，LRU 33 块，LFU 26 块。

被调用于：`nanoserve/kvcache/tiers.py`

题目

容量为 capacity 的缓存。每个键有使用次数：插入时为 1，之后每次 get 命中或 put 覆盖都 +1，并标记为最近使用。get 不存在返回 -1，且不改变任何状态。put 新键时如果已满，先淘汰使用次数最少的键，次数并列时淘汰最久未用的那个，再插入。get 和 put 都要均摊 O(1)。

> **核心洞见**
> 按频率分桶，每个桶是一个按最近性排序的有序字典（OrderedDict 里先插入的就是最旧的），再维护全局 min_freq。min_freq 不需要搜索：触碰一个键只会让它的频率 +1，所以 min_freq 最多 +1，且只在旧桶被搬空时才变；新键插入时 min_freq 一定重置为 1。

算法

1. 数据：vals（键→值）、freq（键→次数）、buckets（次数→OrderedDict）、min_freq。
2. _touch(key)：f = freq[key]，从 buckets[f] 删除 key；若桶空了，删掉该桶，并且若 min_freq == f 则令 min_freq = f+1；然后 freq[key] = f+1，把 key 追加到 buckets[f+1] 的末尾。
3. get：不存在返回 -1；否则 _touch 后返回值。
4. put 已有键：更新值并 _touch。put 新键：如果已满，从 buckets[min_freq] 执行 popitem(last=False) 弹出最旧的键，删除它的 vals/freq；然后插入 freq=1 的新键，令 min_freq = 1。
5. 例（cap=2）：put1, put2, get1（1 的频率为 2），put3 淘汰频率最低的 2；get3 后 1、3 的频率都是 2，put4 淘汰两者中更旧的 1。

时间 get/put 均摊 O(1)　**空间** O(capacity)

参考实现

```python
class LFUBlockCache:
    def __init__(self, capacity: int) -> None:
        self.capacity = capacity
        self.vals: dict[int, int] = {}
        self.freq: dict[int, int] = {}
        self.buckets: defaultdict[int, OrderedDict[int, None]] = defaultdict(OrderedDict)
        self.min_freq = 0

    def _touch(self, key: int) -> None:
        f = self.freq[key]
        bucket = self.buckets[f]
        del bucket[key]
        if not bucket:
            del self.buckets[f]
            if self.min_freq == f:
                self.min_freq = f + 1
        self.freq[key] = f + 1
        self.buckets[f + 1][key] = None

    def get(self, key: int) -> int:
        if key not in self.vals:
            return -1
        self._touch(key)
        return self.vals[key]

    def put(self, key: int, value: int) -> None:
        if self.capacity <= 0:
            return
        if key in self.vals:
            self.vals[key] = value
            self._touch(key)
            return
```

```
        if len(self.vals) >= self.capacity:
            bucket = self.buckets[self.min_freq]
            victim, _ = bucket.popitem(last=False)
            if not bucket:
                del self.buckets[self.min_freq]
            del self.vals[victim]
            del self.freq[victim]
        self.vals[key] = value
        self.freq[key] = 1
        self.buckets[1][key] = None
        self.min_freq = 1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.lru	O(1)，一条双向链表	LRU 只看最近一次访问。一串一次性的 prompt（扫描）就能把热点挤出去，所以 hot vs scan 里 LRU 只恢复 4 块。LFU 记得热点的历史次数，扫描进来的块频率只有 1，先被淘汰。反过来，热点集合迁移后，LFU 会被旧热点的高频率污染（shifting 里 26 块，LRU 33 块）。题面强调“访问次数”就用 LFU，强调“最近”就用 LRU。
最小堆，键为 (freq, last_used_time)	O(log n)，触碰时要懒删除或调整堆	好写，但不满足 O(1)。频率不是每次只 +1、而是任意增减时（例如带衰减的 LFU），堆更自然。
每个频率桶用手写双向链表代替 OrderedDict	O(1)	这是 C++/Java 的标准写法。Python 的 OrderedDict 内部本来就是哈希 + 双向链表，popitem(last=False) 和 del 都是 O(1)，所以参考解直接复用。kv.lru 则要求自己手写链表。
兄弟练习 kv.refcount（全 O(1)）	O(1)	两题都是“按计数分桶”。LFU 的计数只增不减，新键总从 1 开始，所以一个 min_freq 指针就够了；RefCounter 的计数可增可减，还要取最大值，必须把桶串成有序的双向链表。

易错点

- put 覆盖已有键也算一次使用，要 _touch。
- 插入新键要先淘汰再插入，并把 min_freq 置 1。先插入的话，新键自己可能被当成频率最低的键淘汰掉。
- _touch 搬空旧桶时，只有在 f == min_freq 时才把 min_freq 改成 f+1。
- get 未命中不能创建任何条目：用 defaultdict 访问 freq[key] 会悄悄插入新键。

模式信号

“淘汰使用次数最少的，并列时淘汰最久未用的”加上 O(1)：频率桶 + 每桶内按最近性排序 + min_freq 指针。

相关练习：`kv.lru` `kv.random_evict` `kv.refcount`

kv.ring_queue

kv.ring_queue RingQueue

其它 LeetCode　LC 622 · Design Circular Queue　设计（数据结构）

`nanoserve/kvcache/ring_queue.py` · `class RingQueue(k: int): enqueue(x) -> bool; dequeue() -> bool; front() -> int; rear() -> int; is_empty() -> bool; is_full() -> bool`

引擎里的角色

`pool.py::FreePool` 用一个容量为 `staging`（默认 8）的 RingQueue 暂存“释放批次编号”：`stage(ids)` 把一个序列释放的块排序成链（kv.sort_free），放进 `slots[n % staging]`，再 `ring.enqueue(n)`；入队失败就抛出 `RuntimeError("staging ring is full: flush() first")`。`flush()` 不断取 `front()` 和 `dequeue()`，按从旧到新的顺序收集各批次的链，再做单路或 k 路归并。块管理器只在空闲链表耗尽、环满或 sanitize 时才 flush，所以多个批次常常一次 k 路归并完（demo：108 次单路、1 次 k 路）。

被调用于：`nanoserve/kvcache/pool.py`

题目

实现容量固定为 k 的环形 FIFO，用定长数组 + 头尾指针，不允许动态扩容。enqueue 满了返回 False；dequeue 空了返回 False；front/rear 空时返回 -1；还有 is_empty/is_full。所有操作 O(1)。

核心洞见

用 (head, size) 表示状态，空和满就不会混淆：size==0 为空，size==cap 为满。队尾写入位置是 (head+size) % cap，最后一个元素在 (head+size-1) % cap。

算法

1. 初始化：buf=[0]*k，head=0，size=0。
2. enqueue(x)：满则返回 False；buf[(head+size)%cap] = x，size += 1。
3. dequeue()：空则返回 False；head = (head+1)%cap，size -= 1。
4. front = buf[head]；rear = buf[(head+size-1)%cap]（空时都返回 -1）。
5. 例（k=3）：入 1,2,3，再入 4 失败；出队一次后 head=1，入 4 写到下标 (1+2)%3=0，rear=4。

时间　全部 O(1)　空间　O(k)

参考实现

```python
class RingQueue:
    def __init__(self, k: int) -> None:
        self.buf = [0] * k
        self.cap = k
        self.head = 0
        self.size = 0

    def enqueue(self, x: int) -> bool:
        if self.size == self.cap:
            return False
        self.buf[(self.head + self.size) % self.cap] = x
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.size == 0:
            return False
```

```
        self.head = (self.head + 1) % self.cap
        self.size -= 1
        return True

    def front(self) -> int:
        return -1 if self.size == 0 else self.buf[self.head]

    def rear(self) -> int:
        return -1 if self.size == 0 else self.buf[(self.head + self.size - 1) % self.cap]

    def is_empty(self) -> bool:
        return self.size == 0

    def is_full(self) -> bool:
        return self.size == self.cap
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
只存 head/tail 两个指针，多开一格（cap+1）区分空满	O(1)，浪费 1 格	无锁 SPSC 队列常用这种写法（生产者只写 tail、消费者只写 head，没有共享的 size）。单线程下，用 size 计数更直观。
collections.deque(maxlen=k)	O(1)	语义不同：满了以后 append 会悄悄丢掉最旧的元素，而不是拒绝。对 FreePool 来说，丢掉一个批次就是丢掉一批空闲块，等于泄漏，所以必须“满了就拒绝、让调用方先 flush”。
兄弟：eventlog.py 的环形日志（kv.ring_head / kv.ring_search）	O(1) 写	事件日志的环满了就覆盖最旧的事件（demo 里最早保留的是 #232），因为诊断信息丢了也可以接受。暂存环存放的是资源，绝不能覆盖。一个看“满了是拒绝还是覆盖”，就知道该用哪种。
Python list + pop(0)	dequeue O(n)	每次出队都要整体搬移元素，而且容量无上限，两条规格都不满足。

易错点

- rear 下标要取模：(head+size-1) % cap，在 head=0 附近回绕时最容易写错。
- 只用 head == tail 判断，无法区分空和满：要么记 size，要么浪费一格。
- dequeue 只移动 head，不必清零 buf；但 front/rear 必须先判空再读，否则会读到残留的旧值。

模式信号

“容量固定”“不允许动态扩容”“满了拒绝”：定长数组 + head/size 取模。

相关练习：`kv.ring_head` `kv.ring_search` `kv.merge_free_k`

kv.snapshot

kv.snapshot SnapshotTable

其它 LeetCode　LC 1146 · Snapshot Array　二分查找

`nanoserve/kvcache/snapshot.py` · `class SnapshotTable(length: int): set(index, val); snap() -> int; get(index, snap_id) -> int`

引擎里的角色

`manager.py::BlockManager.checkpoint` 在第一次打 checkpoint 时创建 `SnapshotTable(num_blocks + 1)`：槽 0 存已存 token 数 n，槽 i 存块表第 i-1 位的块号，并把当前整张表写入一次。之后 `_append_node`、copy-on-write 和 `rollback` 都只 `set` 改动过的槽。`checkpoint` 调用 `snap()` 并给块加引用（pin 住）；`rollback` 用 `_snapshot` 读回 `get(0, snap)` 和前 ⌈n/bs⌉ 个槽，精确恢复块表。demo 的投机解码段：34 轮、34 次回滚，每次都精确恢复。每轮 checkpoint 的代价是 O(改动)，不是 O(表长)。

被调用于：`nanoserve/kvcache/manager.py`

题目

长度为 length 的整数表，初值全为 0。`set(i, v)` 写槽 i；`snap()` 返回快照号（从 0 开始依次 +1）；`get(i, s)` 返回快照 s 那一刻槽 i 的值（保证 s 已返回过）。空间与 set 次数成正比，get 为 O(log m)（m 为该槽的修改次数）。

核心洞见

每个槽维护一条按快照号递增的修改历史 (snap_id, val)，初始放哨兵 (0, 0)。同一个快照周期内多次 set 只保留最后一次，直接覆盖历史末尾。查询时找最后一个 snap_id ⩽ s 的记录，用 bisect_right 减 1。

算法

1. 每槽两条并行列表：ids=[0]，vals=[0]；全局 snap_id=0。
2. set(i, v)：若 ids[i][-1] == snap_id，就覆盖 vals[i][-1] = v；否则追加 (snap_id, v)。
3. snap()：snap_id += 1，返回 snap_id - 1。
4. get(i, s)：k = bisect_right(ids[i], s) - 1，返回 vals[i][k]。
5. 例：set(0,5)，snap()→0，set(0,6)：槽 0 的历史为 ids=[0,1]、vals=[5,6]；get(0,0) 中 bisect_right([0,1],0)=1，取下标 0，得 5。

时间　set O(1)，snap O(1)，get O(log m)　　空间　O(length + set 次数)

参考实现

```python
class SnapshotTable:
    def __init__(self, length: int) -> None:
        # per index: parallel lists of snap ids and values, starting with (0 -> 0)
        self.ids: list[list[int]] = [[0] for _ in range(length)]
        self.vals: list[list[int]] = [[0] for _ in range(length)]
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        ids, vals = self.ids[index], self.vals[index]
        if ids[-1] == self.snap_id:
            vals[-1] = val
        else:
            ids.append(self.snap_id)
            vals.append(val)
```

```
    def snap(self) -> int:
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect_right(self.ids[index], snap_id) - 1
        return self.vals[index][i]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每次 snap 复制整张表	snap O(length)，空间 O(length × 快照数)	实现最简单。投机解码每轮都要 checkpoint，而一轮通常只改尾部一两个槽，整表复制就是 O(表长) 的浪费。只有快照极少、改动遍布全表时，整表复制才划算（读取是 O(1)）。
兄弟练习 kv.versioned（LC 981）	set O(1)，get O(log m)	两题骨架相同，都是每个键一条有序历史，查询时二分找 floor。区别在版本轴：snapshot 的版本号是内部计数器，只在 snap() 时推进，一个周期内的多次 set 合并为一条（覆盖）；versioned 的时间戳由调用方给，每次 set 都是新版本。snapshot 的所有槽都有隐含初值 0（哨兵），versioned 的键可能从未写过，所以要处理 i==0 时返回 ""。
持久化数据结构（路径复制的线段树或数组）	set/get O(log length)	需要“从旧快照分叉出新分支”（真正的版本树）时才必要。本题的版本是线性的，回滚只是读旧值，不会在旧快照上继续写，所以按槽记录历史更省。

易错点

- snap() 返回推进之前的编号（第一次返回 0）。
- set 只能在 ids[-1] == 当前 snap_id 时覆盖，否则会改写已经冻结的快照。
- 要用 bisect_right(ids, s) - 1，不能用 bisect_left：后者在 s 恰好等于某条记录的 snap_id 时会偏一位。
- 每槽的哨兵 (0, 0) 保证 k ⩾ 0，不需要额外判空。

模式信号

“打快照、按快照号查历史”加上“空间与修改次数成正比”：每个槽一条有序修改历史，查询时二分找 floor。

相关练习： `kv.versioned` `kv.seq_range`

---

kv.versioned VersionedStore

kv.versioned

其它 LeetCode · LC 981 · Time Based Key-Value Store · 二分查找

`nanoserve/kvcache/versioned.py · class VersionedStore: set(key, value, ts); get(key, ts) -> str`

引擎里的角色

`manager.py::BlockManager._event` 每记一条事件，就调用 `self.owners.set(block, owner_rid, ts)`，其中 ts 是事件日志的全局递增编号；块退休时 owner 记为 ""。`owner(block)` 等于 `owners.get(block, log.total)`。`owner_at(block, tick)` 先用 bisect 在 `_ticks` / `_tick_ts` 上把 tick 换成该 tick 的最后一个事件号，再查询。`describe()` 用它诊断泄漏和 double free，demo 里的 `at tick 78, 15 blocks were charged to a request, e.g. 0->r011 …` 也来自这里。

被调用于： `nanoserve/kvcache/manager.py`

题目

键可以是任意可哈希值（整数块号 0 也合法），值是字符串。`set(key, value, ts)` 的 ts 在所有键之间全局严格递增；`get(key, ts)` 返回该键时间戳 ≤ ts 的最新版本，没有则返回 ""。get 为 O(log m)。

> **核心洞见**
> ts 严格递增，所以每个键的版本列表按到达顺序 append，本身就有序，不需要排序。查询就是在有序时间戳上找 floor：i = bisect_right(times, ts)，i==0 表示没有版本，否则取 values[i-1]。

算法

1. times、values：键 → 并行列表。
2. set：times[key].append(ts)，values[key].append(value)。
3. get：先判断 key 不在 times 中则返回 ""；i = bisect_right(times[key], ts)；i == 0 返回 ""，否则返回 values[key][i-1]。
4. 例：set("foo","bar",1)，set("foo","bar2",4)；get("foo",3)：bisect_right([1,4],3)=1，得 "bar"；get("foo",0) 得 i=0，返回 ""。

时间 set O(1) 均摊，get O(log m)　**空间** O(set 次数)

参考实现

```python
class VersionedStore:
    def __init__(self) -> None:
        self.times: defaultdict[Hashable, list[int]] = defaultdict(list)
        self.values: defaultdict[Hashable, list[str]] = defaultdict(list)

    def set(self, key: Hashable, value: str, ts: int) -> None:
        self.times[key].append(ts)
        self.values[key].append(value)

    def get(self, key: Hashable, ts: int) -> str:
        if key not in self.times:
            return ""
        i = bisect_right(self.times[key], ts)
        return self.values[key][i - 1] if i else ""
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.snapshot（LC 1146）	同为 O(log m) 查询	versioned 的版本号由外部给定、可以跳跃（这里是事件编号），每次写都保留；snapshot 的版本号由 snap() 内部生成，周期内的写会合并。引擎里，“谁在第 t 个事件时持有块 b”的事件号天然存在，所以用 versioned；块表 checkpoint 只关心“打快照那一刻”，所以用 snapshot。
从版本列表末尾向前线性扫描	get O(m)	如果查询几乎总是问“现在”（ts = log.total），从末尾扫描第一步就命中，平均 O(1)。owner_at 会问任意历史 tick，二分才稳定。
ts 不保证递增时：bisect.insort 或 SortedDict	insort 插入 O(m)，SortedDict O(log m)	本题“全局严格递增”的保证让 append 就保持有序。日志乱序到达（分布式、多源）时，就必须用有序容器。

易错点

- 键 0 和空字符串都合法，判断“键不存在”要用 `key not in self.times`，不能用真值判断。
- get 里直接访问 defaultdict 会为不存在的键创建空列表，并不报错。参考解先判断成员关系。
- 要用 bisect_right：ts 恰好等于某个版本的时间戳时，应该返回该版本。
- 整数 0 和字符串 "0" 是不同的键（规格示例特意测了这一点）。

模式信号

“时间戳递增写入”“查询不晚于 t 的最新值”：每个键一条有序列表，查询时 bisect_right - 1。

相关练习：`kv.snapshot` `kv.seq_range`

`kv.mem_alloc`

kv.mem_alloc MemoryAllocator

其它 LeetCode · LC 2502 · Design Memory Allocator · 设计（数据结构）

`nanoserve/kvcache/memalloc.py` · `class MemoryAllocator(n: int): allocate(size, owner) -> int; free(owner) -> int`

引擎里的角色

`arena.py::TokenArena` 仿照 SGLang 的 `req_to_token`：一块大小为 `size`（默认 2 · num_blocks · block_size，demo 中为 288）的共享 arena，每个序列持有几段连续区间，用来存它每个 token 的 KV slot 号。`reserve(owner, n)` 调用 `allocate(n, owner)`（first-fit，失败时计入 `failed`）；`release(owner)` 调用 `free(owner)` 一次性归还；`fits()` 先试分配再 free 撤销，供 `BlockManager.can_admit / fits` 判断准入。`_store` 在序列写满已有容量时再 `reserve` 一段 block_size 的区间。demo：峰值 199/288，0 次失败。

被调用于：`nanoserve/kvcache/arena.py`

题目

n 个单元。`allocate(size, owner)`：找到最左侧的 size 个连续空闲单元，标记为 owner 并返回起点，找不到返回 -1，且不做任何改动。`free(owner)`：释放该 owner 的所有单元（可能来自多次分配、彼此不连续），返回释放的总数，没有则返回 0。同一个 owner 可以多次分配。

核心洞见

规格要的就是 first-fit：从左往右扫描，维护当前连续空闲长度 run，遇到占用单元就清零，run 第一次达到 size 时，起点就是 i-size+1。free 按 owner 扫一遍清零。n 在引擎里只有几百，O(n) 扫描完全够用。

算法

1. mem = [0]*n（0 表示空闲）。
2. allocate：run=0；对每个 i，mem[i]==0 时 run+=1，否则 run=0；run==size 时把 mem[i-size+1 .. i] 置为 owner，返回 i-size+1；扫完都没找到则返回 -1。
3. free：扫描，把 mem[i]==owner 的单元置 0 并计数，返回计数。
4. 例：n=10，分配 1,2,3 号各 1 格（下标 0,1,2）；free(2) 释放下标 1；allocate(3,4) 跳过 1 格的洞，从 3 开始；allocate(1,1) 填回下标 1。

时间 allocate O(n)，free O(n)　**空间** O(n)

参考实现

```python
class MemoryAllocator:
    def __init__(self, n: int) -> None:
        self.mem = [0] * n

    def allocate(self, size: int, owner: int) -> int:
        run = 0
        for i, o in enumerate(self.mem):
            run = run + 1 if o == 0 else 0
            if run == size:
                start = i - size + 1
                self.mem[start:i + 1] = [owner] * size
                return start
        return -1

    def free(self, owner: int) -> int:
        freed = 0
        for i, o in enumerate(self.mem):
            if o == owner:
                self.mem[i] = 0
                freed += 1
        return freed
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
best-fit：选能放下的最小空洞	按大小有序的空闲表上 O(log k)，朴素实现 O(n)	尽量保留大空洞，但会留下大量放不下任何东西的碎片。题面要求“最左侧”，所以必须 first-fit。first-fit 还倾向于把占用集中在低地址，和 FreePool 总是弹出最小块号的思路一致。
buddy 分配器（大小取 2 的幂，分裂与合并）	O(log n) 分配和释放	合并很快（伙伴地址只差一位）、外部碎片可控，但请求大小要向上取整到 2 的幂，内部碎片最坏接近 50%。Linux 页分配器用它；arena 的段长是 prompt+block_size 这类任意值，向上取整太浪费。
不要求连续：分页（M3 的块表本身）	O(1) 分配一页	PagedAttention 的核心正是放弃连续性，用块表间接寻址，外部碎片随之消失。arena 保留连续段，是为了演示 first-fit 在序列乱序结束后会碎片化：总空闲足够，却可能找不到一段足够长的连续区间。
free 时用 owner→段列表的映射	O(段数 + 段长)	比全表扫描快。TokenArena 自己就在 rows 里记了 (start, n)，并用它核对 free 返回的数目，可以把它当作参考解的优化方向。

易错点

- 失败时绝不能留下部分标记：先找到位置，再一次性写入。
- 遇到占用单元时 run 要重置为 0，否则会跨过被占单元，凑出一段假的连续区间。
- 起点是 i - size + 1，不是 i - size。
- fits() 的试探分配必须 free 掉，它依赖 owner 当前不持有任何单元，free 才只会回收这一次试探。

模式信号

“分配最左侧的连续空闲段”“按 owner 释放”：first-fit 线性扫描，维护 run 计数。

相关练习：`kv.address_ranges` `kv.free_tracker` `kv.longest_run`

---

kv.refcount

kv.refcount RefCounter

其它 LeetCode | LC 432 · All O(1) Data Structure | 设计（数据结构）

`nanoserve/kvcache/refcount.py` · `class RefCounter: inc(key); dec(key); max_key() -> str; min_key() -> str`

引擎里的角色

`manager.py::BlockManager._ref` / `_unref` 在更新 `refs[b]` 和 `holders[b]` 的同时，调用 `self.refcount.inc(b)` / `dec(b)` 镜像引用计数。`report()` 用 `max_key()` / `min_key()` 在 O(1) 内给出 `most_shared` / `least_shared`；`check()` 核对 RefCounter 和 refs 是否一致。fork demo 里，3 次 fork 之后 prompt 块 [0,1,2] 各被引用 4 次。

被调用于：`nanoserve/kvcache/manager.py`

题目

计数器：`inc(key)`（不存在则以 1 插入）、`dec(key)`（减到 0 就移除，保证 key 存在）、`max_key()` / `min_key()`（返回任意一个计数最大/最小的键，没有键时返回 ""）。键可以是任意可哈希值，块号 0 合法。四个操作都要平均 O(1)。

核心洞见

计数每次只变 ±1，所以一个键只会移动到相邻的计数桶。把桶按计数升序串成双向链表（头尾各放一个哨兵），每个桶存一组键，再用 where[key] 记录键所在的桶。最小值是 head.next，最大值是 tail.prev。

算法

1. _Bucket(count, keys:set, prev, next)；head 和 tail 是哨兵；where：键 → 桶。
2. inc：cur = where.get(key, head)；若 cur.next 不是 count+1 的桶，就在 cur 后面插入一个新桶；把 key 放进去，更新 where；若 cur 不是 head，从 cur 中移除 key，cur 空了就摘掉。
3. dec：cur = where[key]；count==1 时删除 where[key]；否则在 cur 前面找或建 count-1 的桶，放入 key；最后从 cur 移除 key，cur 空了就摘掉。
4. max_key：tail.prev 是 head 时返回 ""，否则返回其中任意一个键；min_key 同理用 head.next。
5. 例：inc(0), inc(7), inc(7) 后链表为 head ⇄ {0}:1 ⇄ {7}:2 ⇄ tail，max=7，min=0。

时间 inc/dec/max_key/min_key 均 O(1) 平均　空间 O(键数)

参考实现

```python
class _Bucket:
    __slots__ = ("count", "keys", "prev", "next")

    def __init__(self, count: int) -> None:
        self.count = count
        self.keys: set[Hashable] = set()
        self.prev: Optional[_Bucket] = None
        self.next: Optional[_Bucket] = None


class RefCounter:
    def __init__(self) -> None:
        self.head = _Bucket(0)          # sentinel: smaller than every count
        self.tail = _Bucket(0)          # sentinel: larger than every count
        self.head.next, self.tail.prev = self.tail, self.head
        self.where: dict[Hashable, _Bucket] = {}

    def _insert_after(self, node: _Bucket, count: int) -> _Bucket:
        b = _Bucket(count)
        b.prev, b.next = node, node.next
        node.next.prev = b
        node.next = b
        return b

    def _remove_if_empty(self, b: _Bucket) -> None:
        if not b.keys:
            b.prev.next, b.next.prev = b.next, b.prev

    def inc(self, key: Hashable) -> None:
        cur = self.where.get(key, self.head)
        nxt = cur.next
        if nxt is self.tail or nxt.count != cur.count + 1:
            nxt = self._insert_after(cur, cur.count + 1)
        nxt.keys.add(key)
        self.where[key] = nxt
        if cur is not self.head:
            cur.keys.discard(key)
            self._remove_if_empty(cur)

    def dec(self, key: Hashable) -> None:
        cur = self.where[key]
        if cur.count == 1:
            del self.where[key]
        else:
            prv = cur.prev
            if prv is self.head or prv.count != cur.count - 1:
                prv = self._insert_after(prv, cur.count - 1)
            prv.keys.add(key)
            self.where[key] = prv
        cur.keys.discard(key)
        self._remove_if_empty(cur)

    def max_key(self) -> Hashable:
        b = self.tail.prev
        return "" if b is self.head else next(iter(b.keys))

    def min_key(self) -> Hashable:
        b = self.head.next
        return "" if b is self.tail else next(iter(b.keys))
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.lfu	O(1)	LFU 也按计数分桶，但计数只增、新键从 1 开始、只关心最小值，所以一个 min_freq 整数就够，桶之间不需要链接。RefCounter 要支持 dec，还要同时取最大和最小，所以桶之间必须用有序双向链表连起来：dec 会让键回到更小的桶，而那个桶可能需要新建在中间。
两个堆（最大堆 + 最小堆）加懒删除	O(log n)	好写，但每次 inc/dec 都要压入新的 (count, key)，过期项堆积，查询时再弹出。不满足 O(1)，内存也会随操作次数增长。
dict 计数，查询时扫描	max/min O(n)	report 只调用一次时，O(n) 也完全可以接受。题目把 max/min 也要求成 O(1)，就逼出了按计数分桶的链表。如果查询极少、更新极多，dict 反而最省。

易错点

- 键可能是 0：查找要用 `where.get(key, head)` 或 `in`，不能写 `where.get(key) or head`。
- 新桶必须建在正确的一侧：inc 建在 cur 之后，dec 建在 cur 之前（cur.prev 之后）。
- 先把键放进新桶，再从旧桶删除并摘掉空桶。顺序反了，旧桶被摘掉后 cur.prev / cur.next 可能已经失效。
- 没有键时要返回 ""，report 用 `top == ""` 判断空。返回 None 或 0 都会出错（0 是合法的块号）。

模式信号

“计数 ±1”加上“O(1) 取最大和最小”：计数桶组成的有序双向链表 + 键到桶的哈希。

相关练习：`kv.lfu` `kv.lru` `kv.block_hash`

`kv.block_hash`

kv.block_hash BlockHashMap

其它 LeetCode · LC 706 · Design HashMap · 哈希表

`nanoserve/kvcache/hashmap.py · class BlockHashMap: put(key: int, value: int); get(key: int) -> int; remove(key: int)`

引擎里的角色

前缀共享的去重表。`manager.py::BlockManager._register` 把满块的链式哈希（`chain_hash`，对素数 HASH_MOD=999_983 取模，保证键 ⩽ 10^6）映射到块号，并且先注册者胜出（`table.get(h) == -1` 时才 put）。`_lookup` 先 get，再核对 `key[b]` 是否和完整 token 前缀一致，所以哈希冲突不会共享错误的 KV。`_clear` 时 remove。它也是 `tiers.py::RandomTier.slot_of` 的“哈希 → 主机槽”表。demo 中 prompt 块命中率 0.784。

被调用于：`nanoserve/kvcache/manager.py`，`nanoserve/kvcache/tiers.py`

题目

不用 dict/set（以及基于它们的 collections 容器），自己实现整数键哈希表：`put(key, value)`（存在则覆盖）、`get(key)`（不存在返回 -1）、`remove(key)`（不存在时什么也不做）。0 ⩽ key, value ⩽ 10^6，总调用次数约 10^4，平均 O(1)。

> **核心洞见**
>
> 桶数组 + 拉链：key % B 选桶，桶内是 [key, value] 对组成的小列表。B 取素数（参考解用 2069），带公共步长的键（例如都是 8 的倍数）也能均匀散开。10^4 个键 / 2069 个桶，平均链长约 5。

算法

1. buckets = [[] for _ in range(2069)]，_bucket(key) = buckets[key % 2069]。
2. put：遍历桶，找到同 key 的对就改 value 并返回；否则追加 [key, value]。
3. get：遍历桶，找到就返回 value，否则返回 -1。
4. remove：找到下标 i 后，用桶末尾元素覆盖 bucket[i] 再 pop。桶内顺序无关紧要，所以这是 O(1) 删除。

时间 平均 O(1 + α)，α = n/B；最坏 O(n)（全部落在同一个桶）　**空间** O(B + n)

参考实现

```python
class BlockHashMap:
    _BUCKETS = 2069  # prime, so keys with a common stride still spread out

    def __init__(self) -> None:
        # separate chaining: each bucket is a list of [key, value] pairs
        self.buckets: list[list[list[int]]] = [[] for _ in range(self._BUCKETS)]

    def _bucket(self, key: int) -> list[list[int]]:
        return self.buckets[key % self._BUCKETS]

    def put(self, key: int, value: int) -> None:
        bucket = self._bucket(key)
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value
                return
        bucket.append([key, value])

    def get(self, key: int) -> int:
        for k, v in self._bucket(key):
            if k == key:
                return v
        return -1

    def remove(self, key: int) -> None:
        bucket = self._bucket(key)
        for i, pair in enumerate(bucket):
            if pair[0] == key:
                bucket[i] = bucket[-1]  # swap-remove: order inside a bucket doesn't matter
                bucket.pop()
                return
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
直接寻址：长度 10^6+1 的数组，-1 表示空	严格 O(1)，约 10^6 个槽	键域小而且已知时最快，最坏情况也是 O(1)。代价是不管存多少键都占满整个键域的内存。键是 64 位哈希时根本不可行，所以引擎才先把链式哈希压到 10^6 以内。
开放寻址（线性探测）	平均 O(1)，删除要用墓碑	数据连续存放、缓存友好，CPython 的 dict 就是开放寻址加扰动探测。删除时不能直接置空（会截断探测链），要放墓碑，墓碑多了还要重建。负载因子要保持在约 0.7 以下。拉链法对删除和高负载更宽容。
固定桶数 vs 动态扩容	扩容均摊 O(1)	本题总操作数有上限，固定 2069 个桶足够。真实系统里键数不可预知，要在 α 超过阈值时把桶数翻倍并重新散列，否则链长线性增长。

易错点

- put 必须先查找已有的 key 再覆盖，直接追加会让同一个 key 出现两份，remove 之后 get 还能读到旧值。
- value 可以是 0：调用方用 `== -1` 判断不存在，实现里不要用真值判断。
- 桶数取 2 的幂（如 1024）时，key % B 只看低位，块哈希低位有规律时会严重冲突。

模式信号

“不许用内置 dict/set”加上“整数键、范围有限”：桶数组 + 拉链（或开放寻址）。

相关练习：`kv.random_evict` `kv.refcount` `kv.lru`

`kv.free_tracker`

kv.free_tracker FreeRangeTracker

其它 LeetCode · LC 352 · Data Stream as Disjoint Intervals · 区间合并

`nanoserve/kvcache/free_tracker.py` · `class FreeRangeTracker: add(block: int); ranges() -> list[list[int]]`

引擎里的角色

`manager.py::BlockManager._evict` 把每个被逐出到主机层的块号 `add` 进本 tick 的 `FreeRangeTracker`（`self._xfer`）。`_close_tick` 在 tick 结束时把 `len(self._xfer.ranges())` 计入 `stats["transfers"]`：每段连续块号合并成一次拷贝。测试 `test_offload_transfers_are_coalesced_per_tick` 在一个 tick 里逐出 6 块，要求 transfers < 6；demo 显示 offloaded 87 blocks in 67 coalesced transfers。

被调用于：`nanoserve/kvcache/manager.py`

题目

在线维护整数集合的不相交闭区间。`add(block)`（0 ⩽ block ⩽ 10^4，重复 add 不产生变化）；`ranges()` 返回按 start 升序、互不相交、互不相邻的极大段 [[s,e],…]（相邻的整数必须合并），每次返回新列表。设段数为 k，add 为 O(log k) 或 O(k)，ranges 为 O(k)。

核心洞见

维护两条有序并行列表 starts / ends。用 bisect_right(starts, block) 找到 i：段 i-1 是起点 ⩽ block 的最后一段，段 i 是第一个起点 > block 的段。新点只可能被段 i-1 覆盖，或者与段 i-1 的右端、段 i 的左端相邻，一共四种情况。

算法

1. i = bisect_right(starts, block)。
2. 若 i>0 且 ends[i-1] ⩾ block，说明已被覆盖，直接返回。
3. join_left = (i>0 且 ends[i-1] == block-1)；join_right = (i<len 且 starts[i] == block+1)。
4. 两侧都相邻：ends[i-1] = ends[i]，删除第 i 段；只与左侧相邻：ends[i-1] = block；只与右侧相邻：starts[i] = block；都不相邻：在 i 处插入 [block, block]。
5. 例：已有 [1,1],[3,3],[7,7]，add(2) 两侧都相邻，合并为 [1,3]；add(6) 只与右侧相邻，变成 [6,7]。

时间 add：查找 O(log k)，list 插入/删除 O(k)（底层是 memmove，很快）；ranges O(k)　**空间** O(k)

参考实现

```python
class FreeRangeTracker:
    def __init__(self) -> None:
        self.starts: list[int] = []  # sorted starts of disjoint, non-adjacent closed intervals
        self.ends: list[int] = []

    def add(self, block: int) -> None:
        i = bisect.bisect_right(self.starts, block)  # intervals [0, i) start at or before block
        if i > 0 and self.ends[i - 1] >= block:      # already covered
            return
        join_left = i > 0 and self.ends[i - 1] == block - 1
        join_right = i < len(self.starts) and self.starts[i] == block + 1
        if join_left and join_right:
            self.ends[i - 1] = self.ends[i]
            del self.starts[i], self.ends[i]
        elif join_left:
            self.ends[i - 1] = block
        elif join_right:
            self.starts[i] = block
        else:
            self.starts.insert(i, block)
            self.ends.insert(i, block)

    def ranges(self) -> list[list[int]]:
        return [[s, e] for s, e in zip(self.starts, self.ends)]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.free_ranges（离线）	排序 O(n log n) + 一遍扫描 O(n)	如果只在最后读一次，攒齐块号后 sort 再套 free_ranges 更简单。tracker 适合块号边到达边查询的场景，还能自动去重；引擎每个 tick 只读一次，两种方案都能用，tracker 的好处是空间只和段数有关。
兄弟练习 kv.address_ranges（LC 715）	O(log k + k)	address_ranges 处理的是实数地址上的半开区间，支持任意区间的增、删和整段查询；free_tracker 只会加单个整数点、从不删除，而且整数的“相邻”（[1,1] 与 [2,2]）也要合并。题面只有“逐个加点”和“列出区间”时用本题的四种情况分类；需要删除或整段查询时，就要用边界列表。
平衡 BST / SortedList（按 start 排序）	add 严格 O(log k)	段数很大（10^5 以上）且 add 很频繁时，list 插入的 O(k) 会成为瓶颈，这时值得用平衡树。块池只有几十到几百块，用 bisect 加 list 常数最小。
并查集：add(x) 时与 x±1 合并	add 近似 O(α)	合并很快，但 ranges() 要把所有根按 start 排序，代价 O(k log k)；每次 add 后都要读区间时就不划算了。

易错点

- 重复 add 要先判覆盖（ends[i-1] ⩾ block），否则会插入重复的段。
- 要用 bisect_right：block 恰好等于某段的 start 时，这段应该算作“左侧段”，才能判定为已覆盖。
- 两侧都相邻时，要把两段合成一段并删除一个元素，只改一边会留下相邻却没合并的两段。
- ranges() 要返回新列表，直接返回内部列表的话，调用方修改后会破坏内部状态。

模式信号

“数据流逐个加入整数”加上“随时列出不相交的连续区间”：有序起点列表 + 二分 + 四种合并情况。

相关练习：`kv.free_ranges` `kv.address_ranges` `kv.longest_run`

`kv.address_ranges`

kv.address_ranges AddressRanges

其它 LeetCode · LC 715 · Range Module · 区间合并

`nanoserve/kvcache/address_ranges.py` · `class AddressRanges: add_range(left: int, right: int); query_range(left: int, right: int) -> bool; remove_range(left: int, right: int)`

引擎里的角色

`arena.py::TokenArena` 的越界检查器。 `reserve` 成功后调用 `bounds.add_range(start, start+n)`； `release` 对每个段调用 `remove_range`； `_checked` （被 `read` / `write` 调用）要求 `query_range(addr, addr+1)` 为真，否则抛出 `ArenaBoundsError`； `check()` 核对每个活段都被完整跟踪。测试 `test_arena_bounds_checker_rejects_stale_access`：读超出行尾的位置会抛错，序列 release 之后再读位置 0 也会抛错，效果类似 ASan 对释放后使用（use-after-free）的检测。

被调用于： `nanoserve/kvcache/arena.py`

题目

维护实数地址集合，操作都是半开区间 [left, right)。`add_range`：把区间标记为已分配（可以和已有区间重叠）；`remove_range`：取消标记（可以从中间挖掉一段）；`query_range`：当且仅当区间内每个地址都已分配时返回 True。首尾相接的 [10,20) 和 [20,30) 视为连续。0 ⩽ left < right ⩽ 10^9，操作约 10^4 次。

> **核心洞见**
> 把并集存成一条有序边界列表 b：已覆盖的部分是 [b0,b1)、[b2,b3)、…。某个位置插入到 b 的下标 i 的奇偶性，就告诉你它在区间内（奇）还是区间外（偶）。增和删的写法统一：把 [left,right] 范围内原有的边界全部删掉，只在覆盖状态真正改变的端点补上 left 或 right。

算法

1. _replace(left, right, keep_inside)：i = bisect_left(b, left)，j = bisect_right(b, right)。
2. new = []；若 (i 为偶) == keep_inside 则 new.append(left)；若 (j 为偶) == keep_inside 则 new.append(right)；然后 b[i:j] = new。add 时 keep_inside=True，remove 时为 False。
3. query：i = bisect_right(b, left)，j = bisect_left(b, right)；返回 i == j 且 i 为奇。
4. 例：b=[10,20]，remove(14,16)：i=1、j=1 都是奇数，补上 14 和 16，得到 [10,14,16,20]。query(13,15)：i=1、j=2，不相等，返回 False。add(20,30) 作用在 [10,20] 上：i=1 为奇，不补 left；j=2 为偶，补 30，结果 [10,30]，首尾相接的段自动合并。

时间 每次操作 bisect O(log k) + 切片替换 O(k)（memmove）；query O(log k)　**空间** O(k)

参考实现

```python
class AddressRanges:
    """Sorted boundary list: addresses in [b[2i], b[2i+1]) are allocated, disjoint and non-touching."""

    def __init__(self) -> None:
        self.b: list[int] = []

    def _replace(self, left: int, right: int, keep_inside: bool) -> None:
        # Boundaries strictly inside the edited span disappear; `left` / `right` become
        # boundaries only where the covered/uncovered state actually changes.
        i = bisect.bisect_left(self.b, left)
        j = bisect.bisect_right(self.b, right)
        new: list[int] = []
        if (i % 2 == 0) == keep_inside:  # left falls outside (add) / inside (remove) a range
            new.append(left)
        if (j % 2 == 0) == keep_inside:
            new.append(right)
        self.b[i:j] = new

    def add_range(self, left: int, right: int) -> None:
        self._replace(left, right, keep_inside=True)

    def remove_range(self, left: int, right: int) -> None:
        self._replace(left, right, keep_inside=False)

    def query_range(self, left: int, right: int) -> bool:
        i = bisect.bisect_right(self.b, left)
        j = bisect.bisect_left(self.b, right)
        return i == j and i % 2 == 1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 kv.free_tracker	add O(log k + k)	free_tracker 是整数点、闭区间、只增不删，用四种情况分类最直接。address_ranges 要支持任意区间的增、删、查，用“边界列表 + 奇偶”能把增和删统一成一个函数，免去大量特判。题面出现 remove 或“整段查询”时，用本题的写法。
有序区间列表 [(s,e),…] 逐个合并或切割	O(k)	最直观，但删除时要把区间拆成两半，加入时要吞并多段，边界条件很多。本质和边界列表等价，边界列表的 bug 面小得多。
动态开点线段树 + 懒标记（区间赋值）	O(log C)，C = 10^9	操作数极大，或者还要统计“区间内已分配长度”这类聚合值时，用线段树。k 很小时，它的常数和代码量都比 bisect 方案大得多。
兄弟练习 kv.mem_alloc 的 owner 数组	按单元 O(n)	mem_alloc 按单元记录归属，负责分配，空间是 O(arena 大小)；address_ranges 只记录边界，负责校验，空间是 O(段数)。地址空间到 10^9 时只能记边界。arena 同时用两者，互相交叉验证。

易错点

- 半开区间：检查单个地址要用 query_range(addr, addr+1)；add(10,20) 之后 20 不算已分配。
- _replace 里 left 用 bisect_left、right 用 bisect_right，这样恰好落在 left/right 上的旧边界也会被删除，首尾相接的段才能合并、挖空的边界才不会重复。
- query 的 bisect 方向和 _replace 相反（left 用 right、right 用 left），这样 left 恰好是某段起点、right 恰好是某段终点时也算完全覆盖。
- 同一个 owner 的相邻两段在 bounds 里会被合并成一段；release 时逐段 remove 仍然正确，因为 remove 不要求和 add 一一对应。

模式信号

“半开区间的增、删、整段是否覆盖”，可以重叠，也可以从中间挖空：有序边界列表 + 奇偶判断（或线段树）。

相关练习：`kv.free_tracker` `kv.mem_alloc` `kv.free_ranges`

M4　前缀缓存与截止时间索引

M4

前缀缓存与 deadline 索引：Trie、BST 索引、持久化

16 题 · ★ Top 150 10 · 其它 6

	练习	LC	LeetCode 题目	模式
★	`prefix.trie`	208	Implement Trie (Prefix Tree)	Trie
★	`prefix.fork_point`	236	Lowest Common Ancestor of a Binary Tree	树的递归
★	`prefix.depth`	104	Maximum Depth of Binary Tree	树的递归
★	`prefix.level_dump`	102	Binary Tree Level Order Traversal	树的 BFS
★	`prefix.same_snapshot`	100	Same Tree	树的递归
★	`index.iter`	173	Binary Search Tree Iterator	BST
★	`index.kth`	230	Kth Smallest Element in a BST	BST
★	`index.validate`	98	Validate Binary Search Tree	BST
★	`index.min_gap`	530	Minimum Absolute Difference in BST	BST
★	`index.build`	108	Convert Sorted Array to Binary Search Tree	分治/归并
	`prefix.codec`	297	Serialize and Deserialize Binary Tree	树的 BFS
	`prefix.hit_sum`	677	Map Sum Pairs	Trie
	`index.range_sum`	307	Range Sum Query - Mutable	树状数组（Fenwick）
	`index.insert`	701	Insert into a Binary Search Tree	BST
	`index.delete`	450	Delete Node in a BST	BST
	`index.restore`	1008	Construct Binary Search Tree from Preorder Traversal	BST

prefix.trie

prefix.trie PrefixTrie

★ Top Interview 150 | LC 208 · Implement Trie (Prefix Tree) | Trie

`nanoserve/prefix/trie.py` · `class PrefixTrie: insert(word); search(word) -> bool; starts_with(prefix) -> bool`

引擎里的角色

前缀缓存的存储结构。`nanoserve/prefix/cache.py::PrefixCache.insert` 在请求完成时（`Server._finish`）把 prompt 的 token 串（每个 token id t 写成字符 `chr(0xE000+t)`）插入；`PrefixCache.match` 在放行时（`Server._admit`）对 `starts_with(key[:mid])` 二分出最长的已缓存 token 前缀，再用 `search(key)` 判断是否是完全重复的 prompt。由于每次 `starts_with` 只与查询长度有关，match 总代价 O(L log L)，与缓存了多少条 prompt 无关——缓存越大越需要这一点。

被调用于：`nanoserve/prefix/cache.py`

题目

实现 `PrefixTrie`：`insert(word)` 加入完整键（重复插入无副作用）；`search(word)` 仅当 word 曾作为完整键插入时为 True；`starts_with(prefix)` 当存在任一已插入键以 prefix 开头（含相等）时为 True。字符串非空，可含任意 Unicode 字符（包括 `$`、`#` 等常被当哨兵的字符），区分大小写；三个操作都应 O(字符串长度)。

核心洞见

每个节点是“一个前缀”，孩子用 dict 按字符索引，节点上的 `end` 标记“恰好有一个键在此结束”。search 与 starts_with 共用同一个“沿路走”的过程，区别只在最后是否要求 `end`。

算法

1. 节点 `_Node` 含 `children: dict[str, _Node]` 和 `end: bool`；根节点代表空前缀。
2. insert：从根逐字符走，缺孩子就新建，走完把最后一个节点的 `end` 置 True。
3. _walk(s)：逐字符走，任何一步缺孩子返回 None，否则返回最后到达的节点。
4. search(w)：节点存在且 `end` 为 True；starts_with(p)：节点存在即可。
5. 例：insert("apple") 后 search("app")=False、starts_with("app")=True；再 insert("app") 后 search("app")=True。

时间 insert/search/starts_with 均 O(L)，L 为参数长度　空间 O(所有键的总字符数)，最坏每个字符一个节点

参考实现

```python
class _Node:
    __slots__ = ("children", "end")

    def __init__(self) -> None:
        self.children: dict[str, _Node] = {}
        self.end = False


class PrefixTrie:
    def __init__(self) -> None:
        self.root = _Node()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
```

```
            nxt = node.children.get(ch)
            if nxt is None:
                nxt = node.children[ch] = _Node()
            node = nxt
        node.end = True

    def _walk(self, s: str) -> _Node | None:
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.end

    def starts_with(self, prefix: str) -> bool:
        return self._walk(prefix) is not None
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
长度 26 的孩子数组（LeetCode 常见写法）	O(L)，每节点 26 个槽	只适用于小写字母。引擎把 token 写成私用区字符 U+E000 起，加上测试里的 `héllo`、`€` 等，数组下标直接越界；dict 是唯一通用的选择。字母表很小且固定时数组更快更省哈希开销。
哈希集合存每个键的所有前缀	starts_with O(L)（含哈希），插入 $O(L^2)$ 时间与空间	实现两行，但一条 L 个 token 的 prompt 要存 L 个前缀串，总内存平方级；Trie 共享公共前缀，正好契合“很多 prompt 共享 system prompt”的场景。
有序列表 + bisect 找第一个 ≥ prefix 的键	starts_with O(L log N)，插入 O(N)	无需自定义节点，适合键少、只读居多；但查询代价随缓存条数增长，违背规格“与键数量无关”。
兄弟练习 stream.multi_stop（LC 1032）与 prefix.hit_sum	—	multi_stop 把停止词反向插入 Trie，用于流式后缀匹配；hit_sum 在节点上多存一个聚合值。三者同骨架，信号分别是“前缀存在性”“流式后缀”“前缀聚合”。

易错点

- search 只检查节点存在而忘了 `end`，导致 search("app") 在只插入 "apple" 时为 True。
- 用 `$` / `#` 之类字符当结束哨兵塞进 children，会与真实键冲突（规格特意列出这些字符）；用独立的 `end` 布尔字段。
- starts_with 中键恰好等于 prefix 也算 True，不要要求“严格更长”。

模式信号

“是否存在以此开头的键”且要求复杂度只与查询长度有关、与键数无关。

相关练习：`prefix.hit_sum` `stream.multi_stop` `tok.common_prefix`

prefix.fork_point

prefix.fork_point `fork_point`

★ Top Interview 150 · LC 236 · Lowest Common Ancestor of a Binary Tree · 树的递归

`nanoserve/prefix/tree_ops.py` · `fork_point(root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode`

引擎里的角色

定位索引损坏的范围。`nanoserve/index/bst_index.py::BSTIndex.repair` 处理「键是对的、但放错了节点」的索引（例如 checkpoint 写入时两个键被交换，由 `checkpoint.py::restore_queue(..., repair=True)` 触发）：先结构化地求中序节点序列，取第一个和最后一个逆序位置上的两个节点，再调用 `fork_point(root, a, b)` ——损坏全部落在这个分叉点的子树里，只需把这棵子树的键排序写回。为什么必须是**通用**二叉树的 LCA：此时树已经不是合法 BST，按键值往下走会走错路。（正常的区间查询 `between` 反而不用它，见对比讲解。）测试 `test_corrupted_checkpoint_with_two_swapped_keys_is_repaired_on_restore` 随机交换两个键，检查修复后出队顺序与损坏前一致； `test_repair_only_touches_the_damaged_subtree` 检查只改写了受损子树。

被调用于： `nanoserve/index/bst_index.py`

题目

给定普通二叉树（不是 BST，值无序但互异）以及树中两个不同节点 p、q（节点对象本身），返回同时以二者为后代、深度最大的节点（节点也是自己的后代，p 是 q 的祖先时答案为 p）。必须返回树中的节点对象。

核心洞见

后序 DFS：子树里“找到了谁”向上汇报。某节点左右子树各汇报一个非空结果，它就是分叉点；若当前节点本身就是 p 或 q，直接返回它——另一个要么在它下面（它就是答案），要么在别处（会被上层合并）。

算法

1. dfs(node)：node 为 None、或 node is p、或 node is q 时返回 node（用 `is` 比较对象，不比较值）。
2. 递归得到 left = dfs(node.left)，right = dfs(node.right)。
3. 两边都非空：p、q 分居两侧，返回 node。
4. 否则返回非空的那一边（都空则返回 None）。
5. 例：树 [3,5,1,6,2,0,8,_,_,7,4]，p=5、q=4：在 5 处直接返回 5，右侧 1 子树返回 None，根返回 5。

时间 O(n)：最坏访问全部节点　空间 O(h) 递归栈

参考实现

```python
def fork_point(root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
    def dfs(node: TreeNode | None) -> TreeNode | None:
        if node is None or node is p or node is q:
            return node
        left, right = dfs(node.left), dfs(node.right)
        if left is not None and right is not None:
            return node
        return left if left is not None else right

    return dfs(root)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
BST 专用 LCA（LC 235）：按值从根往下走	O(h)	在合法的 BST 上，第一个满足 p.val ≤ node.val ≤ q.val 的节点就是分叉点，O(h)。所以框架的区间查询 `between` 用的是按值往下走（再加剪枝中序，O(h + 答案)），而不是这道练习。反过来，`repair` 面对的是序已被破坏的树，按值走会走错，只能用只看结构的通用 LCA——两种 LCA 的分界就是「能不能相信键值」。
父指针 + 把 p 的祖先放进集合，再从 q 往上找	O(h) 查询，需 O(n) 预处理或节点自带 parent	`TreeNode` 没有 parent 字段；为了建父指针表要先遍历整树，单次查询不划算，多次查询同一棵静态树时值得。
欧拉序 + RMQ / 倍增	预处理 O(n log n)，查询 O(1) 或 O(log n)	树不变、查询海量时的标准做法；索引每次插删都会变，预处理立刻作废，不适用。

易错点

- 用 `==` 比较值而不是 `is` 比较节点；规格要求返回节点对象，且框架传进来的是真实节点。
- 找到 p 后还继续往下找 q，或者认为必须两者都找到才返回——“p 是 q 的祖先”时直接返回 p 才是对的。
- 误用 BST 按值比较的写法：题目的树不是 BST，会给出错误答案。

模式信号

两个给定节点、求「共同祖先中最深的」，而且不能（或不该）依赖节点值的顺序。

相关练习：`prefix.depth` `index.iter` `index.range_sum`

`prefix.depth`

prefix.depth `max_depth`

★ Top Interview 150 · LC 104 · Maximum Depth of Binary Tree · 树的递归

`nanoserve/prefix/tree_ops.py` · `max_depth(root: TreeNode | None) -> int`

引擎里的角色

索引平衡的监控器。`nanoserve/index/bst_index.py::BSTIndex._measure` 在每次 `add/discard/reset/adopt` 之后调用 `max_depth(root)`；`_rebalance_if_needed` 发现深度超过 `depth_limit(n) = 2·log2(n)+2` 就触发 `rebuild()`。`BSTIndex.check` 也用它审计（→ `DeadlineQueue.check` → `report()['healthy']`）。正因为平衡完全靠“测量 + 重建”，`insert_key` / `delete_key` 才可以返回任意合法 BST。

被调用于：`nanoserve/index/bst_index.py`

题目

返回二叉树从根到最远叶子路径上的**节点数**。空树 0，只有根为 1。例：[3,9,20,_,_,15,7] → 3；[1,_,2] → 2。

核心洞见

深度 = 1 + max(左深度, 右深度)，空树为 0 的定义让递归无需特判叶子。

算法

1. root 为 None 返回 0。
2. 递归求左右子树深度。
3. 返回 1 + max(left, right)。
4. 例：[3,9,20,_,_,15,7]：9 的深度 1，20 的深度 2，根为 3。

时间 O(n) **空间** O(h) 递归栈

参考实现

```python
def max_depth(root: TreeNode | None) -> int:
    if root is None:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
BFS 按层计数	O(n) 时间，O(宽度) 空间	不受递归深度限制，链状树也不会爆栈。框架的树深度被上限 2 · log2(n)+2 夹住（超限下一刻就重建），递归足够；面对可能是长链的任意输入时选 BFS。
在节点里维护高度（AVL 做法）	每次更新 O(h) 回溯维护，查询 O(1)	能把“每次更新都 O(n) 测深度”降到 O(log n)，但要求插删实现配合更新高度，练习的 insert/delete 不维护任何附加字段，所以框架选择每次重新测。
兄弟练习 prefix.level_dump	O(n)	层序结果的长度就是深度；测试 `test_rebalance_triggers_on_sorted_insertion` 正是断言 `len(levels) == index.height`，两题互为校验。

易错点

- 把深度算成边数（叶子返回 0）：规格是节点数，否则 depth_limit 比较会差 1。
- 只沿一侧走（例如只看右链）：重建后又插入的键可能挂在任意位置。

模式信号

“根到最远叶子”的长度，子问题只依赖左右子树的同类答案。

相关练习：`prefix.level_dump` `index.build` `index.insert`

prefix.level_dump

prefix.level_dump `level_dump`

★ Top Interview 150 · LC 102 · Binary Tree Level Order Traversal · 树的 BFS

`nanoserve/prefix/dump.py` · `level_dump(root: TreeNode | None) -> list[list[int]]`

引擎里的角色

索引的调试视图。 `nanoserve/index/bst_index.py::BSTIndex.levels` 调用它，`queue.py::DeadlineQueue.levels` 再把每个键渲染成 `deadline#seq`；demo 暂停时打印前 5 层（L0: 400#2，L1: 89#6 400#3，…），一眼看出索引是否成了右链。测试用 `len(levels) == height` 与 `max_depth` 交叉核对，用 `levels[0] == [root.val]` 检查根。

被调用于： `nanoserve/index/bst_index.py`

题目

返回二叉树的层序遍历：第 i 项为第 i 层（根为第 0 层）所有节点值，层内从左到右；空树返回 []。例：[3,9,20,_,_,15,7] → [[3],[9,20],[15,7]]。

核心洞见

BFS 时在每层开始前记下队列长度 `len(q)`，恰好就是这一层的节点数；弹出这么多个即完成一层，同时把下一层按左、右顺序入队。

算法

1. 空树返回 []；否则队列 q = deque([root])。
2. 当 q 非空：size = len(q)，新建 level 列表。
3. 循环 size 次：popleft，记录值，左孩子、右孩子依次入队（跳过 None）。
4. 把 level 追加到结果。
5. 例：[3,9,20,_,_,15,7]：第一轮 size=1 得 [3]；第二轮 size=2 得 [9,20]；第三轮得 [15,7]。

时间 O(n)　空间 O(w) 队列（w 为最大层宽）+ O(n) 输出

参考实现

```python
def _levels(root: TreeNode | None) -> list[list[int]]:
    if root is None:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            n = q.popleft()
            level.append(n.val)
            if n.left:
                q.append(n.left)
            if n.right:
                q.append(n.right)
        out.append(level)
    return out


def level_dump(root: TreeNode | None) -> list[list[int]]:
    return _levels(root)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS 带 depth 参数，按 depth 追加到 result[depth]	O(n) 时间，O(h) 栈	先序遍历保证同层内左先于右，结果相同；树很宽（平衡树最后一层约 n/2）而很浅时 DFS 的 O(h) 空间更省，但链状树会受递归限制。
单队列放 (node, depth) 对，或用 None 作层分隔符	O(n)	都可行；“记下 len(q)”最不容易出错，不需要额外标记。
兄弟练习 prefix.codec 的 BFS 序列化	O(n)	同样是 BFS，但 codec 要保留 None 位置以便无损还原；level_dump 只给人看，丢掉空位。

易错点

- 在循环里用变化中的 `len(q)` 当终止条件，会把下一层节点混进本层。
- 把 None 孩子入队后又输出成值，或忘了空树返回 [] 而不是 [[]]。

模式信号

按层分组输出，需要知道“这一层到哪里结束”。

相关练习： `prefix.depth` `prefix.codec`

---

prefix.same_snapshot

prefix.same_snapshot same_tree

★ Top Interview 150 · LC 100 · Same Tree · 树的递归

`nanoserve/prefix/snapshot.py` · `same_tree(a: TreeNode | None, b: TreeNode | None) -> bool`

引擎里的角色

checkpoint 的读回校验。 `nanoserve/index/checkpoint.py::verify(root, blob)` 把刚写出的 blob 用 `load_index` 还原，再 `same_tree(root, restored)` 逐节点比较； `checkpoint_queue` 在校验失败时抛 `RuntimeError` ，拒绝不可信的 checkpoint。demo 中两种格式都显示 `read-back exact` ； `test_verify_detects_a_different_tree` 验证 pop 一个请求后旧 blob 不再通过。

被调用于： `nanoserve/index/checkpoint.py`

题目

判断两棵二叉树是否完全相同：结构一致（每个位置要么都有节点要么都空）且对应节点值相等；两棵都空为 True。例：[1,2,3] vs [1,2,3] → True；[1,2] vs [1,_,2] → False；[1,2,1] vs [1,1,2] → False。

核心洞见

同步递归：两个都空 → 相等；只有一个空 → 不等；否则比值并递归比较左-左、右-右。 `a is b` 一句同时处理了“都空”和“一个空”。

算法

1. 若 a 或 b 为 None：返回 `a is b`（都为 None 才 True）。
2. 比较 `a.val == b.val`。
3. 递归 same_tree(a.left, b.left) 与 same_tree(a.right, b.right)，用 and 短路。
4. 例：[1,2] vs [1,_,2]：根相等；左边 2 vs None，一边为空，返回 False。

时间 $O(\min(n_1, n_2))$，遇到第一处不同即停　**空间** O(h) 递归栈

参考实现

```python
def same_tree(a: TreeNode | None, b: TreeNode | None) -> bool:
    if a is None or b is None:
        return a is b
    return a.val == b.val and same_tree(a.left, b.left) and same_tree(a.right, b.right)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
比较两棵树的序列化字符串（带空位标记的 codec）	O(n) 时间 + O(n) 额外空间	只要序列化包含空位，字符串相等 ⇔ 树相等，可用于跨进程比较；但必须先完整序列化，不能在第一个差异处提前结束。
只比较中序或只比较不带空位的前序	O(n)	中序相同不代表形状相同（任何同键集合的 BST 中序都一样），校验会放过形状变化。对键互异的合法 BST，前序相同确实意味着同一棵树；但校验的目的之一是发现损坏的（非 BST）树，所以要逐节点比较结构。
迭代版：栈里放 (a, b) 对	O(n)，无递归限制	树可能很深时更稳；索引深度有上限，递归版足够。

易错点

- 只比较值序列不比较结构，[1,2] 与 [1,_,2] 会被判相等。
- 先访问 a.val 再判空，空树时报 AttributeError。

模式信号

两棵树同步逐位置比较，判定“结构和值完全一致”。

相关练习：`prefix.codec` `index.restore` `index.validate`

---

index.iter　DeadlineIterator

index.iter

★ Top Interview 150　LC 173 · Binary Search Tree Iterator　BST

`nanoserve/index/deadline.py` · `class DeadlineIterator(root): next() -> int; has_next() -> bool`

引擎里的角色

按 EDF 顺序遍历索引的唯一手段。 `nanoserve/index/bst_index.py::BSTIndex.keys` 用它惰性产出有序键，供 `DeadlineQueue.__iter__`（快照队列顺序）、 `owners()`（checkpoint）、 `check()`（计数）和 `rebuild()`（取中序键再建树）使用。O(h) 空间、惰性产出的价值在于调用方随时可以停下（例如只看队首几个请求）。区间查询 `between` 没有用它，而是写了剪枝中序：迭代器只能从最小键开始，而区间查询需要跳过下界以下的整棵子树。

被调用于： `nanoserve/index/bst_index.py`

题目

实现 BST 迭代器：构造时给根（可空），`next()` 依次返回递增的值（调用方保证有剩余），`has_next()` 判断是否还有。额外空间 O(h)，不得一次性展开成列表；两个操作均摊 O(1)。

> **核心洞见**
> 把递归中序“手动化”：栈里保存从当前节点到其最左后代的路径。弹出栈顶就是下一个最小值，然后把它右子树的左链压栈。每个节点恰好入栈、出栈一次，所以 n 次 next 总代价 O(n)。

算法

1. 构造：_push_left(root)，沿左孩子一路压栈。
2. next：弹出栈顶 node，_push_left(node.right)，返回 node.val。
3. has_next：栈非空。
4. 例：[7,3,15,_,_,9,20]：初始栈 [7,3]；next→3（3 无右子）；next→7，压入 15 的左链 [15,9]；next→9，next→15 压入 20，next→20。

时间 next/has_next 均摊 O(1)，单次 next 最坏 O(h)　**空间** O(h)

参考实现

```python
class DeadlineIterator:
    def __init__(self, root: TreeNode | None) -> None:
        self.stack: list[TreeNode] = []
        self._push_left(root)

    def _push_left(self, node: TreeNode | None) -> None:
        while node is not None:
            self.stack.append(node)
            node = node.left

    def next(self) -> int:
        node = self.stack.pop()
        self._push_left(node.right)
        return node.val

    def has_next(self) -> bool:
        return bool(self.stack)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
构造时中序展开成列表	构造 O(n)，空间 O(n)	代码最短，但违反规格；调用方常常只需要前几个键，展开整棵树是浪费。只遍历一次且 n 很小时其实也无所谓。
Python 生成器递归 `yield from inorder(left)`	每次 next 最坏 O(h) 层生成器转发	写法优雅，空间也是 O(h)，但每个值要穿过 h 层生成器，常数大；显式栈是同一思路的平铺版本。
Morris 遍历（线索化）	O(1) 额外空间，总 O(n)	遍历中临时改写右指针；若调用方中途停下，必须小心恢复，否则索引被破坏。O(h) 栈只有几十个节点，不值得冒险。
兄弟练习 index.kth、index.min_gap	O(h + k) / O(n)	它们都是“受控中序”的变体：kth 在第 k 次弹出时停，min_gap 在弹出时与前驱做差。会写这个迭代器，另两题只是改弹出时做的事。

易错点

- next 里弹出节点后忘记处理其右子树的左链，导致跳过元素。
- 遍历过程中修改树（框架注释明确：Don't mutate while iterating），所以 `DeadlineQueue.__iter__` 先快照成列表再返回。

模式信号

按序逐个取 BST 元素、要求 O(h) 空间与均摊 O(1)，并且可能中途停下。

相关练习：`index.kth` `index.min_gap` `index.build` `prefix.fork_point`

---

index.kth `kth_earliest`

index.kth

★ Top Interview 150 · LC 230 · Kth Smallest Element in a BST · BST

`nanoserve/index/deadline.py` · `kth_earliest(root: TreeNode, k: int) -> int`

引擎里的角色

EDF 队列的“取队首”和“第 k 位”。`nanoserve/index/bst_index.py::BSTIndex.first` 调 `kth_earliest(root, 1)`，支撑 `DeadlineQueue.peek/pop`（每次放行都调用）；`BSTIndex.kth` 支撑 `DeadlineQueue.at(k)`；`BSTIndex.rank` 在 1..n 上对 `kth_earliest` 二分求名次，支撑 `DeadlineQueue.position`，后者在每次到达时统计“新请求超车了几个”（demo：32 次超车，单次最多 4）。

被调用于：`nanoserve/index/bst_index.py`

题目

给定非空 BST（值互异）和 1 ⩽ k ⩽ 节点数，返回第 k 小的值（k 从 1 计）。例：[3,1,4,_,2]，k=1 → 1；[5,3,6,2,4,_,_,1]，k=3 → 3。

核心洞见

BST 的中序就是升序，第 k 个出栈的节点就是答案。用显式栈做中序，数到 k 立即返回，不用走完整棵树。

算法

1. node = root，stack = []。
2. 沿左链压栈直到 None。
3. 弹出栈顶，k -= 1；k 为 0 时返回其值。
4. 转向 node.right，回到第 2 步。
5. 例：[5,3,6,2,4,_,_,1]，k=3：压 5,3,2,1；弹 1(k=2)、弹 2(k=1)、弹 3(k=0) → 3。

时间 O(h + k)；k=1（取队首）为 O(h)　空间 O(h)

参考实现

```python
def kth_earliest(root: TreeNode, k: int) -> int:
    stack: list[TreeNode] = []
    node: TreeNode | None = root
    while True:
        while node is not None:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
```

```
        if k == 0:
            return node.val
        node = node.right
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
顺序统计树：每个节点存子树大小	O(h) 查询，插删时 O(h) 维护 size	比较左子树大小与 k 决定往哪走，k 再大也是 O(h)； `rank` 也能 O(h) 直接算。代价是 insert/delete/重建都要维护 size，练习的接口不带附加字段。框架的 `rank` 因此是 O((h+n) log n) 的二分，对几百个请求可以接受，规模大时应换成它。
完整中序成列表再取下标	O(n) 时间与空间	k=1 时也要 O(n)，而 pop 是最热的路径；提前终止让 first() 只需 O(h)。
堆（M1 `sample.kth_largest`、M7 `metrics.slowest_k`）	O(n log k) 或 O(log k) 增量	数据没有有序结构时用堆；已经有 BST 时中序就是现成的顺序，没必要再建堆。

易错点

- k 是 1-based；先减再判与先判再减差一位。
- 忘记转向右子树，导致只在左链上计数。

模式信号

在 BST 上要“第 k 小”，并希望 k 小时代价小。

相关练习： `index.iter` `index.min_gap` `sample.kth_largest`

`index.validate` is_valid_index

`index.validate`

★ Top Interview 150 · LC 98 · Validate Binary Search Tree · BST

`nanoserve/index/deadline.py` · `is_valid_index(root: TreeNode | None) -> bool`

引擎里的角色

索引的不变量审计。 `nanoserve/index/bst_index.py::BSTIndex.check` 第一步调用 `is_valid_index(root)`，问题汇总进 `DeadlineQueue.check()`； `report()` 的 `healthy` 字段、demo 的 `invariants OK`、 `checkpoint.py::restore_queue` 的恢复后自检都依赖它。因为 insert/delete 的实现自由度很大（规格允许任意合法 BST），外层必须有一个独立的、只看定义的校验器。

被调用于： `nanoserve/index/bst_index.py`

题目

判断二叉树是否为严格 BST：每个节点整个左子树的值都严格小于它、整个右子树都严格大于它，出现相等即非法；空树合法；值可为任意整数（含负数、32 位边界）。例：[5,4,6,_,_,3,7] → False（3 在 5 的右子树里）。

核心洞见

每个节点的合法取值是一个开区间 (lo, hi)，由祖先决定：往左走上界变成父值，往右走下界变成父值。只比较父子是不够的。

算法

1. ok(node, lo, hi)：node 为 None 返回 True。
2. 若不满足 lo < node.val < hi 返回 False。
3. 返回 ok(left, lo, node.val) and ok(right, node.val, hi)。
4. 初始调用 ok(root, -inf, +inf)，用浮点无穷避免整数哨兵与边界值冲突。
5. 例：[5,4,6,_,_,3,7]：3 的区间是 (5, 6)，3 < 5，非法。

时间 O(n) **空间** O(h) 递归栈

参考实现

```python
def is_valid_index(root: TreeNode | None) -> bool:
    def ok(node: TreeNode | None, lo: float, hi: float) -> bool:
        if node is None:
            return True
        if not lo < node.val < hi:
            return False
        return ok(node.left, lo, node.val) and ok(node.right, node.val, hi)

    return ok(root, float("-inf"), float("inf"))
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
中序遍历检查严格递增	O(n) 时间，O(h) 栈	与上下界法等价，且可以复用 index.iter 的迭代器、遇到第一个非递增就停；两种都对，上下界法不依赖遍历顺序，更直接表达定义。
只检查 left.val < node.val < right.val	O(n)	错误：[5,4,6,_,_,3,7] 每对父子都合法，但 3 跑进了 5 的右子树。这正是规格专门给的反例。
用 INT_MIN/INT_MAX 当初始界	O(n)	值恰为 32 位边界时误判；Python 用 float('inf') 或 None 表示无界。规格点名“含 32 位边界值”。

易错点

- 允许相等（≤）：框架的复合键保证互异，任何相等都意味着索引坏了，必须判 False。
- 用整数哨兵当无穷大，碰到边界值出错。

模式信号

校验“整个子树”满足大小关系的不变量，而不是局部父子关系。

相关练习：`index.iter` `prefix.same_snapshot` `index.insert`

index.min_gap

index.min_gap　min_gap

★ Top Interview 150　LC 530 · Minimum Absolute Difference in BST　BST

`nanoserve/index/deadline.py` · `min_gap(root: TreeNode) -> int`

引擎里的角色

“最紧的两个截止时间相隔多远”。`nanoserve/index/queue.py::DeadlineQueue.deadline_gap` 调用 `self.deadlines.min_gap()`，即 `bst_index.py::BSTIndex.min_gap` → `min_gap(root)`；作用在**第二棵**索引上——它只存每个不同 deadline 一个键，所以差值才有意义（在复合键 deadline·SEQ_SPAN+seq 上算出的会是序号差）。每次到达时更新 `tightest_gap` 统计；demo 与测试中该值为 1。

被调用于：`nanoserve/index/bst_index.py`

题目

BST 至少 2 个节点，值互异非负，返回任意两节点值之差绝对值的最小值。例：[4,2,6,1,3] → 1；[1,0,48,_,_,12,49] → 1。

核心洞见

排好序的数组里最小差一定出现在相邻元素之间；BST 的中序就是有序序列，所以只需在中序遍历时与前驱做差。

算法

1. best = +inf，prev = None，用显式栈做中序。
2. 每弹出一个节点：若 prev 不为 None，best = min(best, node.val - prev)。
3. prev = node.val，转向右子树继续。
4. 返回 int(best)。
5. 例：[1,0,48,_,_,12,49] 中序 0,1,12,48,49，相邻差 1,11,36,1 → 1。

时间 O(n)　空间 O(h)

参考实现

```python
def min_gap(root: TreeNode) -> int:
    best = float("inf")
    prev: int | None = None
    stack: list[TreeNode] = []
    node: TreeNode | None = root
    while stack or node is not None:
        while node is not None:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if prev is not None:
            best = min(best, node.val - prev)
        prev = node.val
        node = node.right
    return int(best)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
两两比较	$O(n^2)$	没有利用有序性；n 几百时也要几万次比较，且每次到达都要算。
中序成列表再相邻做差	O(n) 时间，O(n) 空间	一样正确，多 O(n) 内存；显式栈版本把空间降到 O(h)。
只比较每个节点与其左右孩子	O(n)	错误：与节点最近的值是“左子树最大值 / 右子树最小值”，不一定是孩子。例如 [5,1,_,_,4]：父子差为 4 和 3，但 5 与 4 只差 1——4 是 5 的左子树最大值，却是 1 的孩子。
兄弟练习 kv.longest_run（M3）	O(n)	都关心“排序后相邻元素”，但 longest_run 的输入无序，靠哈希集合避免排序；这里输入已在 BST 里，中序免费得到顺序。

易错点

- prev 初值用 0 或 -1 当“无前驱”，会和真实值 0 混淆（值非负，0 合法）；用 None。
- 在复合键索引上调用：框架专门建了去重的 deadline 索引，理解为什么才能用对。

模式信号

BST 上求“任意两值的最小差”，等价于有序序列相邻差。

相关练习：`index.iter` `index.kth` `kv.longest_run`

---

index.build

index.build build_balanced

★ Top Interview 150 · LC 108 · Convert Sorted Array to Binary Search Tree · 分治/归并

`nanoserve/index/deadline.py · build_balanced(sorted_vals: list[int]) -> TreeNode | None`

引擎里的角色

索引的“重新平衡”与批量构建。 `nanoserve/index/bst_index.py::BSTIndex.reset(sorted_keys)` 调用 `build_balanced`； `rebuild()`（深度超过 2 · log2(n)+2 时触发）先用迭代器取出中序键再 reset； `queue.py::DeadlineQueue.from_index` 恢复 checkpoint 时用它重建去重 deadline 索引。demo 第 3 部分 64 个递增 deadline 触发了 8 次重建，深度从可能的 64 压回 10。

被调用于： `nanoserve/index/bst_index.py`

题目

给定严格递增数组，构建高度平衡的 BST（每个节点左右子树高度差 ⩽ 1），中序必须恰好等于原数组；答案不唯一，空数组返回 None。例：[-10,-3,0,5,9] → 例如层序 [0,-3,9,-10,_,5]。

核心洞见

取中点作根，左半递归建左子树、右半递归建右子树：两半大小至多差 1，于是任意节点左右子树高度差至多 1，树高 ⌈log2(n+1)⌉。

算法

1. build(lo, hi)：lo > hi 返回 None。
2. mid = (lo + hi) // 2，以 sorted_vals[mid] 建节点。
3. 左孩子 = build(lo, mid-1)，右孩子 = build(mid+1, hi)。
4. 返回 build(0, n-1)。用下标而不是切片，避免每层 O(n) 的复制。
5. 例：[-10,-3,0,5,9]：mid=2 → 0；左 [-10,-3] mid → -10，右孩子 -3；右 [5,9] mid → 5，右孩子 9。

时间 O(n) **空间** O(log n) 递归栈 + O(n) 节点

参考实现

```python
def build_balanced(sorted_vals: list[int]) -> TreeNode | None:
    def build(lo: int, hi: int) -> TreeNode | None:
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        return TreeNode(sorted_vals[mid], build(lo, mid - 1), build(mid + 1, hi))

    return build(0, len(sorted_vals) - 1)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按顺序逐个 insert_key	$O(n^2)$，结果是一条链	有序输入是 BST 的最坏情况，这正是需要 build_balanced 的原因：deadline 大致递增到达，逐个插入会退化。
切片递归 `build(vals[:mid])`	O(n log n)	结果一样但每层复制数组；n 小时无感，写重建器时用下标更干净。
DSW 算法（原地旋转成链再压缩）	O(n) 时间，O(1) 额外空间	不需要中序列表和新节点，内存紧张时更好；实现复杂得多，而我们本来就需要新的平衡形状，重新建节点代价可忽略。
自平衡树（AVL/红黑）	每次插删 O(log n)	从根本上避免重建；见本章“有序索引”决策——框架用“偶尔 O(n) 重建”换取插删练习的实现自由。

易错点

- 区间端点处理错误（hi 用开区间还是闭区间混用）导致漏元素或死递归。
- 用切片还是下标都行，但不要在每层重新排序——输入已严格递增。

模式信号

有序数组 → 平衡树，“高度平衡”提示取中点二分。

相关练习：`index.iter` `prefix.depth` `index.restore`

prefix.codec

prefix.codec TreeCodec

其它 LeetCode | LC 297 · Serialize and Deserialize Binary Tree | 树的 BFS

`nanoserve/prefix/codec.py` · `class TreeCodec: serialize(root) -> str; deserialize(data: str) -> TreeNode | None`

引擎里的角色

checkpoint 的通用格式。`nanoserve/index/checkpoint.py::dump_index(root, "codec")` 产出 `"T:" + TreeCodec().serialize(root)`，`load_index` 见到 `T:` 标签就调用 `deserialize`；`verify` 用它读回比对。它不假设 BST，因此当索引疑似损坏（重复值、违反 BST）时也能如实转储（`test_codec_dumps_a_tree_that_is_not_a_bst`）。demo 中 12 个等待请求的索引编码为 159 字符，紧凑前序格式为 133 字符。

被调用于：`nanoserve/index/checkpoint.py`

题目

实现二叉树 ↔ 字符串的编解码器，格式自定，只要求 `deserialize(serialize(t))` 结构与值完全相同。值为整数、可负可重复，节点最多约 10^4，树可能是深度上千的单链（不能因 Python 递归限制崩溃）；编解码器无状态；只还原 val/left/right。

核心洞见

BFS 层序输出每个节点的值，空孩子输出 `#`：每个非空节点恰好贡献两个孩子记号，所以解码时按同样的 BFS 顺序“每弹出一个节点就读两个记号”即可。全程迭代，没有递归深度问题。

算法

1. serialize：队列从 root 开始；弹出 None 写 `#`，否则写值并把左右孩子（含 None）入队；用逗号连接。
2. deserialize：按逗号拆分；首记号为 `#` 返回 None，否则建根并入队，指针 i = 1。
3. 每弹出一个节点：tokens[i] 非 `#` 则建左孩子并入队，tokens[i+1] 同理建右孩子；i += 2。
4. 例：[1,2,3,_,_,4,5] 编码为 `1,2,3,#,#,4,5,#,#,#,#`（2n+1 = 11 个记号）。

时间 serialize / deserialize 均 O(n)　空间 O(n) 输出 + O(w) 队列

参考实现

```python
class TreeCodec:
    def serialize(self, root: Optional[TreeNode]) -> str:
        out: list[str] = []
        q = deque([root])
        while q:
            node = q.popleft()
            if node is None:
                out.append("#")
                continue
            out.append(str(node.val))
            q.append(node.left)
            q.append(node.right)
        return ",".join(out)

    def deserialize(self, data: str) -> Optional[TreeNode]:
        tokens = data.split(",")
        if not tokens or tokens[0] == "#":
```

```
        return None
    root = TreeNode(int(tokens[0]))
    q = deque([root])
    i = 1
    while q:
        node = q.popleft()
        if tokens[i] != "#":
            node.left = TreeNode(int(tokens[i]))
            q.append(node.left)
        if tokens[i + 1] != "#":
            node.right = TreeNode(int(tokens[i + 1]))
            q.append(node.right)
        i += 2
    return root
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归前序 + # 空标记	O(n)，O(h) 递归栈	最常见的 LC 297 写法，解码同样优雅；但规格明确要处理深度上千的单链，超过 Python 默认递归上限 1000 会崩。要么改成显式栈，要么用 BFS。
仅前序（index.restore 的紧凑格式）	n 个记号，O(n) 恢复	只对键互异的 BST 成立；这里值可重复、树可以不是 BST，前序无法确定形状，必须带空位信息。
层序后去掉尾部 #（LeetCode 展示格式）	略短	能省几个字符，但解码时要处理“记号提前用完”的边界；上面的读法依赖每个节点两个记号都在，保留完整尾部更不容易出错。

易错点

- 用 `-` 或空格以外的字符做分隔要确认不会与负号冲突；逗号最安全。
- 用类属性/全局变量在 serialize 与 deserialize 之间传信息（违反无状态要求）。
- 空树：serialize(None) 应得到可被 deserialize 识别的串（这里是 `#`）。

模式信号

任意二叉树无损往返、值可重复、树可极深——需要空位标记且不能递归。

相关练习：`index.restore` `prefix.same_snapshot` `prefix.level_dump` `stream.frame_codec`

---

prefix.hit_sum PrefixHitSum

prefix.hit_sum

其它 LeetCode | LC 677 · Map Sum Pairs | Trie

`nanoserve/prefix/hit_sum.py · class PrefixHitSum: insert(key: str, val: int); sum(prefix: str) -> int`

引擎里的角色

前缀缓存的命中面板。`nanoserve/prefix/cache.py::PrefixCache._record` 每次命中时以“被复用的文本”为键，把命中次数与节省 token 数的**累计值**写入两个 `PrefixHitSum`（规格是覆盖，所以框架另用 `_count/_tokens` 两个 dict 维护累计，再 insert 新总数）；`hits_under(prefix)` / `tokens_saved_under(prefix)` 调用 `sum`，`report()['under']` 由此给出“复用了完整 system prompt 的命中”——demo 中为 10 次、495 个 token。

被调用于：`nanoserve/prefix/cache.py`

题目

实现键 → 值映射：`insert(key, val)` 设值（已存在则覆盖，不是累加）；`sum(prefix)` 返回所有以 prefix 开头的键（含等于 prefix）的当前值之和，无匹配为 0。键可含任意字符，区分大小写，val 非负；两个操作都只与字符串长度相关。

> **核心洞见**
> 在 Trie 每个节点上存“经过此节点的所有键的值之和”。覆盖时先算 delta = 新值 − 旧值（旧值查一个普通 dict），沿路径把 delta 加到每个节点；sum 就是走到 prefix 对应节点读它的 total。

算法

1. 维护 Trie（dict 孩子 + total）和 `vals: dict[key, val]`。
2. insert：delta = val − vals.get(key, 0)，更新 vals[key]；沿 key 逐字符走（缺则建），每个经过的节点 total += delta。
3. sum：沿 prefix 走，缺孩子返回 0，否则返回该节点 total。
4. 例：insert("apple",3) → sum("ap")=3；insert("app",2) → 5；insert("apple",1) 覆盖，delta=−2，sum("ap")=3。

时间 insert O(len(key))，sum O(len(prefix))　**空间** O(所有键的总字符数)

参考实现

```python
class _Node:
    __slots__ = ("children", "total")

    def __init__(self) -> None:
        self.children: dict[str, _Node] = {}
        self.total = 0  # sum of the values of all keys passing through this node


class PrefixHitSum:
    def __init__(self) -> None:
        self.root = _Node()
        self.vals: dict[str, int] = {}

    def insert(self, key: str, val: int) -> None:
        delta = val - self.vals.get(key, 0)
        self.vals[key] = val
        node = self.root
        for ch in key:
            nxt = node.children.get(ch)
            if nxt is None:
                nxt = node.children[ch] = _Node()
            node = nxt
            node.total += delta

    def sum(self, prefix: str) -> int:
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return 0
        return node.total
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
dict 存键值，sum 时扫描所有键做 startswith	insert O(1)，sum O(N · L)	键少、查询少时最简单；面板随命中键数增长而变慢，违背“与键数量无关”。
Trie 但 sum 时 DFS 整棵子树求和	insert O(L)，sum O(子树大小)	插入不需要 delta，但 system prompt 这样的公共前缀下面挂着几乎所有键，查询退化成全表扫描。把聚合挪到插入时是典型的“写时多做，读时少做”。
insert 直接把 val 累加到路径上（不算 delta）	O(L)	错误：覆盖语义下重复 insert 同一个键会重复计数。这也是框架为什么另外维护累计 dict——它要的是累加，而练习给的是覆盖。
兄弟练习 prefix.trie / index.range_sum	—	trie 只回答存在性；range_sum 在一维下标区间上做“单点赋值 + 区间求和”，同样靠 delta 更新。hit_sum 是把“区间”换成“前缀子树”。

易错点

- 忘记覆盖语义，直接累加。
- sum(prefix) 走到一半缺孩子时应返回 0，而不是抛 KeyError。
- 框架调用时对查询前缀做了 rstrip：被复用文本在 token 边界结束，末尾空格属于下一个词的 token。

模式信号

“以某前缀开头的所有键的值之和”，且值会被覆盖更新。

相关练习：`prefix.trie` `index.range_sum` `tok.common_prefix`

---

index.range_sum

index.range_sum　RangeCounter

其它 LeetCode　LC 307 · Range Sum Query - Mutable　树状数组（Fenwick）

`nanoserve/index/fenwick.py` · `class RangeCounter(nums: list[int]): update(i, val); sum_range(l, r) -> int`

引擎里的角色

按截止时间窗口统计待生成 token。 `nanoserve/index/queue.py::DeadlineQueue._count_tokens` 以 deadline tick 为桶下标、Σ max_tokens 为桶值：请求入队/离队时读出 `sum_range(d, d)` 再 `update(d, 旧值 ± max_tokens)`（规格是赋值，所以做读-改-写）； `tokens_due(lo, hi)` 直接 `sum_range` ，在每次到达时统计紧急积压（demo：100 tick 窗口内峰值 64 token）； `_grow_due` 在桶不够时容量翻倍并用旧桶值新建实例； `check()` 用全区间和核对。

被调用于： `nanoserve/index/queue.py`

题目

实现支持单点赋值和闭区间求和的整数数组： `RangeCounter(nums)` （n ⩾ 1，元素可负，之后不依赖调用方的列表）， `update(i, val)` 把下标 i 设为 val， `sum_range(l, r)` 返回 l..r 之和。初始化 O(n) 或 O(n log n)，两个操作 O(log n)。

核心洞见

树状数组（Fenwick）：tree[i] 管辖 (i − lowbit(i), i] 这一段的和。前缀和沿 i -= lowbit(i) 向下累加，单点修改沿 j += lowbit(j) 向上传播 delta；区间和 = prefix(r+1) − prefix(l)。赋值语义靠保存一份 nums 求出 delta。

算法

1. 保存 nums 的副本；tree 长 n+1（1-based）。
2. O(n) 建树：tree[i] += nums[i−1]；若父 j = i + (i & −i) ⩽ n，则 tree[j] += tree[i]。
3. update(i, val)：delta = val − nums[i]，nums[i] = val；j = i+1 起，tree[j] += delta，j += j & −j。
4. _prefix(i)：s 累加 tree[i]，i −= i & −i，直到 0。
5. sum_range(l, r) = _prefix(r+1) − _prefix(l)。例：[1,3,5] 和 9；update(1,2) 后为 8。

时间 初始化 O(n)；update / sum_range O(log n)　**空间** O(n)

参考实现

```python
class RangeCounter:
    def __init__(self, nums: list[int]) -> None:
        self.n = len(nums)
        self.nums = list(nums)
        self.tree = [0] * (self.n + 1)
        for i, v in enumerate(nums, 1):
            self.tree[i] += v
            j = i + (i & -i)
            if j <= self.n:
                self.tree[j] += self.tree[i]

    def _prefix(self, i: int) -> int:
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & -i
        return s

    def update(self, i: int, val: int) -> None:
        delta = val - self.nums[i]
        self.nums[i] = val
        j = i + 1
        while j <= self.n:
            self.tree[j] += delta
            j += j & -j

    def sum_range(self, l: int, r: int) -> int:
        return self._prefix(r + 1) - self._prefix(l)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
前缀和数组	查询 O(1)，更新 O(n)	静态数据的最佳选择（M7 `dash.region_sum` 就是不可变的二维前缀和）；这里每次入队/离队都要改桶，更新频率与查询相当，O(n) 更新不可接受。
线段树	两者 O(log n)，空间约 4n	更通用：能做区间最值、区间赋值（懒标记）。只需要可减的“和”时，树状数组代码更短、常数更小。
不维护计数器，直接 due_between 后对 max_tokens 求和	O(n + k)	完全够用且零额外状态，但每次到达都要做一次区间列举；树状数组让窗口求和与窗口内请求数无关。
兄弟练习 prefix.hit_sum	O(L)	同样是“赋值 → 用 delta 更新所有覆盖它的聚合节点”，一个在 Trie 路径上，一个在 lowbit 链上。

易错点

- 把 update 写成“加上 val”：规格是赋值，必须保存旧值求 delta（框架调用方也按赋值使用）。
- 1-based 与 0-based 混用：外部下标 i 对应 tree[i+1]。
- 直接持有调用方传入的列表引用，调用方之后修改会污染内部状态。

模式信号

频繁单点修改 + 频繁区间求和，二者都要 O(log n)。

相关练习：`prefix.hit_sum` `dash.region_sum` `prefix.fork_point`

`index.insert`

index.insert `insert_key`

其它 LeetCode · LC 701 · Insert into a Binary Search Tree · BST

`nanoserve/index/bst_ops.py` · `insert_key(root: TreeNode | None, val: int) -> TreeNode`

引擎里的角色

请求入队。 `nanoserve/index/bst_index.py::BSTIndex.add` 调用 `insert_key(root, key)` ，之后立刻测深度、必要时重建； `queue.py::DeadlineQueue._enter` （ `push` 与抢占回队的 `push_front` ）对请求索引插入复合键，并在某个 deadline 第一次出现时对去重 deadline 索引插入。前置条件“key 不在树中”由队列的 `_key_of` dict 保证。

被调用于： `nanoserve/index/bst_index.py`

题目

向 BST（值互异，可为空）插入不在树中的 val，返回根。必须复用原节点（val 不变），只新建一个节点；结果是任意合法 BST 即可；空树返回单节点；复杂度 O(h)。例：[4,2,7,1,3]，val=5 → [4,2,7,1,3,5]。

核心洞见

沿查找路径往下走，第一次遇到该方向为空的位置就把新节点挂成叶子——所有原节点的相对关系不变，BST 性质自然保持。

算法

1. 建新节点；root 为 None 直接返回它。
2. cur = root，循环：val < cur.val 往左，否则往右。
3. 目标方向为空时挂上新节点并返回 root。
4. 例：[40,20,60,10,30,50,70] 插 25：40→左 20→右 30→左为空，挂在 30 左边。

时间 O(h) **空间** O(1)（迭代）

参考实现

```python
def insert_key(root: TreeNode | None, val: int) -> TreeNode:
    node = TreeNode(val)
    if root is None:
        return node
    cur = root
    while True:
        if val < cur.val:
            if cur.left is None:
                cur.left = node
                return root
            cur = cur.left
        else:
            if cur.right is None:
                cur.right = node
                return root
            cur = cur.right
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归版 `root.left = insert(root.left, val)`	O(h) 时间，O(h) 栈	更短，但在两次重建之间深度可达上限；对任意输入（例如有序插入的长链）有爆栈风险。迭代版没有这个问题，也不重写路径上每个指针。
AVL / 红黑插入（带旋转）	O(log n)，保证平衡	直接解决退化问题，但需要节点高度/颜色字段与旋转逻辑。框架选择让插入保持最朴素，平衡由 `BSTIndex` 测深度 + 重建统一负责。
heapq.heappush	O(log n)	如果只需要取最早，堆插入更便宜；但队列还要任意删除、第 k 位与区间查询，见本章“有序索引”决策。

易错点

- 新建了路径上的节点（复制）而不是复用原节点，违反规格。
- 空树时忘了返回新节点。
- 有序输入下树退化：这道题本身不管平衡，理解为什么外层要测 `max_depth`。

模式信号

向 BST 加一个不存在的键，O(h)，结果任意合法 BST。

相关练习：`index.delete` `index.build` `prefix.depth` `index.restore`

index.delete

index.delete delete_key

其它 LeetCode　LC 450 · Delete Node in a BST　BST

`nanoserve/index/bst_ops.py` · `delete_key(root: TreeNode | None, key: int) -> TreeNode | None`

引擎里的角色

请求离队。 `nanoserve/index/bst_index.py::BSTIndex.discard` 调用 `delete_key(root, key)` 后测深度、必要时重建； `queue.py::DeadlineQueue._leave` 被 `pop` （每次放行）和 `remove` （取消请求）调用，删除请求索引中的复合键，并在某 deadline 的最后一个请求离开时从去重 deadline 索引删除。堆做不到的“删除任意请求”正是选 BST 的理由之一。

被调用于： `nanoserve/index/bst_index.py`

题目

从 BST（值互异，可为空）删除 key 并返回新根（变空返回 None）；key 不存在则原样返回；结果为任意合法 BST，允许就地修改甚至改写节点 val；复杂度 O(h)。例：[5,3,6,2,4,_,7] 删 3 → 例如 [5,4,6,2,_,_,7]。

核心洞见

分三种情况：叶子直接摘掉；只有一个孩子就让孩子顶替；两个孩子时用中序后继（右子树最左节点）顶替——后继比左子树都大、比右子树其余都小，放在原位置 BST 性质不变。

算法

1. 迭代找到目标节点 node 及其父节点 parent；找不到返回 root。
2. 若 node 至多一个孩子：replacement = 那个孩子（或 None）。
3. 否则找右子树最左节点 succ（及其父 succ_parent）；若 succ 不是 node 的直接右孩子，先 `succ_parent.left = succ.right` 、 `succ.right = node.right` ；再 `succ.left = node.left` ，replacement = succ。
4. parent 为 None 时返回 replacement 作新根，否则把 parent 指向 node 的那个指针改为 replacement。
5. 例：[5,3,6,2,4,_,7] 删 3：后继 4 是 3 的直接右孩子，4.left = 2，5.left = 4。

时间　O(h)　空间　O(1)（迭代）

参考实现

```python
def delete_key(root: TreeNode | None, key: int) -> TreeNode | None:
    parent: TreeNode | None = None
    node = root
    while node is not None and node.val != key:
        parent, node = node, (node.left if key < node.val else node.right)
    if node is None:
        return root

    if node.left is None or node.right is None:
        replacement = node.left if node.left is not None else node.right
    else:
        # Splice in the in-order successor (leftmost node of the right subtree).
        succ_parent, succ = node, node.right
        while succ.left is not None:
            succ_parent, succ = succ, succ.left
        if succ_parent is not node:
            succ_parent.left = succ.right
            succ.right = node.right
```

```
        succ.left = node.left
        replacement = succ

    if parent is None:
        return replacement
    if parent.left is node:
        parent.left = replacement
    else:
        parent.right = replacement
    return root
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
复制后继的值到 node，再递归删除后继	O(h)	最常见写法，规格允许改写 val。框架里键→请求用 dict 映射、不依赖节点身份，所以也安全；参考解选择“摘接节点”只是保持节点与值的绑定，调试时 dump 更直观。
用前驱（左子树最右）代替后继	O(h)	完全对称，同样正确；长期总用一侧会让树往另一侧偏，但这里有重建兜底，无所谓。
堆的懒删除（标记失效，弹出时跳过）	O(log n) 摊还，但堆里残留垃圾	只需要“取最小 + 偶尔取消”时可行；队列的长度、名次、区间查询都会被垃圾元素污染，需要额外计数修正。

易错点

- 删除根节点时忘了返回新根（parent 为 None 的分支）。
- 后继恰为 node.right 时仍执行 `succ.right = node.right`，造成自环。
- key 不存在时修改了树或返回 None。

模式信号

从 BST 删除任意键并保持 BST，O(h)。

相关练习：`index.insert` `index.kth` `kv.random_evict`

---

index.restore

index.restore bst_from_preorder

其它 LeetCode | LC 1008 · Construct Binary Search Tree from Preorder Traversal | BST

`nanoserve/index/bst_ops.py` · `bst_from_preorder(preorder: list[int]) -> TreeNode | None`

引擎里的角色

从紧凑 checkpoint 恢复索引。`nanoserve/index/checkpoint.py::load_index` 见到 `P:` 标签就把逗号分隔的键交给 `bst_from_preorder`；`verify` 用它读回比对、`restore_queue` 用它重建后经 `DeadlineQueue.from_index` → `BSTIndex.adopt` **原样接管形状**。因为恢复出的形状与原树完全一致，后续插删与重建时机不变，测试才能要求“恢复后续跑与不中断运行逐 tick 相同”。

被调用于：`nanoserve/index/checkpoint.py`

题目

给定某棵值互异 BST 的前序遍历（保证合法），返回这棵唯一的 BST；空列表返回 None；树高不超过 500；目标 O(n)。例：[8,5,1,7,10,12] → 层序 [8,5,10,1,7,_,12]。

核心洞见

单调栈保存“还可能接收右孩子”的路径，值自底向上递减。新值比栈顶小 → 它是栈顶的左孩子；否则不断弹出比它小的节点，最后弹出的那个就是它的父亲（它是右孩子）。每个节点进出栈各一次。

算法

1. 以 preorder[0] 建根，stack = [root]。
2. 对后续每个 v 建节点：若 v < stack[-1].val，挂为栈顶的左孩子。
3. 否则 parent = stack.pop()，当栈非空且 stack[-1].val < v 时继续弹出更新 parent；挂为 parent 的右孩子。
4. 新节点入栈。
5. 例：[8,5,1,7,10,12]：1 挂 5 左；7 弹出 1、5，挂 5 右；10 弹出 7、8，挂 8 右；12 挂 10 右。

时间 O(n)（每个节点入栈出栈各一次） **空间** O(h) 栈

参考实现

```python
def bst_from_preorder(preorder: list[int]) -> TreeNode | None:
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    stack = [root]  # path of nodes that may still receive a right child, values decreasing
    for v in preorder[1:]:
        node = TreeNode(v)
        if v < stack[-1].val:
            stack[-1].left = node
        else:
            parent = stack.pop()
            while stack and stack[-1].val < v:
                parent = stack.pop()
            parent.right = node
        stack.append(node)
    return root
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
按前序顺序逐个 insert_key	O(n · h)	一个漂亮的事实：按前序插入会得到同一棵树（每个节点插入时它的祖先都已在位）。实现零成本、直接复用 index.insert；索引深度有上限时 O(n log n) 也不慢，但最坏（链）是 $O(n^2)$。
上下界递归：build(bound) 只消费 < bound 的值	O(n) 时间，O(h) 递归栈	同样 O(n)、更短；规格保证树高 ⩽ 500，递归可行。单调栈是它的迭代版，不受递归限制。
排序得中序，再用前序 + 中序重建（LC 105）	O(n log n)	通用二叉树需要两种遍历；BST 的中序可由排序得到，但白白多一次排序和哈希表。
存有序键、恢复时 build_balanced	O(n)	得到的是合法但形状不同的树，破坏“逐 tick 一致”的恢复保证；这就是 checkpoint 用前序的原因。

易错点

- 弹栈时只弹一次：父亲应是最后一个比 v 小的节点，可能要连续弹出多个。
- `P:` 后为空串（空树）时 `"".split(",")` 得到 `[""]`，框架已特判；自己测试时注意空列表。

模式信号

BST 的前序序列（值互异）→ 唯一的树，要求 O(n)。

相关练习：`index.insert` `prefix.codec` `prefix.same_snapshot` `index.build`

M5　模型 IR 与配置

模型 IR 与配置：计算图编译执行、配置解析

17 题 · ★ Top 150 7 · 其它 10

	练习	LC	LeetCode 题目	模式
★	`ir.order_ok`	207	Course Schedule	拓扑排序
★	`ir.order`	210	Course Schedule II	拓扑排序
★	`ir.clone`	133	Clone Graph	图的 BFS
★	`ir.rpn`	150	Evaluate Reverse Polish Notation	栈
★	`config.brackets`	20	Valid Parentheses	栈
★	`config.norm_path`	71	Simplify Path	栈
★	`config.expr`	224	Basic Calculator	栈
	`ir.op_latency`	743	Network Delay Time	最短路（Dijkstra）
	`ir.critical_path`	2050	Parallel Courses III	拓扑排序
	`ir.device_groups`	547	Number of Provinces	并查集
	`config.expand`	394	Decode String	栈
	`config.arith`	227	Basic Calculator II	栈
	`ir.depends_on`	1462	Course Schedule IV	拓扑排序
	`ir.computable`	2115	Find All Possible Recipes from Given Supplies	拓扑排序
	`config.strip_comments`	722	Remove Comments	状态机
	`config.flags`	1106	Parsing A Boolean Expression	栈
	`config.glob`	44	Wildcard Matching	双指针

`ir.order_ok`

`ir.order_ok` `can_schedule`

★ Top Interview 150　LC 207 · Course Schedule　拓扑排序

`nanoserve/ir/graph.py` · `can_schedule(n: int, deps: list[list[int]]) -> bool`

引擎里的角色

`ir/compiler.py::validate` 在 `compile_ir` 的第一步调用 `can_schedule(len(ir.ops), ir.deps())`：输入是由张量名推出的依赖对 [a, b]（a 依赖 b），返回 False 就抛 `GraphError("dependency cycle ...")`。后面的 `precedence`、`topo_order`、`critical_path_time` 都以「图无环」为前提，所以必须先在这里用 O(V+E) 把环挡掉。demo 里 attn 依赖 proj、proj 又依赖 attn 的 3 算子图就是在这一步被拒的。

被调用于：`nanoserve/ir/compiler.py`

题目

算子 0..n−1（n ⩾ 1），deps[i] = [a, b] 表示 a 依赖 b（b 必须先执行）。问是否存在一个顺序能执行完全部 n 个算子，即依赖图是否无环。deps 可为空，无重复对。要求 O(n + len(deps))。例：n=2, [[1,0]] → True；[[1,0],[0,1]] → False。

> **核心洞见**
> 「能否全部执行」⇔「有向图无环」⇔「Kahn 算法能把 n 个点全部弹出」。环上的点入度永远不会降到 0，所以只需数弹出了多少个。

算法

1. 建图：对每个 [a, b] 连边 b → a（b 完成后 a 才能开始），indeg[a] += 1。
2. 把所有 indeg = 0 的算子放入队列。
3. 反复弹出 u、计数，对每个后继 v 执行 indeg[v] −= 1，降到 0 就入队。
4. 队列空时，弹出数 == n 返回 True，否则剩下的点都在环上或依赖环，返回 False。
5. 例：[[1,0],[0,1]] 两点入度都是 1，队列一开始就空，弹出 0 个 → False。

时间 O(n + E)　空间 O(n + E)

参考实现

```python
def _kahn(n: int, deps: list[list[int]]) -> list[int]:
    adj: list[list[int]] = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in deps:
        adj[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order: list[int] = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order


def can_schedule(n: int, deps: list[list[int]]) -> bool:
    return len(_kahn(n, deps)) == n
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS 三色标记（白/灰/黑），遇到灰色节点即有环	O(n + E)	复杂度相同，面试中也常见；但递归深度等于最长依赖链，Python 下需要担心递归上限或手写栈。Kahn 的副产品（拓扑序）正好被兄弟题 ir.order 复用，参考解两题共用一个 `_kahn`。只需判环、又已有 DFS 框架（如同时求 SCC）时 DFS 更顺手。
并查集判环	近 O(E)	只对**无向图**有效：有向图里 a→b、a→c、b→c 没有环，但并查集会在 b–c 处误报。题面「依赖」是有向关系，这个信号排除了并查集（对照 ir.device_groups，那里链路被对称化成无向，才用并查集）。
兄弟练习 ir.order	同为 O(n + E)	同一个 Kahn，本题只要布尔值，ir.order 要排列本身。引擎两者都用：validate 先判环给出精确错误，compile_ir 再取顺序。

易错点

- 边的方向：[a, b] 是 a 依赖 b，所以边是 b → a、入度加在 a 上；与 ir.critical_path、ir.depends_on 里 [a, b] =「a 先于 b」方向相反，本仓库三种约定都出现了。
- 孤立算子（不在任何 deps 里）入度为 0，必须一开始就入队，否则计数会少。
- 用 list.pop(0) 当队列会退化成 $O(n^2)$，用 collections.deque。

模式信号

「依赖」「先修」「是否存在合法顺序」——有向图 + 只问可行性 = Kahn 计数判环。

相关练习：`ir.order` `ir.critical_path` `ir.depends_on`

---

ir.order

ir.order `topo_order`

★ Top Interview 150 · LC 210 · Course Schedule II · 拓扑排序

`nanoserve/ir/graph.py` · `topo_order(n: int, deps: list[list[int]]) -> list[int]`

引擎里的角色

`ir/compiler.py::compile_ir` 在死代码消除之后调用 `topo_order(len(ir.ops), ir.deps())` 得到 `Plan.order`； `model_ir.py::execute` 按这个顺序逐个解释算子， `devices.py::place` 用它决定同一设备上算子的串行先后。另一个调用点是 `config/loader.py::resolve_flags`：flag 可以引用别的 flag，loader 把引用关系建成 [a, ref] 交给 topo_order，返回 [] 就报「flags refer to each other in a cycle」，否则按顺序求值。demo 里的调度是 dist4 → dist3 → dist2 → dist1 → dist0 → base → term4 → … → p → logits。

被调用于： `nanoserve/config/loader.py` ， `nanoserve/ir/compiler.py`

题目

算子 0..n−1，deps[i] = [a, b] 表示 a 依赖 b（b 先执行）。返回 0..n−1 的一个排列，使每个 [a, b] 中 b 都排在 a 前面；有多个合法顺序时任选其一，有环返回 []。O(n + len(deps))。例：4, [[1,0],[2,0],[3,1],[3,2]] → [0,1,2,3]（[0,2,1,3] 也对）。

核心洞见

Kahn 算法弹出的顺序本身就是拓扑序：一个点只有在它所有前驱都弹出后入度才会归零。弹出数不足 n 说明有环，按规格返回空列表。

算法

1. 与 ir.order_ok 相同地建图：b → a，indeg[a] += 1。
2. 所有入度为 0 的点按编号入队（deque）。
3. 弹出 u 追加到 order，后继入度减一，归零入队。
4. 结束时 len(order) == n 返回 order，否则返回 []。
5. 例：上面的 4 点图，队列依次是 [0] → [1,2] → [2] → [3]，得到 [0,1,2,3]。

时间 O(n + E)　空间 O(n + E)

参考实现

```python
def _kahn(n: int, deps: list[list[int]]) -> list[int]:
    adj: list[list[int]] = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in deps:
        adj[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order: list[int] = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order


def topo_order(n: int, deps: list[list[int]]) -> list[int]:
    order = _kahn(n, deps)
    return order if len(order) == n else []
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS 逆后序	O(n + E)	也是标准解；输出是「深度优先」的：会把 dist4 → term4 这样的链连着排，而 Kahn 输出的是波前——同一层可并行的算子挨在一起（demo 里 5 个 lookup 和 base 都排在最前）。引擎关心并行层次，而且不想要递归，所以选 Kahn。
最小堆替代队列（字典序最小的拓扑序）	O((n + E) log n)	需要确定性、可复现的「最小」顺序时用（例如 LC 的变体要求字典序）。本题任意顺序都合法，deque 已经足够；而且 Kahn 按编号初始化队列，在固定输入下结果本来就是确定的。
兄弟练习 ir.critical_path	O(n + E)	同一个 Kahn 骨架，多带一个 start[v] = max(前驱完成时间) 的 DP；本题只要顺序，ir.critical_path 要沿着顺序做最长路。拓扑序是在 DAG 上做 DP 的通用前提。

易错点

- 有环时规格要求返回 []，而不是返回部分顺序；部分顺序会让 execute 漏掉算子。
- 边方向同 ir.order_ok：[a, b] 中 b 先执行。写成 a → b 会得到反序，测试 test_schedule_respects_dependencies 会失败。
- loader 里的 flag 依赖用的也是「a 依赖 b」的约定，自引用（a = !(a)）由 loader 先单独报错，不会作为自环传进来。

模式信号

「给出任一合法执行顺序」「有环返回空」——要排列本身的拓扑排序。

相关练习：`ir.order_ok` `ir.critical_path` `config.flags`

---

ir.clone　clone_graph

ir.clone

★ Top Interview 150　LC 133 · Clone Graph　图的 BFS

`nanoserve/ir/graph.py` · `clone_graph(node: GraphNode | None) -> GraphNode | None`

引擎里的角色

`ir/compiler.py::prune` 的第一步是 `work = clone_graph(plan.view)`： `plan.view` 是 `ModelIR.build_view` 生成的无向算子邻接视图（节点 val = 算子下标 + 1）。剪枝 pass 在副本上把不可计算的节点从邻居表里摘掉，然后从输出节点出发用 `graph_nodes` 收集仍连通的分量作为保留集合。必须深拷贝，因为 `IRModel` 只编译一次原始 plan，再为每种上下文长度各特化一份并缓存，原始视图要留给下一次特化；test_clone_is_deep_and_pass_leaves_original_untouched 检查原图邻接表不变且副本不共享任何节点对象。

被调用于： `nanoserve/ir/compiler.py`

题目

给定无向连通图中值为 1 的节点（val 唯一 1..n，无自环无重边），返回一份深拷贝中对应的节点：结构相同（每个节点的 val、邻居 val 的顺序都一致），但所有节点都是新对象，不得修改原图；输入 None 返回 None。

核心洞见

图里有环，朴素递归复制会无限循环且会把同一节点复制多次；用一个「原节点 → 新节点」的哈希表同时充当 visited 集合和查找表，每个节点只创建一次。

算法

1. node 为 None 直接返回 None。
2. copies = {node: GraphNode(node.val)}，队列放入 node。
3. 弹出 cur，按原顺序遍历 cur.neighbors：邻居不在 copies 中就新建副本并入队。
4. 无论邻居是否新建，都把 copies[nb] 追加到 copies[cur].neighbors——这样邻居顺序与原图完全一致。
5. 返回 copies[node]。例：4 个节点的环 [[2,4],[1,3],[2,4],[1,3]]，每个节点创建一次，每条边在两端各追加一次。

时间 O(V + E)　空间 O(V)

参考实现

```python
def clone_graph(node: GraphNode | None) -> GraphNode | None:
    if node is None:
        return None
    copies = {node: GraphNode(node.val)}
    q = deque([node])
    while q:
        cur = q.popleft()
        for nb in cur.neighbors:
            if nb not in copies:
                copies[nb] = GraphNode(nb.val)
                q.append(nb)
            copies[cur].neighbors.append(copies[nb])
    return copies[node]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS 递归 + 同样的哈希表	O(V + E)，栈深 O(V)	代码更短；但必须在递归进入邻居**之前**把副本放进表里，否则环会导致无限递归。算子图 ⩽ 100 个节点，深度不是问题；大图（长链）时 BFS 更稳。
copy.deepcopy	O(V + E)	deepcopy 自带 memo，能正确处理环；但它递归实现（长链会撞递归上限），并且会把节点上挂的任何东西都复制一遍，语义不受控。题目考的就是 memo 思想；工程里对自定义节点类型一般手写，明确拷贝范围。
只复制邻接表（list of lists）再重建	O(V + E)	如果 pass 只需要 val 结构，复制成整数邻接表更轻；compiler 选择在 GraphNode 视图上编辑，是为了让 pass 代码和 LeetCode 原题的数据结构一致，并能用 graph_nodes 直接遍历连通分量。

易错点

- 邻居已在表中时也要追加 copies[nb]，只在新建时追加会丢边。
- visited 判定要在入队时（创建副本时）做，而不是出队时，否则同一节点可能被创建两次。
- 不能复用原节点对象或原 neighbors 列表（例如 copy.copy 浅拷贝），否则 prune 摘边时会改坏 plan.view。

模式信号

「深拷贝」「图可能有环」「不得共享节点」——哈希表做 old→new 映射 + BFS/DFS。

相关练习：`ir.computable` `ir.device_groups`

ir.rpn

ir.rpn eval_rpn

★ Top Interview 150 | LC 150 · Evaluate Reverse Polish Notation | 栈

`nanoserve/ir/interp.py` · `eval_rpn(tokens: list[str]) -> int`

引擎里的角色

`ir/model_ir.py::fold_constants` 对每个 scale/floor 算子的系数字节码调用 `eval_rpn`，结果除以定点（toy 模型为 100）得到浮点系数，存入 `Plan.consts` ——这是 compile_ir 的最后一步常量折叠；`mass_value` 把所有程序拼成一个程序（prog1 prog2 + prog3 + …）再求一次，得到总概率质量。字节码由 `coef_program` 生成，例如 λ = 0.5 → [5 100 * 10 /] = 50，floor 0.01 → [1 100 * 100 /] = 1；demo 中总质量 = 100/100 = 1。`_check_program` 事先检查过栈深度，所以 eval_rpn 可以假设表达式合法。

被调用于：`nanoserve/ir/model_ir.py`

题目

tokens 是后缀表达式：每项为 + - * / 之一或一个整数字面量（可带负号，如 "-11"）。遇到运算符弹出两个数，先弹出的是右操作数；除法向零截断（-7/2 = -3，6/-132 = 0），保证不除零、结果合法、中间值在 32 位内。返回最终结果。例：[2,1,+,3,*] → 9；[4,13,5,/,+] → 6。

核心洞见

后缀表达式不需要优先级和括号：操作数入栈，运算符作用于栈顶两个元素，结果再入栈。唯一要小心的是操作数顺序和除法的取整方向。

算法

1. 遍历 token：若是四个运算符之一（精确比较字符串），r = pop()，l = pop()。
2. 计算 l op r 入栈；除法用 int(l / r)（向零截断）。
3. 否则 int(token) 入栈（"-11" 是字面量，只有单独的 "-" 才是运算符）。
4. 返回栈顶。例：[4,13,5,/,+]：13/5 = 2，4 + 2 = 6。

时间 O(n) 空间 O(n)

参考实现

```python
def eval_rpn(tokens: list[str]) -> int:
    stack: list[int] = []
    for t in tokens:
        if t in ("+", "-", "*", "/"):
            r = stack.pop()
            l = stack.pop()
            if t == "+":
                stack.append(l + r)
            elif t == "-":
                stack.append(l - r)
            elif t == "*":
                stack.append(l * r)
            else:
                stack.append(int(l / r))
        else:
            stack.append(int(t))
    return stack[-1]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
用 Python 的 // 做除法	O(1)	错误：// 向下取整，-7 // 2 = -4，规格要求 -3。正确做法是 int(l / r)（32 位范围内 float 足够精确）或者 abs 相除后补符号。Python 与 C/Java 的整除语义不同是这道题在 Python 下最大的坑。
兄弟练习 config.arith / config.expr（中缀求值）	O(n)	中缀需要处理优先级（arith）或括号（expr），要么维护「上一个运算符」，要么维护符号栈；后缀已经把优先级编码在顺序里，所以 IR 选用后缀做字节码——解释器最简单，编译器（coef_program）负责生成。
运算符查表 dict[str, Callable]（operator.add 等）	O(n)	更简洁，但除法仍要自定义 lambda 实现截断；if/elif 链在只有 4 个运算符时同样清晰。

易错点

- 操作数顺序：先 pop 的是右操作数，写反会让减法和除法出错。
- 用 t.isdigit() 判断数字会把 "-11" 当成运算符或抛异常；应先判断是否为四个运算符字符串之一。
- 向零截断而不是向下取整；参考解用 int(l / r)。

模式信号

「后缀表达式」「字节码解释器」——操作数栈，遇运算符弹两个算一个。

相关练习：`config.arith` `config.expr` `config.flags`

`config.brackets`

config.brackets　brackets_balanced

★ Top Interview 150　LC 20 · Valid Parentheses　栈

`nanoserve/config/parser.py` · `brackets_balanced(s: str) -> bool`

引擎里的角色

`config/loader.py::split_sections` 在切分节之前对去完注释的整段文本调用 `brackets_balanced`，不平衡就报「unbalanced brackets」，免得后面的解析器在半截结构上产生莫名其妙的错误（test_brackets_checked_before_parsing：`[server` 缺右括号、`(2]` 类型错配都在这里被拦下）。`expand_list` 在展开模板前再查一次 `[ ]` 是否平衡，以满足 expand_template 的输入契约。M6 的 `bench/runner.py::fuzz_brackets` 还把它当被测对象：所有平衡串必须接受，任意单字符替换/删除/插入后必须拒绝。

被调用于：`nanoserve/bench/runner.py`，`nanoserve/config/loader.py`

题目

s 是任意字符串（长度 ⩾ 1，可含换行、字母、= 等），只有 ( ) [ ] { } 参与检查，其余忽略。合法当且仅当每个左括号被同类右括号闭合、后开先闭、每个右括号都有对应左括号；无括号的串合法。例："([])" → True，"(]" → False，"[server]\npool = 2[big,]small" → True。

核心洞见

嵌套结构「后开先闭」正是栈的 LIFO：左括号入栈，右括号必须与栈顶配对；扫描结束栈必须为空。

算法

1. pairs = {')':'(', ']':'[', '}':'{'}，空栈。
2. 遇到右括号：栈空或弹出的不是对应左括号 → False。
3. 遇到左括号：入栈；其它字符跳过。
4. 结束时返回栈是否为空。例："x = (1 + 2]"：(入栈，] 弹出 (不匹配 → False。

时间 O(n) **空间** O(n)（最坏全是左括号）

参考实现

```python
def brackets_balanced(s: str) -> bool:
    pairs = {")": "(", "]": "[", "}": "{"}
    stack: list[str] = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        elif ch in "([{":
            stack.append(ch)
        # every other character is ignored
    return not stack
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每种括号一个计数器	O(n) 时间、O(1) 空间	只对**单一类型**括号正确；多类型时 "([)]" 每个计数器都平衡却不合法。题面「三种括号、要正确嵌套」这个信号要求栈。只有一种括号时（例如只检查圆括号深度，loader 的 _check_plus_minus 就是这样）计数器更省。
反复把 "()" "[]" "{}" 替换为空	$O(n^2)$	写起来短，但每轮扫描整串，最坏 n/2 轮；而且要先过滤掉非括号字符。整份配置文本可能上千字符，没有理由接受平方。
直接交给完整解析器报错	—	解析器当然也能发现，但错误位置和信息往往离真正原因很远；一个 O(n) 的预检把「结构性错误」和「语义错误」分开，报错更精确，这正是 loader 把它放在流水线前端的原因。

易错点

- 右括号到来时栈为空要返回 False，而不是对空栈 pop 抛异常。
- 扫描结束后忘记检查栈空："((" 会被误判为合法。
- loader 在去注释之后才调用，所以注释里的括号不参与检查；如果顺序反过来，注释里的 "(" 会让合法配置被拒。

模式信号

「多种括号」「正确嵌套」「后开先闭」——栈匹配。

相关练习：`config.expand` `config.flags` `config.strip_comments`

config.norm_path

config.norm_path normalize_path

★ Top Interview 150 | LC 71 · Simplify Path | 栈

`nanoserve/config/paths.py` · `normalize_path(path: str) -> str`

引擎里的角色

`config/loader.py::_model_path` 先检查 [model] path 以 / 开头、无空格、可打印，再调用 `normalize_path` 得到 `LoadedConfig.model_path`。规范形式是权重缓存和去重的键：/models/./toy/../toy/ngram-5/ 与 /models/toy/ngram-5 指的是同一份权重，必须得到同一个字符串（server.conf 的 demo 输出即 /models/toy/ngram-5；tiered.conf 的 /srv/weights/./staging/../toy/ngram-5/ → /srv/weights/toy/ngram-5）。test_model_path_normalised 还覆盖了 /../.. → /。

被调用于： `nanoserve/config/loader.py`

题目

输入以 / 开头的 Unix 绝对路径，/ 是唯一分隔符。"." 忽略；".." 回到上一级（根目录时停在根）；连续多个 / 视为一个；"..."、"...." 等是普通目录名。输出以单个 / 开头、目录间恰好一个 /、不以 / 结尾（根除外）、不含 . 或 .. 组件。例："/.../a/../b/c/../d/./" → "/.../b/d"。

核心洞见

按 / 切开后，每个组件只有四种情况：空串或 "."（跳过）、".."（弹栈）、其它（入栈）。栈里剩下的就是规范路径的各级目录。

算法

1. path.split('/') 得到组件列表（连续斜杠产生空串）。
2. 空串或 "." → 跳过；".." → 栈非空则弹出，空栈（在根）则什么都不做。
3. 其它任何名字（包括 "..."）→ 入栈。
4. 返回 '/' + '/'.join(stack)；栈空时自然得到 "/"。例："/home//foo/" → ['', 'home', '', 'foo', ''] → 栈 [home, foo] → "/home/foo"。

时间 O(n) 空间 O(n)

参考实现

```python
def normalize_path(path: str) -> str:
    stack: list[str] = []
    for part in path.split("/"):
        if part == "" or part == ".":
            continue
        if part == "..":
            if stack:
                stack.pop()
        else:
            stack.append(part)
    return "/" + "/".join(stack)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
os.path.normpath / posixpath.normpath	O(n)	绝大多数情况下结果相同，但 POSIX 允许两个前导斜杠有特殊含义，normpath("//models/./a/") 返回 "//models/a"，而规格要求单个 /。题目要求的是精确定义的规范形式，库函数的边角语义不一样。
pathlib.Path.resolve()	访问文件系统	会跟随符号链接、访问磁盘，结果取决于机器状态；配置加载时权重路径可能根本不在本机（会被分发到 worker），我们要的是纯字符串层面的规范化。需要「真实物理路径」时才用 resolve。
正则反复替换 /./、/x/../	最坏 $O(n^2)$	需要循环直到不变，还要特判开头的 /../ 和 "..." 这种不该匹配的名字；栈一次扫描就处理了所有情况。

易错点

- "..." 不是特殊组件，只有恰好 "." 和 ".." 是；用 startswith('.') 会出错。
- 根目录上的 ".." 要忽略而不是报错或弹空栈。
- 结果为空栈时要返回 "/" 而不是空串；不要输出结尾斜杠。

模式信号

「.. 回退一级」「规范化路径」——按分隔符切分 + 栈。

相关练习：`config.brackets` `config.expand`

---

config.expr　eval_expr

config.expr

★ Top Interview 150　LC 224 · Basic Calculator　栈

`nanoserve/config/expr.py` · `eval_expr(s: str) -> int`

引擎里的角色

`config/loader.py::eval_number` 在值里只出现 + - ( ) 时调用 `eval_expr`（出现 * / 则走 eval_arith，两者混用直接报错）。调用前 `_check_plus_minus` 用一个小状态机保证契约：一元负号只能出现在开头、运算符之后或左括号之后，不会有两个连续运算符（`1 + -2` 被拒），括号平衡；并用字面量之和 ⩽ $2^{31}-1$ 保证中间值在 32 位内。server.conf 里 token_budget = 4096 - (512 + 128) = 3456、session_ttl = -(-50) = 50 都由它求值；测试里 -(-(100)) + (-20) = 80。

被调用于：`nanoserve/config/loader.py`

题目

s 由非负整数、+、-、(、) 和空格组成，保证合法；- 可作一元负号（"-1 + 2"、"-(2+3)"、"1-(-2)"），+ 不作一元正号，不会有两个连续运算符。返回整数值，O(len(s))，不许用 eval。例："(1+(4+5+2)-3)+(6+8)" → 23。

核心洞见

只有加减时，每个数对结果的贡献就是 ±它本身，符号 = 当前括号层的符号 × 外层括号的符号。遇到 (把（当前累计值，括号前的符号）压栈、从 0 重新累计；遇到) 弹出并合并。一元负号自然被处理：-(2+3) 相当于 0 − (2+3)。

算法

1. result = 0，sign = 1，num = 0，stack = []。
2. 数字：num = num*10 + d（多位数）。
3. + 或 -：result += sign*num，num = 0，sign = ±1。
4. (：push (result, sign)，result, sign = 0, 1。
5.)：result += sign*num，num = 0；弹出 (prev, s)，result = prev + s*result。
6. 结束返回 result + sign*num。例："1-(-2)"：result=1, sign=−1 遇 (压栈 (1,−1)；括号内 - 使 sign=−1，读 2,) 时 result=−2，合并 1 + (−1)(−2) = 3。

时间 O(n) **空间** O(括号嵌套深度)

参考实现

```python
def eval_expr(s: str) -> int:
    result = 0
    sign = 1
    num = 0
    stack: list[tuple[int, int]] = []
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch in "+-":
            result += sign * num
            num = 0
            sign = 1 if ch == "+" else -1
        elif ch == "(":
            stack.append((result, sign))
            result, sign = 0, 1
        elif ch == ")":
            result += sign * num
            num = 0
            prev, prev_sign = stack.pop()
            result = prev + prev_sign * result
    return result + sign * num
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归下降（expr := term (± term)*，term := num | -term | (expr)）	O(n)，递归深度 = 嵌套深度	更通用，加上 * / 和优先级只需再加一层文法（这就是 Basic Calculator III）；但 Python 递归上限约 1000，恶意配置可以写很深的括号。只有加减时栈版本更短且无递归。
调度场算法（shunting-yard）转后缀再用 ir.rpn 求值	O(n)	两遍、两个栈，适合运算符多、优先级复杂的情况；一元负号要额外区分成另一个运算符。对只有加减的文法是杀鸡用牛刀。
兄弟练习 config.arith	O(n)	arith 有优先级没括号，靠「上一个运算符 + 栈」；expr 有括号没优先级，靠「(结果, 符号) 栈」。loader 按出现的运算符分派到其中一个，并禁止同一个值同时出现括号和 * /——用两道经典题覆盖两种常见写法，而不必实现完整计算器。

易错点

- 遇到) 时要先把当前 num 加进 result 再合并，否则括号里最后一个数会丢。
- 空格不能重置 num，也不能触发结算（与 arith 的逐字符结算写法不同，这里只在 + - ( ) 时结算）。
- 弹栈合并是 prev + s*result，不是 prev + result：-(2+3) 的符号在压栈时保存。

模式信号

「只有加减和括号」「一元负号」「不许 eval」——(累计值, 符号) 栈。

相关练习：`config.arith` `ir.rpn` `config.brackets`

---

ir.op_latency

ir.op_latency broadcast_latency

其它 LeetCode · LC 743 · Network Delay Time · 最短路（Dijkstra）

`nanoserve/ir/latency.py` · `broadcast_latency(edges: list[list[int]], n: int, src: int) -> int`

引擎里的角色

`ir/devices.py::Topology.broadcast_time(island, host)` 把岛内设备重新编号为 1..m、保留岛内的有向链路，然后调用 `broadcast_latency(edges, m, host)`，求权重从 host 广播到岛内所有设备的时间。`choose_island` 对每个候选 host 都调用一次，丢弃返回 −1（有设备不可达）的 host，取最快者。默认拓扑的 demo：host 1/2/3/4 的广播时间为 5/4/3/6 ms，选 host 3（3→2 用 1 ms，3→2→1 用 3 ms，3→4 用 3 ms）。test_device_islands_and_broadcast 中单向链 1→2→3 以 3 为 host 返回 −1。

被调用于：`nanoserve/ir/devices.py`

题目

设备 1..n，edges[i] = [u, v, w] 是 u → v 的有向链路，耗时 w ⩾ 0（无重复对、无自环）。数据在时刻 0 从 src 出发，每台设备收到的时刻是最短路长度；返回所有设备都收到的时刻（最短路的最大值），有设备不可达返回 −1；n = 1 返回 0。要求 O(E log V) 级别。

核心洞见

「广播完成时间」= 单源最短路的最大值，边权非负，所以是 Dijkstra。最后检查是否所有点都被确定了距离即可判断可达性。

算法

1. 建邻接表 adj[u] = [(v, w)]。
2. 最小堆放入 (0, src)，dist = {}。
3. 弹出 (d, u)：u 已确定就跳过（惰性删除）；否则 dist[u] = d，把未确定的邻居 (d + w, v) 入堆。
4. 堆空后，len(dist) == n 返回 max(dist.values())，否则返回 −1。
5. 例：[[2,1,1],[2,3,1],[3,4,1]]，src=2：dist = {2:0, 1:1, 3:1, 4:2}，答案 2。

时间 O(E log E)（惰性删除的二叉堆） 空间 O(V + E)

参考实现

```python
def broadcast_latency(edges: list[list[int]], n: int, src: int) -> int:
    adj: list[list[tuple[int, int]]] = [[] for _ in range(n + 1)]
    for u, v, w in edges:
```

```
        adj[u].append((v, w))
    dist: dict[int, int] = {}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in dist:
            continue
        dist[u] = d
        for v, w in adj[u]:
            if v not in dist:
                heapq.heappush(heap, (d + w, v))
    return max(dist.values()) if len(dist) == n else -1
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
Bellman–Ford	O(V · E)	能处理负权边，这里链路耗时 ⩾ 0 不需要；在小图上代码同样简单，但复杂度高一个量级。
Floyd–Warshall 一次求全源最短路	$O(n^3)$	choose_island 实际上对**每个** host 都算一遍，等价于全源最短路；设备数很小（几台到几十台）且稠密时，一次 Floyd 比 n 次 Dijkstra 更简单、常数也小。框架选择逐 host 调用单源函数，是为了复用 LeetCode 原题接口；设备数一大（上百、稀疏），n 次 Dijkstra 的 O(n · E log V) 更优。
BFS	O(V + E)	只在所有边权相等时正确（0/1 权可用 0-1 BFS 双端队列）。链路耗时不同（1、2、3、5 ms）这个信号排除了 BFS。
兄弟练习 ir.device_groups	—	并查集回答的是对称化后的**无向**连通（哪些设备能组成一个岛），本题回答**有向**可达和最短时间；同一个岛里仍可能有 host 广播不到全部设备（返回 −1），所以两者都需要。

易错点

- 边是有向的，不要把 [u, v, w] 当成双向边加两次。
- 惰性删除：同一节点可能多次入堆，弹出时必须检查是否已确定，否则会用较大的距离覆盖。
- 不可达判断要看确定距离的节点数是否为 n；n = 1 时 dist = {src: 0}，返回 0。

模式信号

「有向带权」「从一点出发到所有点的最早时刻」「权非负」——Dijkstra 求最大最短路。

相关练习：`ir.device_groups` `ir.critical_path`

ir.critical_path

ir.critical_path `critical_path_time`

其它 LeetCode | LC 2050 · Parallel Courses III | 拓扑排序

`nanoserve/ir/latency.py` · `critical_path_time(n: int, deps: list[list[int]], cost: list[int]) -> int`

引擎里的角色

`ir/compiler.py::Plan.critical_path(extra_before)` 把依赖 [a 依赖 b] 转成「b 先于 a」的 1 起编号对，连同额外约束交给 `critical_path_time`，cost 取每个算子的 `op.cost`。不带额外约束时得到无限并行下的下界：toy 图为 dist4(6) → term4(1) → p(2) → logits(2) = 11，而串行和 `serial_time` = 30。`devices.py::place` 把同一设备上按调度顺序相邻的算子对作为 extra_before 传入，得到具体放置的估计 12。test_critical_path_bounds 验证 max cost ⩽ cp ⩽ Σcost，且把整个调度串成一条链时恰好等于 Σcost。

被调用于：`nanoserve/ir/compiler.py`

题目

算子 1..n 构成 DAG，deps[j] = [a, b] 表示 a 完成后 b 才能开始；cost[i] 是算子 i+1 的耗时（正整数）。任意多个就绪算子可同时执行，返回全部完成的最早时刻（关键路径长度）。O(n + len(deps))。例：3, [[1,3],[2,3]], [3,2,5] → 8。

核心洞见

无限并行下，每个算子的最早开始时间 = 所有前驱最早完成时间的最大值（没有资源竞争，只有依赖）。这是 DAG 上的最长路，按拓扑序做一遍 DP：finish[u] = start[u] + cost[u]，start[v] = max(start[v], finish[u])。

算法

1. 建图 a−1 → b−1，统计入度；start = [0]*n。
2. 入度为 0 的点入队（Kahn）。
3. 弹出 u：finish = start[u] + cost[u]，更新全局 best = max(best, finish)。
4. 对每个后继 v：start[v] = max(start[v], finish)，入度减一，归零入队。
5. 返回 best。例：[[1,3],[2,3]]，cost [3,2,5]：start[3] = max(3, 2) = 3，finish = 8。

时间 O(n + E)　空间 O(n + E)

参考实现

```python
def critical_path_time(n: int, deps: list[list[int]], cost: list[int]) -> int:
    adj: list[list[int]] = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in deps:
        adj[a - 1].append(b - 1)
        indeg[b - 1] += 1
    start = [0] * n
    q = deque(i for i in range(n) if indeg[i] == 0)
    best = 0
    while q:
        u = q.popleft()
        finish = start[u] + cost[u]
        best = max(best, finish)
        for v in adj[u]:
            start[v] = max(start[v], finish)
            indeg[v] -= 1
            if indeg[v] == 0:
```

```
                q.append(v)
    return best
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
串行求和 Σcost	O(n)	单设备时是精确值（30），是并行执行的上界；关键路径（11）是下界。引擎同时报告两者，真实放置（12）必须落在中间——这个不变量本身就是测试。
DFS + 记忆化求 longest(u) = cost[u] + max longest(后继)	O(n + E)	同样正确，从终点往回想时更直观；但递归深度 = 最长链，且要对所有起点取最大值。Kahn 版本与 ir.order 共享骨架，额外约束（设备内链）直接当作普通边加入即可。
有限设备数的调度（例如 LC 1494 Parallel Courses II）	NP 难（一般情况）	限制同时执行的数量后问题陡然变难，需要状态压缩 DP 或启发式。框架绕开它：先用贪心（LPT）把算子分到设备上，再把「同设备串行」编码成额外的先后边，问题又变回无限并行的关键路径。

易错点

- 编号是 1 起，cost 是 0 起下标：cost[i] 对应算子 i+1；建图时记得减一。
- 答案是所有算子 finish 的最大值，而不是最后一个弹出算子的 finish；可能有多个汇点。
- 方向：[a, b] 是 a 先于 b，与 ir.order 的 [a 依赖 b] 相反；Plan.critical_path 内部做了 (b, a) 翻转再 +1。

模式信号

「无限并行」「最短完成时间」「依赖」——DAG 最长路 = 拓扑序上的 max DP。

相关练习：`ir.order` `ir.order_ok` `ir.op_latency`

`ir.device_groups`

ir.device_groups　count_device_groups

其它 LeetCode　LC 547 · Number of Provinces　并查集

`nanoserve/ir/topology.py` · `count_device_groups(linked: list[list[int]]) -> int`

引擎里的角色

`ir/devices.py::Topology.group_count` 把有向链路对称化成 0/1 邻接矩阵（对角线为 1）再调用 `count_device_groups`，得到「岛」的个数：只有一个岛内的设备才能组成模型并行组。`Topology.islands` 还把它当 oracle 求成员关系：对设备 d 和已有岛的代表 rep，在矩阵里加一条 d–rep 直连边，若组数不变说明两者本来就连通。默认 6 设备拓扑得到 2 个岛 [1,2,3,4] 和 [5,6]，`choose_island` 选最大的那个放模型。

被调用于：`nanoserve/ir/devices.py`

题目

n×n 的 0/1 矩阵 linked（n ⩾ 1，对称，linked[i][i] = 1），linked[i][j] = 1 表示设备 i、j 直接互联；直接或间接互联的设备属于同一组，返回组数。建议用并查集。例：[[1,1,0],[1,1,0],[0,0,1]] → 2。

核心洞见

连通分量计数。并查集从 n 个单点组开始，每成功合并一次（两个根不同）组数减一；扫完所有 i < j 的边即可。

算法

1. parent = list(range(n))，groups = n。
2. find(x)：沿 parent 上溯到根，途中做路径减半 parent[x] = parent[parent[x]]。
3. 对每个 i < j 且 linked[i][j] = 1：ri, rj = find(i), find(j)，不同则 parent[ri] = rj、groups −= 1。
4. 返回 groups。例：上例中 (0,1) 合并一次，groups = 3 − 1 = 2。

时间 $O(n^2 \cdot \alpha(n))$（矩阵输入必须读完 n^2 个格子）　**空间** O(n)

参考实现

```python
def count_device_groups(linked: list[list[int]]) -> int:
    n = len(linked)
    parent = list(range(n))

    def find(x: int) -> int:
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    groups = n
    for i in range(n):
        for j in range(i + 1, n):
            if linked[i][j]:
                ri, rj = find(i), find(j)
                if ri != rj:
                    parent[ri] = rj
                    groups -= 1
    return groups
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS/BFS 遍历矩阵数连通分量	$O(n^2)$	输入是邻接矩阵时两者渐近相同，DFS 代码同样短。并查集的真正优势在**边逐条到来**的增量场景（合并后随时查询组数），以及不想写递归时。题面「用并查集」是给引擎的提示：islands 反复问「加一条边后组数会不会变」，正是并查集的语义。
Floyd–Warshall 传递闭包后数不同的行	$O(n^3)$	能顺便回答任意两点是否连通，但对单纯计数是浪费。
兄弟练习 ir.op_latency	—	本题把链路对称化，回答弱连通（能不能放在一个组里）；broadcast_latency 回答有向可达与时间。一个岛内仍可能没有任何合法 host（test_no_valid_host_is_an_error），所以两步都要做。

易错点

- 只在 find(i) != find(j) 时才减组数；对已在同组的边重复减会得到错误甚至负数。
- 合并的是根（parent[ri] = rj），不是 parent[i] = j，否则会把 i 从原来的树上摘下来。
- 只需扫上三角 i < j（矩阵对称），对角线恒为 1 不代表合并。

模式信号

「直接或间接相连」「有多少组」「邻接矩阵」——连通分量计数，并查集或 DFS。

相关练习：`ir.op_latency` `ir.clone`

---

config.expand `expand_template`

config.expand

其它 LeetCode · LC 394 · Decode String · 栈

`nanoserve/config/template.py · expand_template(s: str) -> str`

引擎里的角色

`config/loader.py::expand_list` 处理 [workers] pool：值里含 [ ] 时先验证契约（只能有方括号且平衡、每个数字后紧跟 [且 k ⩾ 1、每个 [前都有次数、len × Πk ⩽ 100000 防模板炸弹），然后调用 `expand_template`，再按逗号切开、去掉空项、校验 worker kind 名。server.conf：2[gpu-a,]cpu_b → gpu-a, gpu-a, cpu_b；tiered.conf：3[2[hbm,]ddr,] → hbm,hbm,ddr 重复 3 次共 9 个 worker（展开结果末尾的逗号产生空项，被过滤）。test_templates_expand_nested 中 2[big, 2[small,]]solo 得到 7 个 worker。

被调用于：`nanoserve/config/loader.py`

题目

输入由数字、[ ]、其它可打印字符组成。k[片段] 表示片段重复 k 次，可任意嵌套；保证数字只作为重复次数且后面紧跟 [，k 为正整数可多位（10[a]），方括号严格配对。返回完全展开的字符串；无方括号时原样返回。
例："3[a2[c]]" → "accaccacc"。

核心洞见

嵌套重复是一个栈结构：遇到 [时，把「括号外已经拼好的前缀」和「这一层的次数」压栈，开始新的一层；遇到] 时，把当前层重复 k 次接回前缀后面。

算法

1. cur = ""，k = 0，stack = []。
2. 数字：k = k*10 + d（支持多位）。
3. [：push (cur, k)，cur, k = "", 0。
4.]：(prev, times) = pop()，cur = prev + cur*times。
5. 其它字符：cur += ch。返回 cur。例："3[a2[c]]"：遇内层] 时 cur = "a" + "cc"；遇外层] 时 cur = "" + "acc"*3。

时间 与构造出的字符串总长成正比，最坏 O(d · |输出|)，d 为嵌套深度　空间 O(|输出|)

参考实现

```
def expand_template(s: str) -> str:
    stack: list[tuple[str, int]] = []
    cur, k = "", 0
```

```
    for ch in s:
        if ch.isdigit():
            k = k * 10 + int(ch)
        elif ch == "[":
            stack.append((cur, k))
            cur, k = "", 0
        elif ch == "]":
            prev, times = stack.pop()
            cur = prev + cur * times
        else:
            cur += ch
    return cur
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归下降：decode(i) 返回（展开串, 结束位置）	同上，递归深度 = 嵌套深度	结构上更像文法（seq := (char | k[seq])*），写多层语法时更易扩展；栈版本把递归显式化，避免 Python 递归上限，代码也更短。
正则反复替换最内层 r'(\d+)\[([^\[\]]*)\]'	每轮 O(|当前串|)，轮数 = 深度	能用，但每轮重扫整个（越来越长的）字符串，且依赖 re。配置加载禁止依赖正则做核心解析的理由同 config.glob。
兄弟练习 config.expr	O(n)	同一个「遇左括号把外层上下文压栈、遇右括号弹出合并」的模式：expr 压 (累计值, 符号)，本题压 (前缀串, 次数)，config.flags 压字符直到 (。看到「可嵌套的括号 + 外层需要内层结果」就想到这个栈。

易错点

- 多位数次数：10[a] 要累计 k = k*10 + d，而不是只取最后一位。
- 压栈后要把 k 清零，否则内层的次数会叠加到外层的数字上。
- 展开结果可能指数级增长（3[3[3[…]]]）——练习本身不防，loader 在调用前用 len × Πk ⩽ 100000 拦截。

模式信号

「k[片段] 可嵌套」「重复展开」——(前缀, 次数) 栈。

相关练习：`config.expr` `config.brackets` `config.flags`

config.arith

config.arith　eval_arith

其它 LeetCode　LC 227 · Basic Calculator II　栈

`nanoserve/config/arith.py` · `eval_arith(s: str) -> int`

引擎里的角色

`config/loader.py::eval_number` 在值里出现 * 或 / 时调用 `eval_arith`（同一个值里不能再有括号）。调用前 `_check_arith` 保证 token 严格交替为「数 运算符 数 …」（没有一元负号）、不会除以 0；再用 $\Pi(n_i+1) \le 2^{31}-1$ 这个上界保证任何 + − × ÷ 组合的中间结果都在 32 位内。server.conf：max_batch = 2 * 2 = 4，prefill_chunk = 64 / 4 - 8 = 8；tiered.conf：kv_blocks = 1024 / 8 - 2 * 4 = 120；测试：10 / 3 * 3 + 7 / 2 - 1 = 11。

被调用于： `nanoserve/config/loader.py`

题目

s 只含非负整数、+ - * / 和空格，无括号、无一元负号，保证合法非空。* / 优先于 + −，同级从左到右；整数除法向零截断（与 Python 的 // 不同）；中间值在 32 位内，不许 eval。例："3+2*2" → 7，" 3+5 / 2 " → 5。

核心洞见

把表达式看成若干「项」的和：每遇到一个运算符，就用**上一个**运算符处理刚读完的数——+ 把 num 入栈，− 把 −num 入栈，* / 与栈顶合并。最后把栈求和。乘除被立刻吸收进当前项，于是自然高于加减。

算法

1. stack = []，num = 0，op = '+'（假想开头有一个 +）。
2. 逐字符：数字则 num = num*10 + d。
3. 遇到非空格的非数字字符，或到达最后一个字符时：按 op 处理 num（+：push num；−：push −num；*：push pop()*num；/：push int(pop()/num)），然后 op = 当前字符，num = 0。
4. 返回 sum(stack)。例："14-3/2"：栈 [14, −3]，遇 / 后结算 int(−3/2) = −1，结果 13。

时间　O(n)　空间　O(n)（可优化到 O(1)）

参考实现

```python
def eval_arith(s: str) -> int:
    stack: list[int] = []
    num, op = 0, "+"
    for i, ch in enumerate(s):
        if ch.isdigit():
            num = num * 10 + int(ch)
        if (not ch.isdigit() and ch != " ") or i == len(s) - 1:
            if op == "+":
                stack.append(num)
            elif op == "-":
                stack.append(-num)
            elif op == "*":
                stack.append(stack.pop() * num)
            else:
                stack.append(int(stack.pop() / num))
            op, num = ch, 0
    return sum(stack)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
O(1) 空间：只保留 result 与上一项 last	O(n) 时间、O(1) 空间	栈里真正会被修改的只有栈顶，所以可以用 last 代替：遇 + − 时 result += last、last = ±num；遇 * / 时 last = last op num。面试追问空间时用它；栈版本更容易推广到括号（Basic Calculator III）。
除法写成 stack.pop() // num	O(1)	错误：栈里可能是负数（来自减号），"14-3/2" 用 // 得 −3 // 2 = −2，结果 12；规格要求向零截断得 13。这就是规格特意强调「注意与 Python 的 // 不同」的原因。
兄弟练习 config.expr / ir.rpn	O(n)	expr 处理括号和一元负号但没有优先级；rpn 已经是后缀形式，无需处理优先级。三者合起来正好说明为什么编译器喜欢后缀字节码：中缀的优先级和括号都要靠栈「翻译」掉。

易错点

- 最后一个数没有后继运算符，必须用「i == len(s) − 1」触发结算；参考解里末尾是空格时也会触发，因为空格不会清零 num。
- 空格要跳过但不能触发结算，否则 "3 + 5" 会在空格处把 op 改成空格。
- 向零截断：int(a / b)，不是 a // b。

模式信号

「+ − * / 有优先级、无括号」「不许 eval」——上一个运算符 + 项栈。

相关练习：`config.expr` `ir.rpn`

---

ir.depends_on　depends_queries

ir.depends_on

其它 LeetCode　LC 1462 · Course Schedule IV　拓扑排序

`nanoserve/ir/reach.py · depends_queries(n: int, deps: list[list[int]], queries: list[list[int]]) -> list[bool]`

引擎里的角色

`ir/compiler.py::precedence` 把依赖反转成「b 先于 a」的 runs_first 对，然后对所有 n(n−1) 个有序对一次性调用 `depends_queries`，填出 n×n 的 `Plan.before` 矩阵。它被用了两次：`compile_ir` 用 before[i][out] 判定死算子（不是输出的祖先就删），`Plan.parallel_pairs` 用「两个方向都为假」找可并行的算子对——toy 图 78 对里有 50 对，例如 dist4|dist3；dist4|logits 是传递依赖，不可并行。n ⩽ MAX_OPS = 100，所以查询数 ⩽ 9900，正对应规格里的 len(queries) ⩽ 10^4。

被调用于：`nanoserve/ir/compiler.py`

题目

算子 0..n−1（n ⩽ 100），deps[i] = [a, b] 表示 a 必须先于 b（DAG，无重复，a ≠ b，可为空）；先后约束可传递。queries[j] = [u, v]（u ≠ v）问 u 是否直接或传递地必须先于 v，返回等长布尔列表。例：3, [[1,2],[1,0],[2,0]], [[1,0],[1,2]] → [True, True]。

核心洞见

查询多（$\leqslant 10^4$）、点少（$\leqslant 100$）：先求传递闭包，再 O(1) 回答。在拓扑序上传播「祖先集合」：v 的祖先 = ∪（前驱 u 的祖先 ∪ {u}），用整数位掩码表示集合，一次 | 就是一次集合并。

算法

1. 建 succ[a].append(b)，indeg[b] += 1。
2. before = [0]*n（第 v 个整数的第 u 位表示 u 必须先于 v）。
3. Kahn 顺序弹出 u；对每个后继 v：before[v] |= before[u] | (1 << u)，入度归零则入队。
4. 按拓扑序处理保证弹出 u 时 before[u] 已完整。
5. 回答：before[v] >> u & 1。例：1→2→0：before[2] = {1}，before[0] = {1} ∪ {1,2} = {1,2}，所以 [1,0] 为真。

时间 O((n + E) · n / w + q)，w 为机器字长（Python 大整数按字运算）　**空间** $O(n^2 / w)$

参考实现

```python
def depends_queries(n: int, deps: list[list[int]], queries: list[list[int]]) -> list[bool]:
    succ: list[list[int]] = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in deps:
        succ[a].append(b)
        indeg[b] += 1
    # before[v] is a bitmask of every operator that must run before v.
    before = [0] * n
    q = deque(i for i in range(n) if indeg[i] == 0)
    while q:
        u = q.popleft()
        for v in succ[u]:
            before[v] |= before[u] | (1 << u)
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return [bool(before[v] >> u & 1) for u, v in queries]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每个查询 DFS/BFS 一次	O(q · (n + E))	无预处理，查询很少或图频繁变化时合适；这里 q 可达 n(n−1)，总量是闭包的约 n 倍，而 precedence 恰好每次都要问全部对。
Floyd–Warshall 式可达性 reach[i][j] |= reach[i][k] & reach[k][j]	$O(n^3)$	n = 100 时 10^6 次，完全可以接受、代码最短，而且不需要 DAG 前提；位掩码版本在稀疏 DAG 上快很多，并且与 ir.order、ir.critical_path 共享 Kahn 骨架。
从每个点 BFS 一次求闭包	O(n · (n + E))	也是标准做法（LC 1462 常见解），不依赖拓扑序；位掩码版把 n 次遍历压缩成一次遍历 + 字级并行。

易错点

- 方向：本题 [a, b] 是「a 先于 b」，而 IR 的 deps() 是「a 依赖 b」；precedence 先翻转成 [b, a] 再调用。
- 传播时要同时并上 before[u] 和 u 自己（1 << u），只并 before[u] 会漏掉直接前驱。
- 必须按拓扑序传播；按编号顺序扫一遍会在 u 的祖先尚未传全时就把它传给 v。

模式信号

「传递地先于」「批量查询」「n ⩽ 100」——先求传递闭包（位掩码），再 O(1) 回答。

相关练习：`ir.order` `ir.computable` `ir.critical_path`

`ir.computable`

`ir.computable` `computable_outputs`

其它 LeetCode | LC 2115 · Find All Possible Recipes from Given Supplies | 拓扑排序

`nanoserve/ir/reach.py` · `computable_outputs(outputs: list[str], inputs: list[list[str]], provided: list[str]) -> list[str]`

引擎里的角色

`ir/compiler.py::prune(plan, available)` 用全部 params 加上实际存在的 feeds 作为 provided，把每个算子的输出名和**必需**输入（不含 sum 的 optional 项）交给 `computable_outputs`，得到可计算集合；输出不在其中就抛 GraphError("not computable")。上下文只有 2 个 token 时 ctx4、ctx3 不存在，dist4、dist3 以及依赖它们的 term4、term3 算不出来，被剪掉 4 个算子；`IRModel.specialize` 按 feed 集合缓存结果，长度 0..4 分别得到 5/7/9/11/13 个算子。

被调用于：`nanoserve/ir/compiler.py`

题目

outputs[i] 是第 i 个算子的输出名，需要 inputs[i] 中的全部张量；provided 一开始就有、可被重复使用；算子输出也可作别人的输入。inputs 里可能有既非 provided 也非任何输出的名字；处在依赖环上的输出算不出来。返回所有可算出的输出名（顺序任意，不重复），O(算子数 + Σ|inputs[i]|)。

核心洞见

把张量和算子看成二部图，从 provided 出发做 Kahn 式传播：每个算子维护「还缺几个输入」，一个张量就绪就让所有用到它的算子计数减一，归零即可计算，其输出成为新的就绪张量。环上的算子计数永远到不了 0，自然被排除。

算法

1. users[name] = 用到该张量的算子下标列表；missing[i] = len(inputs[i])。
2. ready = deque(provided)。
3. 弹出张量 name，对每个 i ∈ users[name]：missing[i] −= 1；归零则把 outputs[i] 加入结果并入队。
4. 队列空时返回结果。例：outputs [attn, proj]，inputs [[q,k],[attn,w_o]]，provided [q,k,w_o]：q、k 使 attn 归零，attn 入队；w_o 和 attn 使 proj 归零。

时间 O(m + Σ|inputs[i]| + |provided|)　空间 O(m + Σ|inputs[i]|)

参考实现

```python
def computable_outputs(outputs: list[str], inputs: list[list[str]], provided: list[str]) -> list[str]:
    users: dict[str, list[int]] = defaultdict(list)
    missing = [len(ins) for ins in inputs]
    for i, ins in enumerate(inputs):
        for name in ins:
            users[name].append(i)
    ready = deque(provided)
    result: list[str] = []
    while ready:
        name = ready.popleft()
        for i in users.get(name, ()):
            missing[i] -= 1
            if missing[i] == 0:
                result.append(outputs[i])
                ready.append(outputs[i])
    return result
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
DFS + 记忆化 + 三色判环：can(x) = x ∈ provided 或 (x 是输出且其全部输入 can)	O(m + Σ|inputs|)	LC 2115 的另一种常见解，从「目标」往回问更直观；但必须正确处理环（灰色节点视为不可算）且有递归深度问题。只关心少数几个目标输出时 DFS 只访问相关子图，更省。
不动点迭代：反复扫描所有算子，能算就标记，直到一轮无变化	O(m · Σ|inputs|)	最容易写对，n 小时可用；最坏每轮只新增一个，平方复杂度。
与 compile_ir 的死代码消除对比	—	死代码消除从输出**反向**看「谁对输出有贡献」（ir.depends_on 的 before[i][out]）；本题从输入**正向**看「谁算得出来」。dist4 永远是输出的祖先，只有正向分析知道 ctx4 缺席时它算不出来。prune 之后又调用 compile_ir，两种剪枝串联。

易错点

- inputs[i] 里的未知名字永远不会就绪，不要预先报错，它只是让该算子算不出来。
- 一个张量可能被多个算子使用，users 要用列表而不是单值。
- prune 只传必需输入：如果把 sum 的 optional 项也算进去，任一 term 缺席都会让 p 和 logits 算不出来。

模式信号

「给定初始供给，哪些能做出来」「产物可作原料」「有环则做不出」——二部图上的 Kahn 传播（缺失计数归零）。

相关练习：`ir.depends_on` `ir.clone` `ir.order`

config.strip_comments

config.strip_comments strip_comments

其它 LeetCode　LC 722 · Remove Comments　状态机

`nanoserve/config/comments.py` · `strip_comments(lines: list[str]) -> list[str]`

引擎里的角色

`config/loader.py::clean_lines` 把文本按行切开（先检查可打印 ASCII），在末尾追加两行哨兵 ["<eof>", "*/<eof>"] 后调用 `strip_comments`：若输出末尾不是原样的这两行，说明文本留下了未闭合的块注释，报「unterminated /* block comment」（练习规格保证块注释闭合，哨兵把这个保证变成了检测）。之后每行 strip、丢空行，交给 split_sections。因为注释最先剥离，// 和 /* 在值里也总是注释——所以 tiered.conf 特意注明路径只能写单斜杠。test_comments_are_removed_everywhere 覆盖了行内块注释 max_/* inline block */batch = 3 和跨行块注释。

被调用于：`nanoserve/config/loader.py`

题目

lines 是按行切开的文本。从左到右，块注释外最先出现的起始符决定含义：// 忽略到行尾；/* 开始块注释直到之后第一个 */（从 /* 之后开始找，/*/ 不会结束）；块注释可跨行，开始前的文本与 */ 之后的文本拼成同一行；块注释外单独的 */ 是普通字符；无引号转义。去注释后为空的行不输出，其他字符（包括空格）原样保留。

核心洞见

一个两状态的扫描器（在块注释内 / 外），状态跨行保留；关键在于输出缓冲区 buf 只在「不在块注释内」时才于行首清空，这样块注释前后的文本会自然拼成一行。

算法

1. in_block = False，buf = []，out = []。
2. 每行开始：若不在块内则 buf = []。
3. 逐字符看两字符窗口：块内遇 */ → 出块、跳 2；块外遇 /* → 入块、跳 2；块外遇 // → 跳过本行剩余；否则块外字符加入 buf。
4. 行末：若不在块内且 buf 非空，输出 ''.join(buf)。
5. 例：["a/*comment", "line", "more_comment*/b"]：第 1 行 buf = [a] 后入块，第 2、3 行不清空 buf，*/ 后追加 b，输出 ["ab"]。

时间　O(总字符数)　空间　O(总字符数)

参考实现

```python
def strip_comments(lines: list[str]) -> list[str]:
    out: list[str] = []
    buf: list[str] = []
    in_block = False
    for line in lines:
        if not in_block:
            buf = []
        i, n = 0, len(line)
        while i < n:
            two = line[i:i + 2]
            if in_block:
                if two == "*/":
```

```
                    in_block = False
                    i += 2
                else:
                    i += 1
            elif two == "/*":
                in_block = True
                i += 2
            elif two == "//":
                break
            else:
                buf.append(line[i])
                i += 1
        if not in_block and buf:
            out.append("".join(buf))
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
把全文用换行拼起来，re.sub(r'/*.*?*/|//[^\n]*', '', text, flags=re.S) 再切行去空	O(n)	正则从左到右找最左匹配，恰好实现了「最先出现的起始符决定含义」，非贪婪 .*? 实现「第一个 */」，因此在本题语义下是对的且很短。我们手写状态机是因为：它是 LC 722 本身；而且 loader 需要用哨兵判断未闭合注释，正则遇到未闭合的 /* 只会静默不匹配。
逐行处理、每行独立用 find('//') / find('/*')	O(n)	无法处理跨行块注释，也会搞错 /* 与 // 谁先出现（例如 a /* // */ b）。需要跨行状态这个信号决定了用状态机。
支持引号内的 // 不算注释（真正的词法分析器）	O(n)	用户体验更好（URL、路径里的 // 不会被吃掉），但要引入字符串字面量和转义规则；本格式刻意不支持引号，代价是写配置时要避开 // 与 /*，loader 的文档里明确写了这一点。

易错点

- 行首清空 buf 只能在块外进行；在块内清空会丢掉 /* 之前的文本。
- /* 之后要跳 2 个字符再找 */，否则 /*/ 会被误判为立即闭合。
- 只剩空格的行要保留（练习规格），丢弃空白行是 loader 的 strip 做的，不是本函数。

模式信号

「块注释可跨行」「最先出现的起始符生效」——跨行保留状态的扫描器。

相关练习：`config.brackets` `config.glob`

config.flags

config.flags　eval_flag_expr

其它 LeetCode　LC 1106 · Parsing A Boolean Expression　栈

`nanoserve/config/flags.py` · `eval_flag_expr(expr: str) -> bool`

引擎里的角色

`config/loader.py::eval_condition` 处理所有布尔值（[server] 的 bool 字段、[model] compiled、[flags] 里的每个 flag）：先把 t/true/on/yes 和 f/false/off/no 统一成 t/f，把已解析的 flag 名替换成 t/f、去掉空格，再用 `_check_condition` 验证语法（!( ) 恰好一个参数、深度 ⩽ 200、没有尾随文本），最后调用 `eval_flag_expr`。flag 之间的引用顺序由 `resolve_flags` 用 topo_order 决定。server.conf：canary = &(stable, !(debug)) = t，paranoid = !(|(stable, canary)) = f，所以 prefix_cache = &(paranoid, fast) = f，compiled = fast = t。

被调用于：`nanoserve/config/loader.py`

题目

语法：t 真，f 假，!(x) 取反（恰好一个子表达式），&(x1,⋯,xn) 与，|(x1,⋯,xn) 或（n ⩾ 1）；只含字符 ( ) & | ! t f ,，无空格，保证合法，长度 ⩽ $2 \cdot 10^4$，深度 ⩽ 200。返回 bool。例："!(&(f,t))" → True，"&(|(f))" → False。

核心洞见

后序求值：一个运算符的结果只有在它的) 出现时才能确定。用字符栈：除逗号外全部入栈；遇到) 就弹出直到 (，只需记录弹出的值里「见过 t / 见过 f」，再弹出运算符算出结果压回。& 只关心有没有 f，| 只关心有没有 t，! 的参数只有一个。

算法

1. 遍历字符：逗号跳过；不是) 就入栈。
2. 遇到)：弹出直到 (，记录 seen_t / seen_f；弹出 (；再弹出运算符 op。
3. ! → seen_f；& → not seen_f；| → seen_t；结果以 't' 或 'f' 压栈。
4. 最后返回栈顶是否为 't'。例："!(&(f,t))"：内层) 时 seen_f → & 得 f；外层) 时 ! 作用于 f 得 t。

时间　O(n)（每个字符入栈出栈各一次）　空间　O(n)

参考实现

```python
def eval_flag_expr(expr: str) -> bool:
    stack: list[str] = []
    for c in expr:
        if c == ",":
            continue
        if c != ")":
            stack.append(c)
            continue
        seen_t = seen_f = False
        while stack[-1] != "(":
            if stack.pop() == "t":
                seen_t = True
            else:
                seen_f = True
        stack.pop()  # "("
        op = stack.pop()
        if op == "!":
```

```
            val = seen_f
        elif op == "&":
            val = not seen_f
        else:
            val = seen_t
        stack.append("t" if val else "f")
    return stack[-1] == "t"
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
递归下降 parse(i) → (value, next_i)	O(n)，递归深度 ⩽ 200	最贴合文法，也能做短路求值（& 遇到 f 后跳过剩余子树，但仍需扫描以找到匹配的)，渐近不变）。深度上限 200 在 Python 默认递归限制内，所以递归同样可行；栈版本无递归、只用一个列表，且与 config.expand / config.expr 的「左括号压栈」模式一致。
替换成 Python 语法再 eval	O(n)	安全上不可接受（配置里不能执行任意代码），而且 &(a,b,c) 的变参语义要先转写。这正是题面「feature flag 条件」在配置系统里出现的意义。
flag 名在 eval 时惰性递归求值（带环检测）	O(总长度)	另一种组织方式：求值遇到 flag 名就递归求那个 flag，需要三色标记防环。loader 选择先 topo_order 确定顺序、再逐个代入 t/f，于是 eval_flag_expr 只需处理只含 t/f 的纯表达式，职责分离更清楚。

易错点

- & 与 | 是变参的，不能假设恰好两个参数；用 seen_t / seen_f 汇总即可。
- ! 的结果是 seen_f（参数为 f 时为真）；因为 ! 恰好一个参数，它也等于 not seen_t，但千万别写成 not seen_f。
- 弹完 (之后还要再弹一次才能拿到运算符；忘了这一步会把运算符留在栈里。

模式信号

「前缀运算符 + 括号 + 逗号分隔的变参」「布尔表达式」——字符栈，遇) 归约。

相关练习：`config.expand` `config.expr` `ir.order`

`config.glob`

config.glob　glob_match

其它 LeetCode　LC 44 · Wildcard Matching　双指针

`nanoserve/config/glob.py` · `glob_match(s: str, p: str) -> bool`

引擎里的角色

`config/loader.py::LoadedConfig.route(model)` 按 [routes] 在文件中的顺序逐条调用 `glob_match(model, pattern)`，第一条命中的规则决定 worker 下标；没有规则时返回 worker 0，有规则但都不命中则抛 NoRoute，`config/deploy.py::Deployment.submit_many` 据此在到达 tick 拒绝请求。route 先拒绝含 ? * 的模型名和超过 2000 字符的名字，保证练习的输入契约。server.conf demo：toy-7b-instruct → toy-*-instruct → 1，ToyXBase → Toy?Base → 2，toy-chat-v2 → toy-chat* → 0，mystery → * → 0；测试中 toy-chat-instruct 同时匹配前两条，first-match 取 worker 1。

被调用于：`nanoserve/config/loader.py`

题目

判断 s 是否与模式 p **整串**匹配：? 匹配恰好一个字符，* 匹配任意序列（含空），二者也能匹配 /；其余字符只匹配自身、区分大小写。s 不含 ?*，两者可为空、长度 ⩽ 2000。不许用 fnmatch/re，复杂度不超过 O(|s| · |p|)。例：("aa","a") → False，("aa","*") → True，("Qwen2-7B","qwen*") → False。

> **核心洞见**
>
> 只需记住**最后一个** *：遇到 * 时记下它的位置 star 和此时 s 的位置 mark，先让它匹配空串；之后一旦失配，就让这个 * 多吞一个字符（mark += 1）从 star+1 重试。更早的 * 不必回溯，因为后一个 * 能吸收任何本该由前一个 * 多吃的字符。

算法

1. i = j = 0，star = −1，mark = 0。
2. 当 i < |s|：若 p[j] 是 ? 或等于 s[i]，i、j 同时前进。
3. 否则若 p[j] 是 *：star = j，mark = i，j += 1（先匹配空串）。
4. 否则若 star ≠ −1：mark += 1，i = mark，j = star + 1（让 * 多吞一个字符）；否则返回 False。
5. s 扫完后跳过 p 末尾所有 *，返回 j == |p|。例：s = "adceb"，p = "*a*b"：第二个 * 先匹配空，失配后逐个吞掉 d、c、e，最后 b 对上。

时间 最坏 O(|s| · |p|)，常见输入接近线性　**空间** O(1)

参考实现

```python
def glob_match(s: str, p: str) -> bool:
    i = j = 0
    star = -1  # index in p of the last '*' seen
    mark = 0  # index in s where that '*' currently stops matching
    while i < len(s):
        if j < len(p) and (p[j] == "?" or p[j] == s[i]):
            i += 1
            j += 1
        elif j < len(p) and p[j] == "*":
            star, mark = j, i
            j += 1
        elif star != -1:
            # Let the last '*' swallow one more character and retry.
            mark += 1
            i, j = mark, star + 1
        else:
            return False
    while j < len(p) and p[j] == "*":
        j += 1
    return j == len(p)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
二维 DP：dp[i][j] = s[:i] 与 p[:j] 是否匹配	O(|s| · |p|) 时间，O(|p|) 滚动空间	严格 O(mn)，正确性最直观（* 时 dp[i][j] = dp[i][j−1] or dp[i−1][j]），也容易扩展到更复杂的模式；2000×2000 = 4 · 10^6 格在 Python 里明显慢于贪心。路由在每个请求上都要跑，贪心的常数更重要。
朴素递归回溯（* 尝试吞 0、1、2… 个字符）	指数级	模式里有很多 * 时（如 *a*a*a*a*b 对 aaaa…a）会爆炸，规格明确点名。加记忆化就退化成上面的 DP。
fnmatch / 正则	库实现	被禁止；而且语义不同：fnmatch 支持 [seq] 字符类（模型名里的 [会被误解释），内部把模式翻译成正则，且在某些平台上大小写不敏感。我们的 ? * 语义需要精确、可预测。
路由层面：最长/最具体匹配 vs first-match	—	first-match 让规则顺序成为显式优先级，简单可预测（兜底 * 放最后）；最具体匹配对写规则的人更宽容，但「具体」难定义。这是 loader 的设计选择，与本题算法无关。

易错点

- s 扫完后 p 可能还剩若干 *，必须全部跳过再判断 j == |p|；"" 与 "***" 应匹配。
- 回溯时要从 mark + 1 重新开始，而不是从当前 i，否则会漏掉 * 吞更多字符的可能或死循环。
- 区分大小写：Toy?Base 不匹配 toyxbase；* 也匹配 /，与 shell glob 不同。

模式信号

「? 单字符、* 任意串、整串匹配」「* 很多时回溯会爆炸」——贪心双指针记最后一个 *（或二维 DP）。

相关练习：`config.strip_comments` `config.norm_path`

M6　解码与评测

M6

解码与评测：投机解码草稿树、评测指标、benchmark 与 fuzz

8 题 · ★ Top 150 6 · 其它 2

	练习	LC	LeetCode 题目	模式
★	`eval.edit_distance`	72	Edit Distance	二维 DP
★	`eval.interleave_ok`	97	Interleaving String	二维 DP
★	`decode.beam_pairs`	373	Find K Pairs with Smallest Sums	堆
★	`bench.choose`	77	Combinations	回溯
★	`bench.orders`	46	Permutations	回溯
★	`bench.json_fuzz`	22	Generate Parentheses	回溯
	`eval.rouge_l`	1143	Longest Common Subsequence	二维 DP
	`decode.regex`	10	Regular Expression Matching	二维 DP

`eval.edit_distance`

`eval.edit_distance` edit_distance

★ Top Interview 150　LC 72 · Edit Distance　二维 DP

`nanoserve/eval/metrics.py` · `edit_distance(a: str, b: str) -> int`

引擎里的角色

`eval/harness.py::edit_score` 先用 `normalize`（小写、去标点、压缩空白）处理模型输出与参考答案，再调用 `edit_distance` 得到字符级编辑距离，写进 `ExampleScore.edit`，`EvalReport.mean_edit` 取平均（demo：12 个样例 mean edit 14.6）。它是最严格的指标：只有完全相同才为 0，测试据此断言 `edit == 0 ⇔ exact_match`。每个样例只算一次、串长几十个字符，O(mn) 完全可接受。

被调用于：`nanoserve/eval/harness.py`

题目

给定字符串 a、b（可空、任意 Unicode，区分大小写，不做归一化），允许插入、删除、替换单个字符，每次代价 1，返回把 a 变成 b 的最少操作数。要求 O(len(a)·len(b))。例：horse→ros 为 3，intention→execution 为 5，「The cache.」→「the cache!」为 2。

核心洞见

dp[i][j] = a 的前 i 个字符变成 b 的前 j 个字符的最小代价。最后一步只可能是三种操作之一（或末字符相等免费对齐），于是 dp[i][j] 只依赖左、上、左上三格，可以滚动成两行。

算法

1. 边界：dp[0][j] = j（插入 j 个），dp[i][0] = i（删除 i 个）；参考解用 `prev = list(range(len(b)+1))` 表示第 0 行。
2. 逐行计算 i = 1..m：`cur[0] = i`。
3. 若 a[i-1] == b[j-1]：cur[j] = prev[j-1]（对齐，不花钱）。
4. 否则 cur[j] = 1 + min(prev[j-1] 替换, prev[j] 删除 a[i-1], cur[j-1] 插入 b[j-1])。
5. 一行算完令 prev = cur；答案是 prev[-1]。例：「blocks」→「block」只需删除末尾 s，结果 1（测试用例）。

时间 O(m · n)　**空间** O(n)（两行滚动；保留整张表则 O(m · n)，可回溯出具体编辑序列）

参考实现

```python
def edit_distance(a: str, b: str) -> int:
    prev = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(prev[j - 1], prev[j], cur[j - 1])
        prev = cur
    return prev[-1]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 eval.rouge_l 的 LCS	O(m · n)	只允许插入/删除时，距离 = m + n − 2 · LCS，所以两者是同一张表的两种读法。差别在用途：编辑距离对长度差和每个字符都计费，一个被截断的正确答案也会扣很多分；ROUGE-L 在词级只看公共子序列占比，容忍截断与插入。harness 两个都报，正是因为它们回答不同的问题。
朴素递归（不记忆化）	O(3^(m+n))	三路分支重复计算同一 (i, j)，几十个字符就爆炸。加 `lru_cache` 后与 DP 同阶，但 Python 递归深度受限（默认约 1000），长输出会栈溢出，所以引擎用自底向上。
带宽 DP / Myers O(ND) 差分	O(d · n)	当只关心距离是否 ⩽ d（例如「近似命中」判定）或两串高度相似时，只算对角线附近 2d+1 条带即可，diff 工具就是这么做的。本题要求精确距离且串很短，没必要。
Hamming 距离	O(n)	只数等长串的逐位不同，一次插入就让后面全部错位——模型输出多一个空格就会被判得很远，不适合评测自由文本。

易错点

- 三个方向别搞混：prev[j] 是删除 a 的字符，cur[j-1] 是插入 b 的字符，prev[j-1] 是替换；滚动数组时要在覆盖前读取 prev[j-1]。
- 边界行列必须初始化为 j 和 i，而不是 0；空串到 b 的距离是 len(b)。
- 规格要求逐字符原样比较：不要在函数里 lower() 或 strip()，归一化是 harness 的 `normalize` 负责的。

模式信号

两个序列、三种单字符操作、求最少步数——经典的双序列 DP，状态是两个前缀长度。

相关练习：`eval.rouge_l`　`eval.interleave_ok`　`decode.regex`

eval.interleave_ok is_interleaving

★ Top Interview 150 | LC 97 · Interleaving String | 二维 DP

`nanoserve/eval/streams.py` · `is_interleaving(a: str, b: str, merged: str) -> bool`

引擎里的角色

`eval/harness.py::check_streams` 的核心。服务器把所有正在运行的请求复用进一条事件流（`ServerReport.events`，元素是 (tick, rid, chunk)）。对每一对运行区间重叠的请求（`concurrent_pairs`），把事件流限制到这两个 rid（`restrict`）得到原始文本，用 `is_stream_of` → `is_interleaving` 检查它是否是两路输出的交错；没与任何人重叠的请求与空串配对。demo：30 个并发对，65 个事件，全部通过；`corrupt_events` 制造的 swap / drop / dup 三种破坏都被涉及受损请求的 3 个检查抓到。

被调用于：`nanoserve/eval/harness.py`

题目

给定 a、b、merged，判断 merged 的字符能否分成两组，一组按原顺序恰好是 a，另一组恰好是 b。长度不等直接 False；三者都可为空、任意字符原样比较。要求 O(len(a)·len(b))。例：(aabcc, dbbca, aadbbcbcac) 为 True，(aabcc, dbbca, aadbbbaccc) 为 False。

核心洞见

用了 a 的前 i 个和 b 的前 j 个，就一定用掉了 merged 的前 i+j 个——merged 的下标不是独立维度。dp[i][j] = merged[:i+j] 能否由 a[:i] 与 b[:j] 交错而成，最后一个字符要么来自 a、要么来自 b。

算法

1. 先检查 len(a) + len(b) == len(merged)，否则 False。
2. dp 为长 n+1 的一维数组（按行滚动 i），dp[0] 在 i = j = 0 时为 True。
3. 对每个 (i, j)，k = i + j − 1 是当前要匹配的 merged 下标：from_a = i>0 且 dp[j]（上一行同列）且 a[i-1] == merged[k]；from_b = j>0 且 dp[j-1]（本行左边）且 b[j-1] == merged[k]。
4. dp[j] = from_a or from_b；全部算完返回 dp[n]。
5. 小例：a=「aab」，b=「aac」，merged=「aacaab」。优先从 a 取的贪心取走两个 a 后卡在 c 上；DP 同时保留「这个 a 来自 a / 来自 b」两种可能，dp[0][3]（先取完 b）为真，最终 dp[3][3] = True。

时间 O(m · n)　空间 O(n)（一维滚动）

参考实现

```python
def is_interleaving(a: str, b: str, merged: str) -> bool:
    m, n = len(a), len(b)
    if m + n != len(merged):
        return False
    dp = [False] * (n + 1)
    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 and j == 0:
                dp[j] = True
                continue
            k = i + j - 1
            from_a = i > 0 and dp[j] and a[i - 1] == merged[k]
```

```
            from_b = j > 0 and dp[j - 1] and b[j - 1] == merged[k]
            dp[j] = from_a or from_b
    return dp[n]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
贪心双指针（能从 a 取就从 a 取）	O(m+n)	a 和 b 有相同字符时会选错分支且无法回头（上面的 aab / aac 例子）。兄弟练习 dash.fuzzy 能用贪心，是因为那里只有一个目标串、没有「分给谁」的选择。
按 rid 重组：把每个请求的 chunk 拼起来与其输出比较	O(L)	事件带可信的 rid 标签时，这更便宜也更强（还能发现两请求之间的错标）。交错 DP 解决的是标签丢失后的问题：客户端只看到一条混合的原始字节流时，「是否存在合法的拆分」就是唯一能问的问题。harness 的成对检查就是在验证「同一连接上的混合文本」这一视角。
对 n 个并发请求做 n 维 DP	O(Π len_i)	一次性检查所有并发请求是指数级的状态空间。harness 拆成两两检查：任何丢字、多字、乱序都会出现在包含受损请求的每一对中（demo 中 3 对全部报错），所以成对检查已足够定位，代价是 O(对数 × m · n)。

易错点

- 忘记先比较总长度：长度不等时 DP 的下标 k 会越界或给出错误的 True。
- 一维滚动时 dp[j] 的旧值代表「上一行」，必须在覆盖前用来算 from_a；而 from_b 读的 dp[j-1] 已经是本行的新值，这正是需要的。
- i = 0 或 j = 0 的边界行列要正常走循环（只有一个来源可用），不要全部置 False。
- swap 两个相同的 chunk 是不可观察的，所以 `corrupt_events` 只挑 c1+c2 ≠ c2+c1 的对；校验器本来就不可能、也不需要识别它。

模式信号

三个串、两个源按原顺序合成第三个、字符可能重复导致选择歧义——状态 (i, j) 隐含 merged 的位置 i+j。

相关练习： `eval.edit_distance` `dash.fuzzy` `eval.rouge_l`

`decode.beam_pairs`

decode.beam_pairs `k_best_pairs`

★ Top Interview 150 · LC 373 · Find K Pairs with Smallest Sums · 堆

`nanoserve/decode/beam.py` · `k_best_pairs(a: list[int], b: list[int], k: int) -> list[list[int]]`

引擎里的角色

`decode/speculative.py::SpeculativeDecoder.draft` 构造 Medusa 式草稿树：两个草稿头（bigram 表猜下一个 token、skip-gram 表猜再下一个）各给出一个按代价（−log p，量化为整数）升序的候选列表，`k_best_pairs` 选出代价和最小的 k 个两 token 猜测作为树的叶子，目标模型在一次验证里给整棵树打分。框架把每个代价编码成 cost · W + rank（W = 2m），让值唯一以便从值映射回 token，且不改变排序。测试 `test_draft_tree_is_the_k_cheapest_pairs` 用 V×V 暴力枚举核对。

被调用于： `nanoserve/decode/speculative.py`

题目

a、b 均非递减（可负、可重复），一对是 [a[i], b[j]]（返回元素值），每个下标对至多用一次。返回和最小的 k 对，1 ⩽ k ⩽ len(a)·len(b)，和相同可任选、顺序任意。要求 O(k log k) 级别，不能枚举全部 m·n 对。例：([1,7,11], [2,4,6], 3) → [[1,2],[1,4],[1,6]]。

> **核心洞见**
> 把 m·n 个和看成 m 条有序链：第 i 条是 a[i]+b[0] ⩽ a[i]+b[1] ⩽ ⋯。这就是「合并 m 个有序链表取前 k 个」：堆里放每条链的当前头，弹出一个就把它在链上的后继推进去。

算法

1. 初始化：只把前 min(k, len(a)) 条链的头 (a[i]+b[0], i, 0) 放进堆并 heapify——第 k 条之后的链头至少有 k 个更小的和，不可能入选。
2. 循环直到取满 k 个：弹出最小的 (s, i, j)，输出 [a[i], b[j]]。
3. 若 j+1 < len(b)，把同一条链的下一个 (a[i]+b[j+1], i, j+1) 推入堆。
4. 每个 (i, j) 只会从 (i, j−1) 被推入一次，因此不需要 visited 集合。
5. 例：a=[1,7,11]，b=[2,4,6]，k=3。堆 {3,9,13} → 弹 3 推 5 → 弹 5 推 7 → 弹 7。结果 [1,2],[1,4],[1,6]。

时间 O(min(k,m) + k log min(k,m))　**空间** O(min(k,m))（堆）+ O(k) 输出

参考实现

```python
def k_best_pairs(a: list[int], b: list[int], k: int) -> list[list[int]]:
    heap = [(a[i] + b[0], i, 0) for i in range(min(k, len(a)))]
    heapq.heapify(heap)
    out: list[list[int]] = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append([a[i], b[j]])
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
暴力：生成全部 m·n 个和再排序 / 取前 k	O(mn log(mn))	草稿头的词表 V 有几十到上百个 token 时，V^2 对就是上万；而我们只需 k ⩽ 16 个。框架还把每个列表截到前 m = min(k, V) 个，因为前 k 对不可能用到某一列表的第 k+1 名。
只从 (0,0) 出发、每次推 (i+1,j) 与 (i,j+1)，配 visited 集合	O(k log k)	同阶，但需要哈希集合去重，堆最多涨到 2k。「每行一条链」的写法用结构保证不重复，更简洁。
二分答案：二分阈值 S，双指针数出 ⩽ S 的对数	O((m+n) log(值域))	k 接近 m·n 或只要第 k 小的和时更好（不依赖 k）。但还要再收集具体的对、处理等于阈值的并列，草稿树 k 很小，用堆更直接。
兄弟练习 kv.merge_free_k（合并 k 个有序链表）	O(N log k)	同一个「多路归并 + 堆」模式。这里的链是隐式的（a[i] 加上整个 b），不需要真的建出链表。

易错点

- 返回的是元素值不是下标；框架因此要把值编码成唯一的（cost · W + rank），自己写引擎代码时别假设值唯一。
- 堆里必须带上 (i, j)，只放和会丢失后继信息；元组比较在和相等时会继续比 i, j，不会报错。
- 初始化只放前 min(k, len(a)) 行；全部放进去也对，但堆变大到 m。

模式信号

两个有序列表、组合代价是和、只要最小的 k 个且明确禁止枚举全部——多路归并的隐式链加最小堆。

相关练习：`kv.merge_free_k` `sample.kth_largest` `metrics.slowest_k`

---

bench.choose

bench.choose choose

★ Top Interview 150 · LC 77 · Combinations · 回溯

`nanoserve/bench/sweep.py` · `choose(n: int, k: int) -> list[list[int]]`

引擎里的角色

`bench/runner.py::grid` 把 n 个特性开关（CORE 的 batch8 / budget / prefill4 加上可导入的 kv / prefix / edf）展开成「恰好 k 个打开」的配置，对每个 k ∈ sizes 调用 `choose(n, k)`。框架把每个组合排序后再整体排序（`choose` 的顺序是任意的），并丢掉两个开关改写同一配置字段的组合。demo：6 个开关、sizes = (1, 2) → 1 + 6 + 15 = 22 行，全部 `same-out = yes`。

被调用于：`nanoserve/bench/runner.py`

题目

从 1..n 中选 k 个互不相同的数，列出所有组合（不计顺序，[1,2] 与 [2,1] 是同一个，只能出现一次），组合之间和组合内部的顺序任意。1 ⩽ k ⩽ n ⩽ 20。choose(4, 2) 共 6 个。

核心洞见

规定组合内部严格递增，就把「无序集合」变成「唯一的有序表示」，回溯时下一个数只从上一个数之后选，天然不重复。剪枝：剩余可选的数不够填满时就停。

算法

1. 维护 path（当前部分组合）与 start（下一个可选的最小数）。
2. len(path) == k 时把 path 的副本加入结果并返回。
3. 候选 x 从 start 到 n − (k − len(path)) + 1：上界保证 x 之后还剩至少 k − len(path) − 1 个数可选。
4. 选 x：path.append(x)，递归 bt(x + 1)，回来后 pop。
5. 例：n=4, k=2。path=[1] 时 x 可为 2,3,4；path=[] 时 x 上界是 3（选 4 之后无数可配），共 [1,2],[1,3],[1,4],[2,3],[2,4],[3,4]。

时间 O(k · C(n,k))（每个结果复制 k 个数）　空间 O(k) 递归栈（不计输出）

参考实现

```
def choose(n: int, k: int) -> list[list[int]]:
    out: list[list[int]] = []
    path: list[int] = []
```

```
    def bt(start: int) -> None:
        if len(path) == k:
            out.append(path[:])
            return
        for x in range(start, n - (k - len(path)) + 2):
            path.append(x)
            bt(x + 1)
            path.pop()

    bt(1)
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
itertools.combinations	O(k · C(n,k))	生产代码直接用它（C 实现、惰性生成）。练习手写是为了掌握「有序化去重 + 剪枝」这一回溯模板，bench.orders、bench.json_fuzz 都是它的变体。
位掩码：枚举 0..2^n−1，筛出 popcount == k 的	O(2^n · n)	n = 20 时要扫约 100 万个掩码，而 C(20,2) 只有 190，k 很小或很大时浪费严重；k ≈ n/2 时差距不大，且无递归，适合想要「配置 = 一个整数」的场景。
随机抽样若干配置	O(样本数)	开关多（例如 40 个）时 C(40,20) 无法穷举，只能抽样或用 pairwise 覆盖设计。我们 n ⩽ 20、k ⩽ 2，穷举只有 22 次 server 运行，并且能对每个组合做「输出与 baseline 完全相同」的断言，这比抽样更有说服力。

易错点

- 加入结果时必须复制 path（path[:]），否则所有结果都指向同一个最后被清空的列表。
- 上界写成 n 而不剪枝结果依然正确，只是多走死路；写成 n − (k − len(path)) 会漏掉最后一个可选数（off-by-one）。
- runner 依赖 `sorted(sorted(c) for c in choose(n, k))` 得到确定顺序，所以返回顺序可以任意，但每个组合恰好一次。

模式信号

列出所有「选 k 个、不计顺序」的子集——组合回溯，用递增顺序去重。

相关练习：`bench.orders` `bench.json_fuzz`

bench.orders

bench.orders arrival_orders

★ Top Interview 150 | LC 46 · Permutations | 回溯

`nanoserve/bench/fuzz.py` · `arrival_orders(ids: list[int]) -> list[list[int]]`

引擎里的角色

`bench/runner.py::check_arrival_orders` 的输入生成器。同一个 tick 到达的请求进入队列的顺序是任意的，调度器必须做到：顺序影响谁先等待，但不影响任何请求的输出（每请求独立 seed）。对 m 个同 tick 请求（ServerConfig(max_batch=2)，槽位比请求少），枚举 `arrival_orders([1..m])` 的每一种顺序重放服务器。demo：4 个请求 24 种顺序 → 24 种不同调度、1 种输出集合，OK。

被调用于： `nanoserve/bench/runner.py`

题目

ids 中的整数互不相同，1 ⩽ len(ids) ⩽ 6，返回全部 len(ids)! 个排列，每个恰好一次，排列之间顺序任意。

核心洞见

第 i 个位置轮流放剩下的每个元素：把位置 i 与 j ⩾ i 交换，就把 cur[i:] 中的每个元素依次放到位置 i，然后递归处理 i+1，回来再换回，原数组本身就是「已用 / 未用」的划分。

算法

1. cur 为 ids 的拷贝，从 bt(0) 开始。
2. i == len(cur) 时加入 cur 的副本。
3. 对 j = i..n−1：交换 cur[i], cur[j]，递归 bt(i+1)，再交换回来。
4. 例：[1,2,3]，i=0 依次让 1、2、3 当首位；每个首位下 i=1 再两种，共 6 个排列。

时间 O(n · n!)　空间 O(n) 递归栈（不计输出）

参考实现

```python
def arrival_orders(ids: list[int]) -> list[list[int]]:
    out: list[list[int]] = []
    cur = ids[:]

    def bt(i: int) -> None:
        if i == len(cur):
            out.append(cur[:])
            return
        for j in range(i, len(cur)):
            cur[i], cur[j] = cur[j], cur[i]
            bt(i + 1)
            cur[i], cur[j] = cur[j], cur[i]

    bt(0)
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
used[] 布尔数组 + path	O(n · n!)	同阶；需要额外 O(n) 标记但输出按字典序（若输入有序），调试友好。交换法原地、无标记，但输出顺序不是字典序——本题规格允许任意顺序。
itertools.permutations / Heap 算法 / next_permutation（LC 31）	O(n!) 摊还	Heap 算法每个排列只做一次交换，next_permutation 可无递归地逐个产出，适合流式重放而不必先把 n! 个列表存起来。n ⩽ 6（最多 720 个）时存储不是问题。
随机打乱若干次（random.shuffle）	O(样本 · n)	n 大时只能这样；但「所有顺序输出都相同」是一个全称命题，抽样只能给统计信心。n = 4 时穷举 24 次就是证明。
兄弟练习 bench.choose	—	组合不计顺序，用递增 start 去重；排列计顺序，每层要考虑所有未用元素。题面里「到达顺序」就是信号。

易错点

- 递归返回后忘记把交换换回来，后续分支会在被打乱的数组上继续，产生重复与遗漏。
- 加入结果时要 cur[:] 复制。
- 直接修改调用者传入的 ids：参考解先 `ids[:]` 拷贝，避免副作用。

模式信号

「所有到达顺序」「每个排列恰好一次」，规模 ⩽ 6——全排列回溯，穷举可行。

相关练习：`bench.choose` `bench.json_fuzz`

---

bench.json_fuzz balanced_brackets

bench.json_fuzz

★ Top Interview 150 · LC 22 · Generate Parentheses · 回溯

`nanoserve/bench/fuzz.py` · `balanced_brackets(n: int) -> list[str]`

引擎里的角色

`bench/runner.py::fuzz_brackets` 用它穷举 n 对括号的全部合法结构，再用 `typed` 按嵌套深度着色成 ()[]{}（3 种偏移，所以每个结构得到 3 个不同字符串），对每个合法串生成全部单字符替换 / 删除 / 插入变体（`mutations`），喂给 M5 的配置预检查 `config/parser.py::brackets_balanced`：合法串必须全部接受，变体必须全部拒绝。demo n = 6：132 个结构 → 396 个合法串、52668 个变体，0 误拒 0 误收。

被调用于：`nanoserve/bench/runner.py`

题目

生成所有由 n 对圆括号组成的合法串（长 2n，正确配对），每个恰好一次，顺序任意，1 ⩽ n ⩽ 8。n = 3 时共 5 个：((()))、(()())、(())()、()(())、()()()。

核心洞见

合法串等价于任意前缀中「)」的数量不超过「(」，且总数各为 n。回溯时只在保持这个不变量的前提下追加字符，生成的每个叶子都合法，不需要事后过滤。

算法

1. 状态 (s, opened, closed)。
2. len(s) == 2n 时收集 s。
3. opened < n 时可以追加 「(」。
4. closed < opened 时可以追加 「)」——永远不会产生非法前缀。
5. 结果个数是 Catalan 数 C_n：1, 2, 5, 14, 42, 132, 429, 1430（测试核对前 5 个）。

时间 O(n · C_n)，C_n ≈ 4^n / (n^1.5 · √π)　**空间** O(n) 递归深度（不计输出）

参考实现

```python
def balanced_brackets(n: int) -> list[str]:
    out: list[str] = []

    def bt(s: str, opened: int, closed: int) -> None:
        if len(s) == 2 * n:
            out.append(s)
            return
        if opened < n:
            bt(s + "(", opened + 1, closed)
        if closed < opened:
            bt(s + ")", opened, closed + 1)

    bt("", 0, 0)
    return out
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
枚举全部 2^(2n) 个串再用栈过滤	O(n · 4^n)	n = 8 时 65536 个串中只有 1430 个合法，98% 的工作都浪费；剪枝回溯只走能成功的分支。
按首个闭合位置分解的 DP：(A) B	O(n · C_n)	每个合法串唯一地写成 「(」 + A + 「)」 + B，按 A 的对数枚举并记忆化各规模的结果列表。适合要反复取不同 n 的场景；单次调用回溯更省内存。
随机 fuzz：随机生成 ()[]{} 串	O(样本)	随机长 12 的六字符串中，合法串的比例约为 132 · 3^6 / 6^12 ≈ 4.4×10^-5，于是随机 fuzz 几乎从不触发「接受」路径，也很难恰好生成「只差一个字符」的边界变体。生成式穷举 + 单字符变异系统地覆盖了这两类。
兄弟练习 config.brackets	O(n)	那是被测的检查器（栈），这里是测试数据生成器。runner 之所以不需要第二个检查器作为 oracle，是因为单字符变异必然破坏平衡（替换要么改变开闭数量、要么破坏强制配对，增删让长度变奇数）。

易错点

- 闭括号条件写成 closed < n 会生成 「)(」 这类非法串；必须是 closed < opened。
- Python 中 s + 「(」 每次新建字符串，深度 2n 的路径上是 O(n) 拷贝——n ≤ 8 无所谓；规模大时改用 list 追加/弹出再 join。
- n 的上限是 8：runner 在调用前做了这个前置检查，C_8 = 1430 个结构再乘 3 种着色和数百个变体已经是十万量级。

模式信号

「生成所有合法括号」「每个恰好一次」——带计数约束的回溯，结果个数是 Catalan 数。

相关练习：`config.brackets` `bench.choose` `bench.orders`

`eval.rouge_l`

eval.rouge_l lcs_length

其它 LeetCode | LC 1143 · Longest Common Subsequence | 二维 DP

`nanoserve/eval/rouge.py` · `lcs_length(a: str, b: str) -> int`

引擎里的角色

`eval/harness.py::rouge_l` 计算词级 ROUGE-L：先 normalize 并按空格切词，再把每个不同的词映射成一个 Unicode 私用区字符（chr(0xE000 + 编号)），这样字符级的 `lcs_length` 实际是在词序列上跑 LCS；precision = LCS / 输出词数，recall = LCS / 参考词数，F1 写进 `ExampleScore.rouge_f1` 。demo mean ROUGE-L F1 = 0.39。测试：「queue model」对「model queue」的 F1 是 0.5，说明 LCS 尊重顺序。

被调用于： `nanoserve/eval/harness.py`

题目

返回 a 与 b 最长公共子序列的长度（子序列不要求连续，保持相对顺序），任一为空或无公共字符时返回 0，区分大小写，O(len(a) · len(b))。例：abcde 与 ace → 3，abc 与 def → 0。

核心洞见

dp[i][j] = a[:i] 与 b[:j] 的 LCS 长度。末字符相等时一定可以把它们配对（交换论证），dp = 左上 + 1；不等时至少丢掉其中一个，取上、左较大者。

算法

1. 让 b 是较短的串（参考解在 len(a) < len(b) 时交换），dp 为长 len(b)+1 的一维数组。
2. 外层遍历 a 的字符 ca，用变量 prev 记住 dp[i-1][j-1]（覆盖前的 dp[j-1] 旧值）。
3. 内层 j = 1..len(b)：tmp = dp[j]；相等则 dp[j] = prev + 1，否则 dp[j] = max(dp[j]（上）, dp[j-1]（左））；然后 prev = tmp。
4. 返回 dp[-1]。例：「the cache is a tree」对「the prefix cache is a tree」，LCS 为 5 个词，P = 1，R = 5/6（测试用例）。

时间 O(m · n) 空间 O(min(m, n))

参考实现

```python
def lcs_length(a: str, b: str) -> int:
    if len(a) < len(b):
        a, b = b, a
    dp = [0] * (len(b) + 1)
    for ca in a:
        prev = 0  # dp[i-1][j-1]
        for j, cb in enumerate(b, 1):
            tmp = dp[j]
            dp[j] = prev + 1 if ca == cb else max(dp[j], dp[j - 1])
            prev = tmp
    return dp[-1]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 eval.edit_distance	O(m · n)	同一类 DP，但问的是相似度而非代价。demo 里 sched 样例输出比参考少一个词「queue」：编辑距离 6，而 ROUGE-L F1 = 0.91；blocks 样例输出 other：编辑距离也是 6，ROUGE-L 却是 0。字符级距离分不清「截断的好答案」与「完全错误的短答案」，词级 LCS 能。
最长公共子串（要求连续）	O(m · n)	只奖励一整段原样复现，一个插入词就把匹配切断；评测摘要/生成时连续性太苛刻，这正是 ROUGE-L 选子序列的原因。
n-gram 重叠（ROUGE-N / BLEU）	O(m+n)	更快、可衡量局部流畅度，但不看全局顺序（打乱 n-gram 位置分数几乎不变）。LCS 天然要求顺序一致，且不必选 n。
Hunt–Szymanski（按匹配对 + LIS）	O((r + n) log n)，r 为匹配对数	词汇稀疏、匹配对很少时远快于 O(mn)，diff 工具常用；我们的句子只有几个词，平方 DP 更简单。

易错点

- 一维滚动时必须用 prev 保存左上角的旧值；直接用 dp[j-1] 会读到本行已更新的值。
- 不等时取 max(上, 左)，不要误写成 max(上, 左, 左上 + 1)。
- ROUGE 的 F1 在 LCS = 0 时要直接返回 0，避免除以 0（harness 已处理）。

模式信号

两个序列、找「保持顺序、可不连续」的最长公共部分——LCS 双序列 DP。

相关练习：`eval.edit_distance` `dash.fuzzy` `eval.interleave_ok`

decode.regex

decode.regex　full_match

其它 LeetCode　LC 10 · Regular Expression Matching　二维 DP

`nanoserve/decode/regex.py` · `full_match(s: str, p: str) -> bool`

引擎里的角色

`eval/harness.py::check_pattern` 把每个样例的输出约束（只有 「.」 与 「*」 的简化正则，例如 「a tre*」、「.*operators」）与 normalize 后的整段输出做完整匹配，结果写进 `ExampleScore.pattern_ok`，报告的 constraints 通过率由它算出（demo：11 个带约束的样例中 6 个通过，55%）。框架先检查前置条件：模式不能以 「*」 开头、不能有 「**」，否则抛 ValueError。

被调用于：`nanoserve/eval/harness.py`

题目

判断 s 是否被 p 完整匹配。p 中只有 「.」（任意单字符）和 「*」（前一个元素重复 0 次或多次）是特殊字符，其余都是字面量，无转义；保证每个 「*」 前面都有合法元素；s、p 可空，空模式只匹配空串；不得用 re。例：(aa, a) False，(aa, a*) True，(ab, .*) True。

核心洞见

dp[i][j] = s[i:] 能否被 p[j:] 完整匹配。看 p[j+1] 是否为 「*」：是的话要么跳过 「x*」 整体（0 次），要么在首字符匹配时吃掉 s 的一个字符但模式停在原地（多次）；否则必须首字符匹配并同时前进。

算法

1. dp 为 (m+1)×(n+1) 布尔表，dp[m][n] = True（空对空）。
2. i 从 m 倒序到 0（包括 i = m：「a*」 这类模式可以匹配空后缀），j 从 n−1 倒序到 0。
3. first = i < m 且 p[j] ∈ {s[i], 「.」}。
4. 若 j+1 < n 且 p[j+1] == 「*」：dp[i][j] = dp[i][j+2] 或 (first 且 dp[i+1][j])。
5. 否则 dp[i][j] = first 且 dp[i+1][j+1]。返回 dp[0][0]。
6. 例：s=「aa」，p=「a*」：dp[2][0] = dp[2][2] = True → dp[1][0] = first 且 dp[2][0] = True → dp[0][0] = True。

时间　O(m · n)　　空间　O(m · n)（只依赖 i 与 i+1 两行，可压到 O(n)）

参考实现

```python
def full_match(s: str, p: str) -> bool:
    m, n = len(s), len(p)
    # dp[i][j]: s[i:] matches p[j:]
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[m][n] = True
    for i in range(m, -1, -1):
        for j in range(n - 1, -1, -1):
            first = i < m and p[j] in (s[i], ".")
            if j + 1 < n and p[j + 1] == "*":
                dp[i][j] = dp[i][j + 2] or (first and dp[i + 1][j])
            else:
                dp[i][j] = first and dp[i + 1][j + 1]
    return dp[0][0]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
朴素回溯（不记忆化）	最坏指数级	s = 「aaaa⋯ab」、p = 「a*a*a*⋯c」 时，每个 「a*」 都要尝试吃掉 0..m 个 a，组合爆炸。记忆化 (i, j) 后就等价于这张表。
兄弟练习 config.glob（LC 44 通配符）	O(m · n)	glob 的 「*」 自己就匹配任意串，与前一个字符无关，所以可以用「记住最后一个 * 的位置」的贪心回退达到常见情况线性；正则的 「*」 绑定前一个元素，「x*」 是一个整体，贪心不成立，必须 DP。看到「* 修饰前一个字符」就是信号。
编译成 NFA/DFA 在生成时逐 token 屏蔽 logits	每步 O(状态数)	真正的约束解码（outlines、xgrammar 一类）在生成过程中把正则编成自动机，只允许能继续匹配的 token，保证输出一定合法。我们是事后校验：实现简单、无需改采样器，但只能报告失败（demo 有 5 个 FAIL），不能阻止。

易错点

- 外层 i 要包含 m：s 已耗尽时 p 剩下 「x*y*」 仍可匹配，漏掉这一行会把 (「」，「a*」) 判成 False。
- s 里的字面 「.」 和 「*」 只是普通字符；只有 p 中的才特殊。first 的判断是 p[j] 等于 s[i] 或 p[j] 是 「.」，不要反过来检查 s[i]。
- 必须完整匹配：harness 测试断言「the cache」不能匹配「the cach」——不能在 s 未耗尽时返回 True。

模式信号

带 「*」（修饰前一元素）与 「.」 的完整匹配、禁止用 re——后缀 DP，状态 (i, j)。

相关练习：`config.glob` `eval.edit_distance` `eval.interleave_ok`

M7　可观测性与扩缩容

M7

可观测性与自动扩缩容：指标面板、容量规划

9 题 · ★ Top 150 2 · 其它 7

	练习	LC	LeetCode 题目	模式
★	`dash.mask_failed`	73	Set Matrix Zeroes	设计（数据结构）
	`dash.region_sum`	304	Range Sum Query 2D - Immutable	前缀和
★	`dash.fuzzy`	392	Is Subsequence	双指针
	`metrics.slowest_k`	703	Kth Largest Element in a Stream	堆
	`metrics.corrected`	2034	Stock Price Fluctuation	双堆
	`metrics.busy_time`	2276	Count Integers in Intervals	区间合并
	`autoscale.min_throughput`	1011	Capacity To Ship Packages Within D Days	二分答案
	`autoscale.min_rate`	875	Koko Eating Bananas	二分答案
	`autoscale.min_time`	2187	Minimum Time to Complete Trips	二分答案

`dash.mask_failed`

dash.mask_failed　mask_failures

★ Top Interview 150　LC 73 · Set Matrix Zeroes　设计（数据结构）

`nanoserve/dashboard/health.py · mask_failures(m: list[list[int]]) -> None`

引擎里的角色

`dashboard/health_router.py::HealthMatrix.masked` 对 workers × models 健康矩阵的深拷贝调用 `mask_failures`：格子 (w, m) 是模型 m 在 worker w 上副本的路由权重，0 表示故障。故障的「爆炸半径」是整行（同一主机/GPU/驱动不可信，drain 该 worker）加整列（坏 checkpoint 或配置发布，暂停该模型）。`drained_workers`、`paused_models` 与加权轮询 `route` 都只读屏蔽后的矩阵。demo：3 个 worker 中 fail(2, toy-code) → drained [2]、paused [toy-code]，健康感知的运行 14 个完成、10 个被前端拒绝、drain 掉的 worker 上 0 个请求。

被调用于：`nanoserve/dashboard/health_router.py`

题目

m 为 rows×cols 整数矩阵（任意整数，0 表示故障）。只依据调用前原矩阵中 0 的位置，把这些 0 所在的整行、整列置 0，其余不变（新置的 0 不连锁扩散）。原地修改，返回 None；进阶为 O(1) 额外空间。

核心洞见

需要记住「哪些行、哪些列要清零」——这是 rows + cols 个比特。第 0 行和第 0 列本身就可以充当这两组标记，只要先用两个布尔变量把它们自己原来是否含 0 存下来。

算法

1. first_row_zero = 第 0 行是否有 0；first_col_zero = 第 0 列是否有 0。
2. 扫描内部 (i ⩾ 1, j ⩾ 1)：m[i][j] == 0 时令 m[i][0] = 0、m[0][j] = 0 作为标记。
3. 再扫描内部：若 m[i][0] == 0 或 m[0][j] == 0，则 m[i][j] = 0。
4. 最后按两个布尔变量处理第 0 行、第 0 列——必须放在最后，否则提前清零的标记行会污染内部。
5. 例：[[1,1,1],[1,0,1],[1,1,1]] → 标记 m[1][0]、m[0][1] → [[1,0,1],[0,0,0],[1,0,1]]。

时间 O(rows · cols)　**空间** O(1) 额外

参考实现

```python
def mask_failures(m: list[list[int]]) -> None:
    rows, cols = len(m), len(m[0])
    first_row_zero = any(m[0][j] == 0 for j in range(cols))
    first_col_zero = any(m[i][0] == 0 for i in range(rows))
    for i in range(1, rows):
        for j in range(1, cols):
            if m[i][j] == 0:
                m[i][0] = 0
                m[0][j] = 0
    for i in range(1, rows):
        for j in range(1, cols):
            if m[i][0] == 0 or m[0][j] == 0:
                m[i][j] = 0
    if first_row_zero:
        for j in range(cols):
            m[0][j] = 0
    if first_col_zero:
        for i in range(rows):
            m[i][0] = 0
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
两个集合记录要清零的行与列	时间同阶，O(rows + cols) 空间	最清晰也最不容易错。健康矩阵只有几个 worker × 几个模型，生产代码这样写完全合理；O(1) 版本的价值在于矩阵极大或内存受限，并练习「复用输入作标记」的技巧。
边扫描边清零	O(rows · cols)	错误：新置的 0 会被后续扫描当成故障，连锁把整个矩阵清空。规格明确要求只依据原快照——对应运维语义：屏蔽不能级联，否则一个故障会 drain 整个集群。
用哨兵值（如 None 或 −1）标记待清零格子	O(rows · cols · (rows+cols))	元素可以是任意整数，任何整数哨兵都可能与真实权重冲突；改成 None 又破坏类型。而且每发现一个 0 就扫一整行一整列，最坏是立方级。

易错点

- 第 0 行 / 第 0 列要最后处理，并且依据的是开头记下的布尔值，而不是被标记后的 m[0][0]。
- m[0][0] 同时属于第 0 行和第 0 列，不能用它一个格子表示两者——这就是需要两个独立布尔变量的原因（测试专门检查 fail(0, toy-chat) 这个角落）。
- 框架在深拷贝上调用：`masked()` 不得改动实时矩阵，测试断言 `health.grid` 只有故障格变化。

模式信号

「整行整列置零」「只看原矩阵」「O(1) 额外空间」——用第 0 行 / 第 0 列做原地标记。

相关练习：`dash.region_sum` `dash.fuzzy`

`dash.region_sum`

dash.region_sum RegionSum

其它 LeetCode · LC 304 · Range Sum Query 2D - Immutable · 前缀和

`nanoserve/dashboard/region.py · class RegionSum(grid: list[list[int]]): sum_region(r1, c1, r2, c2) -> int`

引擎里的角色

`dashboard/panel.py::Occupancy.__init__` 把一次运行画成 0/1 矩阵（行 = 批处理槽位，集群时是 (worker, slot)；列 = tick；请求占用期间 admitted_at ⩽ t < finished_at 为 1），一次性构造 `RegionSum`。之后 `Occupancy.region` 的每次框选都是 O(1)：`cells` 把矩阵切成粗粒度热力图格子，`heatmap` 还要算每行占用率，`worker_busy_slots` 查某 worker 在某时段的忙碌槽数。demo：4 槽 × 119 tick，每列 3 tick；测试用 300 个随机矩形对拍暴力求和。

被调用于：`nanoserve/dashboard/panel.py`

题目

构造时给定 m×n 整数矩阵（可负，之后不变）；sum_region(r1, c1, r2, c2) 返回闭矩形内元素和，0 ⩽ r1 ⩽ r2 < m，0 ⩽ c1 ⩽ c2 < n。构造 O(m · n)，查询 O(1)。

核心洞见

P[r][c] = 左上角 r×c 子矩阵的和（多一行一列的 0 边界）。任意矩形 = 大前缀 − 上方 − 左方 + 被减了两次的左上角，即二维容斥。

算法

1. 开 (m+1)×(n+1) 全 0 的 P。
2. 逐行累加 row_sum：P[r+1][c+1] = P[r][c+1] + row_sum（上一行同列的前缀 + 本行到 c 的和）。
3. 查询：P[r2+1][c2+1] − P[r1][c2+1] − P[r2+1][c1] + P[r1][c1]。
4. 例：样例矩阵 sum_region(1,1,2,2) = 6+3+2+0 = 11，四次查表即可。

时间 构造 O(m · n)，查询 O(1)　空间 O(m · n)

参考实现

```
class RegionSum:
    def __init__(self, grid: list[list[int]]) -> None:
        m, n = len(grid), len(grid[0])
```

```
        p = [[0] * (n + 1) for _ in range(m + 1)]
        for r in range(m):
            row_sum = 0
            for c in range(n):
                row_sum += grid[r][c]
                p[r + 1][c + 1] = p[r][c + 1] + row_sum
        self.p = p

    def sum_region(self, r1: int, c1: int, r2: int, c2: int) -> int:
        p = self.p
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每次查询直接累加矩形内的格子	O(面积)/次	只渲染一次完整热力图时，所有粗格子恰好铺满矩阵，总代价也就 O(m · n)，前缀和不占优。它赢在查询重叠且频繁时：每行占用率、每个 worker 每个 tick 的忙碌槽数、运维反复框选——这些都会重复扫同一片格子。题面里「数据一次刷新内不变、查询非常频繁」就是信号。
每行一维前缀和	O(行数)/次	实现更简单，矩形很矮（只看一两个槽位）时足够。框选整个集群 12 个槽位时就是 12 次查表，二维版本始终 4 次。
二维 Fenwick 树（兄弟练习 index.range_sum 是一维版）	查询与单点更新各 O(log m · log n)	矩阵会被增量修改时（实时面板每个 tick 追加/改动格子）才需要。我们的矩阵在一次刷新内冻结，静态前缀和是最优解。

易错点

- 下标偏移：P 比 grid 多一行一列，查询时右下角是 r2+1、c2+1，左上角用 r1、c1（不加 1）。
- 容斥要把左上角加回来，漏掉 + P[r1][c1] 会把重叠区域减两次。
- 框架的 `Occupancy.region` 负责把越界矩形裁剪到矩阵内；`RegionSum` 本身依赖前置条件，不做检查。

模式信号

静态二维矩阵、任意矩形求和、查询 O(1)——二维前缀和 + 容斥。

相关练习：`index.range_sum` `dash.mask_failed` `metrics.busy_time`

---

dash.fuzzy `fuzzy_match`

dash.fuzzy

★ Top Interview 150 · LC 392 · Is Subsequence · 双指针

`nanoserve/dashboard/filter.py · fuzzy_match(query: str, text: str) -> bool`

引擎里的角色

`dashboard/panel.py::filter_requests` 对每个请求行的文本 `row_text`（rid、租户、模型名、prompt 拼成一行）调用 `fuzzy_match(query, text)`，实现 fzf 式过滤：输入「r017」只剩 r017，输入「rb」能筛出所有含 r…b 的行。`Dashboard.render` 用它显示「requests matching 'q': x/y」。测试用正则「.*」.join(...) 作为独立 oracle，对拍 70 多个查询。

被调用于：`nanoserve/dashboard/panel.py`

题目

判断 query 的字符是否按顺序（可不连续）出现在 text 中，即能否从 text 删去若干字符得到 query。区分大小写，重复字符要出现相应多次，空 query 恒为 True，O(len(query) + len(text))。例：(abc, ahbgdc) True，(axc, ahbgdc) False，(lm3, llama-3-8b) True。

> **核心洞见**
> 贪心：query 的下一个字符在 text 中越早匹配越好——匹配得早只会给后面留下更多选择，不可能更差（交换论证）。所以一个指针扫 text，另一个指针在匹配时前进即可。

算法

1. i = 0 指向 query 下一个待匹配字符。
2. 遍历 text 的每个字符 c：若 i < len(query) 且 query[i] == c，则 i += 1。
3. 返回 i == len(query)。
4. 例：query=「lm3」，text=「llama-3-8b」：l 匹配第 1 个 l，m 匹配 m，3 匹配 3，i = 3 → True。

时间 O(len(query) + len(text)) **空间** O(1)

参考实现

```python
def fuzzy_match(query: str, text: str) -> bool:
    i = 0
    for c in text:
        if i < len(query) and query[i] == c:
            i += 1
    return i == len(query)
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
兄弟练习 eval.rouge_l：LCS(query, text) == len(query)	O(|q| · |t|)	正确但浪费：LCS 要在两边都做选择，所以需要 DP；是否子序列只有一边可以跳过，贪心就够。同理，eval.interleave_ok 有两个源要分配字符，贪心不成立。
正则「.*」.join(re.escape(q))	回溯引擎最坏可达多项式甚至更差	测试拿它当 oracle 很方便，但每个查询都要编译正则，且回溯式引擎在病态输入上会退化。面板每输入一个字符就要过滤全部请求，用线性双指针更稳。
预处理 text：每个字符的出现位置列表 + bisect	预处理 O(|t|)，每次查询 O(|q| log |t|)	LC 392 的进阶：同一 text 被大量不同 query 查询时（例如对一份超长日志），查询代价与 text 长度近似无关。我们的场景相反——一个 query 对许多短行，直接扫描最好。
真正的 fzf 打分（Smith–Waterman 式 DP）	O(|q| · |t|)	fzf 不仅判断是否匹配，还按连续性、词首加分来排序结果，这需要 DP。我们的面板只过滤不排序，所以布尔贪心足够。

易错点

- 别在 query 已经匹配完后继续比较 query[i]（越界）；参考解用 i < len(query) 守卫。
- 区分大小写：不要 lower()；「The」 与 「the」 不同。
- 空 query 返回 True，空 text 只有空 query 才 True。

模式信号

「按顺序出现、可不连续」「O(|q| + |t|)」——子序列判定，贪心双指针。

相关练习： `eval.rouge_l` `eval.interleave_ok` `config.glob`

---

metrics.slowest_k

metrics.slowest_k KthSlowest

其它 LeetCode · LC 703 · Kth Largest Element in a Stream · 堆

`nanoserve/metrics/slowest.py · class KthSlowest(k: int, initial: list[int]): add(latency: int) -> int`

引擎里的角色

`dashboard/panel.py::slowest_panel` 把完成的请求按 (finished_at, slot, rid) 重放：先用前 k−1 个延迟构造 `KthSlowest` （满足「每次 add 后总数 ⩾ k」的前置条件），之后每完成一个请求就 `add(latency)` ，得到实时的第 k 慢延迟——新请求进入「最慢 k 个」面板的门槛。demo：k = 5，门槛在 20 次完成中从 12 升到 76 tick；集群运行中从 10 升到 29。

被调用于： `nanoserve/dashboard/panel.py`

题目

KthSlowest(k, initial)：k ⩾ 1，initial 可空、可少于 k 个。add(latency) 记录新值并返回迄今全部值中第 k 大的（重复值分别计数，[5,5,4] 中第 2 大是 5）。保证每次 add 后总数 ⩾ k。add O(log k)，额外空间 O(k)。例：k=3，initial [4,5,8,2]，依次 add 3,5,10,9,4 → 4,5,5,8,8。

核心洞见

只需保留目前最大的 k 个值；其中最小的那个就是第 k 大。维护一个大小为 k 的最小堆，堆顶就是答案，新值不超过堆顶就与答案无关，可以丢弃。

算法

1. 构造：heapify(initial)，然后不断 heappop 直到大小 ⩽ k（弹掉的都是较小值）。
2. add：若堆大小 < k，直接 heappush。
3. 否则若 latency > heap[0]，heapreplace（弹出堆顶并压入新值，一次 O(log k)）；否则忽略。
4. 返回 heap[0]。例：k=3，堆 {4,5,8}；add 3 → 不进，返回 4；add 5 → 替换 4，堆 {5,5,8}，返回 5；add 10 → {5,8,10}，返回 5。

时间 构造 O(n + (n−k) log n)，add O(log k)　空间 O(k)

参考实现

```python
class KthSlowest:
    def __init__(self, k: int, initial: list[int]) -> None:
        self.k = k
        self.heap = list(initial)  # min-heap holding the k largest values seen so far
        heapq.heapify(self.heap)
        while len(self.heap) > k:
```

```
            heapq.heappop(self.heap)

    def add(self, latency: int) -> int:
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, latency)
        elif latency > self.heap[0]:
            heapq.heapreplace(self.heap, latency)
        return self.heap[0]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
保存全部值，每次 add 后排序取第 k 个	O(n log n)/次	请求流是无界的，延迟会累积成千上万条，每完成一个请求就重排一遍不可接受；而面板只关心前 k 个。
有序列表 + bisect.insort（或 SortedList）	O(n) 移动 / O(log n)	能随时回答任意 k 的查询，也能列出完整的最慢 k 个，但要保存全部历史；k 固定时最小堆只用 O(k) 内存。若面板要支持「改变 k」或删除（请求被撤回），SortedList 更合适。
兄弟练习 sample.kth_largest（快速选择）	平均 O(n)，一次性	数组给定、只问一次时快速选择最快；流式场景每次重新选择就是 O(n)/次。题面「流式到达、随时知道」是选堆的信号。
兄弟练习 metrics.median（双堆）	O(log n)/次	中位数的位置 n/2 随 n 移动，所以要两个堆保存全部数据并平衡；第 k 大的 k 固定，只需要一个只保留 k 个元素的堆。

易错点

- 用最小堆而不是最大堆：要随时取「前 k 大中的最小者」。Python 的 heapq 本来就是最小堆，正好不用取负。
- initial 少于 k 个时不能立刻返回 heap[0] 作为答案——add 中「大小 < k 先 push」这一分支必不可少。
- 相等时（latency == heap[0]）替换与否结果相同，但用严格大于可以省一次堆操作。
- 框架用前 k−1 个延迟做 initial，是为了满足前置条件；若用前 k 个，第 k 个请求自己的门槛就丢了。

模式信号

「流式」「第 k 大」「O(log k)、O(k) 空间」——固定大小 k 的最小堆。

相关练习：`sample.kth_largest` `metrics.median` `metrics.corrected`

metrics.corrected

metrics.corrected　CorrectableSeries

其它 LeetCode　LC 2034 · Stock Price Fluctuation　双堆

`nanoserve/metrics/series.py` · `class CorrectableSeries: update(ts: int, value: int); current() -> int; maximum() -> int; minimum() -> int`

引擎里的角色

`dashboard/panel.py::ThroughputMonitor.step` 每推进一个 tick，就从服务器事件流重新计算各 tick 的输出字符数：新 tick 上报一次；若服务器抢占了某请求，会丢弃它已流出的事件（之后重算），于是更早 tick 的总数事后变小，monitor 把这些 tick 作为修正重新 `update(ts, v)`。面板显示 `current()`（最新 tick 吞吐）、`maximum()`（峰值）、`minimum()`。demo（20 个 KV 块）：18 次抢占、18 次迟到修正，最终 current 10、peak 14、min 0。

被调用于：`nanoserve/dashboard/panel.py`

题目

update(ts, value) 记录时间戳 ts 的值，乱序到达；ts 已有值则覆盖（旧值作废，不再参与最大/最小）。current() 返回出现过的最大时间戳上的当前值，maximum()/minimum() 返回所有时间戳当前值中的最大/最小。0 ⩽ ts, value ⩽ 10^9，每次操作 O(log n)（允许均摊）。

核心洞见

哈希表 ts → 当前值是真相；两个堆只是「候选」索引，旧条目不必立即删除（堆不支持任意删除）。查询时检查堆顶 (value, ts) 是否仍等于 at[ts]，不等就是过期条目，弹掉再看——懒删除。

算法

1. 状态：at = {ts: value}，latest = 最大时间戳，max_heap 存 (−value, ts)，min_heap 存 (value, ts)。
2. update：at[ts] = value；latest = max(latest, ts)；两个堆各 push 一条（不管旧条目）。
3. current：返回 at[latest]。
4. maximum：当 at[max_heap[0].ts] != −max_heap[0].value 时 heappop；返回堆顶的值。minimum 对称。
5. 例：update(1,10), update(2,5) → max 10；update(1,3) 修正 → 堆顶 (−10, 1) 与 at[1] = 3 不符被弹掉，max = 5。

时间　update O(log n)；查询均摊 O(log n)（每条入堆的条目至多被弹出一次）　空间　O(n)，n 为 update 次数（过期条目在被弹出前一直占空间）

参考实现

```python
class CorrectableSeries:
    def __init__(self) -> None:
        self.at: dict[int, int] = {}
        self.latest = 0
        self.max_heap: list[tuple[int, int]] = []  # (-value, ts), lazily invalidated
        self.min_heap: list[tuple[int, int]] = []  # (value, ts), lazily invalidated

    def update(self, ts: int, value: int) -> None:
        self.at[ts] = value
        self.latest = max(self.latest, ts)
        heapq.heappush(self.max_heap, (-value, ts))
        heapq.heappush(self.min_heap, (value, ts))
```

```python
    def current(self) -> int:
        return self.at[self.latest]

    def maximum(self) -> int:
        while self.at[self.max_heap[0][1]] != -self.max_heap[0][0]:
            heapq.heappop(self.max_heap)
        return -self.max_heap[0][0]

    def minimum(self) -> int:
        while self.at[self.min_heap[0][1]] != self.min_heap[0][0]:
            heapq.heappop(self.min_heap)
        return self.min_heap[0][0]
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
每次查询 max(at.values()) / min(...)	O(n)/次	实现最短；面板每个 tick 都查三个值，序列越长越慢。数据量很小（几十个点）时它反而更快、更省内存。
SortedList（sortedcontainers）或平衡树保存值的多重集合	O(log n)/操作，严格	修正时先删旧值再插新值，内存严格 O(不同 ts 数)，不会积累过期条目，并且还能回答任意分位数。代价是第三方依赖、常数更大。修正非常频繁时（同一 ts 被改很多次）值得换它。
兄弟练习 metrics.window_max（单调队列）	O(1) 均摊	单调队列要求删除按到达顺序（滑动窗口左端出队），这里的作废是任意位置的，单调队列不适用。「可以修正任意历史点」就是选懒删除堆的信号。

易错点

- current 是最大时间戳的值，不是最近一次 update 的值：修正旧 tick 不能改变 current（测试中 update(0, 0) 后 current 仍是 ts=4 的 2）。
- 过期判定用 at[ts] == 堆中的值：同一 ts 被改成 A、再改成 B、又改回 A 时，旧的 (A, ts) 条目会重新「有效」，这恰好是对的，不会出错。
- latest 初始为 0 而 ts 可以是 0：因为查询只会在至少一次 update 之后发生，at[latest] 一定存在——但若把初值设成 −1 然后忘了更新就会 KeyError。
- 值可以为 0：不要用 `if value:` 之类的真值判断。

模式信号

「乱序到达」「可被修正覆盖」「随时最大/最小」——哈希表存真值 + 两个懒删除堆。

相关练习：`metrics.slowest_k` `metrics.window_max` `metrics.median`

metrics.busy_time

metrics.busy_time BusyTimeCounter

其它 LeetCode | LC 2276 · Count Integers in Intervals | 区间合并

`nanoserve/metrics/busy.py` · `class BusyTimeCounter: add(left: int, right: int); count() -> int`

引擎里的角色

`dashboard/panel.py::cluster_utilization` 把每个 worker 的忙碌区间（`busy_spans`，半开 [s, e) 转成闭区间 [s, e−1]）逐个 worker 地喂给 `BusyTimeCounter`：它们按 worker 而非时间顺序到达、彼此重叠。`count()` 就是「至少一个 worker 在计算」的 tick 数，并与 `Cluster` 用 sched.merge_busy 合并出的时间线交叉核对（`Utilization.consistent`）。demo：3 个 worker 各忙 35、44、65 tick，并集 66/66，与合并时间线一致；测试断言并集严格小于各 worker 之和。

被调用于：`nanoserve/dashboard/panel.py`

题目

add(left, right) 加入闭区间 [left, right]（0 ⩽ left ⩽ right ⩽ 10^9），区间可重叠、相邻、包含或相同；count() 返回至少被一个区间覆盖的整数个数。不能逐个整数存储；count O(1)；add 不得扫描全部已有区间或重建并集，应维护有序不相交区间并用二分定位。

核心洞见

维护两张平行的有序数组 starts、ends（不相交区间），再维护覆盖总数 total。与新区间 [l, r] 重叠的已有区间在下标上是连续的一段 lo..hi−1：lo 是第一个 end ⩾ l 的，hi 是第一个 start > r 的——两次二分就能找到，合并这一段即可。

算法

1. lo = bisect_left(ends, left)：之前的区间都在 left 左侧结束。
2. hi = bisect_right(starts, right)：之后的区间都在 right 右侧开始。
3. 若 lo < hi：left = min(left, starts[lo])，right = max(right, ends[hi−1])，并从 total 中减去这些区间的长度。
4. 切片替换 starts[lo:hi] = [left]、ends[lo:hi] = [right]；total += right − left + 1。count 直接返回 total。
5. 例：add(2,3), add(7,10) → total 6；add(5,8)：lo=1（ends=[3,10]，第一个 ⩾5 的是 10），hi=2，与 [7,10] 合并成 [5,10]，total = 2 + 6 = 8。

时间

add O(log n + j)，j 为被合并的区间数（每个区间只会被合并删除一次，故均摊 O(log n)；list 切片的内存搬移按规格忽略）；count O(1)

空间 O(n) 个不相交区间

参考实现

```
class BusyTimeCounter:
    def __init__(self) -> None:
        # Disjoint, sorted closed intervals [starts[i], ends[i]].
        self.starts: list[int] = []
        self.ends: list[int] = []
        self.total = 0

    def add(self, left: int, right: int) -> None:
        # Intervals overlapping [left, right] are exactly indices lo..hi-1.
```

```
        lo = bisect_left(self.ends, left)
        hi = bisect_right(self.starts, right)
        if lo < hi:
            left = min(left, self.starts[lo])
            right = max(right, self.ends[hi - 1])
            for i in range(lo, hi):
                self.total -= self.ends[i] - self.starts[i] + 1
        self.starts[lo:hi] = [left]
        self.ends[lo:hi] = [right]
        self.total += right - left + 1

    def count(self) -> int:
        return self.total
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
把每个被覆盖的 tick 放进 set	O(区间长度)/次，内存 O(覆盖长度)	测试拿它当 oracle（tick 数只有几十），但区间长度可达 10^9，存储和插入都不可行。规格里「不能逐个整数地存储」就是排除它。
每次 count 时用兄弟练习 sched.merge_busy 排序合并	O(n log n)/次	离线、只查一次时这是最简单的写法——Cluster 本来就用它算合并时间线，所以面板拿两者交叉核对。在线场景每次查询都重排不划算。
兄弟练习 sched.insert_busy（LC 57）：线性扫描插入	O(n)/次	同样维护不相交有序区间，但用线性扫描找重叠段。本题把定位换成二分，剩下的只有真正被合并的区间。kv.free_tracker、kv.address_ranges 也是同一族「有序不相交区间」设计。
动态开点线段树	O(log C)/次，C = 10^9	LC 2276 的另一经典解，适合还要支持删除区间或区间查询的场景；节点多、常数大，本题只加不删，有序区间更省。

易错点

- 二分的左右侧别搞反：对 ends 用 bisect_left（end == left 的区间与新区间共享 left 这个点，算重叠），对 starts 用 bisect_right（start == right 同理）。
- 相邻但不重叠的区间（[2,3] 与 [4,5]）是否合并不影响计数，参考解不合并；但重叠一个点必须合并，否则那个点被数两次。
- 合并前先把被删区间的长度从 total 减掉，再加新区间长度；闭区间长度是 right − left + 1。
- 框架传入的是闭区间：半开的 [s, e) 要转成 [s, e−1]。

模式信号

在线加区间、问并集大小、长度到 10^9、要求 count O(1)——有序不相交区间 + 二分定位 + 维护总数。

相关练习：`sched.merge_busy` `sched.insert_busy` `kv.free_tracker` `kv.address_ranges`

autoscale.min_throughput `min_throughput`

其它 LeetCode | LC 1011 · Capacity To Ship Packages Within D Days | 二分答案

`nanoserve/autoscale/sizing.py · min_throughput(work: list[int], days: int) -> int`

引擎里的角色

`autoscale/autoscaler.py::Autoscaler.size_waves`：有序批任务（结果必须按顺序返回）最多分 D 波执行，每波一次性准入 backlog 中连续的一段，服务器的预留预算 `ServerConfig.token_budget`（在途请求的 Σ prompt + max_tokens）限制一波的大小。work[i] = 第 i 个请求的 prompt token 数 + max_tokens，`min_throughput(work, D)` 给出最小预算 C，`pack_waves` 用同一个贪心把 backlog 打包成波次，第 k 波在 k · L tick 放行（L = 最长请求的服务时长）后真实运行。demo：12 个请求、4 波，C = 108（波次 [3,3,2,4]，最后一波 37+23+28+20 恰好 108），预算 108 时 0 个请求等待，107 时有 1 个请求卡住。

被调用于：`nanoserve/autoscale/autoscaler.py`

题目

work[i] ∈ [1, 10^9] 必须按顺序处理，每个周期处理连续一段、任务不可拆分，每周期总量 ⩽ C（所有周期相同）。返回能在至多 days 个周期内完成的最小整数 C。1 ⩽ days ⩽ n ⩽ 5 · 10^4，O(n log(sum))。例：([1..10], 5) → 15，([3,2,2,4,1,1,4], 3) → 6。

核心洞见

可行性关于 C 单调：C 能完成，C+1 也能。给定 C，「尽量往当前周期塞，塞不下就开新周期」的贪心给出最少周期数。于是对答案二分：区间 [max(work), sum(work)]，找第一个使贪心周期数 ⩽ days 的 C。

算法

1. periods_needed(cap)：used = 1，load = 0；对每个 w，若 load + w > cap 则 used += 1、load = 0；load += w。
2. lo = max(work)（最大的单个任务必须能放进一个周期），hi = sum(work)（一个周期全做完）。
3. while lo < hi：mid = (lo+hi)//2；若 periods_needed(mid) ⩽ days 则 hi = mid，否则 lo = mid + 1。
4. 返回 lo。例：[3,2,2,4,1,1,4]，days 3：C = 6 → [3,2] [2,4] [1,4] 三个周期可行；C = 5 → [3,2] [2] [4,1] [4] 四个周期不行。

时间 O(n log(sum(work)))　空间 O(1)

参考实现

```python
def min_throughput(work: list[int], days: int) -> int:
    def periods_needed(cap: int) -> int:
        used, load = 1, 0
        for w in work:
            if load + w > cap:
                used += 1
                load = 0
            load += w
        return used

    lo, hi = max(work), sum(work)
    while lo < hi:
        mid = (lo + hi) // 2
        if periods_needed(mid) <= days:
```

```
            hi = mid
        else:
            lo = mid + 1
    return lo
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
从 max(work) 开始逐个尝试 C	O(n · (sum − max))	work 可达 10^9，答案区间也可达 10^13 量级，线性搜索不可行。真实场景的 token 预算只有几百，线性也能跑，但二分的代码量一样、复杂度对规模不敏感。
DP：dp[i][d] = 前 i 个任务分成 d 段的最小最大段和（LC 410 的 DP 解）	$O(n^2 \cdot days)$	能处理更一般的代价函数（例如每段代价不是简单求和），但 n = 5 · 10^4 时完全不可行。只要可行性检查可以贪心完成，二分答案就是更好的选择。
公式下界 max(max(work), ⌈sum/days⌉)	O(n)	只是下界：任务不可拆分、必须连续分段。[3,2,2,4,1,4] 分 3 段时下界 6 恰好等于答案，但 [1..10] 分 5 段时下界是 11，真实答案 15。可以用作 lo 的更紧初值，不能代替搜索。
兄弟练习 autoscale.min_rate / autoscale.min_time	—	三题同一骨架，差别只在可行性函数：本题是顺序贪心打包（任务有序、连续分段），min_rate 是 Σ⌈q/r⌉（各队列独立），min_time 是 Σ⌊T/t⌋（各 worker 独立）。「必须按顺序、不可拆分」决定了需要贪心模拟而不是一个求和公式。

易错点

- lo 必须是 max(work) 而不是 1：cap 小于某个任务时贪心仍会「数」出一个周期数，可能误判可行。例：[1,2,3,1,1]、days 4，若 lo = 1，C = 2 时贪心数出 4 个周期，会错误地返回 2（正确答案 3）。
- 贪心中判断用 load + w > cap 再开新周期，并且新周期要先清零再加上 w。
- 二分模板：求「第一个可行」用 hi = mid / lo = mid + 1 与 while lo < hi，避免死循环。

模式信号

「至少需要多大的上限」「必须按顺序、连续分段」「逐个尝试会超时」——二分答案 + 贪心可行性检查。

相关练习：`autoscale.min_rate` `autoscale.min_time` `sched.min_launches`

autoscale.min_rate

autoscale.min_rate　min_drain_rate

其它 LeetCode　LC 875 · Koko Eating Bananas　二分答案

`nanoserve/autoscale/sizing.py` · `min_drain_rate(queues: list[int], hours: int) -> int`

引擎里的角色

`autoscale/autoscaler.py::Autoscaler.size_batch`：多 LoRA 服务中一个 batch 只能装一个租户的 adapter，所以每次 launch 只服务一个租户的队列，至多 max_batch 个请求，占用服务器 S tick（请求相同，S = `service_ticks`）。要在 H tick 内清空各租户积压 q_i，就是在 H // S 次 launch 内清空——`min_drain_rate(queues, H // S)` 给出最小 max_batch。验证：`AdapterBatchQueue` 只允许单租户 batch，真实运行服务器。demo：积压 {7,3,11,5}，H = 40，S = 5 → 8 次 launch，max_batch = 4（2+1+3+2 = 8）在 40 tick 内清空；max_batch = 3 需要 50 tick。

被调用于：`nanoserve/autoscale/autoscaler.py`

题目

queues[i] ∈ [1, 10^9]，长度 ⩽ 10^4。每个时间片只能选一个队列，取出至多 rate 个；剩余不足 rate 时取完即止，剩余能力浪费、不能转给别的队列。保证 len(queues) ⩽ hours ⩽ 10^9。返回 hours 个时间片内清空全部队列的最小正整数 rate，O(n log max(queues))。例：([3,6,7,11], 8) → 4。

核心洞见

给定 rate，清空队列 i 需要 ⌈q_i / rate⌉ 个时间片，各队列互不影响，总数 Σ⌈q_i / rate⌉ 关于 rate 单调不增。对 rate ∈ [1, max(queues)] 二分，找第一个使总数 ⩽ hours 的值。

算法

1. lo = 1，hi = max(queues)（rate = max 时每队列一个时间片，因 hours ⩾ n 必然可行）。
2. mid = (lo+hi)//2，need = Σ (q + mid − 1) // mid（整数上取整）。
3. need ⩽ hours 则 hi = mid，否则 lo = mid + 1；lo == hi 时返回。
4. 例：[3,6,7,11]，hours 8：rate 4 → 1+2+2+3 = 8 可行；rate 3 → 1+2+3+4 = 10 不行；答案 4。

时间　O(n log max(queues))　空间　O(1)

参考实现

```python
def min_drain_rate(queues: list[int], hours: int) -> int:
    lo, hi = 1, max(queues)
    while lo < hi:
        mid = (lo + hi) // 2
        if sum((q + mid - 1) // mid for q in queues) <= hours:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
公式 ⌈Σq / hours⌉	O(n)	这是「容量可以跨队列共享」时的答案，是下界但通常不准：每个时间片只服务一个队列，尾部的零头会浪费。[30,11,23,4,20] 在 hours 5 时下界是 18，真实答案是 30。多 LoRA 的 batch 不能混装不同 adapter，正是这种浪费的来源。
从 1 逐个尝试 rate	O(n · max(queues))	max(queues) 可达 10^9，不可行。
兄弟练习 autoscale.min_throughput	O(n log sum)	那里任务有序、一个周期可以跨多个任务（连续一段），可行性需要贪心模拟；这里一个时间片只服务一个队列、队列之间无顺序，可行性是独立求和。题面「每个时间片只能选一个队列」决定了是 Σ⌈q/r⌉。

易错点

- 上取整写成 (q + r − 1) // r，不要用浮点 math.ceil(q / r)——q 到 10^9 时浮点通常没事，但在其它语言或更大规模下会出精度问题。
- lo 从 1 开始而不是 0，否则会除以 0。
- hi 取 max(queues) 而不是 sum：rate 再大也不会让某队列少于 1 个时间片。
- 框架层面：H // S 次 launch 少于租户数时无解，`size_batch` 先抛 ValueError（测试 horizon = 6 的例子）。

模式信号

「最小速率」「每片只服务一个队列、剩余浪费」「逐个尝试会超时」——二分答案，可行性 Σ⌈q/r⌉ ⩽ H。

相关练习：`autoscale.min_throughput` `autoscale.min_time`

`autoscale.min_time`

autoscale.min_time `min_time_for`

其它 LeetCode · LC 2187 · Minimum Time to Complete Trips · 二分答案

`nanoserve/autoscale/sizing.py · min_time_for(per_request_time: list[int], total: int) -> int`

引擎里的角色

`autoscale/autoscaler.py::Autoscaler.size_pool`：一波相同请求在 tick 0 打到异构单流 worker 池上，worker i 每个请求需要 t_i tick（不同 kind 的 prefill_chunk 不同）。`makespan → min_time_for(times, n)` 给出最短完成时间 T；按 t_i 从快到慢逐个加入 worker，直到 T ⩽ 目标。`plan_burst` 让 worker i 承担 ⌊T/t_i⌋ 个请求，在 `PlannedCluster` 上真实运行。demo：30 个请求、目标 40 tick；1 个 worker 要 150 tick；推荐 5 个 worker（t = 5,5,6,6,8），估计 36，真实运行 36 tick（每 worker 7,7,6,6,4）；4 个 worker 估计和实测都是 42，未达标。

被调用于：`nanoserve/autoscale/autoscaler.py`

题目

per_request_time[i] ∈ [1, 10^7]，长度 ⩽ 10^5；每个 worker 从 0 时刻起不间断地逐个处理，到时刻 T 恰好完成 ⌊T / t_i⌋ 个。返回使总完成数 ⩾ total 的最小整数 T（total ⩽ 10^7）。O(n log(min(t) · total))，答案可达 10^14。例：([1,2,3], 5) → 3。

核心洞见

f(T) = Σ⌊T / t_i⌋ 关于 T 单调不减，所以对 T 二分。上界取 min(t) · total：最快的 worker 独自做完全部请求就需要这么久，有它帮忙只会更快。

算法

1. lo = 1，hi = min(t) · total。
2. mid = (lo+hi)//2；若 Σ mid // t_i ≥ total 则 hi = mid，否则 lo = mid + 1。
3. 返回 lo。
4. 例：t = [1,2,3]，total 5：T = 3 → 3+1+1 = 5 可行；T = 2 → 2+1+0 = 3 不行；答案 3。
5. demo 验证：t = [5,5,6,6,8]、n = 30：T = 36 → 7+7+6+6+4 = 30；T = 35 → 7+7+5+5+4 = 28。

时间 O(n log(min(t) · total))　**空间** O(1)

参考实现

```python
def min_time_for(per_request_time: list[int], total: int) -> int:
    lo, hi = 1, min(per_request_time) * total
    while lo < hi:
        mid = (lo + hi) // 2
        if sum(mid // t for t in per_request_time) >= total:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

对比讲解

另一种做法	代价	为什么（不）用它 / 何时它更好
堆模拟：每次弹出「下一次完成最早」的 worker	O(total log n)	直接构造出调度，适合 total 小、还需要具体分配的时候；total = 10^7 时要 10^7 次堆操作，比二分（约 47 轮 × n）慢得多。框架的 `plan_burst` 选择用二分得到 T，再直接按 ⌊T/t_i⌋ 分配，免去模拟。
连续近似公式 T ≈ total / Σ(1/t_i)	O(n)	忽略了取整：每个 worker 最后一个未完成的请求不计数，所以这只是下界，demo 中 30 / (2/5 + 2/6 + 1/8) ≈ 35.0，真实答案 36。可作为 lo 的紧初值，但必须搜索修正。
兄弟练习 sched.assign_workers（LC 1882 堆模拟）	O(m log n)	那里请求按时间到达、需要逐个分配给空闲 worker，只能模拟；这里请求一次性到达、只问完成时间，单调性让二分直接跳到答案。「只问最小时间」而非「每个任务给谁」是信号。
兄弟练习 autoscale.min_rate	—	两题可行性都是独立求和：min_rate 求 Σ⌈q/r⌉ ≤ H 的最小 r（向上取整、单调不增），本题求 Σ⌊T/t⌋ ≥ total 的最小 T（向下取整、单调不减）。比较方向和取整方向都相反，写错一个就差一。

易错点

- hi 用 min(t)·total，不要用 max(t)·total（也对但范围更大）或 total（当 min(t) > 1 时不可行）。
- 其它语言要用 64 位整数：答案可达 10^14，Σ⌊T/t_i⌋ 也可能溢出，可在累加超过 total 时提前退出。
- 框架层面：size_pool 是「逐个加入 worker 再算一次 T」的线性外层，第二层才是二分；测试还用子集暴力枚举证明没有更小的 worker 集合能达标。

模式信号

「最短时间」「到时刻 T 完成 ⌊T/t_i⌋ 个」「答案可达 10^14」——单调的计数函数上二分答案。

相关练习：`autoscale.min_rate` `autoscale.min_throughput` `sched.assign_workers`

第 三 部 分

按模式索引

训练「认模式」：从题面信号到算法选择。

同一种模式的题放在一起看。每组先给出这种模式的「识别信号」与「它和相邻模式的分界」，再列出仓库里用到它的练习以及各自题面中的信号。做完一个里程碑后回到这里，按模式复盘一遍。

二分查找 9

信号：数据**有序**或分段有序（旋转数组、行优先有序矩阵、按时间戳追加的版本列表），要找位置/边界/前驱后继。关键是写清楚不变量： `lo` 左边都满足 P、 `hi` 右边都不满足 P。

分界：在**答案空间**上二分（「最小的 x 使得可行」）是**二分答案**，单调性来自可行性判定而非数据本身；在线插入+查询的场景，单纯二分会被插入的 O(n) 拖累，需要 **BST/树状数组**。

M1	★	`sample.cdf_index` LC 35 Search Insert Position	「严格递增序列中的位置或插入点」+ O(log n) → lower bound 二分。
M2	★	`metrics.merged_median` LC 4 Median of Two Sorted Arrays	两个已排序数组 + 要求 O(log) + 只关心某个秩（中位数/第 k 小）→ 对划分位置二分。
M2		`sched.maintenance` LC 729 My Calendar I	“新区间不得与已有任何区间重叠，冲突就拒绝”：有序存储 + 只查前驱和后继。
M3	★	`kv.ring_search` LC 33 Search in Rotated Sorted Array	「旋转过的有序数组」+「O(log n) 查找」：每次判断哪半有序，再看目标在不在那半。
M3	★	`kv.ring_head` LC 153 Find Minimum in Rotated Sorted Array	「旋转有序数组」+「找最小值 / 逻辑起点」：mid 与右端比较，收缩到断点。
M3	★	`kv.seq_range` LC 34 Find First and Last Position of Element in Sorted Array	“有序数组”“首尾位置”“O(log n)”：用 lower_bound 和 upper_bound 两次二分。
M3	★	`kv.page_lookup` LC 74 Search a 2D Matrix	二维矩阵“逐行展开后整体有序”加上 O(log(mn)) 的要求：展平成一维再二分。
M3		`kv.snapshot` LC 1146 Snapshot Array	“打快照、按快照号查历史”加上“空间与修改次数成正比”：每个槽一条有序修改历史，查询时二分找 floor。
M3		`kv.versioned` LC 981 Time Based Key-Value Store	“时间戳递增写入”“查询不晚于 t 的最新值”：每个键一条有序列表，查询时 bisect_right - 1。

双指针 9

信号：数组/字符串上「原地」「有序」「两个序列对齐」「保留前 k 个」。两个下标各自单调前进，每个元素只被看常数次，所以 O(n)。

分界：如果需要维护的是一段连续区间内的统计量（计数、和），那是**滑动窗口**（双指针的特例，多了窗口内的状态）；如果比较的是两个序列能否逐字符对齐且有「跳过/匹配」的选择，往往要升级为**二维 DP**（例如 glob 中的 `*`、interleave）。

M1	★	`tok.common_prefix` LC 14 Longest Common Prefix	「一批字符串的最长公共前缀」→ 以最短串为界纵向逐列比较。
M1	★	`stream.holdback` LC 58 Length of Last Word	「最后一个单词」+「只需从尾部扫描」→ 反向两段扫描。

M3	★	`kv.compact` LC 27 Remove Element	「原地删除」+「返回新长度、尾部内容不作要求」：读写快慢双指针。
M3	★	`kv.dedupe` LC 26 Remove Duplicates from Sorted Array	「有序数组」+「原地去重、返回长度」：写指针与最后写入值比较。
M3	★	`kv.merge_arrays` LC 88 Merge Sorted Array	「原地合并两个有序数组」+「目标数组尾部预留空位」：从后往前三指针。
M3	★	`kv.rotate_buffer` LC 189 Rotate Array	「数组循环移位」+「O(1) 额外空间」：三次反转。
M3	★	`kv.free_ranges` LC 228 Summary Ranges	“有序、不重复”加上“把连续的值压缩成区间描述”：输入有序，相邻比较就能分段。
M5		`config.glob` LC 44 Wildcard Matching	「? 单字符、* 任意串、整串匹配」「* 很多时回溯会爆炸」——贪心双指针记最后一个 *（或二维 DP）。
M7	★	`dash.fuzzy` LC 392 Is Subsequence	「按顺序出现、可不连续」「O(|q| + |t|)」——子序列判定，贪心双指针。

BST 7

信号：动态集合上要**有序**操作：按序迭代、第 k 小、前驱/后继、区间查询、删除任意元素。中序遍历就是有序序列，几乎所有题都是「中序 + 剪枝」。

分界：只要极值用**堆**更简单；只要计数/求和用**树状数组**；BST 不平衡时高度可能退化到 n——本仓库用「深度超阈值就整体重建」代替旋转，生产代码会用红黑树/跳表/ `sortedcontainers` 。

M4	★	`index.iter` LC 173 Binary Search Tree Iterator	按序逐个取 BST 元素、要求 O(h) 空间与均摊 O(1)，并且可能中途停下。
M4	★	`index.kth` LC 230 Kth Smallest Element in a BST	在 BST 上要“第 k 小”，并希望 k 小时代价小。
M4	★	`index.validate` LC 98 Validate Binary Search Tree	校验“整个子树”满足大小关系的不变量，而不是局部父子关系。
M4	★	`index.min_gap` LC 530 Minimum Absolute Difference in BST	BST 上求“任意两值的最小差”，等价于有序序列相邻差。
M4		`index.insert` LC 701 Insert into a Binary Search Tree	向 BST 加一个不存在的键，O(h)，结果任意合法 BST。
M4		`index.delete` LC 450 Delete Node in a BST	从 BST 删除任意键并保持 BST，O(h)。
M4		`index.restore` LC 1008 Construct Binary Search Tree from Preorder Traversal	BST 的前序序列（值互异）→ 唯一的树，要求 O(n)。

栈 7

信号：嵌套结构（括号、路径中的 `..` 、 `k[...]` 模板、 `!(&(...))` ）、后缀表达式、「最近一个未匹配的东西」。栈把「回到外层」变成 pop。

分界：只要求「最近的更大/更小元素」是**单调栈**；需要完整语法树、错误恢复或运算符优先级很多时，用**递归下降解析**更清晰——本仓库的配置表达式都足够简单，栈是最省的。

M5	★	`ir.rpn` LC 150 Evaluate Reverse Polish Notation	「后缀表达式」「字节码解释器」——操作数栈，遇运算符弹两个算一个。
M5	★	`config.brackets` LC 20 Valid Parentheses	「多种括号」「正确嵌套」「后开先闭」——栈匹配。
M5	★	`config.norm_path` LC 71 Simplify Path	「.. 回退一级」「规范化路径」——按分隔符切分 + 栈。
M5	★	`config.expr` LC 224 Basic Calculator	「只有加减和括号」「一元负号」「不许 eval」——(累计值, 符号) 栈。
M5		`config.expand` LC 394 Decode String	「k[片段] 可嵌套」「重复展开」——(前缀, 次数) 栈。
M5		`config.arith` LC 227 Basic Calculator II	「+ − * / 有优先级、无括号」「不许 eval」——上一个运算符 + 项栈。
M5		`config.flags` LC 1106 Parsing A Boolean Expression	「前缀运算符 + 括号 + 逗号分隔的变参」「布尔表达式」——字符栈，遇) 归约。

堆 6

信号：反复取「当前最小/最大」，而集合在变（top-k、k 路归并、最小空闲编号、能按时完成的最多任务）。插入/弹出 O(log n)。

分界：只要一次性的第 k 大用**快速选择**平均 O(n)；需要删除任意元素、按名次查询、范围查询时，堆不够，换 **BST/有序容器**（M4 的 EDF 队列就是这个原因）；同时要最大和最小、或要中位数，用**双堆**。

M1	★	`sample.kth_largest` LC 215 Kth Largest Element in an Array	「第 k 大」+「优于全排序」+ k ≪ n → 大小为 k 的小顶堆（或快速选择）。
M2		`sched.slots` LC 1845 Seat Reservation Manager	“总是分配编号最小的空闲 X”，同时有归还操作：最小堆（可加一个未发出计数器）。
M2		`sched.slo_bound` LC 630 Course Schedule III	“每个任务有时长和截止期、单机串行、可放弃、最大化按时完成数” → 按截止期排序 + 大顶堆，超时丢最长（反悔贪心）。
M3	★	`kv.merge_free_k` LC 23 Merge k Sorted Lists	「k 条已排序序列合并」+「O(N log k)」：k 路归并，最小堆维护各链当前头（或分治两两归并）。
M6	★	`decode.beam_pairs` LC 373 Find K Pairs with Smallest Sums	两个有序列表、组合代价是和、只要最小的 k 个且明确禁止枚举全部——多路归并的隐式链加最小堆。
M7		`metrics.slowest_k` LC 703 Kth Largest Element in a Stream	「流式」「第 k 大」「O(log k)、O(k) 空间」——固定大小 k 的最小堆。

设计（数据结构） 6

信号：题目给出一组操作及其复杂度要求（例如「均为 O(1)」），而不是一个输入输出。做法是**组合**：哈希表负责定位，链表/桶/数组负责顺序或计数，必要时再加一个指针（min_freq）或懒删除。

分界：先列出每个操作需要的「定位」「顺序」「极值」三种能力，再为每种能力选结构；操作之间共享同一个节点对象是 O(1) 的关键。

M2		`server.tokens` LC 1797 Design Authentication Manager	“固定 TTL + 续期 + 统计未过期数”，且时间单调：按写入顺序即按过期顺序，用有序哈希表从头过期。
M3		`kv.lfu` LC 460 LFU Cache	“淘汰使用次数最少的，并列时淘汰最久未用的”加上 O(1)：频率桶 + 每桶内按最近性排序 + min_freq 指针。
M3		`kv.ring_queue` LC 622 Design Circular Queue	“容量固定”“不允许动态扩容”“满了拒绝”：定长数组 + head/size 取模。
M3		`kv.mem_alloc` LC 2502 Design Memory Allocator	“分配最左侧的连续空闲段”“按 owner 释放”：first-fit 线性扫描，维护 run 计数。
M3		`kv.refcount` LC 432 All O(1) Data Structure	“计数 ±1”加上“O(1) 取最大和最小”：计数桶组成的有序双向链表 + 键到桶的哈希。
M7	★	`dash.mask_failed` LC 73 Set Matrix Zeroes	「整行整列置零」「只看原矩阵」「O(1) 额外空间」——用第 0 行 / 第 0 列做原地标记。

区间合并 5

信号：并集、利用率、合并重叠/相邻、插入一个新区间。按起点排序后一遍扫描；在线版本用有序结构维护不相交区间，插入时只合并被影响的那几段。

分界：「一次动作覆盖所有包含 t 的区间，最少几次」是**按终点排序的贪心**；「同时重叠的最大数量」是**扫描线**；三者经常被混淆，例如 [[1,2],[2,3],[3,4]] 合并后是一个块，但需要 2 次发射。

M2	★	`sched.merge_busy` LC 56 Merge Intervals	“合并重叠区间”“计算并集/利用率/覆盖时长” → 按起点排序，只和最后一段比较。
M2	★	`sched.insert_busy` LC 57 Insert Interval	“已排序、不重叠的区间列表，插入一个新区间并合并”：一遍扫描分左、中、右三段。
M3		`kv.free_tracker` LC 352 Data Stream as Disjoint Intervals	“数据流逐个加入整数”加上“随时列出不相交的连续区间”：有序起点列表 + 二分 + 四种合并情况。
M3		`kv.address_ranges` LC 715 Range Module	“半开区间的增、删、整段是否覆盖”，可以重叠，也可以从中间挖空：有序边界列表 + 奇偶判断（或线段树）。
M7		`metrics.busy_time` LC 2276 Count Integers in Intervals	在线加区间、问并集大小、长度到 10^9、要求 count O(1)——有序不相交区间 + 二分定位 + 维护总数。

哈希表 5

信号：去重、计数、按键 O(1) 查找、「是否出现过」、建立对象到对象的映射（深拷贝时旧节点 → 新节点）。

分界：需要有序遍历时换 **BST/排序**；键是前缀关系时换 **Trie**；需要同时随机取元素时，哈希表 + 数组（交换到末尾删除）才能做到 O(1)。

M1		`tok.top_pieces` LC 347 Top K Frequent Elements	「出现次数最多的 k 个」+「优于 O(n log n)」+ 次数为有界整数 → 计数 + 桶排序（或堆）。

M3	★	`kv.fork` LC 138 Copy List with Random Pointer	「深拷贝」+「指针可指向任意（包括后面的）结点」：需要旧→新映射，两遍处理前向引用。
M3	★	`kv.longest_run` LC 128 Longest Consecutive Sequence	「无序」+「最长连续值段」+「O(n)、不能排序」：哈希集合，只从段起点向上扩展。
M3	★	`kv.random_evict` LC 380 Insert Delete GetRandom O(1)	“O(1) 插入、删除，并且等概率随机取一个”：用数组 + 值到下标的哈希，删除时和末尾交换。
M3		`kv.block_hash` LC 706 Design HashMap	“不许用内置 dict/set”加上“整数键、范围有限”：桶数组 + 拉链（或开放寻址）。

拓扑排序 5

信号：有向依赖、执行顺序、能否全部完成、关键路径、可达性。Kahn 算法（入度为 0 入队）天然给出顺序并能检测环；关键路径是在拓扑序上做 DP。

分界：只问「有没有环」DFS 三色标记也行；要回答大量「u 是否先于 v」的查询，预先算**传递闭包**（位集）比每次 DFS 划算；无向连通性用**并查集**，带权最短路用 **Dijkstra**。

M5	★	`ir.order_ok` LC 207 Course Schedule	「依赖」「先修」「是否存在合法顺序」——有向图 + 只问可行性 = Kahn 计数判环。
M5	★	`ir.order` LC 210 Course Schedule II	「给出任一合法执行顺序」「有环返回空」——要排列本身的拓扑排序。
M5		`ir.critical_path` LC 2050 Parallel Courses III	「无限并行」「最短完成时间」「依赖」——DAG 最长路 = 拓扑序上的 max DP。
M5		`ir.depends_on` LC 1462 Course Schedule IV	「传递地先于」「批量查询」「n ⩽ 100」——先求传递闭包（位掩码），再 O(1) 回答。
M5		`ir.computable` LC 2115 Find All Possible Recipes from Given Supplies	「给定初始供给，哪些能做出来」「产物可作原料」「有环则做不出」——二部图上的 Kahn 传播（缺失计数归零）。

二维 DP 4

信号：两个序列的对齐：编辑距离、最长公共子序列、交错、带 `*` 的匹配。`dp[i][j]` = 前缀 a[:i] 与 b[:j] 的答案，转移只看左、上、左上。

分界：只有一个序列、或另一维可以贪心掉（例如判断子序列）时，**双指针**就够；空间可以滚动成一维，但要注意转移方向。

M6	★	`eval.edit_distance` LC 72 Edit Distance	两个序列、三种单字符操作、求最少步数——经典的双序列 DP，状态是两个前缀长度。
M6	★	`eval.interleave_ok` LC 97 Interleaving String	三个串、两个源按原顺序合成第三个、字符可能重复导致选择歧义——状态 (i, j) 隐含 merged 的位置 i+j。
M6		`eval.rouge_l` LC 1143 Longest Common Subsequence	两个序列、找「保持顺序、可不连续」的最长公共部分——LCS 双序列 DP。
M6		`decode.regex` LC 10 Regular Expression Matching	带「*」（修饰前一元素）与「.」的完整匹配、禁止用 re——后缀 DP，状态 (i, j)。

堆 + 模拟 4

信号：「谁接这个任务」——多个 worker/槽位、任务按时间到达、有优先规则（最小权重、最小编号、轮询起点）、全忙时延迟/排队/丢弃。标准结构是**空闲堆 + 忙碌堆（按释放时间）**，每个事件先把到期的忙碌 worker 移回空闲堆。

分界：只问「最少需要几个 worker」不需要模拟，用**扫描线**数峰值并发；只问「最多按时完成几个」用**贪心 + 最大堆**（交换论证）。

M2		`sched.sjf` LC 1834 Single-Threaded CPU	单服务者、任务按时间到达、每次从已到达者中按某个键挑一个 → 按到达排序的指针 + 就绪小顶堆 + 时钟跳跃。
M2		`sched.assign_workers` LC 1882 Process Tasks Using Servers	“多台服务器 + 优先级挑空闲服务器 + 任务排队等待 + 输出每个任务去了哪” → 空闲堆 + 忙碌堆的事件模拟。
M2		`sched.busiest_worker` LC 2402 Meeting Rooms III	“固定数量的房间/服务器 + 编号优先 + 全满就延后保持时长 + 问谁最忙” → 空闲编号堆 + (结束, 编号) 堆。
M2		`sched.rr_busiest` LC 1606 Find Servers That Handled Most Number of Requests	“从 i % k 号开始循环找第一个空闲服务器，全忙丢弃，问谁最忙” → 忙碌堆 + 支持“循环后继”的空闲集合（虚拟编号小顶堆或有序集合）。

Trie 3

信号：前缀查询、多个模式串同时匹配、「以某前缀开头的所有键」。插入/查询都是 O(键长)，与键的数量无关。

分界：只做精确查找用**哈希表**；多模式的流式匹配，本仓库用「反向 Trie + 有界缓冲」，通用做法是 Aho–Corasick；节省空间的变体是**基数树**（SGLang 的 RadixAttention）。

M1		`stream.multi_stop` LC 1032 Stream of Characters	「字符流逐个到来」+「是否以若干模式之一结尾」+ 每次 O(最长模式) → 倒序 Trie + 有界缓冲。
M4	★	`prefix.trie` LC 208 Implement Trie (Prefix Tree)	“是否存在以此开头的键”且要求复杂度只与查询长度有关、与键数无关。
M4		`prefix.hit_sum` LC 677 Map Sum Pairs	“以某前缀开头的所有键的值之和”，且值会被覆盖更新。

一维 DP 3

信号：前缀上的最优值/可行性，且第 i 个状态只依赖前面若干个（最大子数组、能否切分成词表中的词）。

分界：最大子数组的 Kadane 是一维 DP 的状态压缩；环形版本 = max(普通最大, 总和 − 最小子数组)，但全负时要特判；切分问题如果词表很大，用 **Trie** 加速内层循环。

M1	★	`tok.segment` LC 139 Word Break	「能否拆成若干个可重复使用的词」+「多项式时间，不能枚举切法」→ 前缀可达性 DP。
M2	★	`metrics.max_burst` LC 53 Maximum Subarray	“连续一段的累计增长最大是多少”，元素有正有负：Kadane / 最大子段和。
M2	★	`metrics.max_burst_circular` LC 918 Maximum Sum Circular Subarray	“循环/周期序列上的最大子段和，可跨过末尾”：max(Kadane 最大, 总和 - Kadane 最小)，注意全负。

二分答案 3

信号：「最小的容量/速率/时间，使得能在期限内完成」。答案空间单调：x 可行则 x+1 也可行。写一个 O(n) 的贪心可行性检查，在 [下界, 上界] 上二分，总 O(n log 范围)。

分界：如果有闭式公式（例如所有任务可任意重排时的平均负载）就不用二分；线性扫描答案空间在值域 10^9 时会超时——本仓库的测试专门卡这一点。

M7	`autoscale.min_throughput` LC 1011 Capacity To Ship Packages Within D Days	「至少需要多大的上限」「必须按顺序、连续分段」「逐个尝试会超时」——二分答案 + 贪心可行性检查。
M7	`autoscale.min_rate` LC 875 Koko Eating Bananas	「最小速率」「每片只服务一个队列、剩余浪费」「逐个尝试会超时」——二分答案，可行性 Σ⌈q/r⌉ ⩽ H。
M7	`autoscale.min_time` LC 2187 Minimum Time to Complete Trips	「最短时间」「到时刻 T 完成 ⌊T/t_i⌋ 个」「答案可达 10^14」——单调的计数函数上二分答案。

位运算 3

信号：异或抵消（成对出现的元素消失）、按位计数、按位模式校验（UTF-8 的 10xxxxxx）、位图。

分界：异或只能找出「恰好一个」落单的元素；落单多个时要先按某一位分组，或者像本仓库的泄漏检测那样二分区间再异或。

M2	`server.utf8_ok` LC 393 UTF-8 Validation	“按首字节高位判断长度、后续字节固定前缀”：位运算 + 计数器状态机。
M3 ★	`kv.popcount` LC 191 Number of 1 Bits	“二进制中 1 的个数”或“统计位图中被占用的块”，并且禁止库函数：用 n & (n-1)。
M3 ★	`kv.leaked` LC 136 Single Number	“其余元素都恰好出现两次”加上 O(1) 额外空间：把所有元素异或起来。

双堆 3

信号：在线中位数、或同时需要两类元素的极值。左边最大堆 + 右边最小堆，维持大小差 ⩽ 1；滑动窗口版本要配**懒删除**。

分界：如果只是固定窗口的最大值，用**单调队列** O(n) 更好；如果数据允许离线排序，直接排序取中位数更简单。

M2 ★	`metrics.median` LC 295 Find Median from Data Stream	“数据流在线中位数 / 分位数”，只增不删：最大堆 + 最小堆各存一半。
M2	`metrics.window_median` LC 480 Sliding Window Median	“滑动窗口的中位数/分位数”：双堆 + 惰性删除（或有序容器）。
M7	`metrics.corrected` LC 2034 Stock Price Fluctuation	「乱序到达」「可被修正覆盖」「随时最大/最小」——哈希表存真值 + 两个懒删除堆。

回溯 3

信号：枚举**所有**组合/排列/合法结构（benchmark 网格、所有到达顺序、所有平衡括号串）。递归做选择 → 递归 → 撤销，并在不合法时剪枝。

分界：只要**数量**用组合数学或 DP；只要一个可行解用 DFS/贪心；输出规模本身是指数级时，回溯已经是最优的。

M6	★	`bench.choose` LC 77 Combinations	列出所有「选 k 个、不计顺序」的子集——组合回溯，用递增顺序去重。
M6	★	`bench.orders` LC 46 Permutations	「所有到达顺序」「每个排列恰好一次」，规模 ⩽ 6——全排列回溯，穷举可行。
M6	★	`bench.json_fuzz` LC 22 Generate Parentheses	「生成所有合法括号」「每个恰好一次」——带计数约束的回溯，结果个数是 Catalan 数。

树的递归 3

信号：答案能由左右子树的答案组合出来（深度、是否相同、最近公共祖先）。写清楚「函数返回什么」是关键。

分界：要按层输出用 **BFS**；在 BST 上找 LCA 可以利用有序性只走一条路径 O(h)，一般二叉树需要后序递归 O(n)。

M4	★	`prefix.fork_point` LC 236 Lowest Common Ancestor of a Binary Tree	两个给定节点、求「共同祖先中最深的」，而且不能（或不该）依赖节点值的顺序。
M4	★	`prefix.depth` LC 104 Maximum Depth of Binary Tree	“根到最远叶子”的长度，子问题只依赖左右子树的同类答案。
M4	★	`prefix.same_snapshot` LC 100 Same Tree	两棵树同步逐位置比较，判定“结构和值完全一致”。

滑动窗口 3

信号：连续子数组/子串上的「最长/最短/计数」，且窗口扩张、收缩时状态可以增量维护（计数表、和）。

分界：窗口内要求**最大/最小值**时，计数表不够，用**单调队列**；要求中位数用**双堆 + 懒删除**；不连续（子序列）问题不是滑动窗口。

M1	★	`stream.unique_run` LC 3 Longest Substring Without Repeating Characters	「最长的连续片段」+「片段内不能重复」+ O(n) → 可变滑动窗口 + 哈希表记录最后位置。
M1	★	`stream.repeat_window` LC 219 Contains Duplicate II	「距离 ⩽ k 内是否有相同值」+ O(n)/O(k) 空间 → 定长窗口集合。
M2		`metrics.qps` LC 933 Number of Recent Calls	“最近 T 时间内的计数”且时间戳单调：队列，头部过期。

贪心 3

信号：局部最优可以被证明不劣于全局最优（交换论证）：按终点排序的区间覆盖、按行装词的两端对齐、冷却时间下的任务排布公式。

分界：贪心的正确性需要证明；当选择之间有后效（选了这个影响以后能选什么的价值）时往往要 **DP**。

M1	★	`stream.justify` LC 68 Text Justification	「每行尽可能多放」+「空格均分、余数靠左」→ 贪心分行 + divmod 分配。
M2	★	`sched.min_launches` LC 452 Minimum Number of Arrows to Burst Balloons	“一次操作命中所有包含某点的区间，求最少操作次数”（射气球、最少发射）→ 按右端点排序，在右端点出手。
M2		`sched.cooldown` LC 621 Task Scheduler	“同类任务之间至少间隔 n、顺序任意、求最少总时长” → 按最高频任务建骨架的计数公式。

链表操作 3

信号：重排/删除/合并节点而**不分配新空间**。几乎都靠哑结点（dummy head）消除头结点特判，靠「先连后断」避免丢链。

分界：判断环/找中点用**快慢指针**；排序整条链用**归并**（链表上归并不需要额外数组）；需要随机访问时链表就是错的结构。

M3	★	`kv.partition_hot` LC 86 Partition List	「按阈值分两组」+「组内保持原相对顺序」：稳定分区，链表上用双哑结点拼接。
M3	★	`kv.merge_free` LC 21 Merge Two Sorted Lists	「两条已排序」+「合成一条有序」：二路归并，哑结点 + 尾指针。
M3	★	`kv.drop_double_free` LC 82 Remove Duplicates from Sorted List II	「有序链表」+「凡重复的全部删除」：哑结点 + prev 只在确认唯一时前进。

分治/归并 2

信号：把问题切成两半独立解决再合并：链表归并排序、从有序数组建平衡 BST（取中点为根）。

分界：数组排序直接用内置排序；链表上快排不稳定且难以选主元，归并是自然选择。

M3	★	`kv.sort_free` LC 148 Sort List	「链表排序」+「O(n log n)」：链表上归并排序，快慢指针找中点。
M4	★	`index.build` LC 108 Convert Sorted Array to Binary Search Tree	有序数组 → 平衡树，“高度平衡”提示取中点二分。

前缀和 2

信号：大量区间求和查询而数据不变（或很少变）；按权重随机选择（前缀和 + 二分）。二维版本用容斥 O(1) 查询矩形和。

分界：数据会单点修改时，前缀和的 O(n) 更新太慢，用**树状数组/线段树**；要区间加、单点查用**差分**。

M2		`sched.weighted_route` LC 528 Random Pick with Weight	“按权重比例随机选”“概率为 w[i]/Σw” → 前缀和 + 二分查找落点。
M7		`dash.region_sum` LC 304 Range Sum Query 2D - Immutable	静态二维矩阵、任意矩形求和、查询 O(1)——二维前缀和 + 容斥。

字符串匹配（KMP） 2

信号：在文本中找模式、求「既是前缀又是后缀」的最长串。前缀函数让匹配失败时不回退文本指针，O(n+m)。

分界：单次短匹配 `str.find` 足够；多模式用 **Trie/Aho-Corasick**；本仓库用前缀函数精确计算流式输出要扣留多少字符，比扣留「最长 stop 长度 − 1」少得多。

M1	★	`stream.find_stop` LC 28 Find the Index of the First Occurrence in a String	「子串首次出现位置」+「禁用内置查找」→ KMP 前缀函数。
M1		`stream.stop_overlap` LC 1392 Longest Happy Prefix	「既是前缀又是后缀的最长子串」+ O(n) → KMP 前缀函数最后一项。

差分/扫描线 2

信号：「任意时刻的总占用是否超过容量」「峰值并发」。把区间拆成 (+w at start, −w at end) 事件，排序后扫一遍。

分界：时间值域小时可以开数组做差分；值域大（真实 tick 会不断增长）时必须排序事件——本仓库因此去掉了原题「时间 ⩽ 1000」的限制。

M2	`sched.capacity_ok` LC 1094 Car Pooling	“每个请求在 [start, end) 占用 w 个单位，任意时刻总量是否 ⩽ C”，时间范围很大：端点事件排序 + 累加。
M2	`sched.min_workers` LC 2406 Divide Intervals Into Minimum Number of Groups	“区间互斥占用资源，求最少资源/最少分组/最少会议室” → 峰值重叠深度：扫描线或按起点排序 + 终点小顶堆。

树的 BFS 2

信号：按层处理：层序输出、序列化（层序 + 空位标记）。队列保证同一层的节点连续出现。

分界：只需要深度或子树信息用 **DFS**；很深的树（例如 3000 层的链）上递归 DFS 会爆栈，BFS 或迭代式更稳。

M4	★	`prefix.level_dump` LC 102 Binary Tree Level Order Traversal	按层分组输出，需要知道“这一层到哪里结束”。
M4		`prefix.codec` LC 297 Serialize and Deserialize Binary Tree	任意二叉树无损往返、值可重复、树可极深——需要空位标记且不能递归。

单调队列 1

信号：滑动窗口最大/最小值，O(n)。双端队列里保存「还可能成为最大值」的下标，入队时弹掉比它小的。

分界：窗口中位数需要**双堆**；不定长窗口的最值仍可用单调队列，只要左端点单调右移。

M2	`metrics.window_max` LC 239 Sliding Window Maximum	“每个固定长度滑动窗口的最大/最小值，要求 O(n)”：单调递减（递增）双端队列。

哈希表 + 双向链表 1

信号：O(1) 的「访问后移到最前」+「淘汰最后」。哈希表定位节点，双向链表维护最近使用顺序，两者指向同一个节点。

分界：按**次数**淘汰是 LFU：再加频率桶和 `min_freq` 指针；随机淘汰只要哈希表 + 数组。

M3	★	`kv.lru` LC 146 LRU Cache	「O(1) 的 get/put」+「淘汰最久未使用」：按键随机访问加按时间顺序增删，必然是哈希表配有序链。

图的 BFS 1

信号：遍历/复制一张图，每个节点只处理一次；配合「旧节点 → 新节点」的哈希表实现深拷贝。

分界：带权最短路用 **Dijkstra**；依赖顺序用**拓扑排序**。

M5	★	`ir.clone` LC 133 Clone Graph	「深拷贝」「图可能有环」「不得共享节点」——哈希表做 old→new 映射 + BFS/DFS。

字符串编码（长度前缀） 1

信号：把任意字符串列表无损编码成一个串（可能包含任何分隔符）。用「长度 + 分隔符 + 内容」，解码时先读长度再按长度截取。

分界：用分隔符 + 转义也能做，但转义规则容易出错；JSON/pickle 是生产选择，练习里禁用。

M2		`stream.frame_codec` LC 271 Encode and Decode Strings	“把任意字符串列表编码成一个串并无损还原，内容可能包含任何字符”：长度前缀分帧。

并查集 1

信号：无向连通性、分组计数、在线合并。路径压缩 + 按秩合并，近似 O(1)。

分界：只算一次连通分量数，DFS/BFS 同样 $O(n^2)$（邻接矩阵）；需要删边时并查集不支持。

M5		`ir.device_groups` LC 547 Number of Provinces	「直接或间接相连」「有多少组」「邻接矩阵」——连通分量计数，并查集或 DFS。

快慢指针 1

信号：链表判环、找中点、找入环点，要求 O(1) 额外空间。

分界：允许 O(n) 空间时用哈希集合记录访问过的节点更直观。

M3	★	`kv.has_cycle` LC 141 Linked List Cycle	「链表是否成环」+「O(1) 额外空间、不得修改」：快慢指针是唯一同时满足两者的标准解。

投票（Boyer–Moore） 1

信号：「出现次数超过一半」的元素，O(n) 时间 O(1) 空间。抵消后剩下的候选者是唯一可能的多数元素。

分界：不保证存在多数时，必须再数一遍确认（本仓库的热点租户检测就是这样做的）；找出现超过 n/3 的元素要维护两个候选。

M2	★	`metrics.dominant` LC 169 Majority Element	“找出现次数超过一半的元素，O(1) 额外空间”：Boyer–Moore 投票 + 一遍确认。

最短路（Dijkstra） 1

信号：非负权有向图，从一个源出发到所有点的最短时间，「全部收到」= 最短路的最大值。

分界：有负权用 Bellman–Ford；所有点对、点数很少时用 Floyd；无权图 BFS 即可。

M5	`ir.op_latency` LC 743 Network Delay Time	「有向带权」「从一点出发到所有点的最早时刻」「权非负」——Dijkstra 求最大最短路。

树状数组（Fenwick） 1

信号：单点修改 + 前缀/区间求和，都要 O(log n)。 `i & -i` 取最低位跳转。

分界：数据不变用**前缀和**；要区间修改+区间查询或区间最值，用**线段树**。

M4	`index.range_sum` LC 307 Range Sum Query - Mutable	频繁单点修改 + 频繁区间求和，二者都要 O(log n)。

状态机 1

信号：逐字符扫描，行为取决于「当前处于什么模式」（在块注释里/不在）。写清楚每个状态遇到每类字符时的转移。

分界：模式能嵌套（括号）时状态机不够，需要**栈**。

M5	`config.strip_comments` LC 722 Remove Comments	「块注释可跨行」「最先出现的起始符生效」——跨行保留状态的扫描器。

附录 A　117 题完整对应表

		练习	LC	LeetCode 题目	位置（nanoserve/…）
M1	★	`tok.segment`	139	Word Break	`tokenizer/segment.py`
M1	★	`tok.common_prefix`	14	Longest Common Prefix	`tokenizer/prefix.py`
M1	★	`stream.find_stop`	28	Find the Index of the First Occurrence in a String	`stream/stop.py`
M1	★	`stream.justify`	68	Text Justification	`stream/render.py`
M1	★	`stream.holdback`	58	Length of Last Word	`stream/detok.py`
M1	★	`stream.unique_run`	3	Longest Substring Without Repeating Characters	`stream/repetition.py`
M1	★	`stream.repeat_window`	219	Contains Duplicate II	`stream/repetition.py`
M1	★	`sample.cdf_index`	35	Search Insert Position	`sampling/search.py`
M1	★	`sample.kth_largest`	215	Kth Largest Element in an Array	`sampling/topk.py`
M1		`tok.top_pieces`	347	Top K Frequent Elements	`tokenizer/train.py`
M1		`stream.multi_stop`	1032	Stream of Characters	`stream/multistop.py`
M1		`stream.stop_overlap`	1392	Longest Happy Prefix	`stream/overlap.py`
M2		`stream.frame_codec`	271	Encode and Decode Strings	`stream/protocol.py`
M2	★	`metrics.median`	295	Find Median from Data Stream	`metrics/latency.py`
M2	★	`metrics.merged_median`	4	Median of Two Sorted Arrays	`metrics/latency.py`
M2	★	`sched.merge_busy`	56	Merge Intervals	`scheduler/timeline.py`
M2	★	`sched.insert_busy`	57	Insert Interval	`scheduler/timeline.py`
M2	★	`sched.min_launches`	452	Minimum Number of Arrows to Burst Balloons	`scheduler/batching.py`
M2	★	`metrics.dominant`	169	Majority Element	`metrics/tenants.py`
M2	★	`metrics.max_burst`	53	Maximum Subarray	`metrics/burst.py`
M2	★	`metrics.max_burst_circular`	918	Maximum Sum Circular Subarray	`metrics/burst.py`
M2		`sched.weighted_route`	528	Random Pick with Weight	`scheduler/routing.py`
M2		`sched.capacity_ok`	1094	Car Pooling	`scheduler/capacity.py`
M2		`sched.sjf`	1834	Single-Threaded CPU	`scheduler/simulate.py`
M2		`sched.assign_workers`	1882	Process Tasks Using Servers	`scheduler/simulate.py`
M2		`sched.busiest_worker`	2402	Meeting Rooms III	`scheduler/simulate.py`
M2		`sched.cooldown`	621	Task Scheduler	`scheduler/cooldown.py`
M2		`metrics.qps`	933	Number of Recent Calls	`metrics/window.py`
M2		`metrics.window_max`	239	Sliding Window Maximum	`metrics/window.py`
M2		`metrics.window_median`	480	Sliding Window Median	`metrics/window.py`
M2		`server.utf8_ok`	393	UTF-8 Validation	`server/utf8.py`
M2		`server.tokens`	1797	Design Authentication Manager	`server/auth_tokens.py`
M2		`sched.slots`	1845	Seat Reservation Manager	`scheduler/batch_slots.py`
M2		`sched.maintenance`	729	My Calendar I	`scheduler/calendar.py`

		练习	LC	LeetCode 题目	位置（nanoserve/…）
M2		`sched.min_workers`	2406	Divide Intervals Into Minimum Number of Groups	`scheduler/planning.py`
M2		`sched.slo_bound`	630	Course Schedule III	`scheduler/planning.py`
M2		`sched.rr_busiest`	1606	Find Servers That Handled Most Number of Requests	`scheduler/lb.py`
M3	★	`kv.lru`	146	LRU Cache	`kvcache/lru.py`
M3	★	`kv.fork`	138	Copy List with Random Pointer	`kvcache/block_table.py`
M3	★	`kv.has_cycle`	141	Linked List Cycle	`kvcache/block_table.py`
M3	★	`kv.partition_hot`	86	Partition List	`kvcache/block_table.py`
M3	★	`kv.merge_free`	21	Merge Two Sorted Lists	`kvcache/free_list.py`
M3	★	`kv.merge_free_k`	23	Merge k Sorted Lists	`kvcache/free_list.py`
M3	★	`kv.sort_free`	148	Sort List	`kvcache/free_list.py`
M3	★	`kv.drop_double_free`	82	Remove Duplicates from Sorted List II	`kvcache/free_list.py`
M3	★	`kv.compact`	27	Remove Element	`kvcache/compact.py`
M3	★	`kv.dedupe`	26	Remove Duplicates from Sorted Array	`kvcache/compact.py`
M3	★	`kv.merge_arrays`	88	Merge Sorted Array	`kvcache/compact.py`
M3	★	`kv.rotate_buffer`	189	Rotate Array	`kvcache/ring.py`
M3	★	`kv.ring_search`	33	Search in Rotated Sorted Array	`kvcache/ring.py`
M3	★	`kv.ring_head`	153	Find Minimum in Rotated Sorted Array	`kvcache/ring.py`
M3	★	`kv.longest_run`	128	Longest Consecutive Sequence	`kvcache/allocator.py`
M3	★	`kv.free_ranges`	228	Summary Ranges	`kvcache/allocator.py`
M3	★	`kv.seq_range`	34	Find First and Last Position of Element in Sorted Array	`kvcache/allocator.py`
M3	★	`kv.page_lookup`	74	Search a 2D Matrix	`kvcache/page_table.py`
M3	★	`kv.popcount`	191	Number of 1 Bits	`kvcache/bitmap.py`
M3	★	`kv.leaked`	136	Single Number	`kvcache/leak.py`
M3	★	`kv.random_evict`	380	Insert Delete GetRandom O(1)	`kvcache/evict.py`
M3		`kv.lfu`	460	LFU Cache	`kvcache/lfu.py`
M3		`kv.ring_queue`	622	Design Circular Queue	`kvcache/ring_queue.py`
M3		`kv.snapshot`	1146	Snapshot Array	`kvcache/snapshot.py`
M3		`kv.versioned`	981	Time Based Key-Value Store	`kvcache/versioned.py`
M3		`kv.mem_alloc`	2502	Design Memory Allocator	`kvcache/memalloc.py`
M3		`kv.refcount`	432	All O(1) Data Structure	`kvcache/refcount.py`
M3		`kv.block_hash`	706	Design HashMap	`kvcache/hashmap.py`
M3		`kv.free_tracker`	352	Data Stream as Disjoint Intervals	`kvcache/free_tracker.py`
M3		`kv.address_ranges`	715	Range Module	`kvcache/address_ranges.py`
M4	★	`prefix.trie`	208	Implement Trie (Prefix Tree)	`prefix/trie.py`
M4	★	`prefix.fork_point`	236	Lowest Common Ancestor of a Binary Tree	`prefix/tree_ops.py`

		练习	LC	LeetCode 题目	位置（nanoserve/…）
M4	★	`prefix.depth`	104	Maximum Depth of Binary Tree	`prefix/tree_ops.py`
M4	★	`prefix.level_dump`	102	Binary Tree Level Order Traversal	`prefix/dump.py`
M4	★	`prefix.same_snapshot`	100	Same Tree	`prefix/snapshot.py`
M4	★	`index.iter`	173	Binary Search Tree Iterator	`index/deadline.py`
M4	★	`index.kth`	230	Kth Smallest Element in a BST	`index/deadline.py`
M4	★	`index.validate`	98	Validate Binary Search Tree	`index/deadline.py`
M4	★	`index.min_gap`	530	Minimum Absolute Difference in BST	`index/deadline.py`
M4	★	`index.build`	108	Convert Sorted Array to Binary Search Tree	`index/deadline.py`
M4		`prefix.codec`	297	Serialize and Deserialize Binary Tree	`prefix/codec.py`
M4		`prefix.hit_sum`	677	Map Sum Pairs	`prefix/hit_sum.py`
M4		`index.range_sum`	307	Range Sum Query - Mutable	`index/fenwick.py`
M4		`index.insert`	701	Insert into a Binary Search Tree	`index/bst_ops.py`
M4		`index.delete`	450	Delete Node in a BST	`index/bst_ops.py`
M4		`index.restore`	1008	Construct Binary Search Tree from Preorder Traversal	`index/bst_ops.py`
M5	★	`ir.order_ok`	207	Course Schedule	`ir/graph.py`
M5	★	`ir.order`	210	Course Schedule II	`ir/graph.py`
M5	★	`ir.clone`	133	Clone Graph	`ir/graph.py`
M5	★	`ir.rpn`	150	Evaluate Reverse Polish Notation	`ir/interp.py`
M5	★	`config.brackets`	20	Valid Parentheses	`config/parser.py`
M5	★	`config.norm_path`	71	Simplify Path	`config/paths.py`
M5	★	`config.expr`	224	Basic Calculator	`config/expr.py`
M5		`ir.op_latency`	743	Network Delay Time	`ir/latency.py`
M5		`ir.critical_path`	2050	Parallel Courses III	`ir/latency.py`
M5		`ir.device_groups`	547	Number of Provinces	`ir/topology.py`
M5		`config.expand`	394	Decode String	`config/template.py`
M5		`config.arith`	227	Basic Calculator II	`config/arith.py`
M5		`ir.depends_on`	1462	Course Schedule IV	`ir/reach.py`
M5		`ir.computable`	2115	Find All Possible Recipes from Given Supplies	`ir/reach.py`
M5		`config.strip_comments`	722	Remove Comments	`config/comments.py`
M5		`config.flags`	1106	Parsing A Boolean Expression	`config/flags.py`
M5		`config.glob`	44	Wildcard Matching	`config/glob.py`
M6	★	`eval.edit_distance`	72	Edit Distance	`eval/metrics.py`
M6	★	`eval.interleave_ok`	97	Interleaving String	`eval/streams.py`
M6	★	`decode.beam_pairs`	373	Find K Pairs with Smallest Sums	`decode/beam.py`
M6	★	`bench.choose`	77	Combinations	`bench/sweep.py`
M6	★	`bench.orders`	46	Permutations	`bench/fuzz.py`
M6	★	`bench.json_fuzz`	22	Generate Parentheses	`bench/fuzz.py`

		练习	LC	LeetCode 题目	位置（nanoserve/…）
M6		`eval.rouge_l`	1143	Longest Common Subsequence	`eval/rouge.py`
M6		`decode.regex`	10	Regular Expression Matching	`decode/regex.py`
M7	★	`dash.mask_failed`	73	Set Matrix Zeroes	`dashboard/health.py`
M7		`dash.region_sum`	304	Range Sum Query 2D - Immutable	`dashboard/region.py`
M7	★	`dash.fuzzy`	392	Is Subsequence	`dashboard/filter.py`
M7		`metrics.slowest_k`	703	Kth Largest Element in a Stream	`metrics/slowest.py`
M7		`metrics.corrected`	2034	Stock Price Fluctuation	`metrics/series.py`
M7		`metrics.busy_time`	2276	Count Integers in Intervals	`metrics/busy.py`
M7		`autoscale.min_throughput`	1011	Capacity To Ship Packages Within D Days	`autoscale/sizing.py`
M7		`autoscale.min_rate`	875	Koko Eating Bananas	`autoscale/sizing.py`
M7		`autoscale.min_time`	2187	Minimum Time to Complete Trips	`autoscale/sizing.py`

附录 B　被删掉的 87 道 Top Interview 150 题

判断标准只有一条：引擎里是否有真实调用点。**换壳**（56 道）= 算法不变、硬套领域故事；**放不进引擎**（23 道）；**无真实调用点**（8 道）= 最初看似自然，但最终架构里用不到。这些题依然值得刷，只是不适合放在这个项目里。

LC	题目	类别	说明
1	Two Sum	换壳	曾被包装为：找两个请求的大小恰好加起来等于剩余预算，返回下标（保证唯一解）
2	Add Two Numbers	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
5	Longest Palindromic Substring	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
6	Zigzag Conversion	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
9	Palindrome Number	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
11	Container With Most Water	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
12	Integer to Roman	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
13	Roman to Integer	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
15	3Sum	换壳	曾被包装为：找出所有三个负载增量之和为 0 的不重复组合（迁移后负载守恒）
17	Letter Combinations of a Phone Number	换壳	曾被包装为：benchmark 配置码展开：每个数字 2-9 对应固定的一组选项字母（电话键盘映射），输出所有组合
19	Remove Nth Node From End of List	换壳	曾被包装为：滑动窗口注意力：从块链中移除倒数第 n 个块
25	Reverse Nodes in k-Group	换壳	曾被包装为：以 k 个块为一组原地反转，不足 k 的尾组保持原样
30	Substring with Concatenation of All Words	换壳	曾被包装为：找出所有由一组等长片段以任意顺序拼接而成的子串起点（乱序 stop 模式）
36	Valid Sudoku	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
39	Combination Sum	换壳	曾被包装为：用请求大小（可重复）凑出 target 的所有组合（不计顺序）
42	Trapping Rain Water	换壳	曾被包装为：显存占用剖面中被两侧高占用夹住、无法利用的碎片容量
45	Jump Game II	换壳	曾被包装为：流水线转发：stage i 最多可把激活转发到后面 hops[i] 个 stage，求最少转发次数（保证可达）
48	Rotate Image	换壳	曾被包装为：热力图原地顺时针旋转 90°（竖屏终端）
49	Group Anagrams	换壳	曾被包装为：按字符多重集把词表项分组，做词表冗余分析
50	Pow(x, n)	换壳	曾被包装为：指数退避系数 base^n（n 可为负），快速幂 O(log n)
52	N-Queens II	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
54	Spiral Matrix	换壳	曾被包装为：面板动画：按顺时针螺旋顺序读出格子
55	Jump Game	换壳	曾被包装为：流水线 stage i 最多可把激活转发到后面 hops[i] 个 stage，能否从第一个到达最后一个
61	Rotate List	换壳	曾被包装为：淘汰环的指针前移：块链向右旋转 k 位
63	Unique Paths II	换壳	曾被包装为：带约束掩码（1 为禁止）的解码格中路径条数
64	Minimum Path Sum	换壳	曾被包装为：解码格（只能向右/向下）从左上到右下的最小代价
66	Plus One	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
67	Add Binary	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
69	Sqrt(x)	放不进引擎	与推理服务器无关（曾放在可选的基本功区）

LC	题目	类别	说明
70	Climbing Stairs	换壳	曾被包装为：投机解码每步接受 1 或 2 个 token，生成 n 个 token 有多少种接受序列
76	Minimum Window Substring	换壳	曾被包装为：引用定位：输出中覆盖所有必需字符（含重复次数）的最短片段
79	Word Search	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
80	Remove Duplicates from Sorted Array II	换壳	曾被包装为：每个块最多保留 2 份引用记录，原地处理
92	Reverse Linked List II	换壳	曾被包装为：把块链第 left..right 个块（1-based）原地反转
101	Symmetric Tree	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
103	Binary Tree Zigzag Level Order Traversal	换壳	曾被包装为：蛇形按层 dump（终端可视化用）
105	Construct Binary Tree from Preorder and Inorder Traversal	无真实调用点	前序+中序恢复缓存树：checkpoint 最终用前序恢复 BST（LC 1008）与通用 codec
106	Construct Binary Tree from Inorder and Postorder Traversal	无真实调用点	中序+后序恢复缓存树：同上
112	Path Sum	换壳	曾被包装为：是否存在根到叶的缓存路径，token 总数恰为 target
114	Flatten Binary Tree to Linked List	无真实调用点	把缓存树展平成链：前缀缓存最终用 Trie 实现
117	Populating Next Right Pointers in Each Node II	换壳	曾被包装为：为按层淘汰扫描建立同层 next 指针（任意二叉树，O(1) 额外空间为进阶）
120	Triangle	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
121	Best Time to Buy and Sell Stock	换壳	曾被包装为：现货算力价格序列，买入一次再卖出一次的最大收益
122	Best Time to Buy and Sell Stock II	换壳	曾被包装为：不限次数（同时最多持有一份）的最大收益
123	Best Time to Buy and Sell Stock III	换壳	曾被包装为：最多两次交易的最大收益
124	Binary Tree Maximum Path Sum	换壳	曾被包装为：缓存树中任意一条路径的收益和最大值（节点值可为负）
125	Valid Palindrome	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
127	Word Ladder	换壳	曾被包装为：版本标签每次只改一个字母，求从 begin 到 end 的最短序列长度（含两端），不可达为 0
129	Sum Root to Leaf Numbers	换壳	曾被包装为：每条根到叶路径的数字拼成路径编码，求所有编码之和
130	Surrounded Regions	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
134	Gas Station	换壳	曾被包装为：环形 worker 轮转，经过 i 获得 gain[i] 配额、去下一个消耗 cost[i]；从哪个 worker 出发能转完一圈，否则 -1
135	Candy	换壳	曾被包装为：每个租户至少 1 个 credit，优先级高于相邻租户的必须拿得比邻居多；求最少总 credit
137	Single Number II	换壳	曾被包装为：三副本系统中每个块号出现三次，唯一只出现一次的块；O(1) 空间
149	Max Points on a Line	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
151	Reverse Words in a String	换壳	曾被包装为：日志倒序视图：单词逆序并规整空白
155	Min Stack	换壳	曾被包装为：解释器作用域栈，O(1) 查询当前最低显存水位
162	Find Peak Element	无真实调用点	资源曲线找峰值：面板最终不需要
167	Two Sum II - Input Array Is Sorted	换壳	曾被包装为：找两个请求的大小恰好加起来等于剩余预算，sizes 已非降序，返回 1-based 编号，O(1) 额外空间
172	Factorial Trailing Zeroes	放不进引擎	与推理服务器无关（曾放在可选的基本功区）

LC	题目	类别	说明
188	Best Time to Buy and Sell Stock IV	换壳	曾被包装为：最多 k 次交易的最大收益
190	Reverse Bits	换壳	曾被包装为：位图序列化时的位序转换：反转 32 位无符号整数
198	House Robber	换壳	曾被包装为：相邻时间片因预热冲突不能同时选，选若干时间片使总价值最大
199	Binary Tree Right Side View	换壳	曾被包装为：每层最右侧节点（最新插入的分支）
200	Number of Islands	换壳	曾被包装为：显存热力图中 '1' 的四连通占用块个数
201	Bitwise AND of Numbers Range	换壳	曾被包装为：地址区间 [lo, hi] 所有地址按位与（求共同对齐前缀）
202	Happy Number	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
205	Isomorphic Strings	无真实调用点	词表重映射校验：最终 tokenizer 不需要
209	Minimum Size Subarray Sum	无真实调用点	凑满 batch 的连续窗口：最终准入逻辑按槽位 + 预算，不需要
211	Design Add and Search Words Data Structure	无真实调用点	通配符缓存键：前缀缓存最终只需精确前缀匹配
212	Word Search II	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
221	Maximal Square	换壳	曾被包装为：'1'/'0' 健康图中全 '1' 最大正方形的面积
222	Count Complete Tree Nodes	换壳	曾被包装为：完全二叉树形态的优先队列节点数，要求低于 O(n)
226	Invert Binary Tree	换壳	曾被包装为：生成镜像视图（左右翻转）
238	Product of Array Except Self	换壳	曾被包装为：去掉第 i 个 stage 时其余 stage 的乘积因子（不允许用除法）
242	Valid Anagram	换壳	曾被包装为：乱序 round-trip 测试：两段文本的字符多重集是否完全一致
274	H-Index	换壳	曾被包装为：缓存条目命中数的 h 指数：至少 h 个条目各被命中 ≥ h 次
289	Game of Life	放不进引擎	与推理服务器无关（曾放在可选的基本功区）
290	Word Pattern	换壳	曾被包装为：对话模板的角色模式（如 'abba'）与空格分隔的角色序列是否一一对应
300	Longest Increasing Subsequence	换壳	曾被包装为：checkpoint 评分序列中最长严格递增子序列长度（O(n log n) 为进阶）
322	Coin Change	换壳	曾被包装为：用可选的 chunk 大小（可重复）恰好凑出 total 个 token 的最少 chunk 数，不可能为 -1
383	Ransom Note	换壳	曾被包装为：字符配额检查：text 的每个字符能否由 pool 提供（pool 中每个字符只能用一次）
399	Evaluate Division	换壳	曾被包装为：单位换算图：由已知比率推导查询比率，不可推导为 -1.0
427	Construct Quad Tree	换壳	曾被包装为：用四叉树压缩 0/1 块稀疏注意力掩码（n 为 2 的幂）
433	Minimum Genetic Mutation	换壳	曾被包装为：8 位配置码（字符 ACGT），每步改一位且中间态必须在 valid 中，求最少步数或 -1
502	IPO	换壳	曾被包装为：最多准入 k 个任务；任务 i 要求当前可用资源 ≥ needs[i]，完成后资源增加 gains[i]；求最终最大资源
637	Average of Levels in Binary Tree	无真实调用点	二叉缓存树的层平均：前缀缓存最终用 Trie 实现
909	Snakes and Ladders	放不进引擎	与推理服务器无关（曾放在可选的基本功区）